Tor 解剖

逐字节解析匿名协议

Alessandro Greco

第一版 — 版本 1.0.0

Tor 解剖——逐字节解析匿名协议
第一版，版本 1.0.0。简体中文译本。

Copyright © 2026 Alessandro Greco.

本作品是自由作品：正文、插图、LaTeX 源文件和实验代码均可依照自由软件基金会发布的 GNU 通用公共许可证（GNU General Public License）第 3 版或（由你选择）任何更新的版本（GPL-3.0-or-later）的条款再分发和修改。发布本作品是希望它有用，但不提供任何担保，甚至不包括对适销性或特定用途适用性的默示担保；详情见许可证：`https://www.gnu.org/licenses/gpl-3.0.html`。

可修改的源形式（LaTeX 源文件、TikZ 插图、实验代码和实验网络）发布于 `https://github.com/aleff-github/onion-routing-anatomy`。

本书免费分发；使用和再分发的权利以上述许可证的规定为准。

本译本完全借助人工智能工具，依据意大利文原版生成；意大利文原版仍是本作品的参考文本。

“Tor” 和 Tor 标志是 The Tor Project, Inc. 的商标。本书是独立作品，未获 Tor Project 认可或资助。

书中提及的产品和服务名称归各自所有者所有，仅用于描述目的。

目录

前言 **xi**

第一部分　历史与基础 **1**

第 **1** 章 匿名、监控与元数据 **3**
1.1 内容与元数据 3
1.1.1 元数据会暴露什么 3
1.2 流量分析 4
1.3 精确的术语 4
1.4 谁需要匿名？ 5
1.4.1 匿名需要同伴 5

第 **2** 章 源流：**Mix** 网络、**Remailer**、**Crowds** 与 **Freedom** **7**
2.1 Chaum 的 Mix（1981） 7
2.2 高延迟与低延迟 8
2.3 Remailer 8
2.4 走向低延迟 9

第 **3** 章 从 **NRL** 洋葱路由到 **Tor Project** **11**
3.1 在 Naval Research Laboratory 11
3.2 三代设计 11
3.3 Tor 网络 12
3.4 一个组织与一个社区 13

第 **4** 章 **Tor** 的威胁模型 **15**
4.1 目标 15
4.2 非目标 15
4.3 对手 16
4.3.1 确认攻击与分析攻击 16
4.4 Tor 能保护什么，又不能保护什么 17

第二部分　底层协议 **19**

第 **5** 章 **Tor** 所需的核心密码学 **21**
5.1 五项任务 21
5.2 加密算法 21
5.2.1 计数器模式下的 AES 21
5.2.2 TLS 22
5.3 哈希函数 22

5.4 MAC 与密钥派生 . . . 23
5.4.1 HMAC . . . 23
5.4.2 密钥派生函数 . . . 23
5.5 密钥交换：X25519 . . . 23
5.6 签名：Ed25519 与 RSA . . . 24
5.7 较新的密码学构造 . . . 24

第 **6** 章 网络架构 **27**
6.1 角色 . . . 27
6.2 协议层次 . . . 28
6.3 一个网页从开始到结束 . . . 29
6.4 实现与版本 . . . 29
6.4.1 两种实现 . . . 29
6.4.2 子协议 . . . 29

第 **7** 章 通道与链路协议 **31**
7.1 什么是通道? . . . 31
7.2 中继的身份 . . . 32
7.3 握手全貌 . . . 33
7.4 TLS 层 . . . 34
7.5 CERTS：证明身份 . . . 34
7.5.1 信元 . . . 34
7.5.2 Tor 的 Ed25519 证书 . . . 35
7.5.3 验证过程 . . . 35
7.5.4 旧式 RSA 链 . . . 36
7.5.5 中继实际发送什么 . . . 36
7.6 AUTH_CHALLENGE 与 AUTHENTICATE . . . 37
7.6.1 谁需要认证? . . . 37
7.6.2 认证挑战 . . . 37
7.6.3 认证响应 . . . 37
7.7 NETINFO：时间与地址 . . . 38
7.8 握手之后 . . . 38
7.9 实验：打开并验证一条通道 . . . 39

第 **8** 章 信元（**Cell**） **43**
8.1 为什么使用信元? . . . 43
8.2 从 TLS 字节流到信元 . . . 44
8.3 信元格式 . . . 44
8.3.1 常量 . . . 44
8.3.2 定长信元 . . . 44
8.3.3 变长信元 . . . 44
8.3.4 如何区分两种格式 . . . 45
8.3.5 VERSIONS 的特殊情况 . . . 45
8.4 CircID . . . 46
8.4.1 谁选择 CircID? . . . 46
8.5 信元命令 . . . 47
8.5.1 DESTROY . . . 48
8.5.2 RELAY_EARLY . . . 48
8.6 RELAY 信元内部 . . . 49
8.6.1 Recognized 与 Digest：两级过滤 . . . 49
8.6.2 RELAY 消息填充 . . . 49

8.6.3 RELAY 命令 50
8.7 走向新格式：提案 340 与 CGO 50
8.8 一次会话的完整结构 51
8.9 实验：信元编码器 51

第 **9** 章 电路构建：**TAP**、**CREATE_FAST**、**ntor** 与 **ntor-v3** **57**
9.1 一次只增加一跳 57
9.2 信元与消息 58
9.2.1 CREATE2 与 CREATED2 58
9.2.2 EXTEND2 与 EXTENDED2 58
9.3 一次握手必须实现什么 59
9.4 TAP：第一代握手 59
9.5 CREATE_FAST：何时不需要公钥 59
9.6 ntor 60
9.6.1 组成部分 60
9.6.2 协议 60
9.7 ntor-v3 61
9.7.1 变化 61
9.7.2 协议 61
9.7.3 扩展 62
9.7.4 今天 tor 会选择哪种握手？ 62
9.8 从共享秘密到电路密钥 62
9.9 实验：建立真实电路 63

第 **10** 章 洋葱：**RELAY** 信元加密与 **CGO** **67**
10.1 传统方案 67
10.1.1 前向：从客户端进入电路 67
10.1.2 反向：从电路返回客户端 68
10.1.3 作为“漏水管道”的电路 68
10.2 识别信元 68
10.3 tor1 的弱点 69
10.3.1 可塑性 69
10.3.2 标记（tagging）攻击 69
10.3.3 其他限制 69
10.4 Counter Galois Onion 69
10.4.1 性质 69
10.4.2 高层工作原理 70
10.4.3 实际部署中的 CGO 70
10.5 实验：三跳电路 70

第 **11** 章 流、**SENDME** 与拥塞控制 **75**
11.1 流 75
11.1.1 打开流：BEGIN 与 CONNECTED 75
11.1.2 传输数据：DATA 76
11.1.3 关闭流：END 76
11.1.4 在出口解析 DNS 77
11.1.5 目录流：BEGIN_DIR 77
11.2 原始流量控制 77
11.2.1 问题 77
11.2.2 窗口与 SENDME 77
11.2.3 经过认证的 SENDME 77

11.3 拥塞控制 . . . 78
11.3.1 信号：往返时间 . . . 78
11.3.2 算法 . . . 78
11.3.3 参数：为什么是 31? . . . 78
11.3.4 不再有流窗口：XON 与 XOFF . . . 78
11.4 实验：通过 Tor 下载文件 . . . 79

第 **12** 章 目录系统 **83**
12.1 问题 . . . 83
12.2 参与者 . . . 84
12.3 文档类型 . . . 84
12.4 投票 . . . 85
12.4.1 投票时间表 . . . 85
12.4.2 合并投票 . . . 85
12.4.3 中继标志（Flags） . . . 86
12.4.4 带宽 . . . 86
12.5 一份共识的结构 . . . 86
12.6 验证共识 . . . 87
12.7 客户端如何保持最新 . . . 88
12.8 实验：下载并验证共识 . . . 88

第 **13** 章 路径选择、入口守卫与 **Vanguards** **93**
13.1 顺序与约束 . . . 93
13.2 按带宽加权 . . . 94
13.2.1 为什么需要加权 . . . 94
13.2.2 不同位置并不等价 . . . 94
13.2.3 真实网络中的权重 . . . 94
13.3 入口守卫（Guard） . . . 95
13.3.1 它们解决什么问题 . . . 95
13.3.2 三个、一个、两个 . . . 95
13.3.3 算法 . . . 95
13.4 Vanguards 防护机制 . . . 96
13.5 实验：观察公共网络 . . . 97

第 **14** 章 洋葱服务 **v3** **99**
14.1 洋葱服务提供什么 . . . 99
14.2 地址 . . . 100
14.3 盲化密钥 . . . 100
14.3.1 问题 . . . 100
14.3.2 解决方案 . . . 100
14.3.3 凭据与离线密钥 . . . 101
14.4 定位描述符 . . . 101
14.4.1 时间周期 . . . 101
14.4.2 HSDir 环 . . . 101
14.4.3 只接受匿名请求 . . . 102
14.5 描述符 . . . 102
14.6 引介与汇合 . . . 103
14.7 受限发现 . . . 104
14.8 防御机制 . . . 105
14.9 实验：寻找并读取描述符 . . . 105

第 **15** 章 **DoS** 防护与资源管理 **109**
15.1 为什么 DoS 也是匿名性问题 . . . 109
15.2 物理资源 . . . 110
15.2.1 内存 . . . 110
15.2.2 CPU . . . 110
15.2.3 带宽 . . . 110
15.3 按地址实施限制 . . . 110
15.3.1 代码与共识 . . . 110
15.4 遭受攻击的洋葱服务 . . . 111
15.4.1 引介点的限速机制 . . . 111
15.4.2 工作量证明 . . . 111
15.4.3 线性工作量 . . . 112
15.4.4 控制工作量 . . . 112
15.5 实验：触发防御机制 . . . 113

第 **16** 章 审查、网桥与可插拔传输 **117**
16.1 Tor 如何遭到封锁 . . . 117
16.2 网桥 . . . 118
16.2.1 分发难题 . . . 118
16.2.2 Lox . . . 118
16.3 可插拔传输 . . . 118
16.3.1 系统架构 . . . 119
16.3.2 tor 与 PT 之间的协议 . . . 119
16.3.3 Lyrebird . . . 120
16.4 不同的规避策略 . . . 120
16.4.1 obfs4：看起来不像任何可识别的协议 . . . 120
16.4.2 meek：隐藏在大型服务之后 . . . 120
16.4.3 Snowflake：大量短暂存在的代理 . . . 120
16.4.4 WebTunnel：看起来像普通网站 . . . 120
16.4.5 Conjure：隐藏在地址空间中 . . . 121
16.5 实验：从审查者的视角观察 obfs4 网桥 . . . 121

第三部分 **Tor** 实践 **125**

第 **17** 章 安装 **Tor** 与配置 **torrc** **127**
17.1 应该安装哪一种 Tor? . . . 127
17.2 torrc 配置文件 . . . 127
17.2.1 使用前先检查 . . . 128
17.3 客户端配置 . . . 128
17.3.1 流隔离 . . . 128
17.3.2 谨慎选择中继 . . . 129
17.3.3 其他端口 . . . 129
17.4 控制端口 . . . 129
17.5 日志 . . . 130

第 **18** 章 **Control Port** 与 **Stem** **133**
18.1 控制协议 . . . 133
18.1.1 从 tor 的视角观察电路 . . . 134
18.1.2 主要命令 . . . 134
18.2 Stem . . . 135

18.3 实验：实时观察流隔离 135

第 **19** 章 私有 **Tor** 网络：**Chutney** 与 **Shadow** **139**
19.1 为什么需要私有网络? 139
19.2 仿真还是模拟? 140
19.3 本书的实验环境 140
19.3.1 运行要求 141
19.3.2 构建镜像并启动网络 141
19.4 网络拓扑 141
19.5 测试网络不等于真实网络 142
19.6 节点目录内部有什么? 143
19.7 使用实验网络 144
19.7.1 通过私有网络浏览网页 144
19.7.2 访问洋葱服务 144
19.7.3 用 Stem 观察电路 144
19.7.4 在容器内部操作 145
19.8 第一次与中继通信 145
19.9 Shadow：模拟器中的网络 146

第 **20** 章 观察流量：**Wireshark** 与日志 **149**
20.1 观察者能够看到什么? 149
20.2 使用的工具 150
20.2.1 解密自己建立的 TLS 连接 150
20.3 实验：TLS 内部的信元 151
20.4 tor 日志 152

第 **21** 章 发布洋葱服务 **155**
21.1 第一个洋葱服务 155
21.1.1 避免泄露服务器位置 156
21.2 Onion-Location 156
21.3 受限发现 157
21.4 其他运维需求 158
21.4.1 个性化地址 158
21.4.2 单跳服务 158
21.4.3 Onionbalance 159

第 **22** 章 运行中继或网桥 **161**
22.1 应该运行哪种节点? 161
22.2 基本配置 161
22.2.1 新中继的成长过程 162
22.2.2 密钥 162
22.3 出口策略 162
22.4 实验：测试出口策略 163
22.5 中继家族 164
22.6 运行网桥 165

第 **23** 章 **Tor Browser**、**Tails** 与 **Whonix** **167**
23.1 为什么只有代理还不够? 167
23.2 Tor Browser 167
23.2.1 每个网站使用独立电路 168
23.2.2 抵御浏览器指纹识别 168

23.2.3 安全级别 . . . 169
23.3 浏览器之外的保护 . . . 169
23.3.1 Tails . . . 169
23.3.2 Whonix . . . 170

第 **24** 章 测量网络：**Tor Metrics** 与 **OONI** **173**
24.1 数据来源 . . . 173
24.2 统计匿名用户 . . . 173
24.2.1 发现网络审查 . . . 174
24.3 性能测量 . . . 175
24.4 实验：读取真实共识 . . . 175
24.5 OONI . . . 176

第 **25** 章 意大利和欧盟的法律与伦理问题 **179**
25.1 工具及其用途 . . . 179
25.2 中继运营者 . . . 179
25.2.1 单纯传输通道（Mere Conduit） . . . 179
25.2.2 民事责任与刑事调查 . . . 180
25.2.3 数据保留 . . . 180
25.2.4 日志属于个人数据 . . . 181
25.3 研究人员 . . . 181

第四部分 攻击与防御 **183**

第 **26** 章 流量关联、指纹识别与 **Sybil** 攻击 **185**
26.1 攻击者的位置 . . . 185
26.2 端到端流量关联 . . . 185
26.3 实验：关联自己产生的流量 . . . 186
26.4 网站指纹识别 . . . 187
26.5 Sybil：大量伪造中继身份 . . . 188
26.6 发现入口守卫 . . . 188

第 **27** 章 针对真实 **Tor** 网络的攻击案例 **191**
27.1 2014 年：利用 Cell 类型传递信号 . . . 191
27.2 实验：验证 RELAY_EARLY 的数量限制 . . . 192
27.3 2020–2021 年：KAX17 与网络健康 . . . 192
27.4 2022–2023 年：拒绝服务攻击 . . . 193
27.5 经验总结 . . . 193

第 **28** 章 操作安全失误与去匿名化：经验教训 **195**
28.1 四个层次 . . . 195
28.2 应用程序层面的失误 . . . 195
28.3 实验：tor 识别 DNS 泄露的警讯 . . . 196
28.4 内容层面的失误 . . . 197
28.5 行为失误：匿名集过小 . . . 197
28.6 自我检查清单 . . . 197

第五部分 研究与未来 **199**

第 **29** 章 **Tor** 学术研究 **201**
29.1 Tor 研究发表在哪里？ . . . 201
29.2 如何衡量匿名性？ . . . 201
29.3 四种研究方法 . . . 202
29.4 实验：首次失守需要多长时间？ . . . 203
29.5 如何安全地开展研究 . . . 203

第 **30** 章 **Arti**：用 **Rust** 实现 **Tor** **207**
30.1 为什么重写 tor？ . . . 207
30.2 架构 . . . 208
30.2.1 对比：编写一个 Cell . . . 209
30.3 支持的功能 . . . 209
30.4 实验：让 Arti 接入私有网络 . . . 210

第 **31** 章 开放问题与正在推进的项目 **213**
31.1 协议如何发生变化？ . . . 213
31.2 量子计算机 . . . 213
31.3 更大的消息，以及更小的消息 . . . 214
31.4 性能：多路径传输 . . . 215
31.5 可扩展性：Walking Onions . . . 215
31.6 其他开放问题 . . . 215
31.7 如何参与贡献？ . . . 216

附录 **A** **Tor** 术语表 **219**

附录 **B** 密码学进阶 **221**
B.1 同一条曲线的两种表示 . . . 221
B.1.1 X25519 密钥的比特钳制 . . . 221
B.1.2 无穷远点 . . . 222
B.2 从 Montgomery 到 Edwards：ntor 交叉证书 . . . 222
B.3 流密码的可篡改性 . . . 222

附录 **C** **torspec** 协议规范阅读指南 **225**
C.1 从哪里找到规范？ . . . 225
C.2 约定 . . . 225
C.3 规范文档地图 . . . 226
C.4 如何阅读规范章节？ . . . 226
C.5 Arti 与规范 . . . 226

附录 **D** 习题解答 **227**

参考文献 **245**

索引 **249**

前言

每天都有数百万人使用 Tor，但人们谈论它时，往往首先想到的是它的用途——无论这些用途是真实存在的，还是出于想象：绕过审查、保护消息来源、所谓的“暗网”。本书采取另一条路径：解释 Tor 究竟如何工作。我们从两个中继之间交换的字节开始，一章一章地向上追踪，直到设计取舍、攻击、现实案例以及仍未解决的问题。更重要的是，本书并不要求读者仅凭文字相信这些解释：技术部分所描述的每一种机制，都会在实验中构建并验证，实验运行在一张普通计算机即可承载的私有 Tor 网络上。

来源、创作方法与勘误邀请

本书以我多年来撰写的学习笔记和研究资料为基础，最初的研究始于本科毕业论文。为了将这些资料整理成一部连贯的技术著作，我在自己的指导下使用人工智能工具，实质性地参与了讲解的扩展、论证、正文撰写，以及插图、习题、代码和实验的制作。人工智能并非只用于润色我写好的文字，其参与也不等于独立专家审查。

尽管已进行技术检查并由我本人审阅，书中仍可能存在错误、简化之处或过时的信息。欢迎读者通过项目仓库报告问题并提出修正。我选择通过项目官方渠道免费提供本书及相关源码，并采用 GNU GPL-3.0-or-later 许可证，以践行“diffusione dei saperi senza fondare poteri”——传播知识，而不建立权力中心。包括商业再分发在内的使用条件由许可证规定。

本书适合谁

本书面向计算机科学与安全专业的学生、工作中会接触 Tor 的专业人员、正在运行或希望运行中继的人，以及研究人员。读者应具备基本的网络知识（TCP/IP，以及 TLS 的基本思想），熟悉命令行；要完成实验，还需要掌握 Python 基础。本书所需的密码学知识会在 第 5 章 中介绍，并在 附录 B 中进一步展开。

如何阅读本书

全书分为五个部分。你可以从头到尾顺序阅读，也可以按照以下路径选择重点：

历史与概念路径 第一部分（第 1–4 章）和第五部分（第 29–31 章）：匿名究竟意味着什么，Tor 从何而来，它针对哪些对手提供保护，以及它正在走向何处。

协议路径 第二部分（第 5–16 章），建议按顺序并结合实验阅读。这是本书的核心。

实践路径 第三部分（第 17–25 章）：配置 tor、观察其流量、发布洋葱服务、运行中继、使用 Tor Browser、测量网络并理解相关法律问题。第 19 章 是所有实验的前置章节。

安全路径 第四部分（第 26–28 章），建议先阅读 第 4 章：从研究成果、现实事件与运维失误三个角度分析攻击。

约定

每一章都会先列出学习目标，并给出一行最后核验信息，说明本章内容是针对哪个日期、软件版本和规范版本核对的。Tor 会不断演进，因此读者必须知道正文所描述的是哪一个版本。全书中，*Tor* 指网络和项目，*tor* 指 C 语言实现，*Arti* 指 Rust 实现。规范引用写作 [document § section]；协议变更提案则以编号标识。正文采用的核心英文协议术语及其简体中文译法统一收录于 附录 A。

为了不打断主线叙述，本书使用不同的框区分深入说明、历史说明和注意事项。练习按星级标注难度：一星表示理解题，两星表示应用题，三星表示实验任务或小型研究项目；完整解答见 附录 D。全书共有 20 个实验。在第二部分中，读者将用 Python 逐步构建一个小型 Tor 客户端（`torlab`）；代码与实验网络随书提供，并可在更新后重新运行自动化测试。所有这些资源——`torlab` 包、20 个实验、Docker 测试网络和验证工具——都可以在本书仓库中获得：`https://github.com/aleff-github/onion-routing-anatomy`。

伦理说明

本书的全部实验要么运行在实验室的私有 Tor 网络上，要么仅以普通客户端身份使用真实网络，不观察也不修改他人的流量。这既符合 Tor Research Safety Board 对研究人员的建议，也涉及法律边界（见 第 25 章）。讨论攻击的章节始终采取分析和防御视角：解释过去发生过什么、如何防御，而不是说明如何针对具体个人实施攻击。

致谢

本书建立在 Tor Project 及其社区长期工作的基础之上。开放规范、`tor` 与 Arti 的源代码、协议变更提案和公开文档，为后续每一项技术论断提供依据。正因为这些材料公开可得，读者才能回到原始来源核验，而不必仅仅相信作者的表述。同样需要感谢用于撰写本书和测试实验的自由软件项目及其维护者，包括 LaTeX、Python、Docker、Chutney、Wireshark，以及正文中引用的许多其他项目。

第一部分

历史与基础

第1章

匿名、监控与元数据

最后核验：*2026* 年 *9* 月 *23* 日 — *Tor Metrics* 数据截至 *2026* 年 *9* 月 *20* 日

本章目标

学完本章后，你将能够：

- 区分机密性、匿名性、假名性、不可链接性和不可观察性；
- 解释为什么通信元数据能够揭示与内容同样多、甚至更多的信息；
- 描述流量分析，并说明为什么仅靠加密无法阻止它；
- 说明哪些人在使用 Tor、为什么使用，以及用户群体的多样性为何本身就是一种保护。

本书讨论的是一种旨在保护特定信息的系统：它主要保护的不是我们在网上说了什么，而是我们在与谁通信。在研究系统的工作方式之前，必须先弄清楚它试图解决的问题，以及这个问题为什么不同于我们更熟悉的“加密”问题。

1.1 内容与元数据

当两台计算机通过互联网通信时，每个数据包都会携带两类信息。内容是应用程序实际交换的数据，例如消息文字或网页内容。元数据描述的则是通信本身：谁在发送、发给谁、什么时候发送、发送得多频繁，以及数据量有多大。有些元数据是网络正常运行所必需的，因为路由器必须知道应把数据包送到哪里。目的 IP 地址以及用于接收回复的源 IP 地址，都会以明文形式出现在每个数据包中。

如今 HTTPS 已十分普及，加密因此能够保护内容，但元数据仍然可见。任何能够观察网络的人——从用户的互联网服务提供商，到大型互联网交换中心的运营者——仍可能看到用户连接了哪个网站、连接发生在什么时候、持续了多久以及产生了多少流量。

1.1.1 元数据会暴露什么

人们很容易把元数据看成无关紧要的信息：“他们只知道我和谁说话，却不知道我说了什么。”这种直觉并不可靠。几个例子就能说明原因：

- 某人在凌晨两点访问家庭暴力援助机构的网站，随后又访问离婚律师的网站；
- 在一篇调查报道发布前几天，一名记者持续与该调查涉及的某政府部门员工交换消息；
- 一名员工经常访问工会网站，或者某种特定疾病患者协会的网站。

在这些情形中，无须读取通信内容，就已经能够推断出大量信息。对收集者而言，元数据还有另一个优势：结构化程度高，易于大规模存储和分析，也便于与其他数据交叉关联。曾先

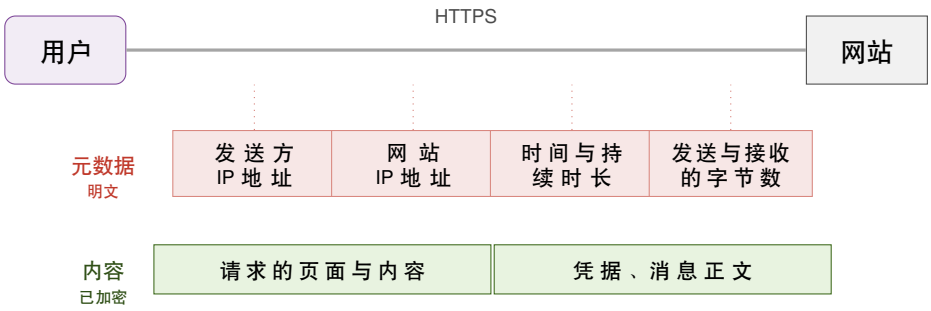


图 1.1: 观察者在 HTTPS 连接上能够看到什么。内容受到加密保护，但元数据仍然可见；Tor 试图保护的正是这类信息。

后担任 NSA 和 CIA 局长的 Michael Hayden 在 2014 年 4 月 1 日约翰斯·霍普金斯大学的一场讨论中，用一句后来广为流传的话概括了这一点：“我们会根据元数据杀人”（本书译文），随后立刻补充说“但我们不会用这些元数据做那件事”[Jus14]。后一句限定并没有改变核心事实：收集元数据的人认为，这些信息本身就足以支撑极其重大的判断。

1.2 流量分析

流量分析通过研究通信元数据来推断信息，而不必读取通信内容。它的历史几乎与电信本身一样悠久。即使在二十世纪的战争中，仅仅观察敌方无线电台多久发一次报、与谁通信、通信节奏如何，也可能在完全无法解密消息的情况下推断出一支军队的组织结构。

在互联网上，流量分析主要有两种形式：

地址分析 链路上的观察者可以看到源 IP 地址和目的 IP 地址。这是最简单的一类，也是 Tor 最直接处理的问题：网络中不存在一个天然的单点能够同时看到两个端点。

流量大小与时序分析 即使看不到完整的地址信息，观察者也可以按数据量和时间模式比较不同流。如果某用户在 10:15:02 下载了 3 MB，而网络另一处的一台服务器也在 10:15:02 发出了 3 MB 数据，那么两次观察很可能属于同一通信。这种流量关联是 Tor 的主要局限，本书会反复回到这个问题（第 4 章和第 26 章）。

1.3 精确的术语

“匿名”一词经常被用于不同含义，而这些含义之间的混淆，在匿名系统的发展史上曾导致不少设计错误。科学文献，尤其是 Pfitzmann 和 Hansen 提出的术语体系 [PH10]，会区分以下几个概念：

机密性 未授权的一方无法读取内容。这是加密主要提供的属性。

匿名性 在一组主体中无法识别某个主体。这组主体称为匿名集。匿名从来不是绝对属性，它总是“相对于其他人”存在。匿名集越大、成员越多样，保护通常越强。

假名性 一个人使用并非真实姓名的标识符活动，而且这个标识符可能长期保持稳定。使用虚构名称的账户是一个假名，而不等于匿名：该账户的所有行为仍可能被彼此关联。

不可链接性 观察者无法判断两个关注对象——例如两条消息、一个用户与一个网站，或两次访问——是否相关。Tor 试图在用户与其目的地之间提供的，正是这种属性。

不可观察性 观察者甚至无法判断通信是否正在发生。这是强得多的属性，而 Tor 本身并不提供它：监视用户网络连接的人通常能够看到该用户正在使用 Tor，除非同时使用可插拔传输（第 16 章）。

历史说明：把身份识别与路由分离

1995 年发明洋葱路由的三位 Naval Research Laboratory 研究人员——David Goldschlag、Michael Reed 和 Paul Syverson——更愿意避免把目标简单描述成“匿名”。他们在 1996 年的一篇论文中写道，他们的目标是“把身份识别与路由分离”；Syverson 后来回顾那段历史时再次强调了这一点 [Syv11]。例如，一个用户可能希望向正在通信的系统（如办公室计算机）证明自己的身份，但并不希望网络中的每一个中间节点都知道“位于这个地点的人正在连接那台办公室计算机”。

1.4 谁需要匿名?

根据 Tor Metrics 的估算，2026 年 8 月和 9 月期间，每天有约 280 万至 350 万用户在不使用网桥的情况下连接 Tor 网络，平均约 320 万。意大利的估算值每天约为 44 000 至 83 000，平均约 60 000。这些数字存在相当大的不确定性，因为网络只能估算用户数量，而不是对用户逐一计数（第 24 章）。这些用户是谁？他们来自非常不同的群体：

- 不希望广告商、互联网服务提供商或所使用网络的运营者追踪自己的普通用户；
- 记者及其消息来源、活动人士和人权捍卫者，特别是生活在威权政府统治国家的人；
- 身处网络审查环境、借助 Tor 访问被屏蔽网站的人（第 16 章）；
- 需要隐藏自身位置的暴力或迫害受害者；
- 不希望向竞争对手暴露研究兴趣的公司；
- 需要在访问网站或收集信息时不暴露身份的执法和军事人员——洋葱路由最初正是在美国海军实验室中为这类需求而诞生。

正如任何通信工具一样，也有一小部分人会利用 Tor 从事违法活动。公共讨论往往把注意力集中在这一群体上。第四部分以及 第 25 章 会基于证据和具体案例讨论这个问题。

1.4.1 匿名需要同伴

用户多样性的重要性还有更深一层原因。如果一种匿名系统只由某一个群体使用——例如只有军方使用——那么它反而无法很好地保护该群体：观察者只要看到某人连接这张网络，就会立即知道这个人属于军方；看到从网络中出来的流量，也会知道它来自军方。Syverson 在讨论美国海军原本可以建设的一张专用网络时，明确指出了这一问题：所有从该网络流出的通信都会被识别为海军流量 [Syv11]。每个人的匿名性都取决于还有多少其他用户，以及这些用户彼此之间有多么不同。概括这一原则的一句话——同时也是 Dingledine 和 Mathewson 一篇论文的标题——是“匿名需要同伴（anonymity loves company）” [DM06]。这也是 Tor 被设计成向所有人开放、免费且开源的原因之一：使用者越多，它越有可能更好地保护每一个人。

小结

- 加密保护内容；元数据（谁、和谁、什么时候、传输了多少）仍然可见，而且可能揭示大量信息。
- 流量分析根据通信的地址、数据量和时序推断信息。
- 匿名性意味着在一个集合中无法识别某个主体；必须把它与假名性、不可链接性和不可观察性区分开。
- Tor 服务于数百万背景各异的用户，而这种多样性本身就是其保护机制的一部分。

练习

练习 **1.1** ⋆　列出互联网服务提供商在其客户阅读 HTTPS 新闻网站文章时可以观察到的元数据。如果客户改用 VPN，哪些信息仍然可见？如果使用 Tor，又会怎样？

练习 **1.2** ⋆　将下列情形分别归类为匿名、假名或两者都不是：使用虚构名称的社交媒体账户；留在公共公告板上的未署名消息；只用姓名首字母签署的信件；投入募捐箱的现金捐款。

练习 **1.3** ⋆⋆　在一小时内，某网站收到 50 名不同 Tor 用户的访问。其中一人发布了一条评论，包含意大利语使用者常见的拼写错误，而且发布时间是意大利当地凌晨 3 点。这会怎样改变该用户的匿名集？这个例子说明用户行为会如何影响匿名性？

练习 **1.4** ⋆⋆　比较商业 VPN 与 Tor：在两种系统中，谁有能力把用户与其访问目的地关联起来？信任分别集中在哪里？

延伸阅读

- A. Pfitzmann, M. Hansen, *A terminology for talking about privacy by data minimization* [PH10]：隐私术语的重要参考。
- P. Syverson, *A Peel of Onion* [Syv11]：由洋葱路由的发明者之一回顾其历史。
- Tor Metrics, `https://metrics.torproject.org/`：按国家估算的 Tor 用户数量。

第 2 章

源流：Mix 网络、Remailer、Crowds 与 Freedom

最后核验：*2026* 年 *9* 月 *23* 日 — 本章所引用的历史资料

本章目标

学完本章后，你将能够：

- 解释 David Chaum 提出的 *mix* 思想，以及它为何成为匿名系统的基础；
- 区分适合电子邮件的高延迟系统与适合网页浏览的低延迟系统；
- 梳理 remailer 从 anon.penet.fi 到 Mixminion 的演进；
- 说明 Crowds、PipeNet 和 Freedom 在通向 Tor 的历史进程中所处的位置。

Tor 并非凭空出现。至少就目前而言，它是四十多年研究与实际部署不断演进的最新阶段。有些系统取得了成功，更多系统最终失败，但每一个都留下了经验。本章将追踪其中最重要的发展脉络，直到洋葱路由出现。

2.1 Chaum 的 Mix（1981）

1981 年，David Chaum 发表了一篇仅五页的论文，却由此奠定了一个完整的研究领域：*Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms* [Cha81]。论文研究的是电子邮件：即使整张网络都处于监控之下，怎样才能发送一封消息，而不让观察者把发送者和接收者联系起来？

Chaum 的答案是 **mix**（混合器）：一种服务器，它接收用自身公钥加密的消息，将其解密，把多条消息收集成一批（*batch*），然后以不同于到达顺序的次序一起转发。以下三种技术阻止观察者把某条输入消息与某条输出消息对应起来：

- mix 去除一层加密后，输出消息的外观与对应的输入消息完全不同；
- 所有消息长度相同；
- 消息被打乱并成批同时释放，因此连时序也无法直接把它们对应起来。

发送者不必只信任一个 mix，而可以让消息经过一条 **mix** 链，并用每个 mix 的密钥逐层加密。每个 mix 只去掉属于自己的那一层，并且只知道下一个 mix 的身份。只要链中至少有一个 mix 是诚实的，输入与输出之间的联系就仍然能够被隐藏。这里已经包含了洋葱路由思想的种子。

在同一篇论文中，Chaum 还提出了匿名返回地址，使接收者可以在不知道发送者身份的情况下回复，以及数字假名的概念。1988 年，他又提出另一种机制 *DC-nets* [Cha88]。它能够以极高成本提供无条件匿名性，因此主要产生了理论影响。

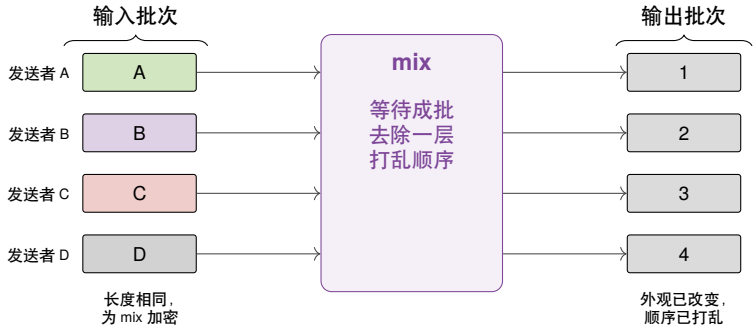


图 2.1: Chaum 式 mix。即使观察者同时监视输入端和输出端，也难以把消息一一对应：所有消息长度相同，mix 去除一层加密后其外观发生变化，而且它们会以不同顺序成批离开。

2.2 高延迟与低延迟

Chaum 的 mix 能提供很强的保护，但它很慢：一条消息在每个 mix 中都必须等待一批消息形成，最终可能需要几分钟甚至几小时才能到达。这对电子邮件尚可接受，却不适合网页浏览——网页通常必须在几秒内返回。

这构成了匿名通信领域最根本的一条分界线：

高延迟系统 mix、remailer 等系统会延迟并打乱消息，因此能够抵抗监视整张网络的强大观察者；但它们只适用于能够容忍延迟的通信。

低延迟系统 洋葱路由和 Tor 等系统会在数据到达后尽快转发，从而支持网页浏览、聊天和其他交互式连接；代价是同时观察通信两端的对手可能通过时间与流量大小进行关联。

Tor 有意识地选择了第二条路线：如果一个系统因为太慢而几乎没人使用，它也就无法保护任何人（第 1.4 节）。

2.3 Remailer

20 世纪 90 年代，Chaum 的思想被实现为真实服务，通常称为 **remailer**（匿名邮件转发器）：服务器接收一封电子邮件，去除与发送者有关的信息后再将其转发。

anon.penet.fi（1993–1996）。 最著名的 remailer 由 Johan “Julf” Helsingius 于 1993 至 1996 年间在芬兰运营。它并不是 mix：系统为每个用户分配一个假名，同时保存一张“假名到真实地址”的映射表，以便能够投递回复。后来，它每天处理数千封消息。最终，正是这张映射表导致服务关闭。1995 和 1996 年，山达基教会在芬兰警方协助下取得了一些曾传播该教会文档的用户身份。Helsingius 于 1996 年关闭服务，理由是当时芬兰针对互联网的法律环境存在不确定性。洋葱路由的设计者后来明确吸取了这一教训 [Syv11]：只要系统保存用户与真实身份之间的映射，就创造了一个单点故障；运营者可能被迫交出它。

Cypherpunk、Mixmaster、Mixminion。 与此同时，*cypherpunk* 社区开发了不保存身份映射表的 remailer：发送者为一条服务器链逐层加密消息。后续几代系统——先是 Mixmaster，后是 Mixminion [DDM03]——加入固定长度消息、真正的批处理，以及针对消息重放的防御。Roger Dingledine 和 Nick Mathewson 都是 2003 年发布的 Mixminion 的作者之一；几乎在同一时期，他们也在设计 Tor。

2.4 走向低延迟

PipeNet（1995）。 1995 年 2 月，Wei Dai 在 cypherpunk 邮件列表中提出 PipeNet：一个低延迟系统，其中所有参与者持续以恒定速率发送流量，从而不给观察者留下可用于关联的变化。理论上它可以抵抗时序分析，但代价无法接受：只要一个参与者停止发送，整张网络都必须停下来 [Syv11]。PipeNet 从未真正实现，却一直是理解“要彻底击败流量关联，需要付出什么代价”的重要参照。

洋葱路由（**1996**）。 同一年，Naval Research Laboratory 正在形成洋葱路由方案。其历史将在 第 3 章 中详细说明。

Crowds（1998）。 Michael Reiter 和 Aviel Rubin 提出了 Crowds [RR98]。用户组成一个“群体（crowd）”，每个网页请求都会先被转发给随机选中的成员。该成员以一定概率把请求发送到目标网站，否则继续转给另一名成员。网站看到请求来自某个任意成员，而任何成员也无法确定把请求传给自己的那个人究竟是原始发起者，还是又一个转发者。Crowds 不使用分层加密，而且只针对网站和其他成员提供保护，不能抵抗网络观察者。不过，它第一次以概率方式严格表达匿名性，并分析了多个恶意成员相互串通的攻击；这些分析预示了后来与守卫中继有关的问题。

Freedom（1999–2001）。 Tor 之前规模最大的匿名网络是加拿大 Zero-Knowledge Systems 公司运营的 Freedom。它于 1998 年末公布，从 1999 年末持续运行到 2001 年末 [Syv11]。Freedom 是商业服务：用户支付订阅费，服务器由公司及其合作伙伴运营。最终，订阅收入不足以维持网络，服务因此关闭。它的经历至少说明了两点，而 Tor 后来实际上都吸取了这些经验：匿名网络由许多相互独立的主体运营，比由单一公司掌控更加稳健；同时，融资模式的重要性并不低于密码学本身（第 3.4 节）。

深入说明：形形色色的“洋葱”

同一时期还出现了许多其他洋葱路由系统，例如完全去中心化的 Tarzan 和 MorphMix，以及 Cebolla。Tor 从 Cebolla 借鉴了“每一跳单独进行 Diffie–Hellman 交换、逐跳扩展电路”的思想 [Syv11]。还有一些项目与 Tor 同期出现，却走上了不同道路：主要面向内部服务、至今仍在运行的 I2P；以及在德累斯顿大学随 JAP 客户端诞生的 AN.ON（它也曾以 JonDonym 之名作为商业服务提供），后者使用由已知组织运营的固定 mix 链。不同项目在去中心化、信任和性能之间选择了不同平衡。

小结

- Chaum 的 mix（1981）通过改变消息外观、统一长度并打乱顺序，隐藏输入消息与输出消息之间的关系。采用分层加密的 mix 链已经预示了“洋葱”结构。
- 高延迟系统能抵抗更强的观察者，但速度慢；低延迟系统适用于交互式通信，却容易受到时序关联。
- anon.penet.fi 展示了保存用户真实身份的危险；Mixmaster 和 Mixminion 则继续发展用于电子邮件的 mix 思路。
- PipeNet、Crowds 和 Freedom 以不同取舍探索低延迟匿名通信。Freedom 尤其说明，多主体运营和可持续融资模式对匿名网络同样重要。

练习

练习 **2.1** ⋆　某个 mix 每批收集 100 条消息。观察者能看到全部 100 条消息进入，也能看到全部 100 条消息离开。如果观察者知道其中一条输入消息的发送者，并随机猜测其对应的输出消息，猜中的概率是多少？如果这 100 条消息中有 99 条都是观察者自己发送的，又会发生什么？

练习 **2.2** ⋆⋆　设计一个允许用户收到回复、但不保存“假名到真实地址”映射表的 remailer。你需要用到 Chaum 提出的哪些思想？

练习 **2.3** ⋆⋆　在 Crowds 中，若继续转发的概率为 $p_f = 3/4$，一个请求在到达网站前平均会经过多少名成员？为什么收到请求的成员无法确定把请求交给自己的人就是原始发起者？

延伸阅读

- D. Chaum, *Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms* [Cha81].
- M. Reiter, A. Rubin, *Crowds: Anonymity for Web Transactions* [RR98].
- G. Danezis, R. Dingledine, N. Mathewson, *Mixminion: Design of a Type III Anonymous Remailer Protocol* [DDM03].
- Free Haven 的 *Anonymity Bibliography*，`https://www.freehaven.net/anonbib/`：该领域研究论文的重要参考集合。

第 3 章

从 NRL 洋葱路由到 Tor Project

最后核验：*2026* 年 *9* 月 *23* 日 — *Syverson*，A Peel of Onion（*2011*）；*Tor Project* 历史页面

本章目标

学完本章后，你将能够：

- 追溯洋葱路由在 Naval Research Laboratory 的起源，并描述其三代设计；
- 解释 Tor 与前两代方案相比有哪些关键差异；
- 概述 Tor Project 从 2002 年至今的主要发展阶段；
- 说明名称 “Tor” 的来源。

3.1 在 Naval Research Laboratory

1995 年，美国 Naval Research Laboratory（NRL）的三位研究人员——David Goldschlag、Michael Reed 和 Paul Syverson——提出了一个问题：能否建立互联网连接，却不暴露通信双方是谁 [Thea]？他们追求的并不是 “为了匿名而匿名”，而是我们已经看到的目标：“把身份识别与路由分离”（第 1.3 节）。他们的第一套设计于 1996 年 5 月在 *Information Hiding Workshop* 上发表 [GRS96]。同年结束前又出现了第二套设计，把负责构建电路的用户应用程序与负责承载电路的节点分离开来 [Syv11]。早在 1996 年，实验室就已经获得公开代码的许可——当时 “开源软件” 这一概念甚至尚未普及。设计者很清楚：如果代码无法接受检查，没有人会信任这样一张网络。

“洋葱” 这个名字来自数据结构本身：一条消息被多层加密包裹，每经过一个节点就去掉一层，就像一层层剥开洋葱。

3.2 三代设计

Syverson 把 NRL 的设计划分为三代 [Syv11]。

第一代。 第一代采用固定的五节点电路，其中一个目标是连接两个组织各自受保护的网络。如果只有三跳，一个恶意中间节点会立即看到通信双方的网络。若为四跳，攻击者只要控制与某个端点相邻的节点，就能知道下一步应该攻击哪一个节点，从而重建整条电路。使用五个节点后，若不先额外攻陷另一个节点，就不存在这样的捷径。客户端与节点集成在一起。网络是静态的，由二十或一百个稳定组织运营的节点组成，配置和密钥依靠人工分发。由于当时 SOCKS 尚未普及，每一种应用都需要自己的代理——网页浏览、远程登录和电子邮件分别处理。一条电路会被客户端一次性构造成一个完整 “洋葱”，按路径上所有节点的公钥逐层加密。

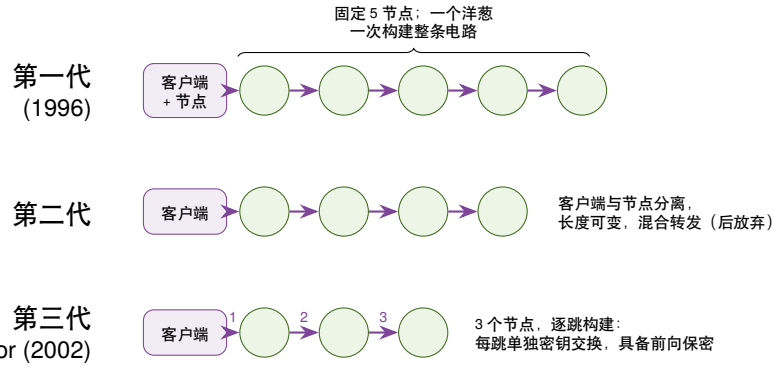


图 3.1: 洋葱路由的三代设计。Tor 的决定性变化并不在跳数，而在电路的构建方式：客户端不再一次性准备完整的“洋葱”，而是逐跳扩展电路，并与每一个节点分别进行密钥交换。

第二代。 客户端与节点被分离，电路长度改为可变，并引入了 mix 和电路间填充等机制，但这些机制后来被放弃。设计中计划加入网络信息的分发方式，却从未完整实现。

第三代：**Tor**。 最重要的变化，是电路改为逐跳构建：客户端与每个节点分别执行 Diffie–Hellman 交换（第 9.1 节）。这一思想借鉴自 Cebolla 项目。逐跳密钥交换提供前向保密，并允许客户端在扩展电路的同时逐个验证每一跳。Tor 还加入目录服务器、SOCKS 支持、使用汇合点的隐藏服务；更重要的是，它从一开始就被设计成可以真正部署，并由世界各地的志愿者共同运行。

历史说明：名称的由来

2004 年论文标题 *Tor: The Second-Generation Onion Router* 带来了两个后来由 Syverson 亲自承认的误解 [Syv11]。首先，Tor 属于第三代，而不是第二代；其次，Tor 中的字母“r”并不代表 *router*。Roger Dingledine 开始参与 NRL 项目时，已经存在许多洋葱路由系统。有人问“是哪一个?”时，他会回答“*the onion routing*”，意思是最初的那一个。Rachel Greenstadt 注意到其首字母正好可以组成一个简洁的名称。Dingledine 还发现它也可以递归解释为 *Tor's onion routing*，于是决定把名字写作“Tor”，而不是“TOR”。它还是一个普通单词，在德语中可表示“门”，例如城门。本书后续统一用 *Tor* 指网络和项目，用小写 *tor* 指程序。

3.3　Tor 网络

2000 年代初，Roger Dingledine 加入 NRL 项目，不久 Nick Mathewson 也加入。Tor 网络在 **2002** 年 **10** 月正式上线，源代码从一开始就以自由软件许可证发布。到 2003 年底，网络中大约有十几个由志愿者运营的节点，几乎都位于美国，另有一个在德国 [Thea]。2004 年 8 月，Dingledine、Mathewson 和 Syverson 在 USENIX Security Symposium 上介绍了该项目 [DMS04]。

此后的主要里程碑如下 [Thea]：

年份	事件
2002	Tor 网络开始运行，源代码自由公开
2004	Electronic Frontier Foundation 开始资助 Dingledine 和 Mathewson 的工作
2006	Tor Project, Inc. 作为美国非营利组织（501(c)(3)）成立
2007	开始开发用于抵抗审查的网桥（第 16 章）
2008	开始开发 Tor Browser（第 23 章）
2010–2011	Tor 在“阿拉伯之春”期间成为重要工具
2013	Edward Snowden 揭露大规模监控项目，Tor 在相关披露中发挥核心作用
2017	v3 洋葱服务（第 14 章）
2021	v2 洋葱服务被禁用
2022–2023	拥塞控制（第 11.3 节）以及面向洋葱服务的工作量证明（第 15 章）
2024	WebTunnel；网桥分发迁移到 rdsys
2025–2026	tor 中的 CGO（第 10.4 节）；Arti 作为客户端并支持洋葱服务（第 30 章）

3.4 一个组织与一个社区

Tor Project, Inc. 负责开发软件、维护规范、与独立运营者共同运行目录权威、测量网络并支持社区。但它并不“拥有”Tor 网络：来自数十个国家的数千名志愿者——个人、大学和协会——运营着中继。这是一项设计选择，也吸取了 Freedom 的经验（第 2.4 节）：任何单一实体都不应该能够控制整张网络，也不应被迫危及用户安全。

Tor Project 的资金来源长期以来都是公共讨论的话题。多年来，它获得过政府机构的资助，尤其是美国政府机构，也接受私人基金会和个人捐赠。该组织每年公开财务报表。“一个部分由政府资助的软件是否值得信任?”这个问题首先是技术问题，而非政治判断：代码与规范公开可验证；设计决策通过提案公开讨论（第 31 章）；系统安全也并不依赖于“信任 Tor Project”。真正的安全基础，是把网络分散在众多彼此独立的运营者之间（第 4 章）。

深入说明：2013 年的披露

Snowden 在 2013 年公开的文件中，包括 NSA 关于 Tor 的内部演示材料。这些材料承认，系统性去匿名化 Tor 用户并不容易，同时描述了一些针对个别用户的定向攻击；那些攻击利用的是浏览器漏洞，而不是 Tor 网络协议本身的弱点。这从侧面支持了本书会不断遇到的一个原则：最危险的薄弱点往往并不位于协议核心，而在围绕协议运行的其他系统中（第 28 章）。

小结

- 洋葱路由于 1995 年诞生在 Naval Research Laboratory，由 Goldschlag、Reed 和 Syverson 提出，目标是把身份识别与路由分离。
- 第一代采用固定五节点电路和静态网络。Tor 属于第三代，逐跳构建电路，并提供前向保密、目录系统和汇合点。
- 名称 “Tor” 来自 *the onion routing*；只有首字母大写。
- Tor 网络自 2002 年运行；Tor Project, Inc. 自 2006 年存在；中继主要由独立志愿者运营。

练习

练习 **3.1** ⋆ 为什么第一代洋葱路由使用五节点电路？重建本章关于三跳和四跳的论证，并解释为什么今天的 Tor 通常只使用三跳。

练习 **3.2** ⋆⋆　从前向保密以及“能否立即识别不响应节点”这两个角度，比较第一代一次性构造完整洋葱的方式与 Tor 的逐跳（telescoping）电路构建方式。

练习 **3.3** ⋆⋆　围绕“考虑到 Tor 的资金来源，它是否值得信任?”写一页短文。明确区分哪些事情能够通过技术手段验证，哪些事情仍然需要信任。

延伸阅读

- P. Syverson, *A Peel of Onion* [Syv11].
- D. Goldschlag, M. Reed, P. Syverson, *Hiding Routing Information* [GRS96].
- Tor Project 历史页面 [Thea]。

第 4 章

Tor 的威胁模型

最后核验：*2026* 年 *9* 月 *23* 日 — *Dingledine*、*Mathewson*、*Syverson*（*2004*），第 *3* 节

本章目标

学完本章后，你将能够：

- 解释什么是威胁模型，以及为什么每一个安全系统都需要明确的威胁模型；
- 列出 Tor 设计者明确提出的目标与“非目标”；
- 描述 Tor 试图抵抗的对手，以及它无法抵抗的对手；
- 区分 Tor 能提供的保护与仍由用户和应用程序承担的责任。

没有任何安全系统能够抵御一切。威胁模型会明确说明一个系统打算保护谁或保护什么、要抵抗哪些对手与威胁，以及建立在哪些假设之上。评价系统时也应以此为标准：如果某种攻击落在威胁模型范围之内且成功了，那说明系统存在缺陷；如果某种攻击被模型明确排除，则不能据此说系统违背了自身安全承诺。不过，这仍然是用户必须理解的局限。本章介绍 Tor 设计者在 2004 年提出的威胁模型 [DMS04]，并说明它之后如何演变。

4.1 目标

Tor 的首要目标，是阻止对手把一次通信的双方关联起来，或把同一用户的多次通信相互关联。2004 年论文提出了四项指导设计的原则：

可部署性 系统必须能在现实世界中运行：不能让志愿者承担过高成本，不能让运营者暴露在过度法律风险之下，不能要求修改操作系统，也不能强迫网站额外安装软件。

可用性 难以使用的系统会吸引更少用户，而用户越少，匿名性越弱。正如作者所强调的，可用性并不仅仅是便利问题，而是“一项安全要求”。

灵活性 协议必须被清晰、完整地规范，才能成为后续研究的基础。

简单性 只使用已经被充分理解、经过验证的技术。每增加一个功能，都会增加安全分析的复杂度。

4.2 非目标

同样重要的是理解 Tor 明确不试图做到什么：

它不是点对点系统 Tor 并不试图依赖成千上万个短时存在、其中许多可能恶意的节点运行。它使用由目录权威列出并验证的一组中继（第 12 章）。

它不能抵御端到端攻击 Tor 从未声称能够击败由同时观察两个端点的对手实施的时序关联或交集攻击。

它不会规范化应用协议 Tor 按应用程序交给它的方式传输应用数据。如果某个应用主动暴露用户身份，Tor 无法阻止。当年这需要过滤代理；今天，这一角色主要由 Tor Browser 承担（第 23 章）。

它不是隐写系统 Tor 本身不会隐藏“用户正在连接 Tor 网络”这一事实。后来出现的可插拔传输可以缓解这一限制（第 16 章）。

4.3 对手

许多匿名系统的理论分析都会假设存在一个全局被动对手：能够观察互联网上全部流量的人。没有任何实用的低延迟系统能够抵御这种对手，Tor 对此明确承认。Tor 假设的对手可见范围较为有限，但在某些方面拥有更强的主动能力 [DMS04]。它可以：

- 观察网络中一部分流量；
- 生成、修改、删除或延迟流量；
- 运营自己的中继；
- 攻陷由其他人运营的一部分中继。

借助这些能力，对手可以尝试多种攻击：在网络边缘同时观察输入和输出流量并进行关联；攻陷中继或密钥；重放流量；对诚实中继发动拒绝服务攻击，迫使用户更多选择恶意中继（第 15 章）；给流量添加标记，使其在网络另一处可以被识别（第 10.3 节）；利用目录系统向不同用户提供彼此不一致的网络视图（第 12 章）；或者让网络变得足够不可靠，迫使用户转而使用安全性更低的系统。

4.3.1 确认攻击与分析攻击

Tor 的设计者区分两类基于流量的攻击。在确认攻击中，对手已经怀疑某种关系存在——例如“Alice 正在与 Bob 通信”——于是同时观察两个端点，验证这个猜测。Tor 与其他低延迟系统一样，无法对此提供完整防护。在分析攻击中，对手研究流量，目标是发现应该去哪里观察，也就是应该攻击网络中的哪些位置。Tor 主要被设计用来抵抗后一类攻击。包括守卫和带宽权重在内的整个路径选择过程（第 13 章），都试图降低对手同时占据电路两端的概率。

注意：必须牢记的原则

Tor 能够针对只观察网络一部分的对手提供保护，却无法抵抗成功观察到一条通信两个端点的对手。如果对手既能控制或观察用户到守卫的连接，又能观察出口到目标网站的连接，就可以根据流量时序和数据量进行关联。对于能够观察一国全部网络流量的组织来说，用户和网站甚至可能都位于同一个国家。这是现实威胁，而不只是理论模型。

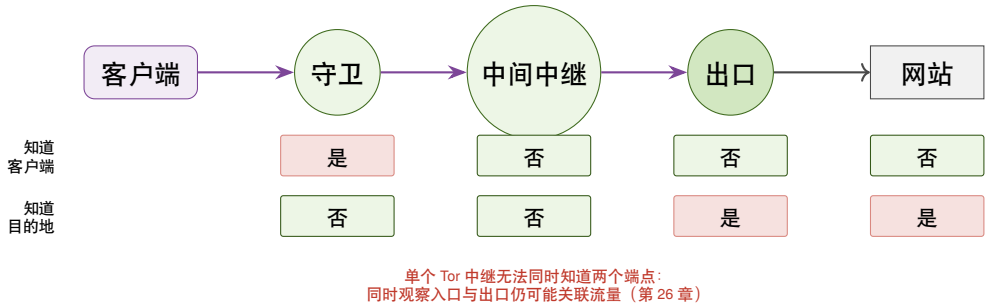


图 4.1: 路径上的各个位置分别知道什么。Tor 的保护来自信息分离：守卫知道客户端，却不知道最终目的地；出口知道目的地，却不知道客户端。

4.4 Tor 能保护什么，又不能保护什么

Tor 能防护	Tor 不能防护
网站试图获知访问者的 IP 地址	用户自己暴露的信息（例如登录个人账户或公开个人资料）
本地网络观察者或只能看到用户连接 Tor 的互联网服务提供商	同时观察通信两端的观察者
最多只知道一个端点的单个中继	同时控制一条电路守卫与出口的对手
出口试图识别请求最初由谁发出	出口读取或修改未加密流量（未使用 HTTPS）
配合网桥时，基于 IP 地址实施的审查	存在漏洞、已被攻陷或配置错误的应用程序与浏览器

随着时间推移，Tor 的模型也扩展到 2004 年论文无法预见的问题：全国级网络审查（第 16 章）、浏览器指纹识别防护（第 23 章）、DoS 防御（第 15 章），以及量子计算机带来的未来威胁（第 31 章）。但有一条原则始终不变：理解威胁模型，是安全使用 Tor 的第一步。

小结

- 威胁模型说明一个系统被设计来抵抗哪些对手和哪些攻击。
- Tor 的目标是阻止用户与目的地被关联，同时优先考虑可部署性、可用性、灵活性和简单性。
- Tor 不是点对点系统；不能击败端到端攻击；不会替应用程序规范化协议；若不使用可插拔传输，也不会隐藏“正在使用 Tor”这一事实。
- 对手可以观察部分网络、运营或攻陷中继，并采取主动措施。Tor 主要用于抵抗分析攻击，对确认攻击的保护则较弱。

练习

练习 **4.1** ⋆ 对下列每种情形，判断 Tor 是否能保护用户，并解释原因：（a）咖啡馆 Wi-Fi 运营者想知道顾客访问了哪些网站；（b）某网站想知道访客的真实 IP 地址；（c）用户通过 Tor 登录自己的个人社交媒体账户；（d）能够控制某国网络基础设施的机构怀疑一名用户正在与同一国家的一家报纸通信。

练习 **4.2** ⋆⋆ 举一个例子，说明为什么在匿名系统中，可用性是一项安全要求，而不仅仅是便利性问题。

练习 **4.3** ⋆⋆ 为一名使用 Tor Browser 与另一个国家的消息来源通信的记者编写威胁模型。指出可能的对手、这些对手能够观察什么，以及哪些风险仍然处于 Tor 的保护范围之外。

延伸阅读

- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04]，第 3、7 节。
- R. Dingledine, N. Mathewson, *Anonymity Loves Company: Usability and the Network Effect* [DM06].

第二部分

底层协议

第 5 章

Tor 所需的核心密码学

最后核验：*2026 年 9 月 23 日 — tor 0.4.9.13*；*torspec* 提交 `8e1f245`

本章目标

学完本章后，你将能够：

- 识别 Tor 使用的主要密码学原语，并说明各自承担什么任务；
- 区分加密、完整性保护、密钥派生、密钥交换和数字签名；
- 理解为什么 Tor 同时使用 SHA-1、SHA-2、SHA-3 和 Blake2b 等不同哈希家族，以及它们之间有何差别；
- 更有把握地阅读后续章节中的密码学公式。

本章是一套工具箱，而不是一门完整的密码学课程。我们只引入理解第二部分所需的必要理论，并在每一种原语出现时指出它会在哪些协议位置被使用。希望进一步深入的读者可以阅读附录 B。所有示例都可以用 Python 的标准库 `hashlib` 和 `cryptography` 库来复现，这也正是实验中使用的工具。

5.1 五项任务

在 Tor 这样的协议中，密码学承担五类不同任务：

机密性 除预期接收者之外，任何人都不应读取数据：使用密码算法（*cipher*）。

完整性 任何人都不应在不被发现的情况下修改数据：使用哈希函数和消息认证码（MAC）。

密钥协商 两个事先没有共享秘密的参与者，必须在公开通信的条件下建立共同秘密：使用 Diffie–Hellman 密钥交换。

身份认证 一方必须证明自己确实掌握与某个身份对应的秘密：使用数字签名。

密钥派生 从一个共享秘密中生成多把彼此独立的密钥：使用密钥派生函数（KDF）。

一个可靠的设计原则是：不要让同一把密钥跨任务或跨上下文复用。Tor 对这一原则的遵守也随着协议演进而越来越严格。

5.2 加密算法

5.2.1 计数器模式下的 AES

Tor 几乎所有加密都建立在 **AES** 上。AES 是标准分组密码，分组长度为 16 字节，可使用 128 位或 256 位密钥。分组密码本身一次只能处理一个分组。为了加密任意长度的数据流，Tor 使



图 5.1: Tor 中的五类密码学任务、承担这些任务的原语，以及本书中会看到它们的章节。

用计数器模式（CTR）：AES 对一个计数器进行加密，每处理一个分组就把计数器加一；产生的密钥流再通过 XOR 与数据结合：

$$C = P \oplus \mathrm{Trunc}_{|P|}\big(\mathrm{AES}_K(IV) \,\|\, \mathrm{AES}_K(IV+1) \,\|\, \cdots\big)$$

解密执行的是同一运算。计数器模式有两个必须始终牢记的性质：

- 同一“密钥 – 计数器”组合绝不能重复使用：如果两条消息用相同密钥流加密，攻击者就能得到两份明文的 XOR；
- 计数器模式具有可塑性：即使不知道密钥，攻击者修改密文中的某一位，也会让明文中的对应位发生可预测变化。

传统方案中的 relay cell 使用 AES-128-CTR 加密（第 10.1 节）；它的可塑性也是某些标记攻击得以实现的原因。AES-256-CTR 用于加密 ntor-v3 消息（第 9.7 节）和洋葱服务描述符（第 14.5 节）。AES-128 还是 CGO 的基础组成部分（第 10.4 节）。

5.2.2 TLS

两个节点之间的通道由 TLS 保护（第 7 章）。TLS 会独立协商带认证的密码套件，例如 AES-GCM 或 ChaCha20-Poly1305。不过，Tor 并不依赖 TLS 来证明中继身份：中继会使用 Tor 自己的证书体系证明身份。

5.3 哈希函数

密码学哈希函数把任意长度输入映射为固定长度的指纹。它的设计目标包括：在计算上难以找到两个具有相同指纹的输入（抗碰撞性），也难以为某个给定指纹反推出一个匹配输入。Tor 同时使用四个哈希家族，这很大程度上反映了它长期演进的历史：

函数	输出	在哪里遇到
SHA-1	20 字节	中继旧式 RSA 指纹、relay cell 的运行摘要（tor1）、KDF-TOR；其抗碰撞性已被认为不足，但在不依赖该性质的旧协议位置仍继续存在
SHA-256	32 字节	ntor 及其 KDF、Ed25519 与 TLS 证书、共识签名、微描述符指纹
SHA3-256、SHAKE-256	32 字节 / 可变	ntor-v3、v3 洋葱服务（地址、盲化密钥、HSDir 环、描述符）、汇合电路摘要
Blake2b	可变	洋葱服务工作量证明（第 15 章）

SHA-3 并不是 SHA-2 的“改进版”，而是完全不同的结构（*sponge*，海绵结构）。SHAKE-256 属于同一家族，是一种可扩展输出函数：需要多少字节就可以输出多少字节，因此很适合用作密钥派生。在 Tor 较新的协议中，每次使用 SHA-3 时都会在输入前加上一个域标签，例如 `"ntor3-curve25519-sha3_256-1:kdf_phase1"`。这样可以防止“为某一种目的计算的哈希”与“为另一种目的计算的哈希”意外落入同一语义空间。

深入说明：运行哈希

很多库都允许应用程序逐步把数据加入哈希状态，并在任意时刻读取摘要，而不必从头重新计算。Tor 正是利用这一能力维护 relay cell 的运行摘要（第 10.2 节）：它概括与某一跳已经交换过的全部 cell。在 Python 中，`hashlib.sha1()` 配合 `update`、`digest` 和 `copy` 方法即可实现同样的模式。

5.4 MAC 与密钥派生

5.4.1 HMAC

MAC 是一种同时依赖密钥的“指纹”：只有知道密钥的人才能计算或验证。**HMAC** 使用哈希函数构造 MAC。Tor 在 ntor 握手（第 9.6 节）及其密钥派生函数中使用 HMAC-SHA256。在基于 SHA-3 的协议中，可以采用更直接的 MAC 构造：先编码密钥长度，再拼接密钥和消息后进行哈希。长度字段能够无歧义地标识密钥在哪里结束、消息从哪里开始 [rend-spec § Cryptographic building blocks]。在 ntor-v3 中，域标签本身也会带长度前缀（第 9.7 节）。

一个实践细节非常重要：比较 MAC 时必须始终采用常数时间实现（Python 中使用 `hmac.compare_digest`）；否则，比较耗时可能泄露已经匹配了多少个字节。

5.4.2 密钥派生函数

一个共享秘密往往需要派生出多把密钥：前向加密一把、反向加密一把，再加上两把完整性密钥（第 9.8 节）。KDF 会把秘密扩展成足够长的输出流，然后从中切分出各把独立密钥。Tor 使用三类构造：

KDF-TOR $\mathrm{SHA1}(K_0\|0)\,\|\,\mathrm{SHA1}(K_0\|1)\,\|\,\ldots$，一种历史遗留构造，**`CREATE_FAST`** 仍在使用；

HKDF-SHA256 RFC 5869 定义的标准构造，由 ntor 使用；

SHAKE-256 在 ntor-v3 和洋葱服务中直接作为 KDF 使用。

5.5 密钥交换：X25519

Diffie–Hellman 密钥交换允许两个参与者只交换公开值，就建立共同秘密。Tor 使用的椭圆曲线版本是 **X25519**，即在 *Curve25519* 上进行密钥交换。每一方都有一个 32 字节私钥 x 和对应公钥 $X = x \cdot G$，其中 G 是曲线上的固定点，乘法表示“整数与点相乘”。给定 x, X 和 y, Y 两组密钥，双方都能计算出同一个值

$$x \cdot Y = y \cdot X = xy \cdot G,$$

而只知道 X 与 Y 的观察者无法推导出该值。密钥只有 32 字节，而且计算速度很快。正是这两个优势，使 Tor 放弃了 TAP 握手中使用的 1024 位模 Diffie–Hellman（第 9.4 节）。

如果一把密钥只用于一次交换，随后立即删除，就称为临时密钥（**ephemeral key**）。把临时密钥与长期密钥结合，正是 Tor 握手能够同时提供身份认证和前向保密的基础（第 9.3 节）。

5.6 签名：Ed25519 与 RSA

数字签名使用私钥生成，并由对应公钥验证。Tor 主要使用 **Ed25519**：它是建立在与 Curve25519 等价曲线上的签名方案，公钥为 32 字节，签名为 64 字节。它用于中继身份、Tor 证书（第 7.5 节）和洋葱服务描述符。Ed25519 还有一个被 Tor 以较特殊方式利用的性质：把公钥 A 乘以因子 h 可以得到另一个有效公钥 $A' = h \cdot A$；知道原始私钥的人也能计算对应的新私钥。这构成洋葱服务盲化密钥的基础（第 14.3 节）。

使用 1024 位密钥的 **RSA** 是 Tor 早期留下的历史遗产：它仍用于中继的旧式身份，以及目录权威给共识签名的密钥。权威密钥更长，而且两种密钥有不同要求：规范要求身份密钥（`KP_auth_id_rsa`）至少为 **3072** 位，签名密钥（`KP_auth_sign_rsa`）至少为 2048 位 [dir-spec § Directory authority operation and formats]。Tor 正在逐步淘汰 RSA。

5.7 较新的密码学构造

近年的协议演进中还会遇到两种更专门的构造：

- **POLYVAL** 是一种通用哈希函数（也用于 AES-GCM-SIV），与 AES 一起构成新的 relay-cell 加密方案 CGO 的基础（第 10.4 节）；
- **Elligator2** 可以把 Curve25519 上的点编码成看起来与随机字符串无法区分的数据；obfs4 用它来隐藏密钥交换公钥（第 16.4 节）。

当量子计算机真正成为现实威胁时，X25519 最终需要由后量子密码学替代。Tor 已在积极研究这一问题，详见 第 31 章。

小结

- 计数器模式下的 AES 用于加密数据；同一密钥流绝不能重复使用，而且 CTR 具有可塑性。
- SHA-1、SHA-256、SHA3-256/SHAKE-256 和 Blake2b 在 Tor 中并存；选择哪一种，取决于协议设计年代以及具体用途。
- HMAC 和 KDF（KDF-TOR、HKDF、SHAKE）分别用于认证和派生密钥；MAC 比较必须使用常数时间实现。
- X25519 是所有现代 Tor 握手使用的密钥交换；Ed25519 用于身份、证书和描述符签名；RSA 正在逐步退出。

练习

练习 **5.1** ⋆ 在 AES-CTR 中，攻击者已知某条消息的明文 P_1，并截获对应密文 C_1；同时还截获另一条错误地使用相同密钥流加密的密文 C_2。攻击者怎样恢复 P_2？

练习 **5.2** ⋆ 在 Python 中计算 `hashlib.shake_256(b"Tor").hexdigest(16)` 和 `hexdigest(32)`。两个输出之间是什么关系？这对把 SHAKE 用作 KDF 意味着什么？

练习 **5.3** ⋆⋆ 使用 `cryptography` 库生成两对 X25519 密钥，并验证双方得到完全相同的共享秘密。如果其中一方把 32 个零字节当成对方公钥，会发生什么？把结果与 ntor 对“无穷远点”的检查联系起来。

练习 **5.4** ⋆⋆ 为什么基于 SHA-3 的 Tor 协议会在每一种输入前加入域标签？如果同一个函数的两种不同用途没有被域分离，请描述一种可能的攻击。

延伸阅读

- J.-P. Aumasson, *Serious Cryptography*（No Starch Press）：本章各类密码学原语的现代实践入门。
- D. J. Bernstein 等关于 Curve25519 与 Ed25519 的原始论文，以及 RFC 7748（X25519）和 RFC 8032（Ed25519）。
- Tor 规范 [Thee] 的 “Preliminaries” 一节，其中定义了密码算法与哈希的约定。

第 6 章

网络架构

最后核验：*2026 年 9 月 23 日 — tor 0.4.9.13*；*2026 年 9 月 23 日 15:00 UTC* 的公共共识

本章目标

学完本章后，你将能够：

- 列出 Tor 网络中各类节点的角色及其责任；
- 描述协议层次，从 TLS 通道一直到应用程序的流（stream）；
- 从整体上跟踪用户通过 Tor 打开一个网页时发生的全部步骤；
- 理解不同实现和不同版本如何在同一张 Tor 网络中共存。

在进入后续章节的底层细节之前，我们先从全局观察这张网络。本章相当于一张地图：其中每一个元素都会在后面的章节中重新出现，并被进一步拆解；相应位置都会给出交叉引用。

6.1 角色

Tor 网络由承担不同角色的程序组成，而且同一个程序往往可以同时承担多个角色。图 6.1 对这些角色进行了概括。

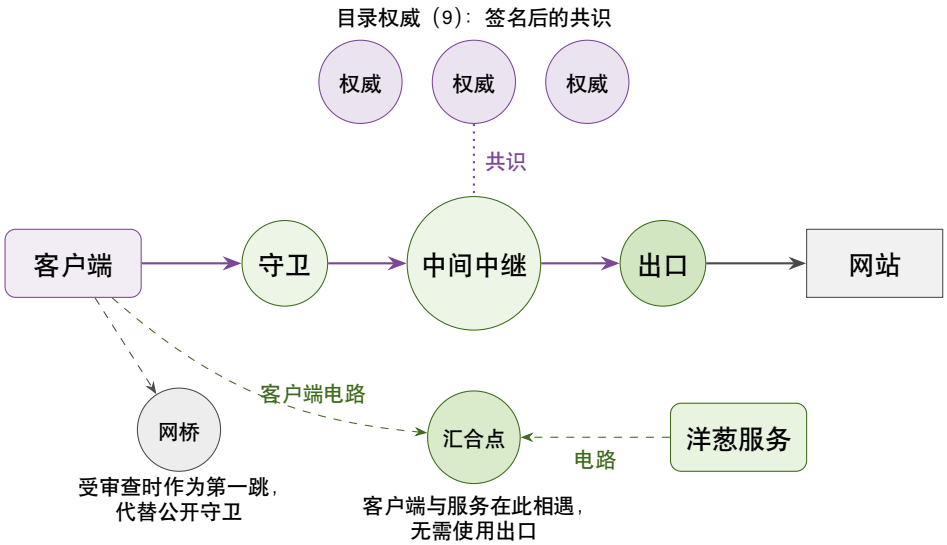


图 6.1: Tor 网络中的主要角色。客户端经由守卫、中间中继和出口构建电路；目录权威发布列出中继的共识；在审查环境下，网桥可以代替公开守卫充当第一跳；访问洋葱服务时，客户端与服务通过汇合点相遇，不需要出口。

客户端　用户侧程序，例如 Tor Browser 中包含的 Tor 客户端。它负责选择路径、构建电路，并向应用程序提供 SOCKS 端口。客户端不接受来自网络的入站连接，也不会向网络认证自己的身份（第 7 章）。

中继　为其他用户转发流量的节点。根据目录权威赋予的标志（flag）以及节点自身配置，它可以承担以下角色：

- 守卫（**guard**）：客户端电路的第一跳；客户端只偶尔重新选择守卫，并会在较长时间内重复使用它（第 13.3 节）；
- 中间中继（**middle**）：位于路径中间，既不知道原始来源，也不知道最终目的地；
- 出口（**exit**）：路径最后一跳，根据自身出口策略打开互联网连接（第 11 章）；
- **HSDir**：保存洋葱服务描述符（第 14.4 节）；
- 目录缓存（**directory cache**）：重新分发共识和描述符（第 12 章）。

目录权威　九台受信任服务器，每小时对网络状态进行投票并给共识签名；其中一些还负责测量中继带宽。

网桥　不公开列入常规中继列表的第一跳中继，供无法访问公开中继的用户使用，通常会配合可插拔传输（第 16 章）。

洋葱服务　只能通过 Tor 访问、同时不暴露自身网络位置的服务（第 14 章）。

在 2026 年 9 月 23 日的公共共识中，共有 9,358 个中继：其中 5,306 个带有 `Guard` 标志，3,177 个带有 `Exit` 标志（1,615 个同时拥有两者）。守卫贡献了总带宽的 70 %，出口贡献了 33 %（第 13 章）。

6.2　协议层次

Tor 协议可以看成由多层组成的协议栈，每一层都建立在下一层之上（图 6.2）。

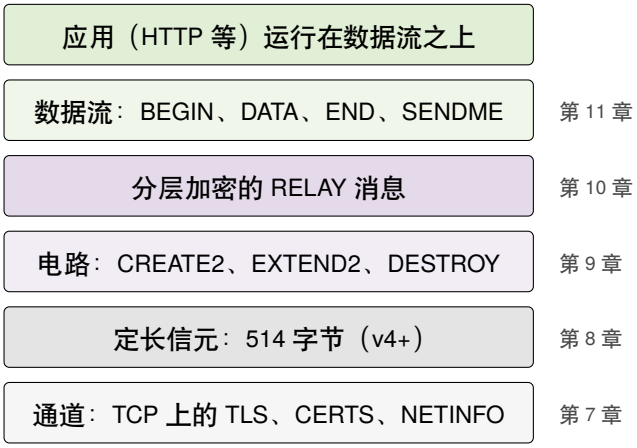


图 6.2: Tor 协议的层次，以及本书介绍各层的章节。每一层都依赖下面一层：应用流以 RELAY 消息的形式传输，RELAY 消息装入经过分层加密的 cell，而 cell 又在 TLS 通道中传输。

通道　两个相邻节点之间的 TLS 连接，同时用于验证中继身份（第 7 章）。

信元（**cell**）通道承载的 514 字节传输单元（第 8 章）。

电路　由多跳组成的路径，通过逐跳握手构建，并为每一跳派生不同密钥（第 9 章）。

分层加密 relay 消息会针对路径上的每一跳包上一层加密（第 10 章）。

应用流 由出口打开的应用 TCP 连接，通过窗口和拥塞控制机制进行管理（第 11 章）。

除这条协议栈之外，还并行存在目录系统和洋葱服务协议。目录系统向客户端提供中继列表；洋葱服务协议则利用电路，让客户端与服务在汇合点相遇。

6.3 一个网页从开始到结束

下面从高层次跟踪一个用户第一次启动 Tor Browser 并打开网页时发生的事情。

1. 启动。tor 联系一个目录权威，或者联系源代码中列出的约两百个备用目录服务器（*fallback directory*）之一，下载共识并验证其签名。随后，它下载中继的微描述符（第 12 章）。
2. 选择守卫。tor 按带宽加权，从守卫中随机选出一个较小样本，并实际使用其中两个（第 13.3 节）。
3. 建立通道。tor 向守卫打开 TLS 连接，并通过 **`CERTS`** cell 验证守卫身份（第 7 章）。
4. 构建电路。tor 选择一个适合目标目的地的出口和一个中间中继，并使用 **`CREATE2`** 与 `EXTEND2` 构建三跳电路（第 9 章和第 13 章）。实际运行时，tor 会提前构建一些电路，以便需要时立即使用。
5. 打开应用流。浏览器通过 SOCKS 端口请求连接到 `www.example.com:443`。tor 向出口发送 `BEGIN` 消息；出口解析域名并建立 TCP 连接（第 11 章）。
6. 传输数据。浏览器通过这条应用流与网站进行 HTTPS 通信。出口能看到网站名称和 TLS 流量，但不知道是谁发起了请求；守卫知道谁发起了请求，却不知道访问目标；中间中继两者都不知道。

整个启动过程通常只需要几秒钟。后续章节介绍的许多机制——包括 padding、拥塞控制和 vanguard——都会在这一流程周围静默运行。

6.4 实现与版本

6.4.1 两种实现

历史最悠久的实现是用 C 编写的 **tor**，它可以承担全部角色：客户端、中继、目录权威和洋葱服务。与此同时，Tor Project 正在开发用 Rust 重写的 **Arti**（第 30 章），目前已可作为客户端并支持洋葱服务。在 2026 年 9 月 23 日的公共共识中，没有中继使用 Arti：6,027 个中继运行 tor 0.4.9.12，2,520 个运行 0.4.9.11，其余大部分仍是 0.4.9 系列的较早版本；另有 47 个中继运行开发版本 0.5.0.0-alpha-dev。

6.4.2 子协议

一张由数千个节点构成、而且所有节点升级时间各不相同的网络，如何逐步引入新功能？Tor 不依赖软件版本号来完成这一点，而是使用子协议（**subprotocol**）：每一项功能属于一个拥有独立版本号的子协议，每个中继都会在共识的 `pr` 行中声明自己支持哪些版本 [tor-spec § Subprotocol-based versioning]。例如，公共网络上的某个中继可能声明：

```
pr Conflux=1 Cons=1-2 Desc=1-4 DirCache=2 FlowCtrl=1-2 HSDir=2 HSIntro=4-5
   HSRend=1-2 Link=3-5 LinkAuth=3 Microdesc=1-3 Padding=2 Relay=2-6
```

如果客户端希望与某个中继使用 ntor-v3，就会检查该中继是否声明 `Relay=4`；CGO 需要 `Relay=6`，拥塞控制需要 `FlowCtrl=2`。这些数字表示彼此独立的能力，并不是累积版本：从形式上说，`Relay=6` 并不自动蕴含 `Relay=4`（提案 346 因此更倾向使用“子协议能力”这一说法）。实践中，tor 因为按顺序实现了这些功能，所以通常会把支持能力声明成连续范围。共识还会列出必需的子协议；如果某个 tor 版本不支持它们，就会知道自己已经过旧，并拒绝继续运行，而不是让用户处于风险之中。

小结

- Tor 网络由客户端、中继（守卫、中间中继、出口、HSDir 和目录缓存）、目录权威、网桥以及洋葱服务组成。
- 协议形成一套分层结构：TLS 通道、信元、电路、分层加密和应用流。
- 守卫知道用户但不知道目的地；出口知道目的地但不知道用户。
- C 实现 tor 与 Rust 实现 Arti 可以共存。网络功能通过共识 `pr` 行中声明的子协议能力进行协商。

练习

练习 **6.1** ⋆ 对于以下每一种角色——守卫、中间中继、出口、HSDir 和目录权威——说明当某个用户通过 Tor 访问网站时，它能够知道哪些有关该用户的信息。

练习 **6.2** ⋆ 根据本章展示的 `pr` 行，这个中继支持第 9–11 章讨论的哪些功能？它能否作为使用 CGO 的出口？

练习 **6.3** ⋆⋆ 通过 Tor 下载一个 100 KB 网页。逐层估算客户端与守卫之间的链路上大约会传输多少字节，并考虑 cell、relay 消息头以及 SENDME 的额外开销。

延伸阅读

- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04]：Tor 的原始架构论文，其中很大一部分设计至今仍然适用。
- Tor Metrics, `https://metrics.torproject.org/`：中继、用户和版本的最新统计数据。

第 7 章

通道与链路协议

最后核验：*2026* 年 *9* 月 *23* 日 — *torspec* 提交 `8e1f245`；*tor 0.4.9.13*；*Arti 2.6.0*；链路协议 *4–5*

本章目标

学完本章后，你将能够：

- 解释什么是通道，以及 Tor 为什么在 TLS 之上建立通道；
- 描述构成中继身份的密钥，以及这些密钥如何被证书绑定在一起；
- 按信元顺序跟踪通道握手：VERSIONS、CERTS、AUTH_CHALLENGE、AUTHENTICATE 和 NETINFO；
- 读取并验证 Tor 的 Ed25519 证书；
- 理解为什么客户端不进行身份认证，以及这一选择对匿名性意味着什么；
- 在实验网络中向一个中继打开经过验证的通道（实验 1）。

电路是 Tor 的核心结构，本质上是一串连续的跳。然而，每一跳都建立在一个更简单的基础之上：两个相邻节点之间直接、加密的连接。规范把这种连接称为通道（**channel**）。本章将说明通道如何建立，更重要的是，通信双方如何确认自己连接到的确实是预期的对端。通道中传输的信元格式将在 第 8 章 中详细介绍，本章会频繁引用那里。

7.1 什么是通道?

通道是两个中继之间，或客户端与中继之间的一条直接加密连接 [tor-spec § Channels]。从技术上看，它是运行在 TCP 上、用于承载信元的 TLS 会话。它具有三个关键特征。

角色不对称 任何一方都可以发起通道，即 发起方（initiator）；但只有中继可以接收通道，即 充当 响应方（responder）。客户端不接受来自 Tor 网络的入站连接，因此没有面向 Tor 网络监听的端口。

身份经过验证 响应方必须以密码学方式证明自己掌握一个或多个中继身份。发起方通常事先已经知道应当看到哪个身份：这个身份可能来自共识、编译进源代码的目录权威列表，或者在为扩展电路而打开通道时，直接来自 `EXTEND2` 消息。如果对端展示的身份并非预期身份，就必须关闭通道。

共享 一个通道可以承载多条电路，不同电路用各自的 CircID 区分（第 8.4 节）。节点更倾向于复用已有通道，而不是向同一中继再打开第二条。没有电路的通道会在随机选择的空闲超时后关闭；范围取决于通道用途（`src/core/or/channelpadding.c` 中的 `channelpadding_get_channel_idle_timeout`）：客户端通道为 3–4.5 分钟，中继之间的通道为 45–75 分钟，其中中心值由共识参数 `nf_conntimeout_relays` 指定，默认为一小时。通道正在使用时，`KeepalivePeriod` 会让节点每五分钟发送一个填充信元，以避免防火墙把连接当作空闲连接关闭。

深入说明：通道还是连接？

较早的文档和许多研究论文把 channel 称为连接（*connection*），尤其是 *OR* 连接（*OR connection*）。规范后来逐渐避免这一叫法，因为它容易与出口中继为外部应用流打开的 TCP 连接混淆。tor 源代码中两个术语至今都存在：`or_connection_t` 表示底层 TLS 连接，`channel_t` 则是建立在它之上的通道抽象。

7.2　中继的身份

在研究握手前，需要先回答：中继究竟要证明什么？中继身份由长期密钥确定。规范采用系统化命名：`KP_` 前缀表示公钥，`KS_` 表示对应私钥 [tor-spec § Relay keys and identities]。与通道有关的主要密钥如下：

`KP_relayid_ed` Ed25519 身份密钥，也称“主身份密钥（master identity key）”。它不会过期，而且只有一个用途：给签名密钥签名，因此可以离线保存。

`KP_relayid_rsa` 1024 位的旧式 **RSA** 身份。Tor Metrics 中显示的 40 个十六进制字符指纹仍以它为基础，但协议安全已不再依赖这把密钥。

`KP_relaysign_ed` 中期使用的签名密钥，由身份密钥认证。中继日常在线运行时主要使用它。

`KP_link_ed` 短期密钥，仅在中继主动发起通道并需要认证自己时使用（第 7.6 节）。

图 7.1 展示了证书如何把这些密钥绑定在一起。其核心是一条从长期身份出发，一直延伸到当前连接 TLS 证书的信任链。

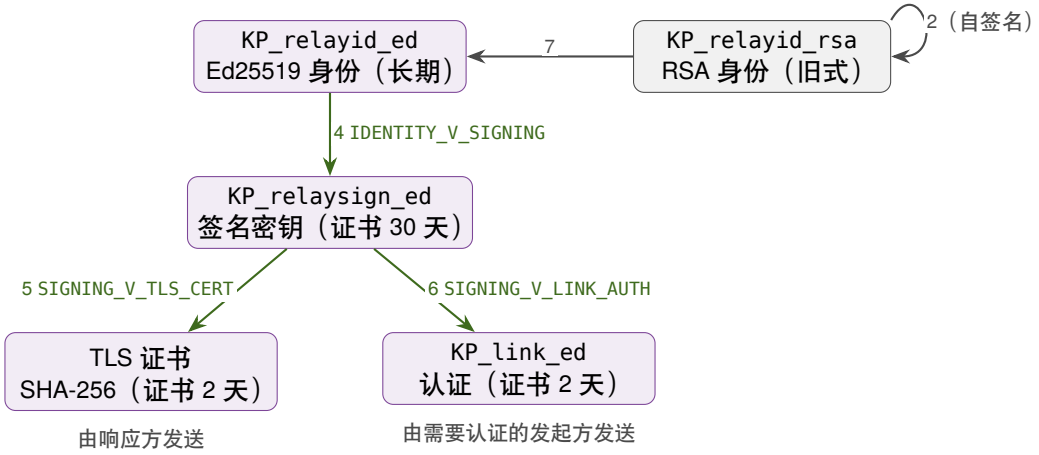


图 7.1: 把中继 Ed25519 身份绑定到当前连接 TLS 证书的证书链。每条箭头代表一个证书：起点密钥为终点处的密钥（或指纹）签名。数字表示 CERTS 信元中使用的 CertType。右侧是旧式 RSA 链，把旧 RSA 身份与 Ed25519 身份绑定起来。

在实验网络中，这些密钥位于每个中继的 `keys/` 目录：`ed25519_master_id_public_key` 及对应的私有身份密钥，签名密钥使用 `ed25519_signing_cert` 和 `ed25519_signing_secret_key`，RSA 身份则使用 `secret_id_key`。节点目录中的 `fingerprint-ed25519` 保存 Base64 编码的 Ed25519 身份。

历史说明：从 RSA 到 Ed25519

直到 2015 年，中继唯一的身份还是 1024 位 RSA 密钥；如今这一长度已被认为不足。提案 220 引入 Ed25519 身份以及本章介绍的证书格式，tor 0.2.7.2-alpha 开始支持。RSA 密钥为了兼容性被保留下来，并通过 cross-certificate 与 Ed25519 身份绑定。

7.3 握手全貌

图 7.2 概括了通道的建立过程 [tor-spec § Negotiating and initializing channels]。

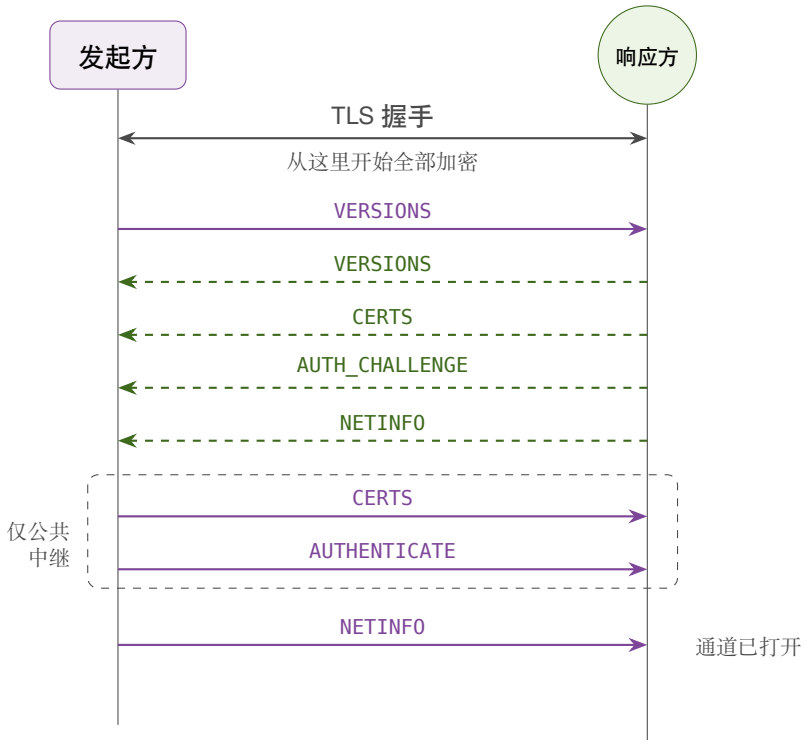


图 7.2: 通道握手。TLS 建立之后，后续内容全部在加密通道中传输。响应方总是依次发送 CERTS、AUTH_CHALLENGE 和 NETINFO。虚线框只适用于发起方是公共中继、因此必须证明自身身份的情况；客户端或网桥只发送 NETINFO。

1. 发起方打开 TLS 连接。
2. 双方各自发送 **VERSIONS** 信元，并协商链路协议版本（第 8.3 节）。
3. 响应方发送 **CERTS**（证明身份的证书）、**AUTH_CHALLENGE**（如果发起方要认证自己，需要对相关上下文签名）以及 **NETINFO**（时间和地址）。
4. 发起方验证证书。如果它是公共中继，则回复 **CERTS** 与 **AUTHENTICATE**；无论是否认证，最后都以 **NETINFO** 结束。

NETINFO 标志握手结束：一方一旦发送它，就不能再发送其他握手信元。响应方正是通过这一点判断发起方是否选择不进行认证。中间可以穿插任意数量的 **VPADDING**，除此之外不能出现其他类型的信元。

7.4　TLS 层

Tor 使用 TLS 的方式比较特殊。在普通网站中，服务器证书由证书颁发机构签发，并包含域名；浏览器会验证这两项。中继则不同：它提供一个没有实际域名意义的自签名证书，而客户端不会按照普通 TLS 规则验证它。因此实验 1 的代码中会出现两行在绝大多数其他场景里都属于严重安全错误的设置：

```
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE
```

这里它们并不是错误，因为真正的身份认证发生在 TLS 建立之后，由通道中的 **CERTS** 完成。TLS 提供通道加密、完整性，并证明响应方掌握 TLS 证书对应的私钥；其余身份关系则由 Tor 信元建立。

规范还给出几条规则 [tor-spec § The TLS handshake]：

- 实现应支持 TLS 1.3，中继还应支持 TLS 1.2，并使用现代密码套件（AES-GCM、ChaCha20-Poly1305，以及基于 X25519 或 P-256 的密钥交换）；
- 不应允许 *session resumption*：它会削弱前向保密，并曾使 *Triple Handshake*（2013）等攻击成为可能；
- 不应使用 TLS 压缩。

在实验网络中，使用 OpenSSL 3.5 的 Python 客户端会与 tor 协商 TLS 1.3，并使用 `TLS_AES_256_GCM_SHA384` 套件。

历史说明：为什么握手如此特殊？

最早的链路协议直接把 Tor 证书放进 TLS 握手（版本 1）；版本 2 则通过 TLS 重新协商隐藏它们，并利用所提供的密码套件列表暗示选择了哪种变体。原因是审查：在可插拔传输出现之前（第 16 章），Tor 试图让自身连接看起来像浏览器与 Web 服务器之间的普通 HTTPS。TLS 1.2 中的证书以明文传输，很容易识别。到了版本 3，证书被移到加密通道内部，以 **CERTS** 信元传输。今天，抵抗审查主要交给可插拔传输，因此更早的变体已经过时；规范指出，截至 2025 年 3 月 tor 仍接受其中一些旧形式，但它们已经计划移除。

7.5　CERTS：证明身份

7.5.1　信元

CERTS 是变长信元（command 129），其载荷是一个证书列表：

字段	字节	内容
N	1	证书数量
以下结构重复 *N* 次：		
CertType	1	证书类型
CertLen	2	证书长度
Certificate	CertLen	证书本身

一条 **CERTS** 不能包含两个相同类型的证书。已定义类型见 表 7.1 [cert-spec § Certificate types (CERT_TYPE field)]。

类型	名称	格式	认证内容
1	TLS_LINK_X509	X.509	TLS 密钥，由 RSA 身份签名（已过时）
2	RSA_ID_X509	X.509	自签名 RSA 身份
3	LINK_AUTH_X509	X.509	用于认证的 RSA 密钥（已过时）
4	IDENTITY_V_SIGNING	Ed	签名密钥，由 Ed25519 身份签名
5	SIGNING_V_TLS_CERT	Ed	TLS 证书指纹，由签名密钥签名
6	SIGNING_V_LINK_AUTH	Ed	KP_link_ed，由签名密钥签名
7	RSA_ID_V_IDENTITY	RSA→Ed	Ed25519 身份，由 RSA 身份签名
8–12	洋葱服务（8、9、11）、ntor 密钥交叉证书（10）、中继家族（12）		

表 7.1: 证书类型。“Ed” 表示 Tor 自己的 Ed25519 证书格式。

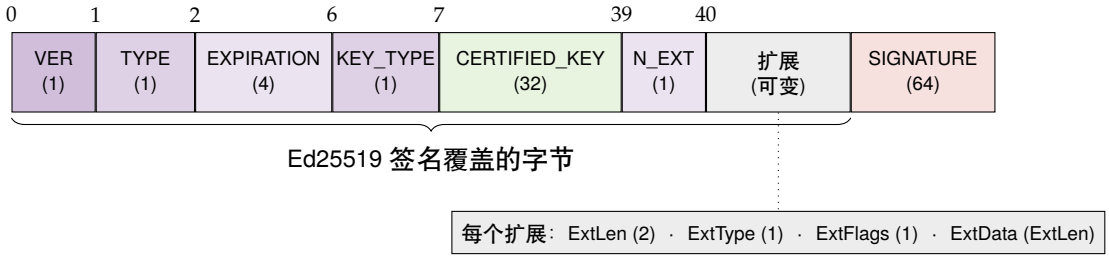


图 7.3: Tor 的 Ed25519 证书。签名前的所有字段都受到该签名保护。过期时间以 Unix epoch 以来的小时数表示；使用无符号四字节字段时，要经过几十万年才会溢出。

7.5.2 Tor 的 Ed25519 证书

Tor 的 Ed25519 证书不使用 X.509，而使用一种简单得多的自定义格式 [cert-spec § Tor Ed25519 Certificates]，如 图 7.3 所示。

VERSION 值为 1。

CERT_TYPE 与 表 7.1 中的 CertType 相同。

EXPIRATION_DATE 表示从 1970 年 1 月 1 日起经过的小时数。

CERT_KEY_TYPE 说明下一个字段包含什么：1 表示 Ed25519 密钥，3 表示 X.509 证书的 SHA-256 指纹。直到 tor 0.4.5.1-alpha，tor 都始终写入 1，因此规范要求实现兼容这种旧行为。

CERTIFIED_KEY 保存被认证的 32 字节值。

Extensions 每个扩展都包含长度、类型、标志位和数据。目前唯一定义的扩展是类型 4，*signed-with-ed25519-key*，其中包含给该证书签名的密钥。标志位 1（AFFECTS_VALIDATION）决定未知扩展如何处理：如果设置了该标志位，则必须拒绝证书；否则忽略未知扩展。

SIGNATURE 是对前面全部字段计算的 Ed25519 签名。

7.5.3 验证过程

响应方必须恰好发送一个类型 4 证书和一个类型 5 证书。发起方按以下步骤验证 [tor-spec § Authenticating the responder from its CERTS]：

1. 类型 4 证书的扩展 4 中包含一个 Ed25519 密钥，即声明的身份 KP_relayid_ed。该证书必须由这把密钥自己签名，也就是自签名；其 CERTIFIED_KEY 是 KP_relaysign_ed。

2. 类型 5 证书必须由 `KP_relaysign_ed` 签名，而它认证的值必须等于 TLS 握手中收到的 TLS 证书的 SHA-256 指纹。

3. 两张证书都不能过期。

4. 最后，必须把得到的身份与预期身份比较。

为什么这条链能证明身份？第 2 步把签名密钥绑定到当前连接的 TLS 证书；TLS 握手此前已经证明响应方确实掌握该证书对应私钥。第 1 步再把签名密钥绑定到长期身份。攻击者当然可以复制真实中继的 **`CERTS`**，因为其中没有秘密；但无法直接利用这些证书冒充中继，因为攻击者还必须在 TLS 握手中展示真实中继的 TLS 证书，并掌握与之对应的私钥。

注意：必须检查预期身份

一条完全有效的证书链只能证明响应方掌握某个身份。如果发起方省略第 4 步，那么任何拥有合法身份的人都能冒充原本希望连接的中继。实验 1 的 `expected_identity` 参数正是为此存在；测试中还专门验证“展示了不同身份的中继”会被拒绝。

7.5.4 旧式 RSA 链

发起方还可以选择验证 RSA 身份。类型 2 证书包含 1024 位 RSA 密钥（自签名，指数 65537）；类型 7 证书采用所谓“cross-certificate”格式，把该 RSA 密钥与已经验证的 Ed25519 身份绑定起来。cross-certificate 中的 RSA 签名，是对字符串 `"Tor TLS RSA/Ed25519 cross-certificate"` 与后续证书字段组合后所得 SHA-256 指纹进行签名。前缀用于防止同一签名被重用到其他上下文。

7.5.5 中继实际发送什么

在实验网络中，中继 `test003r` 发送的 **`CERTS`** 包含五张证书：

```
类型 1  584 字节
类型 2  442 字节
类型 4  140 字节 过期 2026-10-24 16:00  key_type=1  ext=[4]
类型 5  104 字节 过期 2026-09-29 11:00  key_type=3  ext=[]
类型 7  165 字节
```

除了 Ed25519 链（类型 4、5）和 RSA 链（类型 2、7），中继仍会为旧客户端发送类型 1 证书。这张证书不仅和 TLS 证书长度相同，指纹也相同；逐字节比较可确认它们就是同一张证书。

证书长度并不是值得背诵的常数，理解原因更重要。tor 会构造一个虚假的通用名称（common name）：以 `www.` 开头，后接 4–25 个随机字符，再以 `.com` 结尾（`src/lib/tls/tortls_openssl.c` 中的 `crypto_random_hostname`）。仅这一项就可能让不同中继之间的证书长度相差约二十字节。同一中继的长度还会随时间改变，因为 tor 每两小时更新一次 TLS 上下文，同时更新证书（`src/core/or/or.h` 中的 `MAX_SSL_KEY_LIFETIME_INTERNAL`）。我们在四十分钟后重复测量时，**`CERTS`** 载荷从 1,451 字节变成 1,456 字节。因此你的测量值不同完全正常；真正应一致的是结构：所有证书长度之和，加上一个计数字节和每张证书三个头部字节，应当恰好等于信元载荷长度。

两张 Ed25519 证书的过期时间也反映了不同生命周期：签名密钥被认证 30 天，而与 TLS 证书的绑定只持续 2 天。这是 tor 的默认值（选项 `SigningKeyLifetime` 和 `TestingLinkCertLifetime`；后者只能在测试网络中配置），因此同样适用于公共网络。长期身份保持不变，而代表它在线运行的密钥频繁更新，从而限制中继一旦被攻陷时可能造成的损害。

7.6 AUTH_CHALLENGE 与 AUTHENTICATE

7.6.1 谁需要认证?

发起方的身份认证是可选的，而“谁应该进行认证”是整个协议非常重要的设计选择之一 [tor-spec § Channels]：

- 一个公共中继主动向另一个中继打开通道时，应当认证自己；这样响应方就知道对方是网络中的中继，而不是普通客户端。
- 客户端或网桥不应认证自己。

第二条规则的原因就是匿名性。客户端本来没有网络身份，而人为创建一个长期身份反而有害：守卫可能在客户端从另一个地址重新连接时仍认出它。网桥则不应向使用它的客户端暴露自己是网络中继这一事实。代价是：如果通道没有经过发起方认证，中继可能推断自己是某条电路的第一跳（第 8 章, 第 8.6 节）。这也是中继拒绝为单跳电路充当出口的原因之一。

7.6.2 认证挑战

无论发起方最终是否认证，响应方都会发送 **`AUTH_CHALLENGE`**。它包含 32 个随机字节以及响应方接受的认证方法列表：

字段	字节	内容
Challenge	32	随机值，每条通道都不同
N_Methods	2	方法数量
Methods	2× N_Methods	方法列表

已定义方法包括 1（`RSA-SHA256-TLSSecret`，已过时）、2（从未实现）和 3（`Ed25519-SHA256-RFC5705`）。实验网络中继发送的信元载荷长 36 字节，即 $32 + 2 + 2$，表示只提供一个方法：方法 3。

7.6.3 认证响应

需要认证自己的中继，首先返回自己的 **`CERTS`**（使用类型 4 和 6，而不是 4 和 5，因为作为发起方的一方未必向 TLS 层提供过证书），随后发送 **`AUTHENTICATE`**。方法 3 的 **`AUTHENTICATE`** 载荷是一份 352 字节文档，由 `KP_link_ed` 签名：

字段	字节	内容
TYPE	8	字符串 `AUTH0003`
CID, SID	32 + 32	发起方与响应方的 RSA 身份 SHA-256 指纹
CID_ED, SID_ED	32 + 32	发起方与响应方的 Ed25519 身份
SLOG	32	通道上从开始到 AUTH_CHALLENGE（含）的所有接收字节的 SHA-256
CLOG	32	AUTHENTICATE 之前通道上所有发送字节的 SHA-256
SCERT	32	响应方 TLS 证书的 SHA-256
TLSSECRETS	32	从 TLS 会话秘密派生的值（*keying material exporter*）
RAND	24	随机字节
SIG	64	对前面全部字段的 Ed25519 签名

每个字段都把签名绑定到当前这一次连接的某个方面：参与双方的身份（CID、SID 等）、此前的通信内容（SLOG 与 CLOG，其中包含随机 challenge）、TLS 证书（SCERT），以及 TLS 会话本身（TLSSECRETS）。因此，这个签名不能被重用到另一条连接，也不能被位于两条不同 TLS 会话之间的中间人简单转发来冒充身份，因为两侧的 TLSSECRETS 和 SCERT 会不同。

深入说明：被规范保留下来的一个实现错误

TLSSECRETS 通过 TLS *exporter* 计算（RFC 5705；TLS 1.3 对应 RFC 8446 §7.5），使用标签 `"EXPORTER FOR TOR TLS CLIENT BINDING AUTH0003"` 和一个上下文。原本设计意图是把发起方的 Ed25519 身份作为上下文，但实现误用了 CID，也就是 RSA 身份指纹。由于这个错误并没有削弱安全性，规范选择记录真实实现行为，而不是修改已经广泛部署的协议。这是一个很有代表性的例子：对已经存在数千个部署实例的协议来说，真实代码行为有时会反过来成为标准。

7.7　NETINFO：时间与地址

双方发送的最后一条握手信元是 **`NETINFO`**。它是定长信元（command 8），载荷如下：

字段	字节	内容
TIME	4	Unix epoch 起的秒数，大端序
OTHERADDR	2 + ALEN	发送方眼中对端的地址
NMYADDR	1	发送方自身地址数量
MYADDR	NMYADDR × (2 + ALEN)	发送方自己的地址

每个地址编码为：一个字节的类型（IPv4 为 `0x04`，IPv6 为 `0x06`）、一个长度字节（4 或 16）以及地址值。这些字段有两个用途：

- 时间可以帮助发现时钟错误：如果中继发来的时间与本机相差超过一小时，tor 会记录警告（`src/core/or/channeltls.c` 中的 `NETINFO_NOTICE_SKEW`）。
- 地址让中继发现自己从外部看起来使用的公网地址；同时也允许发起方检查自己是否通过中继的某个规范地址到达它，也就是描述符中发布的地址。

注意：客户端不会泄露自己的时钟

中继会发送自己的时间和地址；客户端则应把时间写成 `00 00 00 00`，同时不发送任何自身地址。原因是指纹识别：计算机时钟几乎总存在微小而具有一定稳定性的偏移。如果客户端把自己的时间交给每个守卫，这种偏移可能帮助守卫长期识别同一客户端。这里体现了一条贯穿全书的原则：Tor 客户端不应泄露任何非严格必需的信息。

无符号四字节 TIME 字段将在 2106 年溢出；提案 338 讨论了把它扩展到八字节。收到 **`NETINFO`** 的一方也不应盲目信任其中的值：对端完全可以在时间或地址字段上撒谎。

下面是实验 1 解码从中继 `test003r` 收到的 **`NETINFO`**：

```
Netinfo(timestamp=1790511791, peer_address='127.0.0.1',
        my_addresses=['127.0.0.1'])
```

这个时间对应 2026 年 9 月 27 日 12:23:11 UTC。两个地址都是 loopback，因为整张实验网络都运行在同一台计算机上。我们的回复由 `client_netinfo_cell` 构造，载荷为 `00000000 04 04 7f000001 00`：时间为零，随后是中继地址，最后表示客户端不提供自己的地址。

7.8　握手之后

到这里，通道已经打开，可以开始承载电路。一个很简单的实验会揭示一个有趣细节：如果打开通道后不创建任何电路，只保持监听，中继什么都不会发送。在我们的实验中，连续十五秒没有收到任何信元。

这看起来有些意外，因为我们可能预期出现 **PADDING**。从链路协议第 5 版开始，客户端与守卫会在空闲通道上以 1.5 到 9.5 秒之间的随机间隔交换填充信元，使网络观察者——例如只记录 NetFlow 统计的路由器——难以区分“活动时段”和“静默时段”[padding-spec §Connection-level padding]。tor 源代码解释了为什么这里没有发生。决定是否填充通道的函数 `channelpadding_decide_to_pad_channel`（`src/core/or/channelpadding.c`）只会对已经用于完整电路或用户流量的通道启用 padding。

从中继一侧看，只有真正有多跳电路经过通道之后，才会认为该通道满足这一条件：例如收到第一条用于向第二跳扩展的 **RELAY_EARLY**，或者某条仍要继续越过当前中继的电路上的 **RELAY**（`src/core/or/relay.c`）。因此，新通道保持静默；只承载单跳电路的通道也一样，后者将在 第 9 章 中实际观察到。等到 第 10 章 中客户端把电路扩展到多跳后，padding 才会出现。

这个小实验还带来一个方法论上的教训：没有观察到某个现象，并不等于证明它不存在。在断言 padding 没有工作之前，必须先确认触发 padding 的前提条件是否真的满足。

7.9 实验：打开并验证一条通道

实验 1：向中继打开通道并验证其身份

目标。编写 `labs/zh-Hans/代码/torlab/channel.py`，让它像 Tor 客户端一样打开通道：建立 TLS，发送 **VERSIONS**，验证 **CERTS**，读取 **AUTH_CHALLENGE** 和 **NETINFO**，最后发送自己的 **NETINFO**。

前置条件。实验 2（`cells.py`，第 8 章）：信元处理是通道的基础，因此实现时先写它更合理，尽管书中会在下一章才正式介绍。这里“讲解顺序”是自顶向下（先通道、后信元），而“实现顺序”则自底向上（先信元、后通道），两者相反是正常的。还需要已经运行的实验网络（第 19 章），以及用于 Ed25519 的 `cryptography` 库。

预计时间。2–3 小时。

步骤 **1**：解码 **Ed25519** 证书。 按照 图 7.3，先读取定长头部，再解析扩展，并保留签名实际覆盖的字节：

```
def decode_ed_certificate(data):
    if len(data) < 40 + 64 or data[0] != 1:
        raise ChannelError("证书格式错误")
    _, kind, expiration, key_type = struct.unpack(">BBIB", data[:7])
    key = data[7:39]
    extension_count, pos, extensions = data[39], 40, {}
    for _ in range(extension_count):
        length, extension_type, flag = struct.unpack(">HBB", data[pos:pos+4])
        body = data[pos+4:pos+4+length]
        if extension_type != 4 and flag & 1:     # AFFECTS_VALIDATION
            raise ChannelError("未知扩展影响证书验证")
        extensions[extension_type] = (flag, body)
        pos += 4 + length
    return EdCertificate(kind, expiration, key_type, key,
                         extensions, data[:pos], data[pos:])
```

步骤 **2**：验证证书链。 下面的函数实现 第 7.5.3 节 中的规则，并返回已经得到证明的身份：

```
def verify_responder_identity(certificates, cert_tls_der):
    id_v_sign = decode_ed_certificate(certificates[4])
    sign_v_tls = decode_ed_certificate(certificates[5])
    identity = id_v_sign.signing_key               # extension 4
```

```
    id_v_sign.verify_signature(identity)           # self-signed
    sign_v_tls.verify_signature(id_v_sign.certified_key)
    if sign_v_tls.certified_key != sha256(cert_tls_der).digest():
        raise ChannelError("TLS 证书不匹配")
    for c in (id_v_sign, sign_v_tls):
        if c.expired():
            raise ChannelError("证书已过期")
    return identity
```

使用 `sock.getpeercert(binary_form=True)` 从 socket 取得 DER 格式 TLS 证书。即便这里没有采用标准 TLS CA 流程验证证书，Python 仍会返回它。

步骤 **3**：执行握手。 `open_channel` 发送 **`VERSIONS`**，声明支持版本 4 和 5（与 Arti 一样，不再提供 3）。随后不断读取信元，直到收到 **`NETINFO`**：第一条之后的 **`VERSIONS`** 以及 **`VPADDING`** 会被忽略，其他任何意外类型都会被拒绝。最后验证证书、把对端身份与预期身份比较，并发送客户端自己的 **`NETINFO`**。完整代码在仓库中，核心循环如下：

```
while netinfo is None:
    cell, buffer = _read(sock, buffer, version)
    if cell.command == CellCommand.VERSIONS:
        if version:
            continue
        version = negotiate_version(SUPPORTED_VERSIONS, cell.body)
    elif cell.command == CellCommand.CERTS:
        certificates = decode_certs(cell.body)
    elif cell.command == CellCommand.AUTH_CHALLENGE:
        ...                                        # 接受的认证方法
    elif cell.command == CellCommand.NETINFO:
        netinfo = decode_netinfo(cell.body)
    elif cell.command != CellCommand.VPADDING:
        raise ChannelError("握手期间收到意外 cell")
```

步骤 **4**：不连接网络的测试。 在 `labs/zh-Hans/代码/torlab/test/test_channel.py` 中生成两把 Ed25519 密钥，构造有效证书链并确认代码接受它。然后按照规范逐一破坏证书链：篡改签名、使用未被认证的签名密钥、更换 TLS 证书、制造过期证书、重复 CertType，并确认每一种缺陷都会被检测出来。只测试“合法输入能够通过”几乎证明不了验证器的正确性：一个无条件接受任何输入的错误验证器同样能通过这种测试。

步骤 **5**：连接实验网络。 实验网络运行后执行：

```
from torlab.channel import open_channel, identity_from_file

expected = identity_from_file("labs/zh-Hans/实验室
    /net/nodes/003r/fingerprint-ed25519")
c = open_channel("127.0.0.1", 5103, expected_identity=expected)
print(c.version, c.ed_identity.hex(), c.netinfo, c.auth_methods)
print(c.registry)
```

缩略输出如下：

```
5 785d3199f21d20ad...  Netinfo(time=1790511793, ...)  [3]
['TLS TLSv1.3 TLS_AES_256_GCM_SHA384', '<- VERSIONS (6 字节) ',
 '<- CERTS (1451 字节) ', '<- AUTH_CHALLENGE (36 字节) ',
 '<- NETINFO (509 字节) ', '-> NETINFO: 通道已打开']
```

如果改连端口 5104（中继 `test004r`），却仍期待另一个中继的身份，应得到正确错误：*中继身份与预期身份不同。*

小结

- 通道是两个相邻节点之间的 TLS 会话。只有中继接受通道，而一条通道可以承载多条电路。
- 中继的主要身份是 Ed25519 密钥 `KP_relayid_ed`，它认证签名密钥 `KP_relaysign_ed`；RSA 身份仍为了兼容而存在。
- 中继 TLS 证书是自签名的。真正的身份认证在通道内部通过 **`CERTS`** 完成，证书链把长期身份、签名密钥和 TLS 证书指纹绑定起来。
- 发起方必须验证签名、过期时间、TLS 绑定以及预期身份。
- 主动发起通道的公共中继使用 **`AUTHENTICATE`** 认证自己；该签名绑定 TLS 会话和此前通信。客户端和网桥不认证自己，以避免产生可长期识别的身份。
- **`NETINFO`** 结束握手。客户端发送零时间戳，并且不发送自己的地址。
- 链路 padding 只有在通道已经承载过完整电路或用户流量后才会启动。新通道或只承载单跳电路的通道保持静默。一旦越过这个门槛，即使在第一份应用数据出现前也会开始 padding（`nf_pad_before_usage`，默认值 1）。

练习

练习 **7.1** ⋆　为什么 Tor 客户端不需要面向网络的监听端口？在什么情况下中继会主动发起通道？

练习 **7.2** ⋆　某张 Ed25519 证书的 EXPIRATION_DATE 为 `0x000772F7`。它会在何时过期？为什么规范选择“小时”而不是“秒”作为单位？

练习 **7.3** ⋆　一条 **`AUTH_CHALLENGE`** 的载荷长 38 字节。它提供多少种方法？如果一条 36 字节信元的 N_Methods 等于 2，这是否合法？

练习 **7.4** ⋆⋆　攻击者记录了真实中继的一条 **`CERTS`**，然后在自己的服务器上原样重放，但 TLS 层展示自己的证书。验证过程中的哪一步会失败？如果攻击者同时展示真实中继的 TLS 证书，又会怎样？

练习 **7.5** ⋆⋆　一个中继位于 `203.0.113.5`，在 2026 年 1 月 1 日 12:00:00 UTC 向从 `198.51.100.7` 连接来的客户端发送 **`NETINFO`**。写出该信元载荷的十六进制表示。

练习 **7.6** ⋆⋆　假设 Tor 客户端用一把只生成一次并长期保存在磁盘上的密钥向守卫认证。请描述两种这种设计会具体损害用户匿名性的方式。

练习 **7.7** ⋆⋆　按照 第 7.5 节，为 `labs/zh-Hans/代码/torlab/channel.py` 增加可选的 RSA 身份验证（类型 2 和 7）。把得到的 RSA 公钥 SHA-1 指纹与中继的 `fingerprint` 文件比较。提示：指纹是公钥 DER 编码的 SHA-1。

练习 **7.8** ⋆⋆⋆　启动实验网络后，用 `labs/zh-Hans/代码/torlab/channel.py` 打开一条通道并保持它处于打开状态，同时让客户端 `test008c` 通过实验网络下载一个页面。你的通道会收到 **`PADDING`** 吗？先提出假设，再进行实验，并结合 `channelpadding_decide_to_pad_channel` 解释结果。

练习 **7.9** ⋆⋆⋆　解释为什么 **`AUTHENTICATE`** 中的 TLSSECRETS 字段能够阻止中间人把中继 A 的认证通过另一条 TLS 会话转发给中继 B，即使中间人忠实转发每一个信元。如果没有 TLSSECRETS，会发生什么？

延伸阅读

- *Tor Protocol Specification* 中的 “Channels”、“Negotiating and initializing channels” 和 “Relay keys” 章节，以及 *Tor certificate formats*（cert-spec）[Thee]。
- 提案 220，*Migrate server identity keys to Ed25519*：从 RSA 迁移的原因。
- padding 规范中的 *Tor channel padding*：基于 NetFlow 的对手模型与时间间隔选择。
- RFC 5705 与 RFC 8446（§7.5）：TLS *keying material exporter* 函数。

第 8 章

信元（Cell）

最后核验：*2026* 年 *9* 月 *23* 日 — *torspec* 提交 `8e1f245`；*tor 0.4.9.13*；*Arti 2.6.0*；链路协议 *3–5*

本章目标

学完本章后，你将能够：

- 逐字节描述定长 cell 和变长 cell 的格式；
- 解释 Tor 为什么采用固定大小的传输单元，这一设计能够保护什么、又不能保护什么；
- 理解 CircID，以及每条通道上选择 CircID 的规则；
- 识别所有信元命令和主要 RELAY 命令；
- 读取 RELAY 消息头部，并解释 `Recognized` 与 `Digest` 的用途；
- 编写 Python cell 编码器/解码器（实验 2）。

上一章介绍了两个 Tor 节点如何打开一条通道：它本质上是一条 TLS 连接，双方在其中协商协议版本并验证身份。本章继续回答一个更底层的问题：这条通道内部究竟传输什么？答案很简单：**cell**。从电路建立到下载网页，Tor 承载的一切最终都会被划分为 cell。理解 cell，就相当于掌握了书写整个 Tor 协议的“字母表”。

8.1 为什么使用信元?

低延迟匿名系统无法隐藏两个节点正在通信这一事实，但可以尽量隐藏它们交换了什么，以及中间节点能从通信形态中学到多少信息。假如每个节点都原样转发收到的数据，并保留各不相同的数据包长度，那么观察者可能只需在后续链路上寻找同样长度的消息，就能一路追踪一条 1,337 字节的消息。

因此，最初的洋葱路由设计以及后来的 Tor [DMS04] 都采用了固定大小的传输单元：cell。所有 cell 大小相同时：

- 单个消息的长度不会直接暴露其内容，也不会简单成为某个流的标识；
- 中继可以更简单、高效地管理队列和缓冲区；
- 头部可以保持很小，中继只需读取极少字节就能决定如何处理 cell。

注意：信元不能隐藏什么

Cell 统一的是单个传输单元的大小，并不会统一 cell 的数量或时间模式。同时观察电路两端的对手，仍然可能通过流量大小和时序进行关联。这是 Tor 的根本限制，将在 第 26 章中讨论。通道级和电路级 padding 只能提供部分缓解，分别见 第 7 章和第 26 章 以及第五部分。

历史说明：512 字节

2004 年介绍 Tor 的论文描述的是 512 字节信元：2 字节 circuit identifier、1 字节 command 和 509 字节数据 [DMS04]。其中 509 字节 body 一直保留至今。不过从 link protocol 4 开始，circuit identifier 扩展为 4 字节，因此完整 cell 大小增加到 514 字节。

8.2 从 TLS 字节流到信元

TLS 握手完成以后，双方会在一条连续字节流中按顺序交换 cell [tor-spec § Cells (messages on channels)]。TLS record 的边界和 cell 边界彼此独立：一个 cell 可以被拆到多个 TLS record 中，一个 TLS record 也可以同时包含多个 cell，甚至若干 cell 的片段。规范同时要求，“TLS record 的分段方式不得泄露 cell 类型或内容的信息”（本书译文）。

这对实现者有一个非常现实的影响：不能假设 socket 每次读取正好返回“一个完整 cell”。接收方必须把不断到达的字节积累在缓冲区中，只有当完整 cell 全部到齐后才取出它，并把多出来的字节留给下一次解析。实验 2 的解码器正是采用这种方式。

8.3 信元格式

Cell 有两种格式：定长和变长。两者都从相同的两个字段开始：`CircID` 与 `Command`。

8.3.1 常量

规范定义了以下长度 [tor-spec § Message lengths]，其中 v 表示在当前通道上协商出的 link protocol 版本：

名称	字节	含义
`CELL_BODY_LEN`	509	定长 cell body
`CIRCID_LEN`(v)，$v < 4$	2	CircID 长度
`CIRCID_LEN`(v)，$v \geq 4$	4	
`CELL_LEN`(v)，$v < 4$	512	定长 cell 总长度
`CELL_LEN`(v)，$v \geq 4$	514	

对任意 v 都有 $\texttt{CELL_LEN}(v) = 1 + \texttt{CIRCID_LEN}(v) + \texttt{CELL_BODY_LEN}$。

深入说明：一个旧名称

旧代码和旧文档把 `CELL_BODY_LEN` 称为 `PAYLOAD_LEN`。两个名字表示同一个值：509。

8.3.2 定长信元

图 8.1 展示当前 link protocol（$v \geq 4$）中的定长 cell。

如果要传输的信息少于 509 字节，发送方必须把 body 剩余字节填零，接收方则必须忽略这些填充 [tor-spec § Padding fixed-length cell bodies]。

8.3.3 变长信元

有些 cell，特别是通道握手中使用的 cell，可能需要超过 509 字节——例如携带证书——也可能只需要很少几个字节。变长格式通过增加一个 2 字节大端序整数 `Length` 来处理这类情况，并且不进行固定长度填充（图 8.2）。

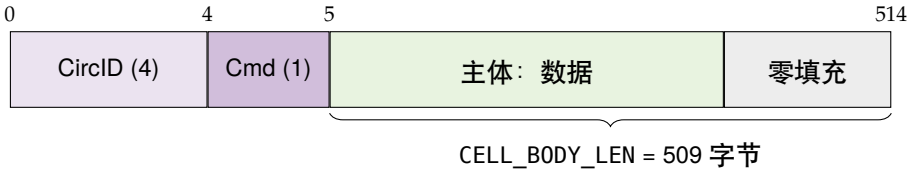


图 8.1: 定长 cell（link protocol ≥ 4）：4 字节 CircID、1 字节 command 和 509 字节 body，总计 514 字节。如果有效内容少于 509 字节，body 的剩余部分用零填充。图中的数字表示从 cell 开头算起的字节偏移。

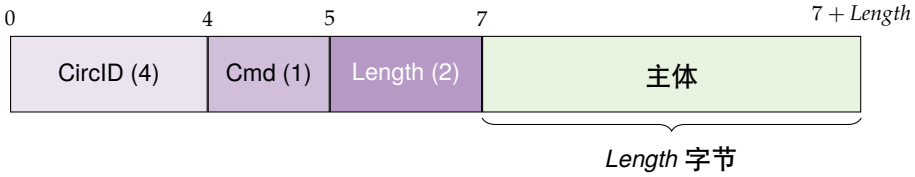


图 8.2: 变长 cell（link protocol ≥ 4）：CircID 和 command 之后增加一个 2 字节大端序 Length 字段，表示随后 body 的实际字节数。因此 body 长度可以在 0 到 65 535 字节之间。

8.3.4 如何区分两种格式

接收方必须在完整 cell 到齐之前就知道总共还需要读取多少字节。判断规则完全由 command 决定 [tor-spec § Cells (messages on channels)]：

- command 7（**VERSIONS**）表示变长 cell；
- 除它之外，小于 128 的 command 都表示定长 cell；
- 大于等于 128 的 command 都表示变长 cell。

换句话说，command 字节的最高位基本上承担“格式标志”的作用，唯一的历史例外就是 **VERSIONS**。

历史说明：VERSIONS 为什么是例外

link protocol 1 中所有 cell 都是定长。link protocol 2 引入版本协商，值为 7 的 **VERSIONS** 被加在原有 command 0–6 之后，因此成为唯一一个低编号的变长 cell。link protocol 3 随后把 ≥ 128 的范围保留给变长 command。版本 1 和 2 如今已经过时，实现不应再支持它们。

8.3.5 VERSIONS 的特殊情况

这里存在一个“先有鸡还是先有蛋”的问题：CircID 长度取决于协商出的版本，而版本本身又要通过 **VERSIONS** 协商。解决办法是：始终把 **VERSIONS** 当成 $v = 0$ 来编码，也就是使用 2 字节 CircID。它的 body 是一串 2 字节大端序整数，每个整数表示一个支持版本。因此，支持版本 3、4、5 的客户端发送：

字节	字段	含义
`00 00`	CircID	0，2 字节 CircID
`07`	Command	**VERSIONS**
`00 06`	Length	6 字节 body
`00 03 00 04 00 05`	Body	版本 3、4、5

双方选择两个 **VERSIONS** 中共同支持的最高版本。如果不存在共同版本，或者 body 长度为奇数，就必须关闭连接 [tor-spec § Negotiating versions with VERSIONS cells]。

版本	说明
1	（已过时）最初基于证书的握手
2	（已过时）TLS renegotiation 握手；引入变长 cell
3	（已过时）现代形式的“协议内部”握手（第 7 章）
4	4 字节 CircID
5	支持链路 padding 及其协商

当前实现中，tor 提供版本 3、4、5（`src/core/or/connection_or.c` 中的 `or_protocol_versions` 数组），Arti 只提供版本 4、5。因此两个最新实现之间总会协商到版本 5。

8.4 CircID

`CircID` 字段标识 cell 属于哪条电路。对于不属于任何电路的 cell，例如 **VERSIONS**、**NETINFO** 和 **PADDING**，它的值为 0。

最关键的一点是：**CircID 只在单条通道内部有效**。一条三跳电路会穿过三条通道（客户端 – 守卫、守卫 – 中间中继、中间中继 – 出口），并且在每条通道上使用不同的 CircID。每个中继维护一张转发表，把进入方向的 `(channel, CircID)` 对映射到出去方向的 `(channel, CircID)` 对，如 图 8.3 所示。

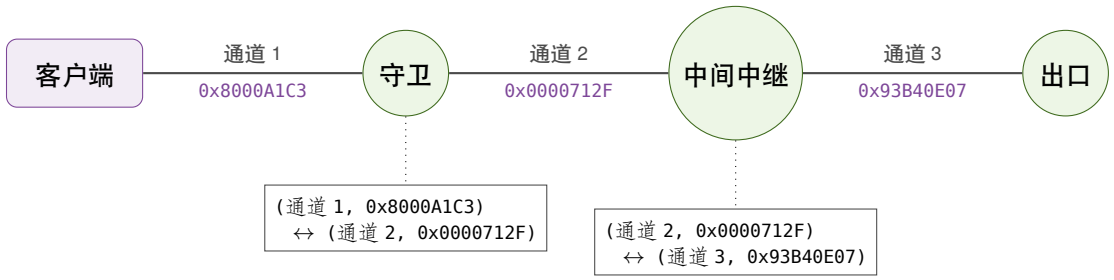


图 8.3: 同一条电路在每条通道上拥有不同的 CircID。守卫和中间中继通过转发表转换标识符，因此一条链路上的观察者不能仅通过比较 CircID 在另一条链路上找到同一电路。图中数值仅为示例，但符合后文介绍的最高位规则：通道 2 上的最高位为零，表示中间中继打开了该连接，而守卫在复用它。

这样设计有两个好处：不同节点无须彼此协调标识符分配；CircID 也不会变成一个能够把同一条电路不同链路段连接起来的全局标签。

8.4.1 谁选择 CircID？

CircID 由发送电路创建 cell（**CREATE2**，见 第 9 章）的一方选择。因为通道两侧都可能创建电路，所以必须避免双方同时选到同一个值。具体规则取决于 link protocol 版本 [tor-spec § Choosing circuit IDs in create cells]：

- **link protocol** ≥ 4：主动打开连接的一方使用最高位（MSB）为 1 的 CircID；另一方使用 MSB 为 0 的 CircID；
- **link protocol** ≤ 3：旧版本通过比较两端中继的身份公钥来划分标识符空间；公钥数值较小的一方使用 MSB 为 0 的一半空间。

值 0 被保留，除此之外没有特别保留值。现有实现会从自己可用的空间中随机选择 CircID，规范也建议新实现这样做，以免不同实现因可预测分配方式而被识别。tor 连续尝试 64 次仍找不到未使用值时，就放弃在该通道上创建新电路。

深入说明：为什么扩展到 4 字节？

CircID 只有 2 字节时，每一方只能使用一半空间，因此每条通道最多只能承载略多于 32 000 条电路，而且随机选择会在空间真正耗尽之前就频繁碰撞。提案 214（2012）直截了当地概括问题："中继正在耗尽 circuit ID"（本书译文）。该提案引入使用 4 字节 CircID 的 link protocol 4，并在 tor 0.2.4.11-alpha 中实现。

8.5 信元命令

`Command` 字节决定 body 应如何解释。下面两张表列出规范定义的全部值 [tor-spec § Interpreting the fields: Command]。C 列中，"是" 表示 command 要求 CircID 非零，"否" 表示要求 CircID 为零。P 列表示首次支持该 command 的 link protocol 版本；空白表示至少自版本 3 起就存在。

值	Command	C	P	用途
0	**PADDING**	否		链路 padding
1	**CREATE**	是		创建电路（TAP，已弃用）
2	**CREATED**	是		CREATE 的响应（已弃用）
3	**RELAY**	是		端到端数据
4	**DESTROY**	是		销毁电路
5	**CREATE_FAST**	是		不使用公钥密码学创建电路
6	**CREATED_FAST**	是		CREATE_FAST 的响应
8	**NETINFO**	否		时间和地址
9	**RELAY_EARLY**	是		与 RELAY 类似，但数量受限
10	**CREATE2**	是		创建电路（可扩展握手）
11	**CREATED2**	是		CREATE2 的响应
12	**PADDING_NEGOTIATE**	是	5	padding 协商

表 8.1: 定长信元的命令。

值	Command	C	用途
7	**VERSIONS**	否	link protocol 协商
128	**VPADDING**	否	变长 padding
129	**CERTS**	否	证书
130	**AUTH_CHALLENGE**	否	身份认证 challenge
131	**AUTHENTICATE**	否	initiator 身份认证
132	**AUTHORIZE**	否	保留

表 8.2: 变长信元的命令。

未知命令。 除上述值之外，其他值都不被允许。接收方遇到未知 command 时，可以静默丢弃 cell，也可以关闭通道。之所以允许如此严格，是因为每个新 command 都会和新的 link protocol 版本一起引入：正确实现不会发送对方没有声明支持的 command。在 link protocol 尚未版本化的年代则不同，当时未知 command 往往会被忽略，因为它可能只是来自更新版本的 Tor。

这些 command 可以按用途分组：

通道管理 **VERSIONS**、**CERTS**、**AUTH_CHALLENGE**、**AUTHENTICATE**、**NETINFO**：即 第 7 章 中的握手。

填充 **PADDING**、**VPADDING**、**PADDING_NEGOTIATE**：用于增加虚假流量，降低链路流量分析能力。

电路生命周期 **CREATE2**/**CREATED2**（以及历史变体）、**DESTROY**：建立与销毁电路（第 9 章）。

传输 **RELAY** 和 **RELAY_EARLY**：承载所有在电路内部传输的内容。

8.5.1 DESTROY

DESTROY 的 body 只有一个字节，用来说明关闭电路的原因 [tor-spec §Tearing down circuits]。节点收到 **DESTROY** 后会释放该电路资源；如果自己不是电路端点，还会向下一节点继续发送 **DESTROY**。

0	NONE	7	OR_IDENTITY
1	PROTOCOL	8	CHANNEL_CLOSED
2	INTERNAL	9	FINISHED
3	REQUESTED	10	TIMEOUT
4	HIBERNATING	11	DESTROYED
5	RESOURCELIMIT	12	NOSUCHSERVICE
6	CONNECTFAILED		

注意：原因码也可能成为侧信道

规范建议始终发送原因 NONE，并且不要原样传播收到的 reason code。假如中继把原始原因在两个方向继续转发，那么同一条电路上两个串通的中继就可能把这个字节当成信号，用来确认彼此是否位于同一条电路。这说明哪怕只有一个字节，也可能成为侧信道。

8.5.2 RELAY_EARLY

RELAY_EARLY 的格式与 **RELAY** 完全相同，但目的在于限制电路长度 [tor-spec §Handling RELAY_EARLY cells]。规则如下：

- 客户端必须只在 **RELAY_EARLY** 中发送用于扩展电路的消息（EXTEND2）；从 0.2.3.11-alpha 起，中继会拒绝通过普通 RELAY 收到的扩展请求；
- 如果一个中继在同一条电路上收到超过 8 个前向 **RELAY_EARLY**，应关闭该电路；如果收到哪怕一个朝客户端方向的反向 RELAY_EARLY，则必须立即关闭；
- 为了不直接泄露电路长度，客户端还应把大约前 8 个“不以第一跳为目的地”的 cell 也封装成 **RELAY_EARLY**。

如果没有这一限制，攻击者可以构建极长电路，让流量反复穿过相同中继，从而放大自己的流量并造成拥塞。这就是 Evans、Dingledine 和 Grothoff 描述的长路径拥塞攻击 [EDG09]。

历史说明：一个意外的信号

禁止反向 **RELAY_EARLY** 的规则来自 2014 年的一起事件：一些中继曾向客户端方向发送 RELAY_EARLY，把它作为隐蔽信道，对洋葱服务用户的电路做标记。第 27 章 将详细分析该事件。

8.6 RELAY 信元内部

RELAY 与 **RELAY_EARLY** 承载的是 **relay message**：客户端和电路节点之间交换的命令，从扩展电路一直到 TCP stream 中的应用数据。它们的 509 字节 body 会针对每一跳进行分层加密，这就是 第 10 章 中真正的 “洋葱”。从通道视角看，一个 RELAY cell 的 body 因而像完全被随机数据填满一样，看不到普通定长 cell 的零填充。

去除所有相关加密层以后，明文 body 采用 图 8.4 所示的传统格式 [tor-spec § Relay cells]。

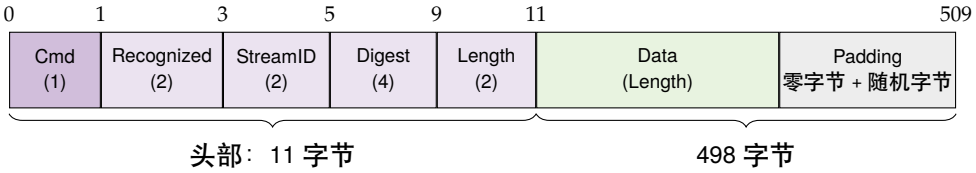


图 8.4: 传统 RELAY 信元的解密后主体：11 字节头部、最多 498 字节数据，以及如有剩余空间则填充至 509 字节。规范建议先填充至多四个零字节，再使用随机字节；不足四字节时全部填零，数据恰有 498 字节时无须填充。

RELAY 命令（**1** 字节）relay message 的类型（表 8.3）。

Recognized（**2** 字节）明文消息中应为零。中继可以借此快速判断 cell 是否以自己为目标：去掉自己的加密层后，如果该字段非零，那么 cell 肯定仍处于其他层的加密状态，应继续转发。

StreamID（**2** 字节）该消息所属的 stream，由客户端选择；如果消息针对整条电路而不是某个 stream，则为零。

Digest（**4** 字节）与该跳之间已经交换的全部消息的运行摘要前 4 字节（普通电路使用 SHA-1，洋葱服务电路使用 SHA3-256）。它确认 cell 确实以当前中继为目的地，并且没有被修改。

Length（**2** 字节）Data 中实际有效字节数。

Data 消息内容。

填充 剩余部分，使整个 body 达到 509 字节。

11 字节 header 意味着每个 cell 最多可以承载 $509 - 11 = 498$ 字节实际数据。

8.6.1 Recognized 与 Digest：两级过滤

为什么需要两个字段？如果中继对经过自己的每个 cell 都计算 digest，成本会更高。Recognized 是一个廉价过滤器：一个仍然有后续加密层的 cell 在当前层解密后，这两个字节近似伪随机，恰好等于零的概率只有 2^{-16}。只有 Recognized 为零时，中继才计算 digest；digest 也匹配，才认为 cell 被当前中继 “识别（recognized）” 并进行处理，否则继续转发。具体计算方法及其安全弱点将在 第 10 章 说明。

8.6.2 RELAY 消息填充

relay message 的 padding 与定长 cell 的零填充不同。规范建议先放 4 个零字节，后续使用随机字节 [tor-spec § Relay cells]。随机 padding 可以让 cell 内容不可预测，这对 提案 289 引入的经过认证的 SENDME 很重要：SENDME 通过包含所接收 cell 的 digest，证明数据确实被接收（第 11 章）。接收方绝不能依赖 padding 的具体内容。

8.6.3 RELAY 命令

表 8.3 列出定义的 relay command。Dir. 列表示方向：F（*forward*，只能从客户端向电路内部）、B（*backward*，只能朝客户端）、F/B（两个方向均可）。C 表示控制消息，这类消息的 StreamID 必须为零。

值	Command	Dir.	用途
核心协议			
1	BEGIN	F	打开 stream
2	DATA	F/B	stream 数据
3	END	F/B	关闭 stream
4	CONNECTED	B	stream 成功打开
5	SENDME	F/B, C?	接收确认（流量控制）
6–7	EXTEND/EXTENDED	F/B, C	使用 TAP 扩展（已过时）
8–9	TRUNCATE/TRUNCATED	F/B, C	截断电路（未使用）
10	DROP	F/B, C	端到端 padding
11–12	RESOLVE/RESOLVED	F/B	远程 DNS 解析
13	BEGIN_DIR	F	到目录缓存的 stream
14–15	EXTEND2/EXTENDED2	F/B, C	扩展电路
16–18	保留		UDP over Tor（提案 339）
Conflux			
19–22	CONFLUX_...	F/B, C	LINK、LINKED、LINKED_ACK、SWITCH：多电路协同（提案 329）
洋葱服务			
32	ESTABLISH_INTRO	F, C	建立引介点
33	ESTABLISH_RENDEZVOUS	F, C	建立汇合点
34–35	INTRODUCE1/INTRODUCE2	F/B, C	引介请求
36–37	RENDEZVOUS1/RENDEZVOUS2	F/B, C	加入汇合
38–40	..._ESTABLISHED, INTRODUCE_ACK	B, C	确认
电路 *padding* 与流量控制			
41–42	PADDING_NEGOTIATE/...D	F/B, C	电路 padding
43–44	XOFF/XON	F/B	stream 流量控制（提案 324）

表 8.3: RELAY 命令。SENDME 只有在针对整条电路而不是单个 stream 时才是控制消息（第 11 章）。

如果 cell 已被 recognized，但其中的 relay command 未知，就必须丢弃该 cell；不过，它的内容仍会计入 digest 和流量控制窗口。

8.7 走向新格式：提案 340 与 CGO

传统格式存在两个已知限制。第一是效率：大消息至少占用一个完整 cell，而多个很小的消息又不能共享同一 cell。第二是安全性：4 字节 digest 加上未认证的 AES-CTR 加密，为 *tagging* 攻击留下空间，这将在 第 10 章 中分析。

提案 340（*Packed and fragmented relay messages*）把 *cell* 和 *message* 两个概念分离：一条消息可以被分片到多个 cell 中，也可以把多条消息打包进同一个 cell。提案 359（*Counter Galois Onion, Updated*）引入新的 relay 加密方案 CGO，并采用 Proposal 340 的 cell 格式。CGO 解密后的 cell 结构如下：

字段	字节
authentication tag	16
body	$509 - 16 = 493$
body 内部：command	1
length	2
StreamID（仅部分 command）	2
data	*length*
padding（4 个零，之后随机）	剩余部分

只有确实需要 StreamID 的 command 才带该字段：BEGIN、DATA、END、CONNECTED、RESOLVE、RESOLVED、BEGIN_DIR、XON、XOFF。Recognized 和 Digest 都消失，改由 authentication tag 取代。只有客户端和中继都支持 CGO（子协议 `Relay=6`），并在电路建立过程中协商成功时，客户端才会对该中继使用 CGO。

深入说明：每个 CGO cell 能装多少数据？

由格式可知，一条 DATA 消息可用的数据空间为 $493 - 1 - 2 - 2 = 488$ 字节，比传统格式少 10 字节。这正是 16 字节 authentication tag 带来的代价。

CGO 已经不再只是纸面提案：tor 从 0.4.9 系列开始支持它（首次加入 0.4.9.3-alpha，2025 年 9 月）。Arti 中该支持始终启用，`counter-galois-onion` build option 仅为兼容性保留。规范仓库仍把两个 proposal 标为 “Open”；proposal 的状态反映标准化流程，并不总与代码实现状态同步。

8.8 一次会话的完整结构

现在可以把前面的组件组合起来，逐个 cell 跟踪客户端如何向守卫建立一条三跳电路并下载网页。图 8.5 汇总了整个序列，后续章节会逐步展开每一个步骤。

1. 通道握手：**`VERSIONS`**，随后守卫发送 **`CERTS`**、**`AUTH_CHALLENGE`** 和 **`NETINFO`**；客户端回复 **`NETINFO`**。这些 cell 的 CircID 始终为 0。
2. 第一跳：客户端用新的 CircID 发送 **`CREATE2`**（因为客户端打开了连接，所以 MSB 为 1）；守卫回复 **`CREATED2`**。
3. 第二跳和第三跳：两条包含 `EXTEND2` 的 **`RELAY_EARLY`**；守卫以及之后的中间中继把扩展请求转换为发往下一节点的 **`CREATE2`**。响应以包含 `EXTENDED2` 的 **`RELAY`** 返回。
4. **Stream**：向出口发送带 `BEGIN` 的 relay message，出口回复 `CONNECTED`；之后双向交换 `DATA`，并穿插 `SENDME`。
5. 关闭：用 `END` 关闭 stream；当整条电路生命周期结束时发送 **`DESTROY`**。

8.9 实验：信元编码器

实验 2：编码和解码信元

目标。编写 `labs/zh-Hans/代码/torlab/cells.py` 模块；后续所有实验都会使用它。
前置条件。Python 3.10 或更新版本，`pytest`。本实验不需要网络，只处理人工构造的字节序列。

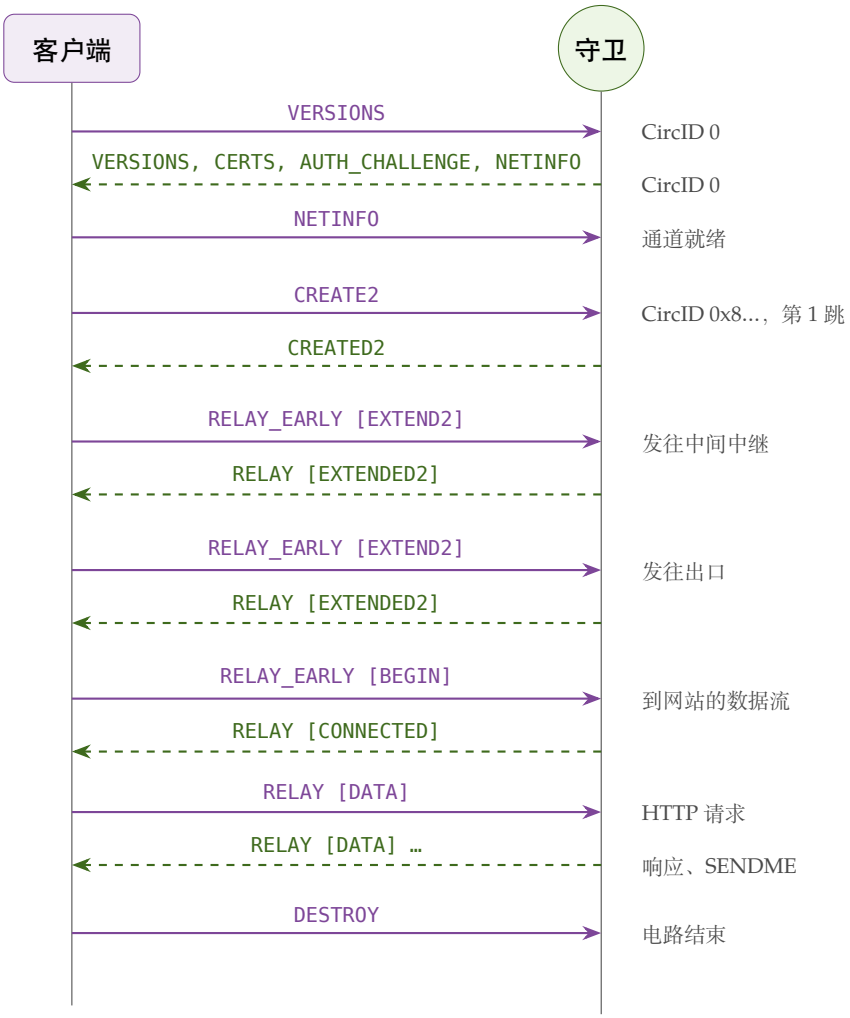


图 8.5: 客户端与守卫在 TLS 通道上交换的 cell 序列，用于建立三跳电路并打开 stream。守卫只能看到自己所在通道上的 cell；面向中间中继和出口的 RELAY 明文内容使用守卫并不掌握的密钥加密，因此守卫无法读取。

预计时间。1–2 小时。

步骤 **1**：常量与命令。 把规范表格转换成 Python 枚举，让每个数值只在一个地方定义：

```
CELL_BODY_LEN = 509
RELAY_HEADER_LEN = 11
RELAY_DATA_MAX = CELL_BODY_LEN - RELAY_HEADER_LEN   # 498

class CellCommand(IntEnum):
    PADDING = 0
    CREATE = 1
    # ... 表 8.1 和 8.2 中的所有值
    AUTHORIZE = 132

def circid_len(version):
    return 2 if version < 4 else 4

def is_variable_length(command):
    return command == CellCommand.VERSIONS or command >= 128
```

步骤 **2**：编码。 下面的函数接收 CircID、command、body 和 link protocol 版本，生成要发送的字节。注意 **`VERSIONS`** 的特殊规则和零填充：

```
def encode(cell, version):
    if cell.command == CellCommand.VERSIONS:
        version = 0                  # CircID 始终为 2 字节
    n = circid_len(version)
    header = cell.circ_id.to_bytes(n, "big") + bytes([cell.command])
    if is_variable_length(cell.command):
        return (header + struct.pack(">H", len(cell.body))
                + cell.body)
    return header + cell.body.ljust(CELL_BODY_LEN, b"\x00")
```

步骤 **3**：解码字节流。 TLS socket 送来的字节可能被任意分片，因此完整 cell 尚未到齐时，解码器返回 `None`；否则返回解析后的 cell 和尚未消费的剩余字节：

```
def decode(buffer, version):
    n = circid_len(version)
    if len(buffer) < n + 1:
        return None
    circ_id = int.from_bytes(buffer[:n], "big")
    command = buffer[n]
    if is_variable_length(command):
        if len(buffer) < n + 3:
            return None
        (length,) = struct.unpack(">H", buffer[n+1:n+3])
        start, end = n + 3, n + 3 + length
    else:
        start, end = n + 1, n + 1 + CELL_BODY_LEN
    if len(buffer) < end:
        return None
    return Cell(circ_id, command, buffer[start:end]), buffer[end:]
```

步骤 **4**：**VERSIONS** 与版本协商。

```
def negotiate_version(ours, received_body):
    if len(received_body) % 2:
        raise CellError("VERSIONS 的字节数为奇数")
    theirs = {v for (v,) in struct.iter_unpack(">H", received_body)}
    common = set(ours) & theirs
    if not common:
        raise CellError("没有共同支持的 link protocol 版本")
    return max(common)
```

中文版实验环境中的教学诊断信息同样使用简体中文；协议常量、函数名和数据格式保持不变。

步骤 **5**：**RELAY** 消息。 最后构造 11 字节 header。`struct` 使用的 `">BHH4sH"` 格式与 图 8.4 中的字段完全对应：

```
def encode_relay(msg, random_bytes=os.urandom):
    header = struct.pack(">BHH4sH", msg.command, msg.recognized,
                                msg.stream_id, msg.digest, len(msg.data))
    space = CELL_BODY_LEN - RELAY_HEADER_LEN - len(msg.data)
    padding = b"\x00" * min(4, space) + random_bytes(max(0, space - 4))
    return header + msg.data + padding
```

`random_bytes` 参数允许测试把真正的随机字节替换成可预测值，从而得到可重复结果。

验证。 仓库在 `labs/zh-Hans/代码/torlab/test/test_cells.py` 中包含测试。从仓库根目录运行，并让 `labs/zh-Hans/代码` 位于 `PYTHONPATH`：

```
$ PYTHONPATH=labs/zh-Hans/代码 python3 -m pytest -q labs/zh-Hans/代码
    /torlab/test/test_cells.py
........                                                 [100%]
8 passed
```

（如果不指定文件，`pytest` 会运行仓库中已经包含的所有实验测试。）例如，其中一个测试会逐字节检查“支持版本 3、4、5 的 **VERSIONS**”是否与 第 8.3 节 中的示例完全一致：`0000 07 0006 0003 0004 0005`。

扩展。 在实验 1（第 7 章）中，本模块将用于和私有网络上的真实中继通信。实验 4 将在此基础上加入 digest 计算和分层加密。

小结

- Cell 是 Tor 在通道上的基本传输单元。从 link protocol 4 起，定长 cell 为 514 字节（CircID 4 + command 1 + body 509）；更早版本为 512 字节。
- 变长 cell 增加 2 字节 Length 字段。**VERSIONS**（7）以及所有 command ≥ 128 都采用变长格式。
- **VERSIONS** 始终使用 2 字节 CircID；双方选择共同支持的最高版本。
- CircID 只在当前通道中有效。打开连接的一方随机选择 MSB 为 1 的值。
- **DESTROY** 应始终使用 reason NONE，避免创建侧信道；**RELAY_EARLY** 用来限制电路长度。

- 传统 RELAY 明文 body 有 11 字节 header（command、Recognized、StreamID、Digest、Length），最多承载 498 字节数据；padding 为 4 个零字节加随机字节。
- 提案 340 和 359（CGO）引入带 16 字节 authentication tag 和可选 StreamID 的新格式。

练习

练习 **8.1** ⋆ 在协商 link protocol 3 的通道上，一条 **`CREATE2`** 占多少字节？版本 5 呢？body 为 `00 04 00 05` 的 **`VERSIONS`** 总共占多少字节？

练习 **8.2** ⋆ 对命令值 0、7、9、12、129、200，逐一说明对应 cell 是定长还是变长。哪个值不被协议允许？

练习 **8.3** ⋆ 客户端使用 link protocol 5 打开一条到守卫的通道。以下哪些 CircID 可用于新电路：`0x00000000`、`0x12345678`、`0x92345678`？为什么？

练习 **8.4** ⋆⋆ 手工解码在 link protocol 4 通道上收到的下列字节序列。给出每个 cell 的 CircID、command 和 body 长度：
`00 00 00 00 81 00 03 aa bb cc 80 00 00 07 04 00 …`
（第二个 cell 的 body 尚未完整显示。）

练习 **8.5** ⋆⋆ 一条 stream 5 上的 relay DATA 消息包含 300 字节 data。在计算 digest 之前，写出其明文 body 前 11 个字节的十六进制值。之后还有多少 padding 字节？其中多少字节保证为零？

练习 **8.6** ⋆⋆ 一个中间中继处理 1 000 000 个实际上并不以它为目标的 cell。平均有多少个会需要进一步计算 digest？为什么只看 Recognized 不足以判断 cell 是否真正以该中继为目的地？

练习 **8.7** ⋆⋆ 描述一个具体场景：同一条电路上的两个串通中继如何利用 **`DESTROY`** 的 reason code 判断彼此位于同一电路？为什么统一使用 NONE 能阻止这种做法？

练习 **8.8** ⋆⋆ 客户端下载一个 1 MiB 文件（1 048 576 字节）。传统格式需要多少 DATA cell？CGO 需要多少？在两种情况下，cell header 带来的额外开销分别占有效数据的百分之多少（link protocol 4，不计 TLS）？

练习 **8.9** ⋆⋆⋆ 为实验 2 增加一个基于属性的测试（*property-based test*），例如使用 Hypothesis 库：随机生成定长和变长 cell，编码后连接起来，再把结果随机切成不同长度的片段。验证解码器最终恢复出的 cell 与原始输入完全一致。

练习 **8.10** ⋆⋆⋆ 在 Arti 源代码的 `tor-cell` crate 中找到编码和解码 channel cell 的函数。比较它们处理“不完整 cell”的方式，与实验 2 的 `decode` 函数有何异同。

延伸阅读

- *Tor Protocol Specification* 中的 “Cells”、“Relay cells” 和 “Negotiating channels” [Thee]：本章的一手规范来源。
- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04]，第 4 节：最初的 cell 格式及设计动机。
- N. Evans, R. Dingledine, C. Grothoff, *A Practical Congestion Attack on Tor Using Long Paths* [EDG09]：促成 **`RELAY_EARLY`** 的攻击。
- torspec 中的提案 340 与 359：新一代 cell 格式。

第 9 章

电路构建：TAP、CREATE_FAST、ntor 与 ntor-v3

最后核验：*2026* 年 *9* 月 *23* 日 — *torspec* 提交 `8e1f245`；*tor 0.4.9.13*；提案 *216*、*332*、*350*

本章目标

学完本章后，你将能够：

- 描述客户端如何使用 CREATE2 和 EXTEND2 逐跳构建电路；
- 解释一次电路握手必须保证什么：中继认证、秘密性与前向保密；
- 逐步跟踪 CREATE_FAST、ntor 和 ntor-v3，并理解 TAP 为什么被淘汰；
- 从协商出的密钥材料中派生某一跳使用的四个值（Df、Db、Kf、Kb）；
- 使用官方测试向量验证实现；
- 在实验网络上向真实中继建立电路（实验 3）。

有了已经打开的通道（第 7 章）以及 cell 这套“协议字母表”（第 8 章），我们终于可以构建 Tor 最核心的结构：电路（**circuit**）。本章将看到客户端如何分别与路径上的每一个中继建立只属于双方的共享秘密，以及怎样把这些秘密转换成实际加密流量所需的密钥。下一章第 10 章 会说明这些密钥如何真正用于分层加密。

9.1 一次只增加一跳

客户端以增量方式，也就是所谓“telescoping（伸缩式）”方式构建电路 [DMS04]：

1. 选择一条路径，例如守卫 R_1、中间中继 R_2 和出口 R_3（第 13 章）。
2. 通过与 R_1 的通道，发送包含密码学握手前半部分的 **`CREATE2`**。R_1 回复 **`CREATED2`**；此后客户端与 R_1 拥有共同的密钥材料。
3. 客户端向 R_1 发送一条为 R_1 加密的 `EXTEND2`，要求它把电路扩展到 R_2。消息中包含一份新的握手前半部分，这一次目标是 R_2。
4. R_1 向 R_2 打开或复用通道，把握手数据放进 **`CREATE2`** 发送过去。R_2 的 **`CREATED2`** 返回 R_1，随后 R_1 再把结果封装在 `EXTENDED2` 中转回客户端。
5. 客户端完成与 R_2 的握手，再以相同方式通过 R_1 和 R_2 把电路扩展到 R_3。

这种设计带来一个根本结果：与 R_3 的握手虽然会经过 R_1 和 R_2，但它是针对 R_3 的公钥设计的，因此 R_1 和 R_2 都无法推导出对应共享秘密。不同跳又使用不同秘密，因此任何中继都无法读取客户端与其他中继交换的内容。守卫 R_1 知道客户端是谁，却不知道流量最终去向；出口 R_3 知道流量去往哪里，却不知道原始客户端是谁。

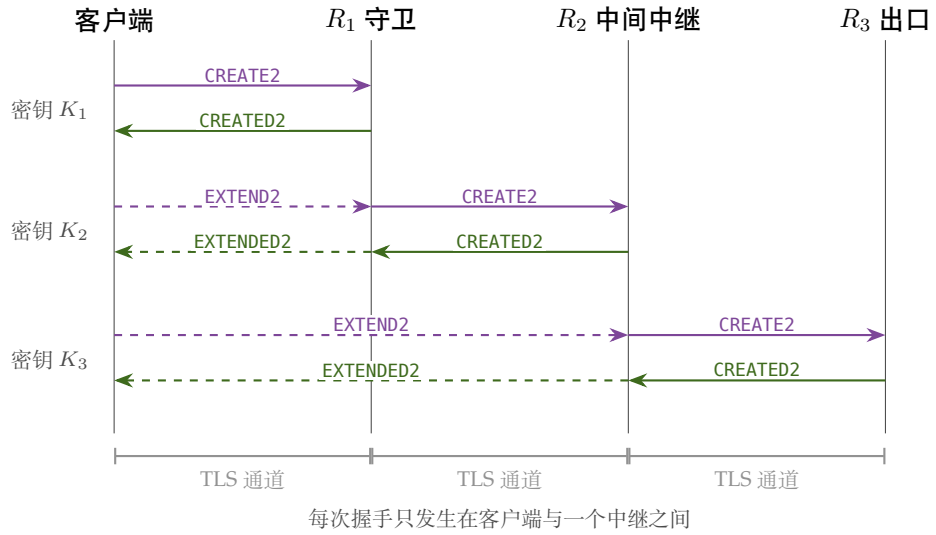


图 9.1: 三跳电路的伸缩式构建。每一次握手只发生在客户端与一个中继之间；中间的中继只是运输握手消息。虚线箭头表示消息在已经建立的电路内部传输，并使用此前跳已经协商出的密钥进行加密。

历史说明：第一代洋葱路由

最初 Naval Research Laboratory 的系统让客户端一次性构造一个完整“洋葱”：消息按所有中继的公钥逐层加密，每个中继剥掉自己的一层，得到下一跳地址和属于自己的会话密钥（第 3 章）。如果攻击者在未来攻陷某个中继，就可能使用长期密钥解密过去记录下来的洋葱。Tor 改为让每一跳都进行临时 Diffie–Hellman 握手，因此引入前向保密：会话密钥一旦被删除，即使未来获得长期密钥，也无法解密过去的流量 [DMS04]。

9.2　信元与消息

9.2.1　CREATE2 与 CREATED2

`CREATE2` 的格式刻意保持通用 [tor-spec § Create and Created cells]：2 字节握手类型 HTYPE、2 字节长度 HLEN 和 HDATA。**`CREATED2`** 只需要 HLEN 与 HDATA，因为握手类型已经由请求确定。所有内容都必须放进一个定长 cell，所以 **`CREATE2`** 中握手数据最多为 $509 - 4 = 505$ 字节。

HTYPE	握手	状态
`0x0000`	TAP	已过时；tor 0.4.9.1-alpha 中移除
`0x0001`	—	保留
`0x0002`	ntor	仍在使用
`0x0003`	ntor-v3	仍在使用；优先选择

如果握手成功，中继回复 **`CREATED2`**；失败则回复 **`DESTROY`**。新 cell 的 CircID 遵循 第 8.4 节：link protocol 4 及以后，打开通道的一方使用最高位为 1 的值。

9.2.2　EXTEND2 与 EXTENDED2

RELAY 消息 `EXTEND2`（RELAY 命令 14）除了包含 **`CREATE2`** 的握手字段，还包含一组 *link specifier*，告诉当前中继怎样联系并识别下一跳 [tor-spec § Extend and Extended messages]：

类型	内容	长度
00	IPv4 地址和 ORPort	6
01	IPv6 地址和 ORPort	18
02	旧式 RSA 身份（SHA-1，20 字节）	20
03	Ed25519 身份	32

负责扩展电路的中继必须验证消息中指出的每一个身份；如果新连接的中继没有在 第 7 章所述通道握手中证明自己掌握这些身份，就不能继续扩展电路。这样，即便客户端从未直接与 R_2 通信，也能确信自己的握手确实到达预期中继。当前中继还会拒绝把电路扩展到自己，或者扩展回刚刚把请求发来的那个中继。

两个细节可以看出协议对不可区分性的重视。首先，规范建议 link specifier 始终按照同一顺序发送（00、02、03、01），避免顺序本身暴露客户端实现。其次，EXTEND2 总是封装在 **RELAY_EARLY** 中（第 8.5 节）；这种 cell 数量受到限制，因此不能任意构造极长电路。

EXTENDED2（RELAY 命令 15）的 body 与 **CREATED2** 完全相同。

9.3 一次握手必须实现什么

在研究不同握手之前，先明确目标。握手结束时，客户端和中继必须拥有一个共享秘密，并满足：

中继认证 客户端必须知道它与那个特定中继共享秘密，而不是其他人。反过来，中继不能通过握手知道客户端身份：认证只发生一个方向，否则每个中继都能识别客户端。

秘密性 网络观察者以及电路中负责转发握手的前一跳，都不能推导出共享秘密。

前向保密 即使未来有人获得中继长期密钥，也不能重建过去的会话秘密。

效率 高负载中继每秒需要执行数千次握手，因此计算成本至关重要。

接下来看到的几种握手，在这些目标之间采用了不同取舍。

9.4 TAP：第一代握手

Tor 最初使用 TAP（*Tor Authentication Protocol*）。它把 1024 位模群上的传统 Diffie–Hellman 密钥交换与 1024 位 RSA 公钥加密结合起来：客户端用中继的 RSA “onion” key 加密自己的 Diffie–Hellman 半边，只有目标中继能够解密并回复。随后使用基于 SHA-1 的 KDF-TOR 派生密钥（第 9.8 节）。

TAP 能满足基本要求，但安全余量不断缩小：1024 位对 RSA 与模 Diffie–Hellman 都已经不足；其混合加密结构也被认为较脆弱，而且握手计算成本较高。从 2013 年开始，ntor 与 TAP 并存；到了 tor 0.4.9.1-alpha，中继彻底停止支持 TAP（提案 350）。它仍留下一个兼容性痕迹：中继描述符中还会发布旧 RSA “onion” key。在实验网络的 microdescriptor 中，可以看到 `onion-key` 与真正使用的 `ntor-onion-key` 同时存在。

9.5 CREATE_FAST：何时不需要公钥

有一种情况不必再进行公钥握手：第一跳。客户端刚刚向该中继建立 TLS 通道并验证过它的身份，TLS 已经提供加密且经过认证的信道。**CREATE_FAST** 正是利用这一事实 [tor-spec §CREATE_FAST/CREATED_FAST cells]：

1. 客户端发送 20 个随机字节 X。

2. 中继回复 20 个随机字节 Y 和 20 字节 KH。

3. 双方从 $K_0 = X \| Y$ 使用 KDF-TOR 派生密钥材料。输出最前面的 20 字节必须等于 KH，从而证明中继确实收到了 X。

`CREATE_FAST` 的安全性完全依赖 TLS：任何能够读取 TLS 流量的人，也能同时得到 X 与 Y。因此它只应使用于单跳电路。典型场景是客户端刚启动，需要先从某个中继下载目录信息，却尚未知道该中继的 onion key。tor 源代码对此非常明确：`src/core/or/circuitbuild.c` 中的 `should_use_create_fast_for_circuit` 只有在客户端没有第一跳可用 onion key 时才选择 **`CREATE_FAST`**。

注意：CREATE_FAST 会暴露第一跳

中继收到 **`CREATE_FAST`** 后，可以确定自己就是电路第一跳，因此通道另一端就是客户端。规范建议中继正是利用这一点，拒绝让这类单跳电路继续作为互联网出口。

9.6 ntor

ntor 由 Goldberg、Stebila 与 Ustaoglu 提出 [GSU13]，通过 提案 216 集成进 Tor（tor 0.2.4.8-alpha，2013）。它以 **Curve25519** 替代 RSA 和模 Diffie–Hellman。Curve25519 使用只有 32 字节的密钥，并能提供高效的椭圆曲线密钥交换（第 5 章）。

9.6.1 组成部分

中继拥有一把中期 Curve25519 onion key：私钥 b 和公钥 $B = bG$，并在描述符中以 `ntor-onion-key` 发布。客户端知道 B，也知道中继身份——在 ntor 中是 20 字节 RSA 指纹 `ID`。我们用 $\mathrm{EXP}(P, k)$ 表示点 P 与标量 k 的 Curve25519 Diffie–Hellman 运算，用 $H(x, t)$ 表示以 t 为 key、消息为 x 的 HMAC-SHA256。各个 t 都是从协议标识符 `"ntor-curve25519-sha256-1"` 派生的固定字符串。

9.6.2 协议

1. 客户端生成临时密钥对 (x, X)，并在 **`CREATE2`** 中发送 `ID`（20 字节）、B（32 字节，用来说明它使用的是哪把 onion key）以及 X（32 字节），共 84 字节。

2. 中继检查 `ID` 和 B，生成临时密钥对 (y, Y)，然后计算

$$\mathit{secret_input} = \mathrm{EXP}(X, y) \,\|\, \mathrm{EXP}(X, b) \,\|\, \texttt{ID} \,\|\, B \,\|\, X \,\|\, Y \,\|\, \texttt{PROTOID}.$$

   从中派生 $\mathit{KEY_SEED} = H(\mathit{secret_input}, t_{key})$ 和确认值 AUTH，随后返回 Y（32 字节）与 AUTH（32 字节）。

3. 客户端使用自己的秘密计算相同的 $\mathit{secret_input}$：$\mathrm{EXP}(Y, x) \,\|\, \mathrm{EXP}(B, x) \,\|\, \ldots$，然后验证 AUTH 是否匹配。

握手核心由两个 Diffie–Hellman 交换组成：

- $\mathrm{EXP}(X, y) = \mathrm{EXP}(Y, x)$ 把两把临时密钥结合在一起，提供前向保密；握手结束后 x 与 y 立即删除。
- $\mathrm{EXP}(X, b) = \mathrm{EXP}(B, x)$ 使用中继 onion key，提供认证：只有知道 b 的一方才能计算正确结果并产生正确的 AUTH。

两个交换缺一不可。没有第一个，未来偷到 b 的攻击者就能重建全部会话秘密；没有第二个，中间节点就可能冒充目标中继。与此同时，客户端完全没有使用长期密钥，因此中继只知道自己与“发送 X 的某个实体”建立了共享秘密。

深入说明：检查全零共享秘密

规范要求双方检查任何 Diffie–Hellman 运算结果都不能是这里用全零表示的无效结果。攻击者可能发送某些小阶点，让 $\mathrm{EXP}(Y, x)$ 落入可预测值。对 Curve25519 而言，检查全零结果是完整验证 Y 所在群的简单替代方案，而且足以满足这里的安全要求。在实验 3 中，这对应 `x25519_dh` 函数里的三行代码。

9.7 ntor-v3

ntor 有一个限制：握手本身除了密钥之外无法携带其他协商信息。随着协议发展，在建立某一跳时逐渐需要协商额外功能，例如拥塞控制（第 11 章）以及 CGO（第 10 章）。ntor-v3（提案 332）在两个方向都增加加密消息，同时改变了若干密码学选择。

9.7.1 变化

- 中继身份改为 32 字节的 **Ed25519** 身份，而不是 RSA 指纹。
- 哈希家族改为 SHA-3：SHA3-256 用于 hash 与 MAC，SHAKE-256 用于可变长度密钥派生。每个输入都带有经过长度编码的域标签（ENCAP），防止一个用途计算出的值被误解释成另一个用途的数据。
- 双方都知道的验证字符串 `VER` 参与所有计算，因此某个上下文中完成的握手不能直接重用于另一个上下文。普通电路中 tor 使用 `"circuit extend"`，对应 `src/core/crypto/onion_crypto.c` 中的 `NTOR3_CIRC_VERIFICATION`。
- 客户端可以发送加密消息 CM，中继也可以返回加密消息 SM。

9.7.2 协议

1. 客户端生成 (x, X)，并立即计算 $B_x = \mathrm{EXP}(B, x)$。根据 B_x 与公开参数派生两把“第一阶段”密钥：一把用 AES-256-CTR 加密 CM，另一把用 MAC 对它认证。随后发送 `ID`（32）、B（32）、X（32）、加密后的 CM 和 MAC（32）。
2. 中继计算 $\mathrm{EXP}(X, b) = B_x$，得到相同的第一阶段密钥，验证 MAC 并解密 CM。然后生成 (y, Y)，把 $\mathrm{EXP}(X, y)$ 和 $\mathrm{EXP}(X, b)$ 两个交换结果组合成 *key_seed*。再用 SHAKE-256 派生用于加密响应 SM 的密钥和电路密钥流。回复 Y（32）、$AUTH$（32）和加密后的 SM。
3. 客户端验证 $AUTH$，解密 SM，并派生相同的密钥流。

注意：客户端消息没有前向保密

CM 的加密密钥只依赖 B_x，而 B_x 使用中继的 onion key 派生。记录握手、并在未来得到 b 的攻击者，可以解密过去的 CM。规范对此明确说明：只有中继返回的数据具备前向保密。因此 CM 只能包含协商参数，绝不能包含用户数据。

9.7.3 扩展

CM 与 SM 采用统一格式 [tor-spec § Sending extensions in the circuit extension handshake]：第一个字节是扩展数量；每项扩展依次包含 1 字节类型、1 字节长度和数据。当前电路建立阶段定义的扩展包括：

类型	方向	含义
1	client → relay	`CC_FIELD_REQUEST`：客户端请求 提案 324 拥塞控制；body 为空
2	relay → client	`CC_FIELD_RESPONSE`：中继接受；1 字节 `sendme_inc`
3	client → relay	子协议请求，例如 CGO（`Relay=6`）使用 `[02 06]`

9.7.4 今天 **tor** 会选择哪种握手？

在 tor 0.4.9.13 中，每一跳分别选择握手类型（`circuit_pick_create_handshake`）。当中继支持拥塞控制且该功能已启用——当前公共网络通常如此——或者需要协商 CGO 时，使用 ntor-v3。否则退回 ntor。正如规范所指出，现代客户端实际上几乎总会选择 ntor-v3；ntor 主要作为兼容性方案保留。

9.8 从共享秘密到电路密钥

每种握手最终都会得到一串伪随机字节，从中提取传统 relay-cell 加密方案（称为 “tor1”）使用的密钥 [tor-spec § Setting circuit keys]：

密钥	字节	用途	方向
Df	20	累积摘要的初始值（SHA-1）	client → relay
Db	20	累积摘要的初始值（SHA-1）	relay → client
Kf	16	AES-128-CTR 密钥	client → relay
Kb	16	AES-128-CTR 密钥	relay → client

总计 72 字节，对应 tor 源代码中的 `CPATH_KEY_MATERIAL_LEN`。具体怎样产生字节流取决于握手：

KDF-TOR 用于 CREATE_FAST，也曾用于 TAP：$K = \mathrm{SHA1}(K_0 \| \texttt{00}) \,\|\, \mathrm{SHA1}(K_0 \| \texttt{01}) \,\|\, \ldots$。前 20 字节构成 KH，之后依次为 Df、Db、Kf、Kb。单字节计数器把输出限制在 5,120 字节以内，但远高于实际需求。

KDF-RFC5869 ntor 使用基于 SHA-256 的 HKDF，以 `KEY_SEED` 作为密钥；Df、Db、Kf、Kb 占最前面的 72 字节。

SHAKE-256 ntor-v3 使用输出的前 32 字节加密 SM，之后依次是 Df、Db、Kf、Kb。

两个方向使用不同密钥绝非多余。如果客户端与中继在两个方向共用同一 AES-CTR 密钥，就会重复使用同一密钥流；把两个密文 cell 做 XOR，就会直接得到两份明文的 XOR。

CGO 会改变密钥布局，详见 第 10 章。

9.9 实验：建立真实电路

实验 3：电路握手

目标。编写 `labs/zh-Hans/代码/torlab/ntor.py`，实现 CREATE_FAST、ntor 和 ntor-v3；使用官方测试向量验证实现；在实验网络上向真实中继建立单跳电路。
前置条件。实验 1 和 2；已经运行的实验网络；`cryptography` 库（X25519、AES）和标准库 `hashlib`（SHA-3、SHAKE）。
预计时间。3–4 小时。

步骤 **1**：用少量代码实现 **ntor-v3**。 实现几乎可以逐行对应规范。先定义辅助函数：

```
NTOR3_PROTOID = b"ntor3-curve25519-sha3_256-1"
def _t(name):  return NTOR3_PROTOID + b":" + name.encode()
def _encap(s): return struct.pack(">Q", len(s)) + s
def _h3(s, t): return hashlib.sha3_256(_encap(t) + s).digest()
def _mac3(k, msg, t):
    return hashlib.sha3_256(_encap(t) + _encap(k) + msg).digest()
def _kdf3(s, t, n): return hashlib.shake_256(_encap(t) + s).digest(n)
```

客户端握手如下：

```
def ntor3_client_start(ID, B, message, ver=b"circuit extend", x=None):
    sk, X = x25519_generate(x)
    Bx = x25519_dh(sk, B)
    phase1 = _kdf3(Bx + ID + X + B + NTOR3_PROTOID + _encap(ver),
                   _t("kdf_phase1"), 64)
    enc_k1, mac_k1 = phase1[:32], phase1[32:]
    encrypted = aes_ctr(enc_k1, message)
    mac = _mac3(mac_k1, ID + B + X + encrypted, _t("msg_mac"))
    state = Ntor3State(sk, X, B, ID, Bx, mac, ver)
    return state, ID + B + X + encrypted + mac

def ntor3_client_finish(st, response, length=72):
    Y, auth, msg = response[:32], response[32:64], response[64:]
    secret_input = (x25519_dh(st.x, Y) + st.Bx + st.ID + st.B + st.X + Y
                    + NTOR3_PROTOID + _encap(st.ver))
    key_seed = _h3(secret_input, _t("key_seed"))
    verify = _h3(secret_input, _t("verify"))
    auth_input = (verify + st.ID + st.B + Y + st.X + st.mac
                  + _encap(msg) + NTOR3_PROTOID + b"Server")
    if not hmac.compare_digest(_h3(auth_input, _t("auth_final")), auth):
        raise HandshakeError("ntor-v3 AUTH 无效")
    stream = _kdf3(key_seed, _t("kdf_final"), 32 + length)
    return aes_ctr(stream[:32], msg), stream[32:]
```

AUTH 使用 `hmac.compare_digest` 比较，无论两个值从第几个字节开始不同，执行时间都尽量保持一致。普通比较可能通过时序泄露前缀中已经匹配了多少字节。

步骤 **2**：测试向量。 密码学握手不能仅靠“两端能够互通”来验证：如果两个实现犯了同一个错误，它们仍然可能完美互通。正确方法是使用规范提供的测试向量：固定输入——包括那些正常情况下应该随机生成的密钥——以及逐字节规定的期望输出。提案 332 给出完整 ntor-v3 向量，提案 216 给出多组 ntor HKDF 向量。`labs/zh-Hans/代码/torlab/test/test_ntor.py` 复现这些数据，例如：

```
def test_ntor3_vectors_proposal_332():
    st, hdata = ntor3_client_start(h(V["ID"]), h(V["B"]), h(V["CM"]),
                                   ver=h(V["VER"]), x=h(V["x"]))
    assert hdata.hex() == V["client"]
    sm, stream = ntor3_client_finish(st, h(V["server"]), length=256)
    assert sm == h(V["SM"])
    assert stream[:32].hex() == V["keystream_start"]
```

其他测试还会验证：修改 *AUTH*，或者换一个 `VER` 字符串，都必须使握手失败。

步骤 **3**：中继密钥。 执行握手需要中继 Ed25519 身份与 `ntor-onion-key`。真实客户端从共识和 microdescriptor 取得它们（第 12 章）；实验中也从同类数据源读取，即实验客户端的 `cached-microdescs.new`。每个中继都有类似条目：

```
onion-key
-----BEGIN RSA PUBLIC KEY-----
MIGJAoGBAJet1Oxp+cvg/vgryEmJuVv2s++WxQt5Id1BQ9UNwCOVFcYvEN9KVklM
...
-----END RSA PUBLIC KEY-----
ntor-onion-key gsi2OisnQE96UkjOVvBKtWDLfwRdYG9d4DScg2wz4Qs
id ed25519 Kz1dj/u6/g7WvhHrt9Ds0jVZjKC+KHo26hWViXAJ1Pg
```

可以注意到：旧 TAP key 仍在发布但已不使用；真正需要的是后面两行 Base64 数据。

步骤 **4**：连接真实中继。 使用实验 1 打开到 `test003r` 的通道，然后分别为各种握手类型发送 **`CREATE2`**：

```
c = open_channel("127.0.0.1", 5103, expected_identity=ed)
st, hd = ntor3_client_start(ed, B, encode_extensions([]))
c.send(Cell(0x80000001, CellCommand.CREATE2,
              create2_body(HTYPE_NTOR_V3, hd)))
r = c.receive()
sm, stream = ntor3_client_finish(st, hdata_created2(r.body))
keys = CircuitKeys.from_stream(stream)
```

用各个变体重复这一测试，得到：

```
<- CircID=0x80000001 CREATED2 消息体=509 字节
   ntor-v3 成功，中继消息：[]
<- CircID=0x80000002 CREATED2 消息体=509 字节
   带 CC_FIELD_REQUEST 的 ntor-v3 成功，响应：[(2, b'\x1f')]
<- CircID=0x80000003 CREATED2 消息体=509 字节
   ntor 成功
<- CircID=0x80000004 CREATED_FAST 消息体=509 字节
   CREATE_FAST 成功
<- CircID=0x80000005 DESTROY 消息体=509 字节（MAC 已修改）
   原因：1
```

未修改时所有握手都成功。第二种情况下，中继接受拥塞控制并返回扩展 2，其中 `sendme_inc=31`（第 11 章）。最后一种情况故意翻转了 MAC 中的一位，中继用 reason 1、即 `PROTOCOL` 的 **`DESTROY`** 拒绝握手。这里可以暴露真实错误原因，因为错误由当前中继自己产生，而不是转发另一个节点的 **`DESTROY`**（第 8.5 节）。

一个意外观察。 第 7 章 中我们发现新通道不会收到 **`PADDING`**。现在，同一通道已经有四条打开的电路，但继续等待二十秒仍然没有收到任何 cell。原因仍在 tor 源代码中（`src/core/`

or/relay.c）：只有当某条电路“使用”通道并继续扩展到当前中继之外时，该通道才会成为 padding 的候选。我们的单跳电路不满足条件。到 第 10 章 真正扩展多跳电路时，padding 就会出现。

小结

- 电路逐跳构建：第一跳使用 **CREATE2/CREATED2**，之后通过已建立的电路发送 EXTEND2/EXTENDED2。每次握手只涉及客户端与一个中继。
- EXTEND2 包含链路说明符（地址、RSA 身份和 Ed25519 身份）；执行扩展的中继必须验证这些身份。
- 电路握手认证中继而不认证客户端，同时必须提供秘密性和前向保密。
- TAP（1024 位 RSA 与 DH）已在 tor 0.4.9.1-alpha 中移除。**CREATE_FAST** 依赖 TLS，只应使用于单跳电路。
- ntor 组合两次 Curve25519：临时 – 临时交换提供前向保密，临时 –onion-key 交换提供身份认证。
- ntor-v3 使用 Ed25519 身份和 SHA-3 家族，并加入加密消息来协商拥塞控制与 CGO 等功能；客户端消息本身不具备前向保密。
- 密钥流提供 Df、Db（各 20 字节）以及 Kf、Kb（各 16 字节），也就是每个方向各一把加密密钥和一个摘要状态。

练习

练习 **9.1** ⋆ 计算 ntor 的 **CREATE2** 中 HDATA 长度；再计算 ntor-v3 在 CM 为 3 字节时的 HDATA 长度。一个 **CREATE2** 最多能够承载多大的 ntor-v3 CM？

练习 **9.2** ⋆ 为什么不能在电路第二跳使用 **CREATE_FAST**？第一跳中继能够做什么？

练习 **9.3** ⋆ 为什么客户端与中继在两个方向使用不同 AES key（Kf、Kb）？如果只使用一把 key，会发生什么？

练习 **9.4** ⋆⋆ 为了把电路扩展到实验中继 test004r，写出一个 EXTEND2 body 头部（specifier 数量与 IPv4 specifier）的十六进制值。使用 ntor-v3 且 CM 只有 1 字节时，完整消息共多少字节？

练习 **9.5** ⋆⋆ 在 ntor 中，假设客户端只使用 EXP(Y, x)，完全省略 EXP(B, x)。请描述一个由转发握手的中间中继实施的具体攻击。反过来，如果省略 EXP(Y, x)，失去的是哪一种安全属性？

练习 **9.6** ⋆⋆ ntor-v3 中 VER 字符串的目的是什么？设想 Tor 之外的另一个协议使用相同密钥执行相同握手；如果没有 VER，可能出现什么问题？

练习 **9.7** ⋆⋆ 重复实验步骤 4，并在 CC_FIELD_REQUEST 之外加入子协议请求 [02 06]（CGO）。中继是否接受？根据 提案 359 解释为什么 CGO 还要求拥塞控制。

练习 **9.8** ⋆⋆⋆ tor 仓库包含 ntor 的 Python 参考实现 src/test/ntor_ref.py。用它生成完整 ntor 握手测试向量（客户端与中继两端），并加入 test_ntor.py。

练习 **9.9** ⋆⋆⋆ 实现 ntor-v3 的中继端（函数 server_part1 和 server_part2），使用 提案 332 的向量验证。然后在内存中让你自己的客户端实现与中继实现完成一次完整握手。

延伸阅读

- *Tor Protocol Specification* 中的 “Creating and extending circuits” 与 “Setting circuit keys” [Thee]。
- I. Goldberg, D. Stebila, B. Ustaoglu, *Anonymity and one-way authentication in key exchange protocols* [GSU13]：定义 ntor 并给出其安全性论证的论文。
- 提案 216（ntor）、提案 332（ntor-v3）和提案 350（移除 TAP）。

第 10 章

洋葱：RELAY 信元加密与 CGO

最后核验：*2026* 年 *9* 月 *23* 日 — *torspec* 提交 `8e1f245`；*tor 0.4.9.13*；提案 *359*

本章目标

学完本章后，你将能够：

- 描述客户端如何为每一跳包上一层加密，以及中继如何逐层去除；
- 解释 Recognized 与累积摘要的作用，以及为什么两者都需要；
- 识别传统方案的弱点：可塑性与 *tagging* 攻击；
- 描述正在取代传统方案的 Counter Galois Onion（CGO）的基本工作方式；
- 使用自己的代码把真实电路扩展到三跳（实验 4）。

在 第 9 章 中，客户端与电路中的每个中继分别协商出四个值：用于加密的 Kf、Kb，以及用于完整性检查的 Df、Db。本章让这些密钥真正开始工作。这就是“洋葱路由”名称的来源：一条消息会按中继数量包上多层加密，每个中继只去掉自己的一层，如同剥洋葱。

10.1 传统方案

Tor 从早期一直使用的方案，在新代码中称为 *tor1*。对每一跳、每个方向，它组合两种工具：

- 使用 Kf 或 Kb 的 **AES-128-CTR** 流式加密；
- 以 Df 或 Db 初始化的累积 **SHA-1** 摘要。

两者在整条电路生命周期内都保持状态。AES-CTR 的计数器不会为每个信元重新从零开始：第一条信元消耗密钥流的前 509 字节，第二条接着使用后 509 字节，以此类推。摘要同样不是对单个信元单独计算，而是涵盖迄今为止与该跳交换过的所有信元。因此客户端和中继必须以完全相同的顺序处理信元；若丢失或重复一条信元，两边的加密和摘要状态都会失去同步。底层 TLS 保证通道上的顺序和完整性，使这种状态机可行。

10.1.1 前向：从客户端进入电路

要向电路第 d 跳发送消息，客户端按如下步骤处理 [tor-spec § Routing relay cells]：

1. 构造 RELAY 消息明文载荷（第 8.6 节），先把 Recognized 与 Digest 设为零；
2. 用该载荷更新第 d 跳的 forward 摘要，并把结果的前 4 字节写入 Digest；
3. 先用第 d 跳 Kf 加密，再用第 $d-1$ 跳 Kf 加密，依次回到第 1 跳；最外层属于守卫。

每个中继收到信元后，都用自己的 Kf 解密一次，去掉一层，再判断信元是否以自己为目的地；如果不是，就继续转发。图 10.1 展示了这一过程。

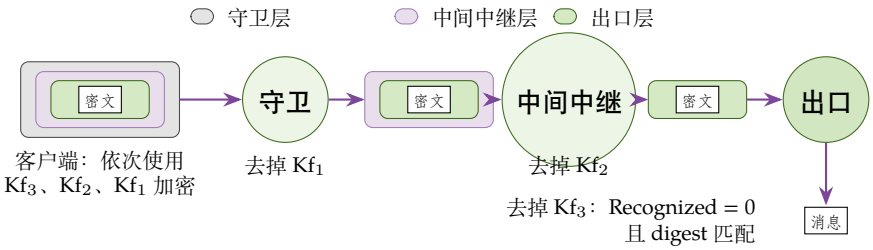


图 10.1: 传统 tor1 RELAY 信元的分层加密：客户端应用 Kf_3、Kf_2、Kf_1，中继依次去除对应层。出口在 Recognized = 0 且运行摘要正确时识别协议消息；应用数据仍可能受到 HTTPS 保护。内部方框示意仍处于加密状态的数据。

10.1.2 反向：从电路返回客户端

反方向上角色相反。产生消息的中继先用自己的 backward 摘要对消息签名，并使用 Kb 加密。之后朝客户端方向经过的每个中继都会用自己的 Kb 再加一层。客户端掌握所有密钥，所以从第一跳开始依次去掉各层，并在每次去层后检查该信元是否由这一跳产生。

10.1.3 作为“漏水管道”的电路

客户端可以把消息发送到电路中的任意一跳，而不仅仅是最后一跳：只需在目标跳停止继续增加加密层即可。发送给第二跳、要求扩展到第三跳的 `EXTEND2` 就是这样工作的。规范把这一结构称为 *leaky pipe topology*（漏水管道拓扑）。Tor 原始论文 [DMS04] 还提出一种防御用途：让一部分数据流从中间节点流出，可以增加出口观察者判断数据流属于哪条电路的难度。不过现实中的 tor 只让数据流从最后一跳退出。

10.2 识别信元

中继去掉自己的一层后，怎样判断信元是否发给自己？它不能简单地“看看内容是否像正常消息”：对程序而言，仍被加密的信元和明文信元都只是字节串。传统方案按顺序做两项检查 [tor-spec § Relay cells]：

1. **Recognized**：两个字节必须为零。仍处于加密状态时，这两字节近似伪随机，恰好为零的概率只有 1/65 536。这项检查成本极低，可以立即过滤掉绝大多数经过的信元。

2. **Digest**：只有 Recognized 为零时，中继才暂时假设信元属于自己，把 Digest 字段清零后计算累积摘要，并把前四字节与信元内的 Digest 比较。匹配则说明信元“已识别”，此时中继正式更新摘要状态并处理消息。

一个很容易忽略的细节是：如果 Recognized 为零但 Digest 不匹配，这只是一次误判（false positive），摘要状态绝不能改变。实验 4 会在临时副本上计算（`self.digest.copy()`），只有验证成功后才保存新状态。

摘要覆盖完整载荷，包括填充，因此填充也必须使用随机数据（第 8.6 节）。对于洋葱服务电路，客户端和服务之间存在一个共享密钥的“虚拟跳”（第 14 章），该位置使用 SHA3-256 而不是 SHA-1。

如果最终没有任何人识别信元会怎样？前向方向上，如果最后一个中继仍无法识别，就说明发生错误，它用 **`DESTROY`** 销毁电路；反向方向上，如果客户端去掉所有跳的层后仍无人匹配，它也销毁电路。

10.3 tor1 的弱点

10.3.1 可塑性

CTR 模式有一个不方便的性质：密文只是 $C = P \oplus S$，其中 S 是密钥流。攻击者无需知道密钥，只要翻转 C 中一位，就能让 P 中对应位发生同样翻转。如果攻击者知道或能猜出某段明文，还可以把它替换为选择的内容：只需把密文 XOR 上两段明文的差值。实验 4 包含一个演示测试，把“支付 10 欧元”改成“支付 99 欧元”。

在 tor1 中，完整性只在最终目标处检查：目标中继验证摘要，中间中继并不会验证。因此被修改的信元可以一路无障碍传到目标，直到那里摘要才失败。

10.3.2 标记（tagging）攻击

这些性质组合起来，就允许一种特定的 *tagging* 攻击。假设攻击者同时控制某条电路的守卫和出口：

1. 守卫按某种可识别模式修改经过信元的若干位，也就是加上“标记”（*tag*）。
2. 信元经过中间中继；中间中继无法发现这项修改。
3. 出口在验证摘要之前，先检查预期标记是否出现在信元中。如果存在，它就知道这条电路经过与自己串通的守卫，于是可以把出口看到的目的地和守卫知道的客户端地址关联起来。

如果出口并未串通，摘要会失败、电路被销毁，客户端随后新建电路。攻击者可以再次尝试，而每次失败也可能泄露信息。标记同样可以从出口向客户端的反方向施加。

深入说明：Tagging 的实际重要性

同时控制守卫和出口的攻击者，即使不加任何标记，也已经可以通过流量时序和数据量进行关联（第 26 章）。因此 Tor 设计者长期把 tagging 视为真实但优先级较低的问题 [DMS04]。不过 tagging 比统计关联更可靠，而且极少流量也足以实施。十多年间出现过多种解决方案：提案 202、261（基于 AEZ 密码）、295、308，最终由引入 CGO 的 359 接续。

10.3.3 其他限制

Digest 只有 4 字节，而且底层 SHA-1 已不再适合新的密码学设计。此外，传统方案不提供电路内部的前向保密：如果攻击者在电路仍活动时得到某一跳的当前密钥，就可以解密此前记录过的全部信元，因为密钥流完全由初始密钥决定。

10.4 Counter Galois Onion

CGO（*Counter Galois Onion*）是 Degabriele、Melloni、Münch 与 Stam 设计并由 提案 359 采用的 RELAY 信元加密方案。本章只介绍功能结构，完整密码学构造见提案及作者论文。

10.4.1 性质

根据提案，CGO 带来三项核心改进：

- 不可塑：任何形式的信元修改都会让当前信元以及电路后续所有信元无法正确解密。标记既不能被读取，也不能被清除，只能让电路失效。

- 信元级前向保密：处理完一条信元后，参与方不再保留再次解密它所需的旧材料。
- 认证强度达到 **128** 位，而不是依赖传统方案中很短的检查值。

CGO 也不再使用 SHA-1，因此整体性能预期更好。

10.4.2　高层工作原理

基础组件是 AES-128 和 POLYVAL；后者是一种通用哈希函数，与 AES-GCM-SIV 中使用的函数属于同一家族。提案用它们构造一种宽分组、可调整（*wide-block, tweakable*）密码：把整个 509 字节信元看成一个整体分组。任意位置即使只改变一位，都会让整个结果不可预测地变化。解密后的信元由 16 字节认证标签和 493 字节载荷构成，这一格式已经在 第 8.7 节 看到。

对于每一跳和每个方向，客户端与中继共享：

- 从电路握手派生出的 64 字节密钥 K 和 16 字节值 N；
- 上一条已处理信元的认证标签 T'，初始为零。

三项机制共同抵抗 tagging：

基于认证标签的识别　中继去掉自己的层之后，如果结果认证标签等于 N，说明信元属于它。这里用 16 字节检查，取代 Recognized 的两字节加四字节摘要。

链式依赖　前一条信元的认证标签会作为密码参数参与下一条信元。被修改的信元产生错误认证标签，随后每一条信元都将无法正确解密，电路无法继续运行。

密钥更新　每当某一跳识别或产生一条信元，就从原来的 K 和 N 派生新值，并擦除旧值，因此获得信元级前向保密。

信元的命令字节（`RELAY` 或 `RELAY_EARLY`）也以关联数据（*associated data*）参与计算；攻击者不能在两种类型之间转换而仍保持可解密性。

注意：两个方向不能随意互换

CGO 底层构造在一个方向上不可塑，而在另一个方向上是可塑的。提案精确规定客户端和中继分别应使用哪个方向，并明确警告实现者不能反过来。这说明一个看似只是实现细节的选择，也可能破坏整套密码学方案。

10.4.3　实际部署中的 CGO

- 密钥：握手输出中，每个方向得到 64 字节 K 和 16 字节 N，取代 Df、Db、Kf、Kb。
- **SENDME**：提案 289 的认证流量控制原本使用累积摘要来证明数据已被接收。CGO 不再有该摘要，因此 SENDME 改为携带最后一条被确认信元的认证标签（第 11 章）。
- 协商：只有中继声明子协议 `Relay=6`，而客户端又在 ntor-v3 握手中提出请求时，才使用 CGO（第 9.7 节）。第 9 章 中已经看到，如果没有同时请求拥塞控制，中继会拒绝 CGO。
- 部署：tor 从 0.4.9 系列起支持 CGO，Arti 一直启用。同一条电路可以混合使用 CGO 与仍使用 tor1 的跳，因为选择是逐跳独立完成的。

10.5　实验：三跳电路

实验 4：分层加密与电路扩展

目标。编写 `labs/zh-Hans/代码/torlab/encryption.py`，实现 tor1 客户端逻辑，并用它把真实电路一路扩展到出口。
前置条件。实验 1–3；正在运行的实验网络。
预计时间。3 小时。

步骤 **1**：某一跳的一个方向。 实验核心是一个小类，把一个方向上的 AES-CTR 密码状态与 SHA-1 摘要组合起来：

```
class Direction:
    @classmethod
    def new(cls, key, digest_seed):
        d = hashlib.sha1(); d.update(digest_seed)
        return cls(_aes_ctr(key), d)

    def apply(self, data):              # 加密/解密：CTR 使用同一操作
        return self.cipher.update(data)

    def sign(self, body):               # 写入 Digest 字段
        self.digest.update(_without_digest(body))
        return body[:5] + self.digest.digest()[:4] + body[9:]

    def recognizes(self, body):
        if body[1:3] != b"\x00\x00":
            return False
        trial = self.digest.copy()
        trial.update(_without_digest(body))
        if trial.digest()[:4] != body[5:9]:
            return False                # 误判：状态保持不变
        self.digest = trial
        return True
```

`cryptography` 提供的密码状态对象会在多次调用之间保留计数器状态，这正是协议需要的行为。

步骤 **2**：客户端。 对每一跳的两个方向各维护一个 `Direction` 后，加密和解密都变成循环：

```
def encrypt(self, msg, destination):
    body = self.hops[destination].forward.sign(encode_relay(msg))
    for i in range(destination, -1, -1):
        body = self.hops[i].forward.apply(body)
    return body

def decrypt(self, body):
    for i, hop in enumerate(self.hops):
        body = hop.backward.apply(body)
        if hop.backward.recognizes(body):
            return i, decode_relay(body)
    raise CellError("没有任何一跳识别该 cell")
```

步骤 **3**：内存测试及其局限。 `labs/zh-Hans/代码/torlab/test/test_encryption.py` 构造三跳模拟电路，验证每一跳只识别属于自己的信元、中间节点只能看到伪随机数据、修改会被最终目标检测，同时演示 AES-CTR 的可塑性。但这类测试有一个重要局限：模拟客户端和

中继使用的是同一份代码。如果摘要更新顺序写错，两端可能一起犯同样错误，测试仍然通过。最终验证必须与一个独立实现真正通信。

步骤 **4**：扩展真实电路。 先像实验 3 一样，通过 ntor-v3 向 `test003r` 建立第一跳，再扩展到 `test004r`，最后到出口 `test006e`。`labs/zh-Hans/代码/torlab/laboratory.py` 从实验网络文件读取中继身份和 ntor 密钥：

```
for r, extensions in ((middle, []), (exit_, [(1, b"")])):
    st, hd = ntor3_client_start(r.id_ed, r.ntor,
                                encode_extensions(extensions))
    body = extend2_body("127.0.0.1", r.port, r.id_rsa, r.id_ed,
                        HTYPE_NTOR_V3, hd)
    msg = RelayMessage(RelayCommand.EXTEND2, 0, body)
    c.send(Cell(CIRC, CellCommand.RELAY_EARLY, circ.crypto.encrypt(msg)))
    i, response = circ.crypto.decrypt(c.receive().body)
    sm, k = ntor3_client_finish(st, hdata_created2(response.data))
    circ.hops.append(Hop.from_keys(CircuitKeys.from_stream(k)))
```

`EXTEND2` 的 StreamID 为零，因为它属于控制消息，并且必须放进 **`RELAY_EARLY`**。和 tor 一样，对出口还会请求拥塞控制。运行结果如下：

```
第 1 跳：003r
  <- CircID=0x80000100 RELAY 消息体=509 字节
  来自第 1 跳 EXTENDED2, 67 字节
第 2 跳：004r  中继扩展：  []
  <- CircID=0x80000100 RELAY 消息体=509 字节
  来自第 2 跳 EXTENDED2, 70 字节
第 3 跳：006e  中继扩展：  [(2, b'\x1f')]
```

此时电路已经有三跳。每一行“来自第 n 跳”都证明：我们的代码针对正确中继完成了加密，而且该中继也能正确为我们返回数据。只要密钥、摘要中任意一字节错误，或者层次顺序错误，中继就会销毁电路。`EXTENDED2` 长度也与 第 9.7 节 一致：2 字节 HLEN、32 字节 Y、32 字节 $AUTH$，再加 SM；中间节点没有扩展，因此 SM 为 1 字节，出口返回一个单字节扩展，因此 SM 为 4 字节。

步骤 **5**：终于出现填充（**PADDING**）。 电路扩展完以后继续监听通道：

```
 4.1 s  <- CircID=0x00000000 PADDING 消息体=509 字节
 7.1 s  <- CircID=0x00000000 PADDING 消息体=509 字节
12.8 s  <- CircID=0x00000000 PADDING 消息体=509 字节
21.5 s  <- CircID=0x00000000 PADDING 消息体=509 字节
28.3 s  <- CircID=0x00000000 PADDING 消息体=509 字节
```

这正是第 7 章和第 9 章中所预期的：现在通道上真正承载了一条继续延伸到守卫之外的多跳电路，守卫会把通道视为“正在使用”，并在空闲时随机发送 **`PADDING`**。这里观察到的间隔在 3 到 8.7 秒之间，落入规范定义的 1.5–9.5 秒范围。

小结

- tor1 中，每一跳在每个方向都有一个 AES-128-CTR 密钥流状态和一份累积 SHA-1 摘要，两者贯穿电路生命周期。
- 前向方向上，客户端从最后目标跳开始向第一跳逐层加密，每个中继去掉一层；反向方向上，中继逐层增加加密，客户端全部去除。

- 客户端可以直接寻址电路中的任意一跳，即 *leaky pipe*。
- 当 Recognized 为零且累积摘要匹配时，信元才被识别；Recognized 的误判（false positive）绝不能改变摘要状态。
- AES-CTR 可塑，而且完整性只在目标端检查，因此传统方案允许 tagging。
- CGO 让信元不可塑，通过认证标签把前后信元链接起来，每条消息后更新密钥，并提供 128 位认证；它通过 ntor-v3 对每一跳独立协商。

练习

练习 **10.1** ⋆ 拥有三跳电路的客户端想向第二跳发送消息。它使用哪些密钥，以什么顺序加密？第三跳会看到什么？

练习 **10.2** ⋆ 客户端已经沿电路发送 1,000 个信元后，第一跳的 AES-CTR 密钥流已经消耗到哪里？为什么同一把密钥不能再用于第二条电路？

练习 **10.3** ⋆⋆ 如果中继在 Recognized 为零但摘要不匹配时仍然更新累积摘要，会发生什么？平均处理多少个信元后会遇到这种问题？

练习 **10.4** ⋆⋆ 一个串通的守卫希望给前向信元加标记，使串通出口能够识别。根据明文 `DATA` 消息的结构，修改哪些字节比较合适？如果出口是诚实的，它会看到什么结果？

练习 **10.5** ⋆⋆ 使用 CGO 重复上一题场景。串通出口会看到什么？电路后续信元会发生什么？

练习 **10.6** ⋆⋆ 计算实验步骤 5 中各个 **`PADDING`** 的时间间隔。按照规范，等待时间来自两个均匀随机变量最大值的分布。对 1.5 到 9.5 秒区间，它的期望值是多少？与观测是否一致？

练习 **10.7** ⋆⋆⋆ 扩展实验，向第二跳发送一条 `DROP`（RELAY 命令 10，端到端填充）。中继会回复吗？怎样验证它确实识别了消息且没有销毁电路？

练习 **10.8** ⋆⋆⋆ 在实验中故意翻转一条包含 `EXTEND2` 的加密 **`RELAY_EARLY`** 中某一位。哪个中继会发现，如何响应？先预测，再实际验证。

延伸阅读

- *Tor Protocol Specification* 中的 “Relay cells” 和 “Routing relay cells” [Thee]。
- 提案 359，*Counter Galois Onion, Updated*，以及其中引用的 Degabriele、Melloni、Münch、Stam 论文。
- 提案 202、261、295、308：十多年来针对 tagging 的不同防御路线。
- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04]，第 4 节：电路、leaky pipe 与完整性检查。

第 11 章

流、SENDME 与拥塞控制

最后核验：*2026* 年 *9* 月 *23* 日 — *torspec* 提交 `8e1f245`；*tor 0.4.9.13*；提案 *289*、*324*

本章目标

学完本章后，你将能够：

- 使用 BEGIN、CONNECTED、DATA 和 END 通过电路打开、使用和关闭 TCP 流；
- 解释为什么域名解析发生在出口，以及 RESOLVE 如何工作；
- 描述 Tor 原始的流量控制机制，包括电路窗口、流窗口和 SENDME；
- 理解为什么 SENDME 必须经过认证，以及它究竟证明了什么；
- 描述提案 324 引入的拥塞控制，以及 XON、XOFF 的作用；
- 通过自己构建的三跳电路下载文件，并测量真实出口的窗口行为（实验 5）。

第 10 章 中的电路已经是一条通向出口的加密通道，但还没有承载真正有用的应用连接。本章让它开始工作：把客户端的 TCP 连接通过电路送往互联网，这些连接在 Tor 中称为流。随后我们处理每个网络都必须面对、而在 Tor 中尤其棘手的问题：如何防止发送方比网络或接收方能够消化的速度更快地发送数据，从而把中继队列塞满。

11.1 流

流是客户端的一条 TCP 连接，例如到 Web 服务器的连接，通过某条电路承载。一条电路可以同时承载多个流；它们由 RELAY 消息中的 **StreamID** 区分（第 8.6 节）。StreamID 由客户端选择，而且必须非零。图 11.1 展示了完整生命周期。

11.1.1 打开流：BEGIN 与 CONNECTED

客户端用一个新的 StreamID 向出口发送 `BEGIN`（RELAY 命令 1），以打开流。消息载荷指定目标 [tor-spec § Opening streams and transmitting data]：

字段	长度	内容
ADDRPORT	可变	`address:port`，以零字节结尾
FLAGS	4（可选）	IPv4/IPv6 偏好

地址可以是 DNS 名称、IPv4 地址，或放在方括号中的 IPv6 地址。两个看似细小的规则体现了协议对不可区分性的重视：字符串必须以小写发送，避免大小写习惯暴露客户端实现；FLAGS 为零时必须完全省略。FLAGS 的第 1 位表示接受 IPv6，第 2 位表示拒绝 IPv4，第 3 位表示优先 IPv6。

收到 `BEGIN` 后，出口会：

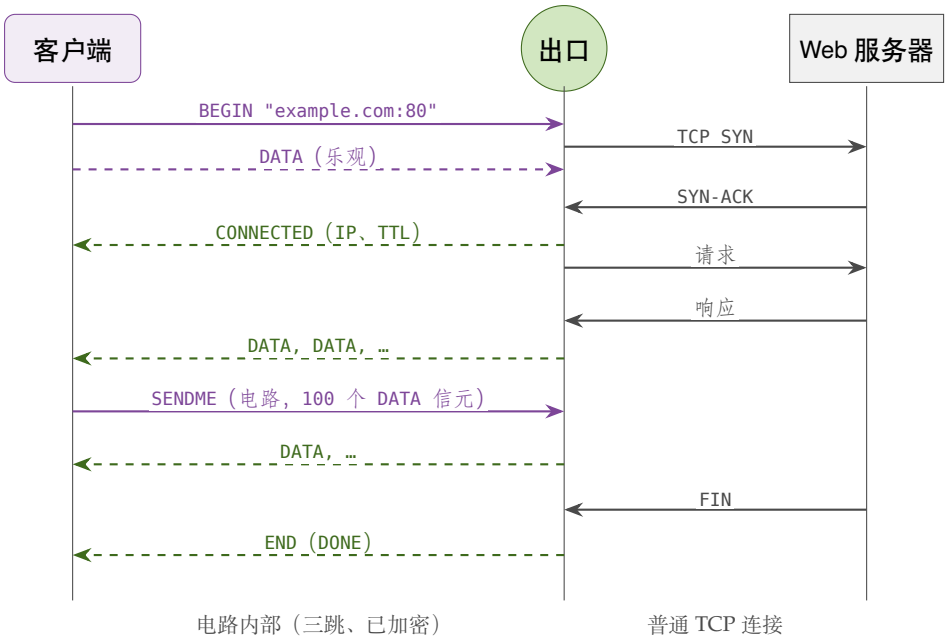


图 11.1: 到 Web 服务器的一条流的生命周期。客户端与出口之间的消息通过电路并受到分层加密；出口与服务器之间则是普通 TCP 连接，如果应用本身没有使用 TLS，这一段不会自动加密。客户端可以在收到 CONNECTED 之前先发送数据，这称为乐观数据（*optimistic data*）。

1. 检查自己不是这条电路的第一跳：电路到达出口的通道必须由共识中列出的、已认证中继打开。Tor 不能被当作单跳代理使用，否则出口的法律和滥用风险会更高；这样的尝试会用 **DESTROY** 拒绝；

2. 应用自身的出口策略（*exit policy*），也就是允许连接的地址与端口规则（第 22 章）；

3. 如有需要解析域名，然后建立 TCP 连接。

成功后，出口回复 `CONNECTED`（RELAY 命令 4）。载荷通常含到达的 IPv4 地址（4 字节）以及客户端可以缓存它的秒数 TTL（4 字节）。IPv6 格式略有不同，也允许空载荷。客户端可以忽略地址；如果缓存，则不得在其他电路复用，因为恶意出口可能故意返回一个唯一地址，以便后来重新识别该客户端。

11.1.2　传输数据：DATA

应用数据使用 `DATA`（RELAY 命令 2）传输；传统格式中每条最多承载 498 字节。出口把 DATA 的内容写入 TCP 连接，并把服务器返回的数据反向封装为 DATA。客户端不必等到 `CONNECTED` 才能开始发送：在 `BEGIN` 后可以立即发出 `DATA`，出口先把它们排队，连接成功后立即转发。这种乐观数据省掉一次完整电路往返延迟，而 Tor 电路的 RTT 很容易达到数百毫秒。

11.1.3　关闭流：END

主动关闭连接或遇到错误的一方发送 `END`（RELAY 命令 3），载荷以 1 字节原因码开头 [tor-spec § Closing streams]。常见原因包括 `DONE`（6，正常结束）、`CONNECTREFUSED`（3）、`RESOLVEFAILED`（2）、`TIMEOUT`（7）和 `EXITPOLICY`（4）；最后一种后面还会带被拒绝的地址与 TTL。对于由自己发起的流，客户端应只发送 `MISC`（1）：与 **DESTROY** 一样，没有必要向另一端暴露更多内部状态。

11.1.4 在出口解析 DNS

在 `BEGIN` 中，客户端可以直接提供域名而不是 IP 地址，由出口负责解析。这不仅是便利性设计，更是安全要求。如果客户端在本地解析，它就会在 Tor 之外向自己的 DNS 解析器查询即将访问的每一个站点；任何观察这些查询的人——首先就是网络接入提供商——都能知道它准备访问什么。这就是常见的 “DNS 泄漏（DNS leak）”，练习 19.3 会演示它，实验 18（第 28.3 节）则会从 tor 侧进行观察。

如果客户端只想查询地址而不建立连接，可以使用 `RESOLVE`（RELAY 命令 11），载荷是一个零终止的名称。出口以 `RESOLVED`（12）返回一个或多个 “类型、长度、值、TTL” 结构：`0x04` 表示 IPv4，`0x06` 表示 IPv6，`0x00` 表示名称（反向查询），`0xF0` 和 `0xF1` 分别表示临时和永久错误 [tor-spec §Remote hostname lookup]。tor 客户端能够通过本地 `DNSPort` 向应用程序提供 DNS 服务，其底层正是这一机制。

11.1.5 目录流：BEGIN_DIR

`BEGIN_DIR`（RELAY 命令 13）不会把流打到互联网，而是连接中继自身的目录服务。客户端可以借此下载共识与描述符（第 12 章），同时避免建立未加密的外部连接。它的载荷为空，而且出口策略不适用，因为流从未离开 tor 进程。

11.2 原始流量控制

11.2.1 问题

设想出口的互联网连接很快，而客户端很慢。如果出口不断从服务器读取数据并全部塞进电路，中间中继的队列会持续增长，消耗内存并拖慢其他电路。TCP 能在每一条独立链路上解决拥塞，但一条 Tor 电路是多条不同 TCP 连接的串联：中间中继与客户端之间出现的 TCP 反向压力（backpressure），不会自动传播到出口。因此需要一种端到端机制。

11.2.2 窗口与 SENDME

Tor 原始流量控制使用按 `DATA` 信元计数的窗口 [tor-spec §Flow control]：

- 电路级：发送方拥有初始为 1000 信元（或共识参数 `circwindow` 指定值）的发送窗口。每发送一条 DATA 就减一，降到零必须停止。接收方维护对称的交付窗口；每收到 100 条 DATA，就发送一条 StreamID 为零的 `SENDME`（RELAY 命令 5），让发送方窗口恢复 100。

- 流级：窗口初始为 500 信元，每收到一条带有该流 StreamID 的 `SENDME`，恢复 50。

因为客户端可以直接与电路中的任意跳通信（第 10.1 节），所以会为每一跳维护一对窗口。这些窗口只统计由该端产生或消费的 DATA，而不统计纯转发信元。

因此，在任意时刻每条电路最多有 1000 条 DATA 在途，每条流最多 500 条，约分别为 500 KB 和 250 KB。这样能够限制中继内存占用，却也限制吞吐量：若 RTT 为半秒，一条电路最多只能每半秒补满 1000 信元，也就是约 1 MB/s，无论物理带宽多高。反过来，在拥塞网络中，1000 条在途信元又可能太多，队列和延迟都会增长。

11.2.3 经过认证的 SENDME

恶意客户端可以伪造 `SENDME`，即使根本没有真正接收数据，也不断告诉出口 “可以继续发”。出口于是持续注入数据，这些数据在网络中积压，使客户端可以很低成本地耗尽中继内存或放大流量。从 0.4.1.1-alpha 开始，电路级 `SENDME` 因此需要认证（提案 289）。载荷包含版本号（1）、长度（20）以及触发该 `SENDME` 的最后一条 `DATA` 信元对应的累积摘要。SENDME 的接收者在发送那条 DATA 时已经计算过该摘要，因此可以比较；不匹配就关闭电路。

摘要取决于此前收到的所有信元内容，但如果客户端预先知道数据，例如下载一份已知文件，它仍可能预测摘要。正因如此，每 100 个信元中至少必须有一条包含不可预测的随机字节；RELAY 消息中的随机填充（第 8.6 节）也承担了这一作用。

深入说明：究竟是哪一个摘要？

规范写的是“位于 SENDME 之前的信元摘要”。在 tor 实现中，它具体是反向方向在最后一条被识别信元之后的完整 20 字节 SHA-1 运行摘要，由 `src/core/crypto/relay_crypto_tor1.c` 中的 `tor1_relay_digest_matches_v0` 保存到 `sendme_digest`。使用 CGO 时，则由最后一条信元的认证标签取代（第 10.4 节）。

11.3　拥塞控制

固定窗口有两个相反问题：快速电路上太小，拥塞电路上又太大。Mike Perry 的 提案 324 为 Tor 引入真正类似 TCP 的拥塞控制，从 tor 0.4.7.7（2022 年 4 月）开始部署。

11.3.1　信号：往返时间

出口怎样判断一条电路正在拥塞？答案是把 `SENDME` 当成时钟。出口记录发送那条将触发下一次 SENDME 的 `DATA` 的时间；SENDME 返回时，就得到电路的往返时间（RTT）。如果 RTT 相比观察过的最小值变大，说明沿途队列正在积压，发送方应减速。发送方据此调整拥塞窗口（*cwnd*），试图让它接近电路的带宽时延积，也就是在不产生额外排队时电路可以容纳的数据量。

11.3.2　算法

提案描述了三种受 TCP 启发的算法：Tor Westwood、Tor Vegas、Tor NOLA。经过 Shadow 仿真（第 19 章）以及真实网络评估后，tor 默认选择 **Vegas**，对应 `src/core/or/congestion_control_common.c` 中的 `CC_ALG_DFLT`。与 TCP Vegas 类似，它通过当前窗口和最低观测 RTT 所需窗口之间的差异，估计有多少信元正处于队列中，再增减 cwnd，把估计排队量维持在目标阈值附近。

11.3.3　参数：为什么是 31？

拥塞控制通过 ntor-v3 对每一跳分别协商（第 9.7 节）。客户端发送 `CC_FIELD_REQUEST`，中继回复 `CC_FIELD_RESPONSE`，其中包含 `sendme_inc`：每收到多少条 `DATA` 后要求一条 `SENDME`。第 9 章 中实验出口返回 31。这个值并非随意选择；tor 把它定义为一个最大 TLS 记录能容纳的信元数量：

$$\texttt{TLS_RECORD_MAX_CELLS} = \left\lfloor \frac{16384}{514} \right\rfloor = 31.$$

初始窗口为 $4 \times 31 = 124$ 信元。与旧方案每 100 信元一次相比，每 31 信元一次 SENDME 能更频繁地测量 RTT，也能更快对拥塞变化作出反应。

11.3.4　不再有流窗口：XON 与 XOFF

启用拥塞控制后，流级 `SENDME` 消失。旧的流窗口存在两难：窗口小，会限制单个下载速度；窗口大，恶意客户端又可以同时打开许多流，让出口为每条流缓冲一个完整窗口。提案用两条消息取而代之：

XOFF（RELAY 命令 43）：“暂停在这条流上发送”；当面向应用程序的队列超过阈值时发出；

XON（RELAY 命令 44）：“恢复发送”，并携带 4 字节字段说明该队列当前的排空速率，以便对端自适应。字段名为 `kbps_ewma`，但 提案 324 没有定义单位。tor 源码（`src/core/or/congestion_control_flow.c`）实际以每秒 **1000** 字节为单位计算和解释，而不是 1000 比特；变量名具有误导性，若按比特/秒解释就会产生八倍误差。

相比之下，电路级 `SENDME` 在信元被解密并识别后立即发送，发生在交给应用程序之前，因此测量的是网络而不是应用处理速度。

注意：拥塞控制与匿名性

任何会根据流量作出反应的机制，都可能成为侧信道。观察窗口行为的攻击者可能推断电路内部状态。提案 324 专门讨论这些风险和缓解措施，其中包括限制 XON/XOFF 的发送频率。这与 第 8 章 中统一让 **`DESTROY`** 使用原因码 NONE 属于同一种防御思路：任何依赖内部状态、又能被外部观察到的变化，都可能泄露信息。第 26 章 会讨论利用流量线索的攻击。

11.4 实验：通过 Tor 下载文件

实验 5：流与流量控制

目标。编写 `labs/zh-Hans/代码/torlab/circuit.py`，把前面实验组合进 `Circuit` 类；再编写 `labs/zh-Hans/代码/torlab/stream.py` 打开流并传输数据。通过三跳电路从本地 Web 服务器下载文件，并测量出口窗口。
前置条件。实验 1–4；正在运行的实验网络；本地 Web 服务器（`python3 -m http.server 8080`）。
预计时间。3 小时。

步骤 **1**：一行构建电路。 `build` 打开到守卫的通道、建立第一跳，再逐步扩展；和 tor 一样，只在最终跳请求拥塞控制：

```
circ = build([relay("003r"), relay("004r"), relay("006e")])
print(circ.exit_extensions)             # [(2, b'\x1f')]: sendme_inc = 31
```

步骤 **2**：打开流。

```
def begin_body(address, port, flag=0):
    body = f"{address.lower()}:{port}".encode() + b"\x00"
    return body + (struct.pack(">I", flag) if flag else b"")

s = Stream(circ, 1)
s.open("127.0.0.1", 8080)       # BEGIN，然后等待 CONNECTED 或 END
```

出口以 `CONNECTED` 回复 8 字节载荷：`7f000001 00000708`，即地址 127.0.0.1 与 TTL `0x708`=1800 秒。

步骤 **3**：**SENDME**。 电路统计从出口收到的每一条 `DATA`，每累计 `sendme_inc` 条就发送一条经过认证的 `SENDME`：

```
def count_received_data(self):
    est = dict(self.exit_extensions)
    sendme_inc = est[2][0] if 2 in est else 100
    self.data_to_ack += 1
```

```
    if self.data_to_ack == sendme_inc:
        self.data_to_ack = 0
        digest = self.crypto.hops[-1].backward.digest.digest()
        body = struct.pack(">BH", 1, 20) + digest
        self.send(RelayMessage(RelayCommand.SENDME, 0, body))
```

注意：一个很有教育意义的错误

实验最初版本把这个计数器放在流对象里。第一条流只返回两条信元，所以运行正常；第二条流下载 1 MB 文件时，却被出口用 **DESTROY** 中断。原因是电路级 SENDME 统计的是这条电路上所有流的 DATA。出口已经把第一条流的两条信元算进去，因此期待第二条流的第 29 条信元后收到 SENDME；我们却等到第 31 条才发，而且摘要已不是出口期待的那个值，于是出口严格按照 提案 289 关闭电路。把计数器移动到 `Circuit` 类后问题消失。

步骤 **4**：下载。 发送 HTTP 请求并一直读取到 END：

```
s.send(b"GET /grande.bin HTTP/1.0\r\nHost: 127.0.0.1\r\n\r\n")
response = s.receive_all()
```

对一个 1 048 576 字节的随机文件，我们的一次运行得到：

```
流 1: 209 字节, 0.05 s; DATA 信元 2; 截至目前 circuit SENDME 0;
        其他 ['END DONE']
流 2: 1048781 字节, 0.18 s; DATA 信元 2120; 截至目前 circuit SENDME 68;
        其他 ['END DONE']
```

接收文件的 SHA-256 与原文件一致。68 条 SENDME 正好等于 $\lfloor (2 + 2120)/31 \rfloor$：计数器确实属于整条电路。DATA 数量是 2120 而不是理论最小值 $\lceil 1048781/498 \rceil = 2106$，因为出口不会一定等到凑满 498 字节才发送，而是按从服务器实际读取到的数据块进行封装。

步骤 **5**：观察窗口。 接下来做一个普通客户端不会故意做的实验：下载同一个文件，但完全不发送 SENDME，观察出口发送多少 DATA 后停住；分别在启用和禁用拥塞控制时测试：

```
circ = build(path, cc=cc)
circ.count_received_data = lambda: None      # 不发送 SENDME
```

```
congestion control=True: 出口在 124 条 DATA 后停止
congestion control=False: 出口在 500 条 DATA 后停止
```

启用拥塞控制时，124 正是初始拥塞窗口 4×31。禁用时，出口在 500 而不是 1000 条停止，因为旧式流量控制下，流 window（500）先于 circuit window（1000）耗尽，客户端还必须发送流级 SENDME。两种情况下出口都只是等待，数据留在自身缓冲区中，而不是继续塞进 Tor 网络。

小结

- 流用 BEGIN 打开（address:port 必须小写），由 CONNECTED 确认，以 DATA 携带数据，以 END 关闭；客户端可以在 CONNECTED 前先发送乐观数据。
- 域名解析发生在出口，以避免向本地 DNS 解析器暴露目的地；RESOLVE/RESOLVED 可以在不打开连接的情况下解析名称。
- 原始流量控制使用每条电路 1000 信元、每条流 500 信元的窗口，并分别每 100/50 信元发送 SENDME。

- 电路级 `SENDME` 使用触发它的信元的累积摘要认证（提案 289）。
- 使用拥塞控制时（提案 324，默认 Vegas），窗口根据通过 SENDME 测得的 RTT 自适应；`sendme_inc`=31，即一个 TLS 记录可装下的信元数，初始窗口 124。
- XON 与 XOFF 取代流窗口。

练习

练习 **11.1** ⋆　为一个同时接受 IPv6 的客户端，写出连接 `Example.COM:443` 的 `BEGIN` 载荷的十六进制内容。

练习 **11.2** ⋆　解释 `CONNECTED` 载荷 `c0000201 0000012c`。客户端应该怎样处理这些信息？

练习 **11.3** ⋆　一条电路 RTT 为 600 ms，一个页面只需要一次 HTTP 请求。估算 optimistic data 能节省多少时间。若连接最终失败，为什么出口必须丢弃已经排队的乐观数据？

练习 **11.4** ⋆⋆　在原始流量控制下，一条 RTT 为 400 ms 的电路上，单条流的最大速度大约是多少？若同一电路有多条流呢？把这些数值与拥塞控制的初始窗口比较。

练习 **11.5** ⋆⋆　描述一种利用未认证 `SENDME` 耗尽出口内存的攻击。为什么 DATA 信元中的随机填充对认证有效性很重要？

练习 **11.6** ⋆⋆　为 `stream.py` 增加 `resolve(name)`：发送 `RESOLVE` 并解析 `RESOLVED`。在实验网络中用 `localhost` 测试。不存在的名称会让出口怎样回复？

练习 **11.7** ⋆⋆⋆　修改实验网络出口 `test006e` 的 `torrc` 出口策略并执行 `chutney hup`，然后尝试打开一个被拒绝端口。`END` 返回哪个原因码？解释后续附加字段。

练习 **11.8** ⋆⋆⋆　修改实验代码，在客户端测量发送每条 `SENDME` 到收到下一条 `DATA` 的时间。你实际测量的是什么？再在同一条电路并行进行第二个下载，比较结果。

延伸阅读

- *Tor Protocol Specification* 中的 “Opening streams and transmitting data”、“Closing streams”、“Remote hostname lookup” 与 “Flow control” [Thee]。
- 提案 289，*Authenticating sendme cells to mitigate bandwidth attacks*。
- 提案 324，*RTT-based Congestion Control for Tor*：设计动机、算法、参数与安全分析。

第 12 章

目录系统

最后核验：*2026 年 9 月 23 日* — *torspec* 提交 `8e1f245`；*tor 0.4.9.13*；共识方法 *36*

本章目标

学完本章后，你将能够：

- 解释 Tor 为什么需要目录系统，以及这一系统必须避免哪些风险；
- 区分描述符、微描述符、投票、共识和权威证书；
- 描述目录权威如何每小时投票并把各自结果合并成共识；
- 理解各种 flag 的分配标准；
- 逐行阅读一份共识并验证其签名；
- 使用自己的代码下载并验证实验网络的共识（实验 6）。

在前面的实验中，我们总是提前知道要连接哪些中继、它们使用哪些密钥、监听哪些端口，因为这些信息可以直接从实验网络文件读取。真实客户端没有这种便利。它必须发现网络中有哪些中继、它们位于哪里、使用哪些密钥，以及哪些中继可以信任。这正是目录系统的职责。

12.1 问题

Tor 的目录系统需要同时满足几项彼此存在张力的要求：

正确性 中继的地址、密钥等信息必须真实。若攻击者让客户端相信某个中继使用了另一把密钥，就可能截获流量。

共同视图 所有客户端必须看到同一份中继列表。如果攻击者能向每个客户端提供不同列表，例如只包含少量自己控制的中继，那么客户端就可能因为所选中继集合而被识别。这称为分区（*partitioning*）攻击，也称认知（*epistemic*）攻击。

抗滥用 任何人都可以向网络加入中继，因此系统必须限制一次加入大量中继的攻击者的影响，也就是 *Sybil* 攻击，并能够排除行为异常的中继。

可扩展性 数百万客户端每天多次下载目录信息，分发成本必须能够承受。

Tor 在三代目录协议演进后形成的解决方案，是由少量受信任服务器——目录权威——共同完成。它们每小时投票，联合产生并签署一份统一文档：共识（**consensus**）[dir-spec § Outline]。

历史说明：三代目录协议

第一版协议中，每个权威各自发布中继列表，因此单个恶意权威就可能欺骗客户端。第二版要求客户端下载所有权威的列表并自行合并，带宽成本成倍增加。第三版已经使用十五年以上，它把合并工作交给权威：权威共同生成一份文档，客户端只需验证签名。与此同时，权威也不再使用必须长期明文驻留内存的中继密钥来签署共识。

12.2 参与者

目录权威 由 Tor 社区中受信任成员运营的服务器。公共网络有九个，另有一个专门服务于网桥（第 16 章）。它们的列表直接编译进 tor（`src/app/config/auth_dirs.inc`），因此所有客户端从同一组信任锚开始。每个权威拥有离线保存的长期身份密钥，用于认证有效期 3–12 个月的签名密钥；后者用于给投票和共识签名。

目录缓存 从权威获取共识与描述符并重新分发给客户端的中继，用来降低权威负载。它们拥有 `V2Dir` 标志，并通过 `BEGIN_DIR` 流提供服务（第 11 章）。

备用目录（**Fallback directory**）大约两百个稳定中继的列表，同样编译进 tor（tor 0.4.9.13 的 `fallback_dirs.inc` 含 200 个条目）。新安装的客户端优先从这些中继之一获取第一份共识，而不是让所有新客户端直接压向目录权威。

中继 周期性把自己的描述符发布给权威。

客户端 下载并验证共识与微描述符，然后用它们选择路径（第 13 章）。

图 12.1 展示这些文档如何流动。

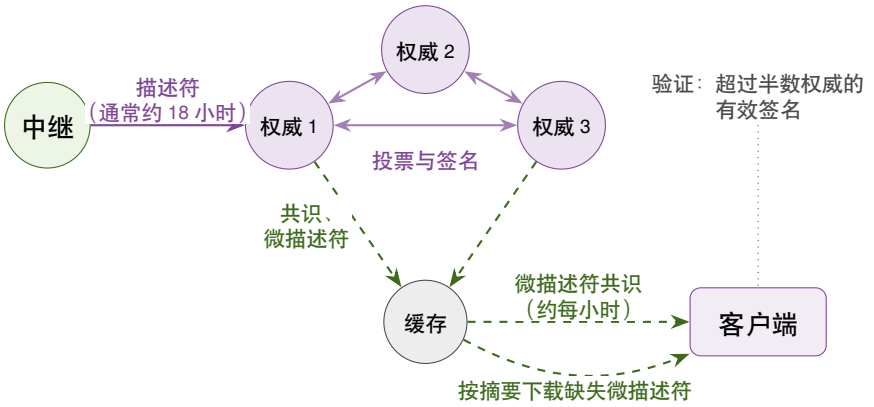


图 12.1: 目录文档的流动。中继向目录权威发布描述符；权威交换投票，依据收到的投票计算共识，再交换签名；目录缓存下载共识和微描述符，并向客户端提供这些文档。

12.3 文档类型

所有目录文档都采用规范称为 *netdoc* 的文本格式：由若干行条目（*item*）组成，每行以关键词和参数开始，某些条目后还会跟随 Base64 编码对象，包在 `-----BEGIN ...-----` 与 `-----END ...-----` 之间。这种格式既容易人工阅读，也容易程序解析，实验中会直接处理它。

中继描述符 （*server descriptor*）由中继编写并签名，包含地址和端口、身份与 ntor onion key、Ed25519 证书、声明带宽、完整出口策略、中继家族、平台和版本等。至少每 18 小时重新发布一次（tor 源码中的 `FORCE_REGENERATE_DESCRIPTOR_INTERVAL`），重要属性改变时会更早更新。

Extra-info 文档 保存协议运行本身不需要的统计数据，例如按国家划分的网桥流量；单独发布是为了避免让描述符膨胀。这些数据会进入 Tor Metrics（第 24 章）。

微描述符 客户端实际需要、且变化不频繁的描述符子集：ntor onion 密钥、中继家族、出口策略摘要和 Ed25519 身份。由权威计算，每份微描述符使用其文本的 SHA-256 digest 作为标识。

投票 某个权威对网络的评估：每个中继的标志、带宽与版本。

共识 多份投票合并后的结果，由权威签名。它有两种版本类型（*flavor*）：`ns` 为每个中继列出完整描述符摘要，`microdesc` 则列出微描述符摘要。现代客户端只使用更紧凑的后者。

权威密钥证书 把权威的签名密钥绑定到其长期身份。

深入说明：为什么使用微描述符？

完整描述符往往有数 KB，而且例如声明带宽变化时就会更新。如果数百万客户端每天多次下载数千个中继的完整描述符，尤其对移动网络成本很高。微描述符只保留很少变化的信息；带宽与标志等频繁变化的信息放在客户端本来就要下载的共识里。客户端按摘要请求微描述符，而不是按中继身份请求，因此目录缓存也不能轻易单独向某个客户端提供一份和其他人不同的微描述符。

12.4 投票

12.4.1 投票时间表

每份共识包含三个时间戳 [dir-spec § Voting timeline]：

`valid-after` 从何时开始有效；

`fresh-until` 到何时不再是最新鲜的共识；通常等于下一份共识的 `valid-after`；

`valid-until` 超过该时间后客户端不能继续使用。

公共网络每小时投票一次，共识持续三个时段有效，对应 `V3AuthVotingInterval` 和 `V3AuthNIntervalsValid`。因此任意时刻通常会有多份仍在有效期内的共识，单次权威延迟不会立即阻断客户端。每轮开始前，权威有五分钟交换投票，再有五分钟交换签名（`V3AuthVoteDelay` 与 `V3AuthDistDelay`）。实验网络为了教学大幅加速：每 20 秒投票一次，两个阶段各 4 秒（第 19.5 节）。

12.4.2 合并投票

每个权威收集其他权威的投票，并使用确定性算法计算共识，因此所有权威应独立得到同一份文档并能够给它签名 [dir-spec § Computing a consensus from a set of votes]。主要规则包括：

- 中继只有被足够多权威认识时才会进入共识；在当前共识方法（24 及以后；实验网络为 36）中，它还必须同时具有 `Running` 和 `Valid`；
- 某项标志若获得参与该标志投票的权威中过半数支持，就赋给中继；
- 带宽 `w` 通常取投票值的较低中位数；如果至少三个权威对其带宽进行了测量，则优先采用测量值的中位数；
- 网络参数 `params` 只有在全部权威中过半数，或者至少三个权威支持时才进入共识；源码条件是 `i > total_authorities/2 || i >= 3`，最终值取较低中位数；

- 时间戳取提议值的中位数。

因为不同权威可能运行不同 tor 版本，文档还会声明正在使用的共识方法（*consensus method*）：一个精确指定算法规则集合的编号，由足够多权威共同支持的最高方法决定。实验网络使用方法 36。

12.4.3　中继标志（Flags）

标志概括权威对中继的评估 [dir-spec § Assigning flags in a vote]。规范明确提醒：具体标准可能随时间变化，客户端不能依赖这些标准永远不变。表 12.1 总结当前规则。

标志	标准（摘要）
`Running`	权威在过去 45 分钟内成功连接该中继全部 ORPort
`Valid`	tor 版本未被认定有缺陷，且中继未被权威排除
`Fast`	带宽位于最快的八分之七，或至少 100 KB/s
`Stable`	按时间加权的平均故障间隔至少达到中位数，或至少 7 天
`Guard`	同时满足 `Fast`、`Stable`、长期稳定（*familiar*）（已在网络存在数周）、可用性与带宽要求，并具有 `V2Dir`
`Exit`	对端口 80 和 443 都允许至少一个 /8 地址块的出口流量
`HSDir`	`Stable` 且 `Fast`，连续运行至少 96 小时，可保存洋葱服务描述符
`V2Dir`	提供目录信息，即目录缓存
`Authority`	是目录权威
`BadExit`	不应作为出口使用的中继，例如会修改流量
`MiddleOnly`	只应作为中间中继使用（0.4.7.2-alpha 引入）
`StaleDesc`	描述符发布时间已超过 18 小时
`Sybil`	超过单个 IP 地址允许的最大中继数量，通常为 2

表 12.1: 共识标志以及权威分配这些标志的标准摘要。

`BadExit` 和 `MiddleOnly` 允许权威限制可疑中继，而不必把它们彻底从网络删除。`Sybil` 则限制单个地址运行大量中继的影响。在实验网络中，所有中继都位于 127.0.0.1，因此通过 `AuthDirMaxServersPerAddr=0` 禁用该限制；实验会在 `params` 行看到这一点。

12.4.4　带宽

中继到底有多快？描述符里自报的值并不可信，恶意中继可以声称拥有极高带宽以提高被选中的概率。因此部分权威作为带宽测量权威（*bandwidth authorities*），使用专门工具——目前是 *sbws*（第 24 章）——通过中继传输流量以测量带宽，并在投票中以 `Measured` 提交。如果至少存在三份测量，共识优先取它们的中位数；否则 `w` 行带有 `Unmeasured=1`，并使用受限值。共识中的 bandwidth 并不是严格意义上的速度，而是用于路径选择概率的权重（第 13 章）。

12.5　一份共识的结构

下面是实验 6 下载的一份 microdesc 共识节选。仓库保存了副本 `labs/zh-Hans/代码/torlab/test/data/consensus-microdesc.txt`，实验测试也使用它，因此无需启动网络就可以逐行检查。副本采集于 2026 年 9 月 23 日，当时实验中继仍运行 tor 0.4.9.12；这就是条目 `v` 行显示的版本，并不等于本书当前实验网络的参考版本。

头部。

```
network-status-version 3 microdesc
vote-status consensus
```

```
consensus-method 36
valid-after 2026-09-23 15:37:20
fresh-until 2026-09-23 15:37:40
valid-until 2026-09-23 15:38:00
voting-delay 4 4
known-flags Authority Exit Fast Guard HSDir ... Running Stable ... Valid
params AuthDirMaxServersPerAddr=0 cc_alg=2
```

`params` 行是共识参数：权威可以通过它调整整张网络行为，而不要求软件更新。这里除了单地址限制外，还指定拥塞控制算法 `cc_alg=2`，即 Vegas（第 11.3 节）。公共网络包含数十个参数。

一个中继条目。

```
r test006e ml1aPg9UFy4VCx85YcpsRs/9shc 2038-01-01 00:00:00 127.0.0.1 5106 0
m FfrgO+s4VYS6WKBq3pjfMDs+uZ2uVsvgG88JilbhIOo
s Exit Fast Guard HSDir Running Stable V2Dir Valid
v Tor 0.4.9.12
pr Conflux=1 Cons=1-2 Desc=1-4 ... Link=3-5 ... Relay=2-6
w Bandwidth=4096
```

`r` 中继昵称、RSA 身份摘要（Base64）、一个如今在共识中已无实际意义的日期字段、IPv4 地址、ORPort 和 DirPort；

`m` 需要下载的微描述符摘要；

`s` 标志；

`v`、**`pr`** tor 版本与支持的子协议；客户端例如从这里知道该中继支持 CGO（`Relay=6`）或链路协议 3–5；

`w` 以 KB/s 表示的带宽权重。

在 `ns` 版本中，每个条目还会有一行 `p`，按端口概括出口策略；microdesc 版本则把这一摘要放在微描述符中，因此共识不再重复。

尾部与签名。

```
directory-footer
bandwidth-weights Wbd=821 Wbe=0 Wbg=5000 Wbm=10000 Wdb=10000 ...
directory-signature sha256 3776EF73A9214587AEFB3D732FA4A443CFF491BA
                           79E102D226590BC5D1D1EBBAD8A16122EC0F9801
-----BEGIN SIGNATURE-----
...
-----END SIGNATURE-----
```

`bandwidth-weights` 是客户端平衡不同电路位置中继使用量的系数（第 13 章）。之后每个权威各有一份签名，包含哈希算法、权威身份摘要、签名密钥摘要与签名本身。

12.6 验证共识

客户端只有在一份共识包含超过半数已知权威的有效签名时才接受它。tor 源码阈值是 `n_v3_authorities/2 + 1`（`src/feature/nodelist/networkstatus.c`）：公共网络 9 个权威需要 5 个签名，实验网络 3 个权威需要 2 个。验证签名需要：

1. 权威证书：客户端下载 `/tor/keys/all`，并依据预先知道的权威身份验证；
2. 被签文本：从文档开头一直到第一处 `directory-signature` 关键词后面的那个空格，并包含该空格；
3. 签名方案：RSA + PKCS#1 v1.5 填充，直接对摘要（SHA-1 或 SHA-256）签名，不包含普通 PKCS#1 签名通常期待的算法标识符 [dir-spec § Signing documents]。

深入说明：到空格，还是到换行？

规范关于已签名文档的通用章节说签名覆盖签名行之后的换行；但共识专门章节规定覆盖到 `directory-signature` 后的空格。后者保证各权威即使拥有不同的签名行，也对完全相同的字节签名。实验 6 同时用两种解释验证真实实验网络共识：按空格规则，三份签名全部有效；按换行规则，一份也无效。遇到规范表述存在歧义时，对独立实现进行实验验证仍然是最终裁决。

注意：权威列表是信任锚

整个系统建立在编译进软件的权威列表之上。用户技术上可以更改它，但规范不建议这样做：使用不同权威的客户端会看到与其他人不同的共识，因此本身变得可识别。权威同样是高价值攻击目标。如果多数权威被攻陷，它们就可能共同签署包含恶意中继的共识。正因如此，权威由不同国家、不同人员运营，而且长期身份密钥离线保存。

12.7　客户端如何保持最新

客户端不会在每个整点同时下载新共识，否则目录缓存会承受同步洪峰。它会在共识不再 fresh 之后第一个 interval 的四分之三处，与距离过期剩余时间的八分之七处之间随机选择更新时间 [dir-spec § Client operation]。规范给出的例子是：共识 1:00 开始有效，2:00 不再 fresh，4:00 过期，那么客户端会在 2:45 到 3:50 之间随机寻找新共识。

为了节省带宽，客户端可以只请求相对于当前共识的差异，即共识差异（*consensus diff*），并用当前共识的 SHA3-256 摘要标识基线。随后只下载缺失微描述符，按摘要请求：`/tor/micro/d/` 后接以 `-` 分隔的摘要。

12.8　实验：下载并验证共识

实验 6：实验网络的共识

目标。编写 `labs/zh-Hans/代码/torlab/directory.py`，通过电路从权威下载共识、证书和微描述符，解析并验证签名。
前置条件。实验 1–5；正在运行的实验网络。
预计时间。3 小时。

步骤 **1**：使用 **BEGIN_DIR** 下载。 与 tor 一样，使用到目录节点的单跳电路，不启用拥塞控制，并打开 `BEGIN_DIR` 流；在其中发送 HTTP/1.0：

```
def download(circ, path, stream_id):
    s = Stream(circ, stream_id)
    circ.send(RelayMessage(RelayCommand.BEGIN_DIR, stream_id, b""))
    _, msg = s._next()                          # CONNECTED, 空 body
    s.send(f"GET {path} HTTP/1.0\r\n\r\n".encode())
    header, _, body = s.receive_all().partition(b"\r\n\r\n")
```

```
    return body

circ = build([relay("000a")], cc=False)
text = download(circ, "/tor/status-vote/current/consensus-microdesc", 1)
```

步骤 **2**：**netdoc** 格式。大约二十行的 `items` 函数把文档拆成条目，并收集 `-----BEGIN/-----END` 对象。在它之上构建两个解析器：`parse_consensus` 为每个中继产生含 flag、带宽、策略和微描述符摘要的条目；`parse_certificates` 解析权威证书。

步骤 **3**：验证签名。

```
def signed_text(text):
    start = text.index("network-status-version ")
    pos = text.index("\ndirectory-signature ") + len("\ndirectory-signature ")
    return text[start:pos].encode()

def verify_signature(consensus, signature, cert):
    h = hashlib.sha256 if signature.algorithm == "sha256" else hashlib.sha1
    expected = h(signed_text(consensus.text)).digest()
    recovered = cert.signing_key.recover_data_from_signature(
        signature.signature, padding.PKCS1v15(), None)
    return recovered == expected
```

`recover_data_from_signature` 使用 algorithm=None 时会直接返回签名中携带的摘要，而不会要求算法标识符，正好符合 Tor 的格式。

步骤 **4**：结果。在实验网络上的一次运行中（日期、名称和数量取决于网络实例）：

```
consensus: 6035 字节; 2026-09-23 15:37:20 -> 2026-09-23 15:38:00
method 36; signatures 3; certificates 3
params: {'AuthDirMaxServersPerAddr': 0, 'cc_alg': 2}
  test006e 127.0.0.1:5106 w=4096 flags=['Exit', 'Fast', 'Guard', ...]
  test000a 127.0.0.1:5100 w=20   flags=['Authority', 'Fast', 'Guard', ...]
  ...
有效签名: 3 (要求: 2)
microdescriptors: 8; 与 consensus m 行匹配: True
```

这份共识覆盖 40 秒，也就是两个 20 秒时段，由三个权威全部签名；每份下载的微描述符 SHA-256 都与共识对应 `m` 行一致。信任链从权威密钥一路延伸到每个中继的 ntor key。

步骤 **5**：负向控制。永远不会失败的验证器等于什么都没有验证。把共识中的 `Bandwidth=4096` 改成 `4097`，有效签名数立刻降到零；如果只知道一个权威，则只能验证出一个签名。`test/test_directory.py` 使用保存下来的共识副本重复这些检查，因此无需启动网络也能运行。

步骤 **6**：真实共识。同样的代码也适用于公共网络。从 CollecTor 下载同一时期的共识与匹配的权威证书，然后验证：

```
c = parse_consensus(open("consensus-microdesc-2026-09-23-15").read())
cert = parse_certificates(open("certs").read())
valid, required, _ = verify_consensus(c, cert, 9)
print(valid, required)          # 9 5
```

示例中九份签名全部有效，高于所需的五份。第一次尝试时有一个容易误判为密码学错误的格式陷阱：CollecTor 文件开头有一行 `@type network-status-microdesc-consensus-3 1.0`，它不属于被签文档。如果把这一行也计入摘要，所有签名都会失败。因此 `signed_text` 必须从 `network-status-version` 开始，测试也应覆盖这一行为。

小结

- 目录系统必须让所有客户端看到相同且真实的网络视图，同时抵抗 Sybil 攻击，并能扩展到数百万用户。
- 九个目录权威（另加一个网桥权威）被写入代码，每小时投票并签署一份统一共识；公共网络共识有效三小时。
- 中继发布描述符；现代客户端使用 microdesc 共识和按摘要下载的微描述符。
- 某项标志只有获得参与该项投票的过半数权威支持才会赋予；至少三个权威的测量带宽会优先于中继自报值。
- 客户端只接受超过半数已知权威有效签名的共识；签名文本覆盖到 `directory-signature` 后的空格。
- 共识参数允许权威在不发布软件更新的情况下调整整张网络的行为。

练习

练习 **12.1** ⋆ 一份共识 `valid-after`=10:00、`fresh-until`=11:00、`valid-until`=13:00。客户端会在哪个时间区间寻找替代共识？如果 13:00 仍没有找到新共识会怎样？

练习 **12.2** ⋆ 某中继带宽 500 KB/s，已在线两天，出口策略为 `accept *:443, reject *:*`。它可能获得哪些标志？为什么不能获得 `Exit`？

练习 **12.3** ⋆⋆ 描述一种分区攻击：恶意目录缓存只向某个特定客户端发送一份较旧但仍有效的共识。什么因素限制这种攻击？为什么按摘要请求微描述符有所帮助？

练习 **12.4** ⋆⋆ 三个权威测得某中继带宽分别为 5000、5200、90000。共识采用哪个值？如果第三个与中继串通的权威是唯一进行测量的权威，又会怎样？

练习 **12.5** ⋆⋆ 在实验网络某个权威目录中找到 `v3-status-votes`，把该权威的投票与最终共识比较：哪些行不同？为什么？

练习 **12.6** ⋆⋆ 从实验下载的共识中选一个中继，使用 `sha256sum` 手工计算其微描述符摘要，并与 `m` 行比较。被哈希文本从哪一行开始，到哪里结束？

练习 **12.7** ⋆⋆⋆ 停止实验网络三个权威中的一个，一分钟后重新下载共识。它包含几个签名？验证器是否接受？再停止第二个权威，解释共识过期后观察到的行为。

练习 **12.8** ⋆⋆⋆ 同时下载 `ns` 版本（`/tor/status-vote/current/consensus`）并验证签名。它使用哪些哈希算法？与 microdesc 版本相比，`r` 行多了哪些信息？

延伸阅读

- *Tor directory protocol, version 3*（dir-spec）[Thee]：格式、投票、中继标志分配以及客户端/缓存运行方式。

- 共享随机性协议文档（shared-randomness protocol，srv-spec）：目录权威每天生成共同随机值的协议；第 14 章 的洋葱服务需要它。
- Tor Metrics 与 CollecTor，`https://metrics.torproject.org/`：共识、投票和描述符的历史归档。

第 13 章

路径选择、入口守卫与 Vanguards

最后核验：*2026* 年 *9* 月 *23* 日 — *torspec* 提交 `8e1f245`；*tor 0.4.9.13*；*2026* 年 *9* 月 *23* 日 *15:00 UTC* 的公共共识

本章目标

学完本章后，你将能够：

- 描述 tor 为电路选择中继的顺序与约束；
- 解释共识带宽和 *bandwidth-weights* 如何平衡不同电路位置的负载；
- 解释为什么使用 guard，并梳理它从三个变成一个、再到两个的历史；
- 描述入口守卫算法中的已采样、已过滤、已确认和主要守卫节点；
- 解释 vanguard 是什么，以及为什么洋葱服务需要它；
- 使用真实公共网络共识模拟路径选择（实验 7）。

目录系统（第 12 章）给客户端一份中继列表。剩下的问题几乎比任何其他问题都更直接决定用户能受到多好保护：到底应该使用哪些中继？如果对手同时控制一条电路的第一跳和最后一跳，它就可以关联进入和离开的流量，判断谁在和谁通信（第 26 章）。因此，路径选择很大程度上就是：在不过度牺牲性能的前提下，让这种事件尽可能少发生。

13.1 顺序与约束

对于三跳电路，tor 先选择出口，然后选择 guard，最后选择 middle [path-spec § Path selection and constraints]。之所以先选出口，是因为它受到的限制最多：出口策略必须允许目标目的地。如果客户端只知道目标域名而不知道 IP 地址，它会选择一个可能接受该目标的出口，也就是在接受所请求端口之前，没有先拒绝它的出口。

除少数例外，所有路径都满足：

- 不使用没有 `Fast` 标志的中继；
- 同一个中继不能出现两次；
- 每个 IPv4 /16 网络（或 IPv6 /32 网络）最多一个中继，因为同一数据中心中的两个中继可能由同一运营者控制；
- 每个中继家族最多一个中继：运营多个中继的人应把它们声明为同一家族，tor 会避免把两个成员放进同一条电路；
- 第一跳必须是 guard（第 13.3 节）；
- 不能选择带有 `BadExit` 标志的出口。

预计会保持较长时间的连接——tor 会把某些端口归类为这类连接，例如 SSH 的 22 和 IRC 的 6667——还要求使用 `Stable` 中继。洋葱服务另有独立于端口的要求：服务端的引介电路和汇合路径会按 `Stable` 电路构建，端点还不能是 `MiddleOnly` [path-spec § Path selection and constraints]。所谓“少数例外”主要出现在使用 vanguard 的洋葱服务电路中（第 13.4 节）：它们有意放宽部分限制，例如允许拥有 `Guard` 标志的中继出现在 middle 位置。若禁止这些组合，反而会告诉对手哪些中继没有被选中，从而泄露 guard 选择信息。

深入说明：两种声明中继家族的方式

传统做法是在每个中继描述符中列出同一运营者的其他全部中继；只有两个中继彼此都列出对方时，它们才属于同一家族。中继很多时列表会非常大，而且一次配置失误就可能破坏家族关系。较新的 *happy families* 机制为一个中继家族分配一把密钥，每个中继用该密钥签名的证书证明成员身份：描述符中的 `family-cert` 与微描述符中的 `family-ids`。共识参数 `use-family-lists` 和 `use-family-ids` 决定客户端使用哪些机制。

13.2　按带宽加权

13.2.1　为什么需要加权

如果客户端均匀随机选择中继，那么 1 MB/s 的中继与 1 GB/s 的中继获得流量的概率完全相同：前者会过载，后者则利用不足。因此 Tor 让每个中继的选择概率与共识中的带宽（`w` 行，第 12.5 节）成比例，以便按容量分配负载。

13.2.2　不同位置并不等价

这里还有一个问题。拥有 `Exit` flag 的中继相对稀缺，而真正能把流量送出 Tor 网络的带宽更是稀缺资源。如果把出口拿去当 guard 或 middle，就会消耗原本应该留给出口流量的容量。guard 也存在类似但较弱的问题。因此权威会为每份共识计算 **bandwidth-weights**：根据中继的类型（纯 guard、纯 exit、guard+exit、两者都不是）和目标位置，对中继带宽乘上不同系数 [path-spec § Weighting node selection]。

每个权重写作 `W`*pt*：*p* 表示位置（`g`=guard、`m`=middle、`e`=exit、`b`=目录请求），*t* 表示类型（`g`=guard、`e`=exit、`d`=同时为 guard 和 exit、`m`=两者都不是）。数值按万分比表示：`Wmg=3805` 表示一个 guard 被选作 middle 时，只计算其 38.05 % 带宽。

13.2.3　真实网络中的权重

下面这些值来自实验 7 从 CollecTor 下载的 2026 年 9 月 23 日 15:00 UTC 公共共识：

位置 \ 类型	guard (`g`)	middle (`m`)	exit (`e`)	guard+exit (`d`)
guard (`Wg`)	6195	6195	—	0
middle (`Wm`)	3805	10000	0	0
exit (`We`)	10000	10000	10000	10000

结果很直观：exit 以及同时具有 Guard 和 Exit flag 的中继，在 guard 和 middle 位置权重都是零。这张网络的出口带宽只占总带宽 33 %，非常宝贵，因此全部保留给出口位置。guard 带宽则占 70 %，更充足：它们作为 guard 时按 62 % 加权，作为 middle 时按 38 % 加权。实验会看到，约 60 % 电路的 middle 本身带有 `Guard` 标志。

权威计算这些权重的公式定义在目录规范中。它们会根据哪类带宽最稀缺采用不同情况，并尽可能让三个位置可用容量保持平衡。

13.3 入口守卫（Guard）

13.3.1 它们解决什么问题

假设对手控制了可作为第一跳和出口的带宽比例 c。如果客户端每条电路都重新随机选择 guard 和 exit，那么单条电路两端都落入对手手中的概率大约是 c^2。构建 C 条电路后，至少一次被攻陷的概率为 $1-(1-c^2)^C$，并且随着 C 增大趋近于 1 [guard-spec § Introduction and motivation]。每天使用 Tor 的用户会构建数千条电路；如果第一跳不断重新随机选择，迟早几乎必然有流量落入对手控制的两端。

Guard（研究文献早期称 *helper node*）的思想是：第一跳只选择一次，并在很长一段时间内重复使用。如果该 guard 是诚实的，那么这一类攻击下的所有电路都安全；如果它属于对手，则很多电路都会暴露。至少一次被攻陷的概率由接近 1 降到约 c。

电路数	不使用固定 guard	使用固定 guard
1	0.0025	0.0025
100	0.221	0.050
1000	0.918	0.050
10 000	1.000	0.050

实验 7 对 $c = 5\,\%$ 的计算展示了这种取舍。guard 并不会让攻陷变得不可能，而是把一个长期几乎必然发生在所有人身上的事件，变成只影响少量用户的低概率事件。这个模型刻意简化，例如没有考虑 guard 随时间更换的情况。

13.3.2 三个、一个、两个

Guard 于 2006 年引入，背景是 Øverlier 与 Syverson 展示了如何反复诱导洋葱服务连接攻击者控制的中继，从而定位服务 [ØS06]。多年间，客户端使用三个 guard，并每两三个月更换。之后研究发现，三个 guard 和频繁轮换都会增加暴露面。提案 236（2014）建议只使用一个 guard，并保留九到十个月。但单一 guard 会让连通性更脆弱，也更容易被攻击者强迫替换。提案 291（2018）改为两个 guard。tor 内建默认值其实仍是一个（`src/feature/client/entrynodes.h` 中的 `DFLT_N_PRIMARY_GUARDS_TO_USE`），但公共共识携带 `guard-n-primary-guards-to-use=2`，从而覆盖所有客户端默认值。这再次说明权威如何通过共识调节网络行为。

13.3.3 算法

当前算法由 提案 271 和 guard 规范定义，它维护多个集合 [guard-spec § The algorithm]：

已采样守卫（**sampled guards**）按带宽加权随机选出并持久化到磁盘的一组 guard。只有需要时才扩展，最多 60 个或共识中 guard 总数的 20 %。限制样本大小可以防止攻击者让客户端一个接一个尝试整张网络的所有 guard，直到最终碰到攻击者控制的中继。

已过滤守卫（**filtered guards**）样本中仍存在于共识、并且在当前配置下可用的 guard，例如防火墙只允许某些端口时仍能连接的中继。

已确认守卫（**confirmed guards**）真正用于用户流量的 guard，按偏好顺序排列。

主要守卫（**primary guards**）默认从 filtered 集合选择三个守卫，先按优先级选取同时属于 confirmed 的守卫，不足时再加入其他已过滤但尚未确认的守卫。客户端优先使用可达的主要守卫（或根据前述共识参数使用前两个）；只有它们不可达时才尝试其他守卫。

一个 guard 在 sample 中保留多久取决于其状态。从未确认过的守卫节点（`CONFIRMED_ON_DATE` 未设置）在被采样 `GUARD_LIFETIME=120` 天后移除；真正被客户

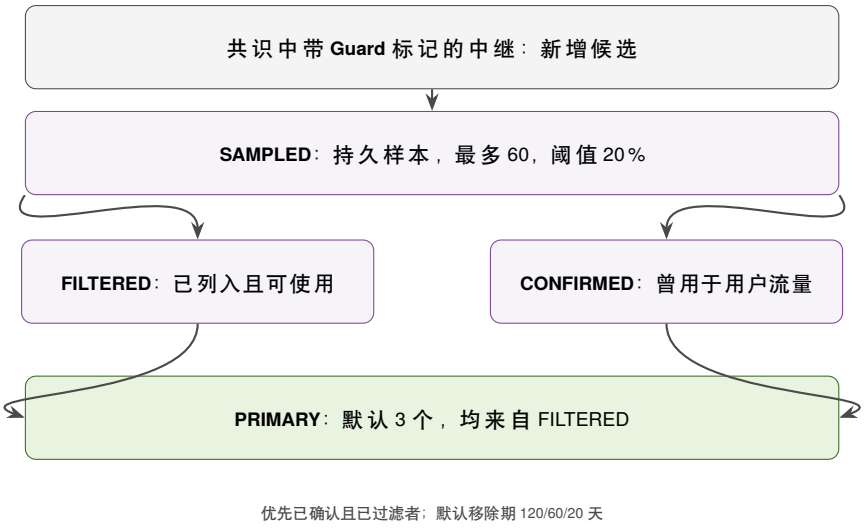


图 13.1: Tor 守卫算法的列表关系：持久样本来自共识中的守卫，而 FILTERED 与 CONFIRMED 是样本中不同且可能重叠的两个子集。PRIMARY 必须取自 FILTERED，优先已确认的守卫，再补充其他已过滤守卫。限制样本大小可以减少接触新守卫的机会。

端使用过的已确认守卫节点则在确认后 `GUARD_CONFIRMED_MIN_LIFETIME=60` 天移除。如果连续 20 天不在共识中，也会被删除。保存到状态文件的时间戳（`ADDED_ON_DATE` 和 `CONFIRMED_ON_DATE`）还会随机向过去偏移最多 `GUARD_LIFETIME/10=12` 天，这样状态文件不会准确暴露客户端从何时开始使用 Tor，也使已确认守卫节点的实际寿命约为 48–60 天。因此，大约每两个月轮换的是已确认的守卫节点，而不是每四个月。这一点对 第 29 章 的模拟很重要。

注意：更换 Guard 本身就是线索

Tor 很谨慎地区分 guard 无法连接与普通电路失败。如果客户端每当电路失败就换 guard，攻击者就可以故意让诚实 guard 上的电路失败，直到客户端最终选择恶意 guard。2026 年 9 月 23 日发布的 tor 0.4.9.13 安全更新正修复了这一点：客户端现在只在无法建立到 guard 的连接时把失败归因于 guard，而不会因为电路、数据流或目录请求失败就这样做。

13.4 Vanguards 防护机制

Guard 保护的是第一跳。洋葱服务（第 14 章）还有另一个问题：恶意客户端可以不断诱导服务向攻击者选择的 rendezvous point 构建大量电路。如果每条电路的 middle 都重新随机选，那么迟早会出现攻击者控制的 middle，此时攻击者就能观察服务正在使用哪个 guard，再进一步攻击该 guard 或设法取得其数据。**Vanguards** 把 guard 的思想延伸到后续跳：第二层（某些配置还包括第三层）也固定使用一小组中继。

Vanguards-lite（提案 333，tor 0.4.7.1-alpha）客户端和洋葱服务默认启用。它使用四个第二层中继，不写入磁盘，每个具有随机寿命。这里代码与规范存在差异：tor 0.4.9.13 使用 1–12 天范围，平均约一周，由 `src/feature/client/entrynodes.c` 中 `MIN_SECOND_GUARD_LIFETIME` 和 `MAX_SECOND_GUARD_LIFETIME` 定义。共识参数 `guard-hs-l2-lifetime-min` 与 `-max` 可以覆盖这些值，但 2026 年 9 月 23 日共识中并不存在。vanguards 规范仍写 1–22 天，平均两周。该模式面向活动时间不超过几周的服务和用户。

Full vanguards 第二层和第三层都固定使用中继，寿命更长，面向长期运行且高风险的洋葱服务。tor 通过 *vanguards* 扩展组件实现，设置 `HSLayer2Nodes` 和 `HSLayer3Nodes`；Arti

则直接实现，同时提供 `Full` 和 `Lite` 模式。

在实验网络（第 19.7.3 节）中，我们曾看到用途标记为 `HS_VANGUARDS` 的电路：它们是预先用 vanguard 构建好的电路，可随时进一步扩展到引介点或汇合点。

13.5 实验：观察公共网络

实验 7：按带宽加权的路径选择

目标。编写 `labs/zh-Hans/代码/torlab/path_selection.py`，用真实公共网络共识分析路径选择。
前置条件。实验 6（`directory.py`）；从 Tor Project 的公开归档 CollecTor（`https://collector.torproject.org/`）下载的一份公共共识。这些数据本来就公开用于研究，下载它们不会接触任何网络用户。
预计时间。2 小时。

步骤 **1**：中继权重。

```
def _relay_type(r):  # g=纯 Guard, e=纯 Exit, d=两者兼有, m=两者都不是
    g, e = "Guard" in r.flag, "Exit" in r.flag and "BadExit" not in r.flag
    return {(1, 1): "d", (1, 0): "g", (0, 1): "e", (0, 0): "m"}[(g, e)]

def weight(r, position, weights):
    key = f"W{position[0]}{_relay_type(r)}"          # 例如 "Wmg"
    return (r.bandwidth or 0) * weights.get(key, 10000) / 10000
```

步骤 **2**：选择路径。 `Selector` 类为每个位置预先计算一次权重。约束通过拒绝采样施加：抽到一个中继，如果违反约束（同一中继或同一 /16），就重新抽取。

```
def path(self, rng, guard=None):
    e = self.choose("exit", [], rng)
    g = guard or self.choose("guard", [e], rng)
    m = self.choose("middle", [e, g], rng)
    return g, m, e
```

最初版本每次选择都过滤九千多个中继，并重新计算权重和网络，构建 2000 条路径需要 310 秒。这个版本构建 20 000 条只需 7 秒。在统计模拟中，性能不是次要问题。

步骤 **3**：公共网络的数字。 对 2026 年 9 月 23 日 15:00 UTC 的共识：

```
中继：9358
带 Guard 标志的带宽：70.1%    带 Exit 标志的带宽：33.1%
guard : 合格  5306；前 10 = 1.7%；前 100 = 12.6%
middle：合格  8961；前 10 = 2.8%；前 100 = 13.7%
exit  : 合格  3126；前 10 = 2.3%；前 100 = 13.7%
20000 条路径：gge 30.9%  ggd 29.4%  gme 20.4%  gmd 19.3%
```

合格中继数量少于拥有相应 flag 的中继数（第 6 章 报告有 3177 个 `Exit`，这里为 3126）。因为 `eligible` 函数还要求 `Running`、`Valid` 和 `Fast`，并排除 `BadExit` 与 `MiddleOnly`。差值包括 48 个没有 `Fast` 的出口和 3 个标记为 `BadExit` 的出口。

最后一行使用随机种子 7。若不固定随机种子，每次结果都会波动零点几个百分点，两个实验将难以直接比较。字母代码表示 guard、middle、exit 的类型，例如 `gme` 表示 guard + 没有两种标志的普通中间节点 + 纯 exit。正如权重所预示，没有路径把 exit 用于第一或第二位

置；约 60 % 情况下，middle 自身带有 `Guard` 标志。集中度也不高：最常被选中的 10 个出口合计只占 2.3 %，前 100 个占 13.7 %。

步骤 **4**：**Guard** 与对手。 `Adversary` 类计算 第 13.3 节 中的表格。尝试改变对手份额：当 $c = 1\,\%$ 或 $c = 20\,\%$ 时，guard 的效果是否仍然明显？

小结

- tor 先选出口，再选 guard 和 middle，并避免重复中继、同一 /16 或同一 family 的中继出现在同一条电路。
- 每个中继的选择概率与“带宽 × 位置/类型权重”成比例；在本章分析的网络中，exit 只用于出口位置。
- Guard 把“长期重复随机选择第一跳最终几乎必然被攻陷”的风险，转变为影响少量用户的低概率事件；当前客户端使用两个。
- Guard 算法维护有限 sample、confirmed guard 和 primary guard，并且谨慎判断何时把失败归因于 guard。
- Vanguard 还为洋葱服务固定第二层；vanguards-lite 默认启用。
- 共识参数可以覆盖软件内建默认值。

练习

练习 **13.1** ⋆ 为什么 tor 要先选择出口？如果先选择 guard，可能出现什么问题？

练习 **13.2** ⋆ 某中继同时具有 `Guard` 和 `Exit` 标志，共识带宽为 20000。按照本章给出的真实网络权重，它在三个电路位置上的权重分别是多少？

练习 **13.3** ⋆⋆ 对 $c = 1\,\%$ 与 $c = 20\,\%$ 重复 guard 表格计算。两种情况下，不使用 guard 时至少需要多少条电路，攻陷概率才会超过 50 %？

练习 **13.4** ⋆⋆ 攻击者希望提高自己的中继被选为出口的概率。它必须投入哪些资源？带宽测量、family、单地址中继数量限制以及 bandwidth weight 分别如何增加攻击难度？

练习 **13.5** ⋆⋆ 为什么 guard sample 被限制为最多 60 个或全部 guard 的 20 %？描述这一限制阻止的攻击。

练习 **13.6** ⋆⋆⋆ 扩展 `Selector`，利用微描述符中的 `family` 和 `family-ids` 字段实施家族约束。从 CollecTor 下载与共识匹配的微描述符，统计多少次抽样因为 family 冲突而被拒绝。

练习 **13.7** ⋆⋆⋆ 按声明的中继家族对公共共识中继分组，计算最大的十个中继家族占出口选择概率的份额，并与单个中继中最常被选择的十个出口份额比较。你能对“经营大量中继的运营者”带来的风险得出什么结论？

延伸阅读

- *Tor Path Specification* 与 *Tor Guard Specification* [Thee]。
- L. Øverlier 与 P. Syverson，*Locating Hidden Servers* [ØS06]：促成 guard 的攻击。
- 提案 236、271、291、333：从单 guard 到双 guard、当前算法与 vanguards-lite。
- vanguards 规范（vanguards-spec），其中包括 Sybil 攻击时间线表格。

第 14 章

洋葱服务 v3

最后核验：*2026 年 9 月 23 日* — *torspec* 提交 `8e1f245`；*tor 0.4.9.13*；*rend-spec v3*

本章目标

学完本章后，你将能够：

- 解释洋葱服务提供什么能力，以及它与通过出口访问普通网站有何不同；
- 从服务的 `.onion` 地址推导公钥，并由公钥生成地址；
- 描述盲化密钥、时间周期，以及客户端如何定位负责的 HSDir；
- 读懂描述符的分层结构；
- 逐步跟踪引介与汇合协议；
- 在实验网络上下载、验证并解密一个洋葱服务的描述符（实验 8）。

到目前为止，我们使用 Tor 隐藏的是谁在连接某台服务器。服务器本身却是公开的：它的 IP 地址已知，出口中继通过公共网络直接连接到它，因此知道目的地。如果应用没有使用 HTTPS 之类的端到端加密，内容还会以明文出现。洋葱服务把这种关系反过来：它允许某人提供一个服务，例如网站，而不暴露服务的网络位置。客户端与服务在网络中间相会，双方都由自己的电路保护，并且彼此都不知道对方的地址。

14.1 洋葱服务提供什么

服务匿名性 服务器的 IP 地址不会暴露：服务只通过自己主动建立的 Tor 电路与网络通信。

自认证地址 `.onion` 地址编码了服务的公钥，以及校验和与版本字节。任何连接到该地址的人都可以通过密码学确认，对端持有相应私钥，而不需要证书颁发机构。

端到端加密 客户端与服务之间的流量使用只有双方知道的密钥加密；不存在能够观察明文流量的出口中继。

可达性 服务不需要接受来自公网的入站连接。只要 NAT 或防火墙允许出站连接，它就能工作。

代价是路径更长。在基本模型中，客户端通过三跳电路到达一个会合点，服务再通过另一条三跳电路到达同一点：总共经过六个中继，而访问普通网站时只有三跳。默认启用 vanguards-lite 后（第 13.4 节），路径不再对称。服务端会插入一个固定的第二层中继，因此其汇合侧电路变为四跳（`S` → `G` → `L2` → `M` → `Rend`），而客户端仍为三跳。

历史说明：从 v2 到 v3

隐藏服务（*hidden services*）自 2004 年起就已存在。第一个广泛部署的版本 v2 使用 16 字符地址：从 1024 位 RSA 密钥出发，将 SHA-1 摘要截断到 80 位后生成。它还允许 HSDir 收集自己托管描述符的服务地址。版本 3（提案 224）自 tor 0.3.2.1-alpha（2017）起可用，引入了 Ed25519 密钥、包含完整公钥的 56 字符地址、加密描述符和盲化密钥。V2 服务在 2021 年 10 月随 tor 0.4.5.11 和 0.4.6.8 被彻底禁用。名称也发生了变化：如今使用的术语是洋葱服务（*onion service*）。

14.2 地址

v3 地址把服务的身份公钥、一个短校验和以及版本号进行 base32 编码 [rend-spec § Encoding onion addresses]：

$$\texttt{address} = \text{base32}(\mathit{PUBKEY} \,\|\, \mathit{CHECKSUM} \,\|\, \mathit{VERSION}) \,\|\, \texttt{".onion"}$$

其中，PUBKEY 是 32 字节 Ed25519 公钥，$\mathit{VERSION}$ 是字节 `0x03`，$\mathit{CHECKSUM}$ 则取 SHA3-256(`".onion checksum"` $\|$ PUBKEY $\|$ $\mathit{VERSION}$) 的前 2 个字节。最终的 35 字节恰好编码成 56 个 base32 字符。校验和并不能抵御攻击者，因为攻击者和任何人一样都能重新计算它；它的作用是发现输入错误。

在一次实验运行中，洋葱服务得到如下地址。这是保存在 `labs/zh-Hans/代码/torlab/test/data/` 中的示例地址，实验测试会使用它。本节所有输出都针对这个示例，因此无需联网也可以复现。当读者启动自己的网络时，会生成新的密钥，因此地址也会不同（第 19.6 节）。

```
wdjtp7qktdjkkgjjbbx2axvg2gri5p4tuhlc4bq23n7o3uwu4rveppqd.onion
```

实验 8 中的 `key_from_address` 函数会从该地址提取公钥并验证校验和。

注意：不能靠记忆识别地址

随机生成的 56 字符地址既难以记忆，也很难凭肉眼快速辨认。用户通常从自己信任的来源复制地址，而一个只差几个字符的地址很可能不会被注意到。这是针对洋葱服务最主要的钓鱼（*phishing*）方式。个性化地址（*vanity address*）通过尝试数百万把密钥，直到地址开头能拼出一个单词，只能部分缓解这一问题：攻击者同样可以生成具有相同前缀的地址。我们将在 第 21 章 再讨论这个问题。

14.3 盲化密钥

14.3.1 问题

服务必须在某处发布如何找到自己的信息，也就是描述符（*descriptor*）。如果描述符直接以身份密钥发布，那么任何运行 HSDir 的人都能看到自己托管描述符的全部服务地址，从而枚举这些服务，或者把同一服务在不同时间的描述符关联起来。这正是 v2 的情况。

14.3.2 解决方案

在 v3 中，服务使用盲化密钥（**blinded key**）对描述符签名，并且每个时间周期（*time period*）使用不同的盲化密钥 [rend-spec § Appendix A: Signature scheme with key blinding]。给定身份密钥 $A = aB$（其中 B 是 Ed25519 基点），先计算

$$h = \text{SHA3-256}(\texttt{"Derive temporary signing key\0"} \,\|\, A \,\|\, B \,\|\, N),$$
$$N = \texttt{"key-blind"} \,\|\, \mathit{period} \,\|\, \mathit{period_length}$$

（其中部分位会像 Ed25519 密钥一样被强制设置），随后得到盲化公钥：

$$A' = h \cdot A.$$

这种构造具有几个重要性质：

- 任何知道地址、因此知道 A 的人，都能为每个周期计算 A'，从而知道应该到哪里查找描述符；
- 只看到 A' 的人，例如 HSDir，无法恢复 A，也无法判断不同周期中的两个盲化密钥是否属于同一个服务；
- 服务可以使用私钥 $a' = h \cdot a$ 在 A' 下签名，生成的签名可以像普通 Ed25519 签名一样用 A' 验证。

一个不寻常的细节说明规范与代码贴得有多近：常量字符串末尾包含一个零字节，因为 tor 用 `sizeof` 而不是 `strlen` 计算它的长度。源码注释（`src/feature/hs/hs_common.c`）承认，与其修改代码，不如把这个“意外”记录进规范。

14.3.3 凭据与离线密钥

身份密钥生成一个凭据（*credential*）：SHA3-256(`"credential"` $\|$ A)。每个周期再由此生成一个子凭据（*subcredential*）：SHA3-256(`"subcredential"` $\|$ *credential* $\|$ A')。子凭据用于描述符加密和引介握手：它证明请求方知道服务地址，而 HSDir 并不知道该地址。

密钥层次结构还允许身份密钥保持离线。每个周期的盲化密钥会认证一个短期描述符签名密钥（类型 8 的 Ed25519 证书，见 表 7.1）。原则上，服务器可以预先接收未来周期的签名密钥与证书，而从不持有与其地址对应的身份私钥。并非所有实现都支持这种模式。

14.4 定位描述符

14.4.1 时间周期

时间被划分为若干周期；在公共网络上，每个周期持续一天（由共识参数 `hsdir-interval` 决定）[rend-spec § Dividing time into periods]。周期并不是在 UTC 午夜开始，而是在 UTC 正午开始。它与权威的共享随机数协议错开 12 小时；该协议每天午夜在共识中发布新的共享随机值（**shared random value, SRV**）。周期编号由从 1970 年 1 月 1 日起经过的分钟数减去偏移量，再除以周期长度得到。规范给出的示例是：2016 年 4 月 13 日 11:15:01 UTC 时，时间周期为 16903，该周期从前一天正午开始。实验 8 正好测试这个例子。

在测试网络中，如果每 20 秒投票一次，等待一天显然不现实。因此 tor 把周期缩短为一轮完整的 SRV 协议周期，也就是 24 次投票（函数 `get_time_period_length`）。在实验网络中，一个周期持续 8 分钟，并偏移 4 分钟。

14.4.2 HSDir 环

哪个 HSDir 保存描述符？每个带有 `HSDir` 标志的中继在一个 256 位环（*ring*）上占据一个位置 [rend-spec § Where to publish a hidden service descriptor]：

$$\textit{relay_index} = \text{SHA3-256}(\texttt{"node-idx"} \,\|\, \textit{Ed25519 identity} \,\|\, \textit{SRV} \,\|\, \textit{period} \,\|\, \textit{period_length})$$

对于两个副本（*replica*）中的每一个，服务都计算一个索引：

$$\textit{service_index}(r) = \text{SHA3-256}(\texttt{"store-at-idx"} \,\|\, A' \,\|\, r \,\|\, \textit{period_length} \,\|\, \textit{period}).$$

服务把描述符上传到环上每个索引之后的前四个 HSDir（参数 `hsdir_spread_store`）；客户端则从前面三个中随机选择一个（`hsdir_spread_fetch`）。把 SRV 纳入索引计算，可以防止攻击者提前很久预测哪些中继将负责某个服务，进而把恶意中继放到恰当位置来审查该服务或观察描述符请求。服务会同时发布两个连续周期的描述符，这样即使客户端的共识稍微领先或落后，服务仍然可达。具体使用哪两个周期取决于当前处在循环中的哪个阶段：在 SRV 轮换前后，可能是“上一周期+当前周期”，也可能是“当前周期+下一周期” [rend-spec § Generating and publishing hidden service descriptors]。

深入说明：应该使用哪个 SRV？

由于时间周期与 SRV 错开半天，客户端必须谨慎选择 SRV。在一个周期开始到下一次 SRV 发布之间的半天里，它使用当前 SRV；另一半天则使用前一个 SRV。如果共识中缺少所需 SRV，例如网络刚启动时，tor 会计算一个备用值，即灾难 *SRV*（*disaster SRV*）：SHA3-256(`"shared-random-disaster"` ‖ *period_length* ‖ *period*)。选择逻辑由 `hs_get_current_srv` 和 `hs_get_previous_srv`（`src/feature/hs/hs_common.c`）实现。如果共识缺少所需值，tor 会静默退回到 disaster SRV。新建实验网络时会出现这种情况，但不会持续太久：在我们的网络中，权威日志显示启动 115 秒后出现第一个 SRV；在此之前，共识中没有 `shared-rand-current-value` 行。要观察这一回退过程，可以查看新网络建立后的前几分钟。一旦网络运行一段时间，如 第 14.9 节 中的实验那样，就会使用真正的 SRV。

14.4.3 只接受匿名请求

客户端通过一个 `BEGIN_DIR` stream 向 HSDir 发起 HTTP 请求，路径为 `/tor/hs/3/` 加上去掉末尾 = 的盲化密钥 base64 表示，以下载描述符。但有一个条件：请求必须经由匿名电路抵达。HSDir 会拒绝来自客户端直接建立电路的请求，也就是单跳电路。我们在实验中实际观察到了这一点：

```
HTTP/1.0 503 Rejecting single hop HS v3 descriptor request
```

检查逻辑位于 `connection_dir_is_anonymous`（`src/feature/dircommon/directory.c`）。这样可以防止 HSDir 把某次描述符请求与发起请求的客户端 IP 地址关联起来。

14.5 描述符

描述符包含三层 [rend-spec § Generating and publishing hidden service descriptors]，如 图 14.1 所示。

外层　是明文且带签名。它包含版本、有效期（`descriptor-lifetime`，实验中为 180 分钟）、由盲化密钥签发的签名密钥证书、修订计数器、加密的 `superencrypted` 块以及签名。签名是 Ed25519 签名，由描述符签名密钥对前缀为 `"Tor onion service descriptor sig v3"` 的文本进行签名。该前缀防止为描述符生成的签名在其他语境中同样有效。每个新版本的修订计数器都必须递增，这样 HSDir 才不会接受比自己已存版本更旧的描述符。

第一加密层　使用由盲化密钥和子凭据派生的密钥加密。不知道服务地址的 HSDir 无法读取它。该层包含受限发现（*restricted discovery*）字段（第 14.7 节）以及第二层。为避免泄露是否启用了受限发现，这些字段始终存在；不需要时会填充随机数据。

第二层　包含建立连接所需的信息：支持的握手格式、流量控制参数、可能存在的抗 DoS 参数，以及每个引介点（**introduction point**）的连接信息，包括到达它所需的 link specifier

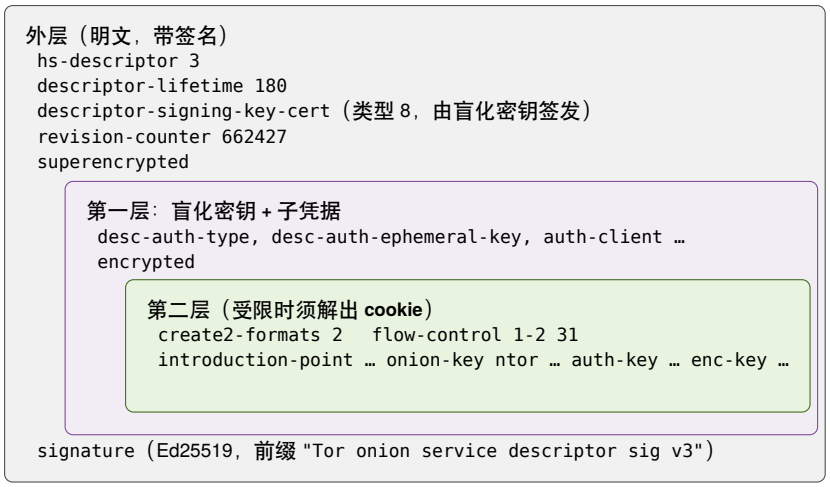


图 14.1: Tor v3 洋葱服务描述符结构：HSDir 可以读取已签名的外层；知道 onion 身份的客户端能解密第一层。启用受限发现时，获授权客户端使用自己的 X25519 密钥恢复 descriptor cookie，继而解密第二层。

（base64）、ntor 密钥、经过认证的身份验证密钥和加密密钥。默认情况下，一个服务使用三个引介点。

两层使用同一种加密方案。一个随机的 16 字节盐值（salt）、秘密数据和层专用常量一起输入 SHAKE-256，派生出 AES-256 密钥、初始化向量和认证密钥。正文使用计数器模式加密，并通过 SHA3-256 认证。第一层明文会填充到 10 000 字节的整数倍，`auth-client` 行数也会填充到 16 的整数倍，从而尽量减少描述符大小对引介点数量和授权客户端数量的泄露。

14.6 引介与汇合

拿到描述符后，客户端就可以连接服务。协议如 图 14.2 所示，涉及三类“特殊”中继：由服务选择的引介点、由客户端选择的汇合点，以及 HSDir。

1. 服务准备引介点。服务选择若干中继，为每个中继建立电路，并发送 `ESTABLISH_INTRO`（中继命令 32），其中为每个引介点携带不同的身份验证密钥和签名。中继回复 `INTRO_ESTABLISHED`（38）；此后，带有该密钥的请求会被转发给服务。

2. 服务发布描述符。按照前面所述，将描述符上传到负责的 HSDir。

3. 客户端准备汇合。客户端随机选择一个中继作为汇合点，建立到它的电路，并发送 `ESTABLISH_RENDEZVOUS`（33）以及一个随机 20 字节随机令牌（*cookie*）。中继回复 `RENDEZVOUS_ESTABLISHED`（39）。

4. 客户端进行引介。客户端建立到某个引介点的电路并发送 `INTRODUCE1`（34）。明文部分指定引介点的身份验证密钥；只有服务能读取的加密部分包含 cookie、汇合点的 link specifier 和 ntor 密钥，以及握手的前半部分。引介点向客户端发送 `INTRODUCE_ACK`（40），并把消息以 `INTRODUCE2`（35）的形式转发给服务。

5. 服务连接汇合点。服务解密 `INTRODUCE2`，检查该消息是否与之前见过的请求重复（即重放缓存，*replay cache*），然后建立到指定汇合点的电路，并发送 `RENDEZVOUS1`（36），其中包含 cookie 和握手后半部分。

6. 汇合点连接两条电路。它使用同一个 cookie 找到客户端的电路，把握手后半部分通过 `RENDEZVOUS2`（37）转发过去，此后就在两条电路之间转发信元。

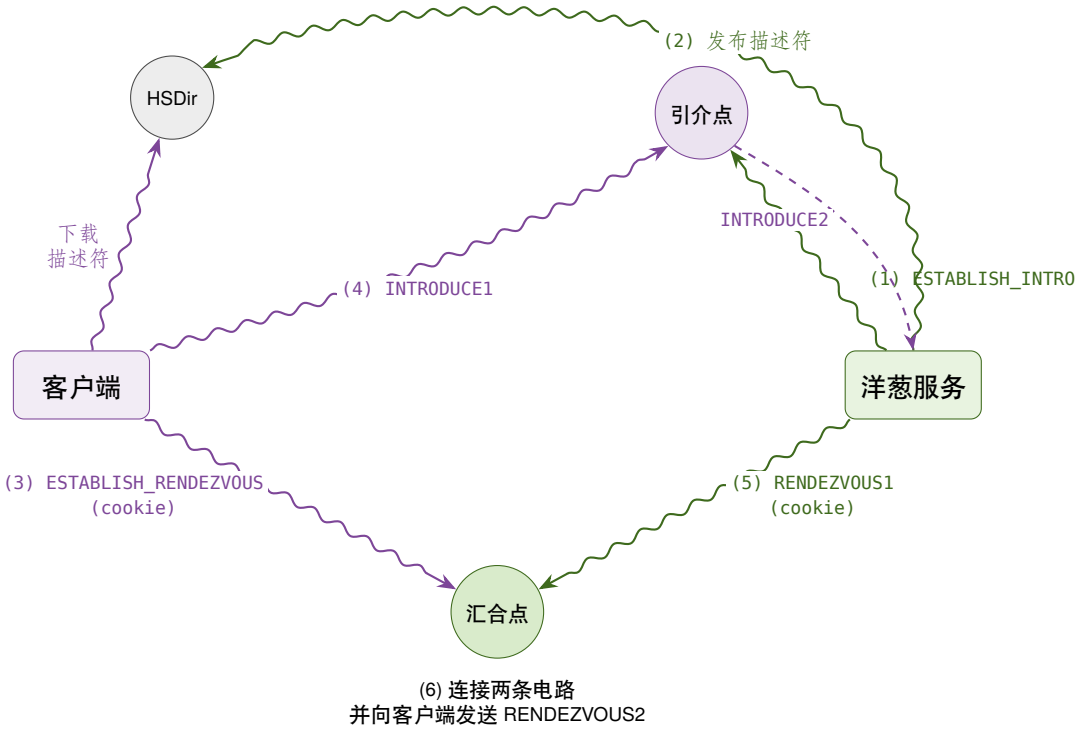


图 14.2: 汇合协议。每条箭头都代表一条多跳电路；客户端、服务与中间节点之间不存在直接连接。数字表示正文描述的步骤顺序。

客户端与服务之间的握手称为 *hs-ntor*，它是 ntor（第 9.6 节）的一个变体，其中加入了子凭据以及引介点的身份验证密钥。只有知道服务地址的人才能生成服务会接受的 `INTRODUCE1`。派生出的密钥再增加一层加密，就像在客户端电路末端增加一个虚拟跳（*virtual hop*），其运行摘要使用 SHA3-256 而不是 SHA-1（第 10.2 节）。无论引介点还是汇合点，都无法读取通信内容。

在拼接后的电路上，客户端像通过出口一样用 `BEGIN` 打开数据流（第 11 章）。服务则把这些数据流转发到通过 `HiddenServicePort` 配置的本地端口。

深入说明：为什么需要这么多中继？

引介点眼中的服务只是“某条电路的身份验证密钥”；它不知道自己服务的是哪个 `.onion` 地址。汇合点只看到一个随机 cookie 和两条电路；它不知道自己正在把哪个服务连接给哪个客户端。每个角色只看到信息的一小部分，也没有任何一个角色知道客户端或服务的 IP 地址，因为它们都隐藏在各自的 guard 后面。

14.7 受限发现

有些服务只希望特定客户端能够访问，例如管理面板。在受限发现（**restricted discovery**）机制下（旧称客户端授权，*client authorization*），每个授权客户端拥有一个 X25519 密钥对，而服务知道其公钥。服务会在第一层的 `auth-client` 行中，分别为每个客户端加密描述符第二层的密钥。即使某人知道服务地址，只要没有授权密钥，就无法读取引介点列表，因此甚至无法尝试连接。在 tor 中，可通过把客户端公钥加入服务的 `authorized_clients` 目录来配置此功能。

14.8 防御机制

洋葱服务天然容易成为攻击目标：因为服务看不到攻击客户端的 IP 地址，所以无法简单按 IP 封禁。主要防御措施包括：

- **vanguards**：保护服务的 guard，抵御利用按需构建电路实施的攻击（第 13.4 节）；
- 重放缓存（**replay cache**）：阻止攻击者让服务重复执行同一份工作；
- 服务可通过 `ESTABLISH_INTRO` 扩展在引介点设置抗 DoS 参数，限制每秒允许的 `INTRODUCE2` 消息数量；
- 工作量证明（**proof of work**）：当服务遭到攻击时，可以要求客户端在获得服务前先解决一个小型计算问题（第 15 章）。

14.9 实验：寻找并读取描述符

实验 8：实验洋葱服务的描述符

目标。编写 `labs/zh-Hans/代码/torlab/onion.py`：处理地址、盲化密钥、HSDir 环、描述符下载、验证与解密。
前置条件。实验 1–6；实验网络正在运行。
预计时间。4 小时。

步骤 **1**：盲化密钥。 常见密码学库提供 Ed25519 签名，但不提供计算 $A' = h \cdot A$ 所需的“点乘标量”操作。因此，本模块包含一份很小的 Ed25519 算术实现，大约三十行，基于 RFC 8032 中的公式。它足以完成这个实验，但不适合通用场景：速度慢，也没有抵抗时序攻击的保护。

```
def blinding_factor(A, period, duration_minutes):
    n = b"key-blind" + struct.pack(">QQ", period, duration_minutes)
    h = bytearray(hashlib.sha3_256(
        b"Derive temporary signing key\x00" + A + BASE_STR + n).digest())
    h[0] &= 248; h[31] &= 63; h[31] |= 64
    return int.from_bytes(h, "little")

def blinded_key(A, period, duration_minutes):
    return _encode_point(_multiply(blinding_factor(A, period, duration_minutes),
                                   _decode_point(A)))
```

步骤 **2**：周期与 **SRV**。 `time_period_info` 函数根据共识中的 `valid-after` 时间戳与投票间隔复现 tor 的计算，判断应使用当前 SRV 还是前一个 SRV。如果共识缺少所需值，则使用 disaster SRV。

步骤 **3**：追踪错误。 第一次运行时，我们在网络中的八个 HSDir 上都没有找到描述符。既然八个都尝试过，问题就不可能只是选错 HSDir；剩下的可能性只有盲化密钥或请求本身。为了区分这两种情况，我们让网络中的真实客户端通过控制端口下载描述符（先发送 `HSFETCH`，再发送 `GETINFO hs/client/desc/id/...`）。从返回证书中读取服务实际使用的盲化密钥后，发现它与我们的计算完全一致。因此问题出在请求上，而 tor 源码解释了原因：HSDir 会拒绝单跳请求。把到 HSDir 的路径改为三跳电路后，就成功获得了描述符。这是一种普遍有用的调试方法：实验失败时，应寻找一个中间检查点，把自己的结果与参考实现进行比较。

步骤 **4**：验证并解密。 此示例中的 `path` 是从实验网络获得的三个 `RelayInfo` 对象组成的列表，末尾是 HSDir；`A` 是身份公钥，`Ap` 是对应周期的盲化公钥。这里不展示这些对象的构造过程。

```
d = download_descriptor(Ap, path)
desc = parse_descriptor(d, A, Ap)
```

`parse_descriptor` 会验证签名密钥证书确实由预期盲化密钥签发，再验证描述符签名，并解密两个加密层。以下是此前一次实验运行的缩略输出：

```
有效期 180 修订号 662427 引介点 3
create2-formats 2
flow-control 1-2 31
introduction-point AwAGfwAAARPyAhQXE5GoH+wp3MoxW9M1QV6XSxb0fgM...
onion-key ntor yRURxDb2eeEmF/p5M/bmHA+giwVyxjApKYyNY1KkCDo=
auth-key
-----BEGIN ED25519 CERT-----
...
```

其中 `flow-control 1-2 31` 告诉客户端，该服务支持拥塞控制，且 `sendme_inc` 的值仍是第 11 章 中见过的 31。

步骤 **5**：一致性检查。 实验网络共有八个 HSDir，而服务会把描述符上传到 $2 \times 4 = 8$ 个 HSDir，也就是全部八个。因此“能够找到描述符”本身并不能证明环计算正确。不过，客户端日志（`info.log`）会记录它实际联系了哪个 HSDir：16:07 时为 `test002a`。在那个时刻，它确实位于我们代码为相应副本返回的三个 HSDir 之一。一次观察当然不是证明，但结果一致。练习 14.6 提出了一种更严格的测试。

小结

- 洋葱服务可以在不暴露 IP 地址的情况下提供服务；其 `.onion` 地址包含并认证服务公钥。
- 描述符使用每个时间周期独有的盲化密钥签名，只有知道地址的人才能把这些盲化密钥与服务关联起来。
- 描述符在 HSDir 环上的位置取决于盲化密钥、时间周期和共享随机值；客户端通过匿名请求下载描述符。
- 描述符包含一个带签名的外层和两个加密层；第二层列出引介点。
- 客户端和服务在客户端选择的汇合点相会，请求通过引介点送达服务。hs-ntor 握手再增加一层端到端加密。
- 受限发现、vanguards、重放缓存、速率限制与工作量证明共同保护洋葱服务。

练习

练习 **14.1** ⋆ 一个 v3 地址编码多少字节？为什么恰好生成 56 个 base32 字符？随机输入错误仍能通过校验和验证的概率是多少？

练习 **14.2** ⋆ 计算公共网络在 2026 年 1 月 1 日 09:00 UTC 时的时间周期编号。该周期何时开始？

练习 **14.3** ⋆⋆ 攻击者希望通过控制负责的 HSDir 来阻断某个洋葱服务。它必须预测什么？最多能提前多久预测？SRV 在其中起什么作用？

练习 **14.4** ⋆⋆ 分别列出引介点、汇合点和 HSDir 能了解到客户端与服务的哪些信息。对于试图估计某个服务受欢迎程度的攻击者来说，哪一种角色最有价值？

练习 **14.5** ⋆⋆ 为什么 HSDir 会拒绝通过单跳电路到达的描述符请求？否则它可能了解到什么？

练习 **14.6** ⋆⋆⋆ 更严格地测试环计算。向实验网络增加中继（练习 19.7），直到 HSDir 数量超过八个，然后验证描述符是否只存在于你的代码选出的 HSDir 上。

练习 **14.7** ⋆⋆⋆ 从实验描述符中解码某个引介点的 link specifier（即 `introduction-point` 行中的 base64 值）。它对应哪一个实验中继？`auth-key` 行中的证书包含什么？

延伸阅读

- *Tor Rendezvous Specification - Version 3*（rend-spec）[Thee]：地址、盲化密钥、描述符、引介与汇合协议，以及 hs-ntor 测试向量。
- 提案 224：v3 服务的设计。
- 共享随机数规范（srv-spec）：权威如何生成 SRV。

第 15 章

DoS 防护与资源管理

最后核验：*2026 年 9 月 23 日 — torspec 提交 `8e1f245`；tor 0.4.9.13；dos-spec、hspow-spec*

本章目标

学完本章后，你将能够：

- 解释为什么针对 Tor 的拒绝服务攻击也会威胁匿名性；
- 描述 tor 如何保护中继的内存、CPU 处理能力和带宽；
- 理解按 IP 地址限制连接与电路的机制，以及共识参数的作用；
- 解释洋葱服务如何抵御大量引介请求；
- 描述 v1 工作量证明机制（Equi-X 和 Blake2b）及其难度调节方式；
- 在实验网络中触发并观察中继和洋葱服务的防御机制（实验 9）。

一个开放、匿名且由志愿者运营的网络，自然容易成为试图使其瘫痪者的目标。本章研究 Tor 对拒绝服务（denial-of-service，DoS）攻击的防护：攻击者试图耗尽中继或洋葱服务的资源，使其他人无法使用。近年来发生的具体事件，包括 2022–2023 年针对洋葱服务的一系列攻击，将在 第 27 章 中讨论。本章重点分析防御机制本身。

15.1 为什么 DoS 也是匿名性问题

在普通网络中，DoS 攻击“只是”可用性问题；在 Tor 中，它还可能成为攻击匿名性的武器 [dos-spec § Overview]：

- 能够按需使某个中继失效的对手，可以迫使以该中继为入口守卫的客户端重新选择守卫，并希望新守卫受其控制。Jansen、Tschorsch、Johnson 和 Scheuermann 曾展示，攻击者利用很少的资源就能耗尽中继内存，并借此对洋葱服务去匿名化。这就是狙击手攻击（Sniper Attack）[Jan+14]。

- 使诚实中继变慢或无法访问，会把流量推向恶意中继，从而增加攻击者能够观察到的流量比例。

- 选择性地使特定电路失效，还可以形成一个信号，例如用来判断哪些电路经过了某个中继。

因此，防御的目标不能只是“让中继继续运行”：防御本身也必须避免产生可供攻击者利用的可观察行为。

15.2 物理资源

15.2.1 内存

内存是最敏感的资源，也是已有防护机制最成熟的资源 [dos-spec § Memory exhaustion]。在 Linux 上，进程申请过多内存时，通常不会马上收到错误。操作系统可能先承诺提供内存，等进程真正使用时再将其终止。因此，等待内存分配失败并不能有效防护。tor 自行施加 `MaxMemInQueues` 上限；该值可由运营者设置，也可按系统内存的一定比例计算。计入的对象包括所有内存消耗项：电路中排队的信元（cell）、连接缓冲区、压缩状态、缓存的洋葱服务描述符以及 DNS 查询缓存。

内存占用超过上限时，tor 分两个阶段释放内存。首先，它从占总量超过 20 % 的缓存中逐出最旧条目，直到这些缓存的占用降到 10 % 以下。如果用量仍超过上限的 90 %，它就按各电路和连接中最旧排队数据的年龄依次关闭它们，最先关闭那些数据滞留最久的对象。这样做旨在优先处理持续积累数据却不读取的参与者——这正是此类攻击的典型特征。

15.2.2 CPU

CPU 处理能力是 C 语言实现中的一个薄弱点，因为其网络处理依赖单个主线程。最耗时的操作是电路握手（第 9 章），它们会排队并交给工作线程池处理。然而，如果握手请求过多，队列仍会增长，合法电路便可能超时。Tor 不直接测量 CPU 使用率，而是通过限制耗费 CPU 的操作间接保护它，包括创建电路和处理洋葱服务引介。

15.2.3 带宽

运营者可以通过 `RelayBandwidthRate` 等选项限制带宽，但这些限制主要用于避免超出其网络接入方案的额度，本身并不能算作攻击防御。面对异常带宽消耗，Tor 依靠流量控制和拥塞控制（第 11 章），以及在电路之间公平调度信元。

15.3 按地址实施限制

针对简单洪泛攻击，最有效的防御十分直接。入口守卫能看到建立连接的客户端 IP 地址，因此可以限制每个地址能够请求的资源。网络中只有这一位置能实施此类限制，因为后续跳点被刻意设计为不知道流量的来源。Tor 针对客户端（而非经过认证的中继）提供三类限制，可通过 `DoS…` 选项配置：

连接 （`DoSConnection…`）限制每个地址同时建立的 TLS 连接数以及新连接的建立速率。超过阈值的连接会被关闭。

电路 （`DoSCircuitCreation…`）为每个地址设置令牌桶：新建一条电路消耗一个令牌。令牌按固定速率补充，直至达到最大容量（突发容量，burst）。当某地址的令牌耗尽，且同时持有至少规定数量的连接时，该地址会被标记；随后一段时间内，它发起的新电路请求将被拒绝。

单跳汇合 （`DoSRefuseSingleHopClientRendezvous`）拒绝来自单跳电路的汇合请求，例如 Tor2web 网关为在不保护客户端匿名性的情况下访问洋葱服务而使用的电路。这类请求能够以较低成本给网络带来负载。

15.3.1 代码与共识

在 tor 的源码中，这些防御默认关闭（`src/core/or/dos.h` 中的 `DOS_CC_ENABLED_DEFAULT 0`）。全网使用的设置来自共识参数。2026 年 9 月 23 日的公开共识包含以下数值：

参数	共识值	代码默认值
DoSCircuitCreationEnabled	1	0
DoSCircuitCreationMinConnections	2	3
DoSCircuitCreationRate	2	3
DoSCircuitCreationBurst	60	90
DoSConnectionEnabled	1	0
DoSConnectionMaxConcurrentCount	50	100
DoSRefuseSingleHopClientRendezvous	1	—

全网实际采用的数值比代码默认值更加严格：这是多年攻击和随后调整的结果。能够通过共识在数小时内修改这些数值，而无须更新数千个中继，是 Tor Project 在应对攻击时最有价值的手段之一。

注意：按地址限制的局限

拥有大量 IP 地址的攻击者，例如控制僵尸网络的人，可以绕过按地址设置的限制。反过来，过于严格的限制会伤及共用同一 IP 地址的合法用户，例如位于移动运营商或大学 NAT 后面的用户。因此，参数值始终是一种折中。

15.4 遭受攻击的洋葱服务

洋葱服务尤其容易受到攻击。对客户端而言，每次引介请求的成本很低：发送一条 INTRODUCE1 即可（第 14.6 节）。对服务而言，同一请求却代价高昂：它必须解密请求、执行握手，并向指定的汇合点建立一条多跳电路。这样就产生了放大效应：攻击者只需相对较少的资源，就能迫使服务及其电路所经过的所有中继进行大量额外工作。由于服务从来看不到攻击者的 IP 地址，也无法按其地址进行封锁。

15.4.1 引介点的限速机制

第一层防御是让引介点限制请求速率。服务通过 HiddenServiceEnableIntroDoSDefense 及相关的 …RatePerSec 和 …BurstPerSec 选项，在 ESTABLISH_INTRO 消息中使用 DOS_PARAMS 扩展，告知各引介点每秒最多愿意接收多少条 INTRODUCE2 消息。这能保护服务，却无法区分攻击者和合法用户：在攻击期间，双方的请求都可能被拒绝。

15.4.2 工作量证明

较新的防御方法是工作量证明（proof of work，PoW），最初由 提案 327 提出，自 tor 0.4.8.4（2023 年 8 月）起提供支持 [hspow-spec § Motivation]。客户端必须证明自己进行了一定量的计算，才会得到服务；证明了更高计算工作量的请求具有更高优先级。对合法用户而言，花几秒钟计算通常可以接受。但对于每秒发出数千次请求的攻击者，将成本乘以请求数量就可能变得难以承受。

该机制由四个部分构成 [hspow-spec § Common protocol]：

1. 描述符参数。在描述符第二层（第 14.5 节）中，服务发布一条 pow-params 记录，包括方案名称（v1）、随机的 32 字节种子（seed）、建议工作量和种子的到期时间。种子每 105–120 分钟更新一次。

2. 客户端求解难题。客户端选定工作量 E 后，针对以下挑战搜索 Equi-X 难题的解：

$$\texttt{"Tor hs intro v1\0"} \,\|\, A' \,\|\, \mathit{seed} \,\|\, \mathit{nonce} \,\|\, E,$$

其中 A' 为服务的盲化公钥。仅有 Equi-X 解还不够。客户端还要计算挑战与解的 32 位 Blake2b 哈希值 R；只有在 $R \cdot E \leq 2^{32} - 1$ 时，该解才有效。否则就更换 nonce 并重新尝试。

3. 引介消息中的证明。客户端在 `INTRODUCE1` 加密部分的扩展中放入 nonce、工作量、种子的前 4 个字节和求得的解；只有服务能读取这些内容。

4. 优先队列。服务验证证明——这一过程远快于寻找证明——然后按照工作量将请求放入优先队列。工作量最高的请求先得到处理；没有证明的请求其工作量为零。如果队列已满，服务会立即丢弃工作量最低的一半请求。

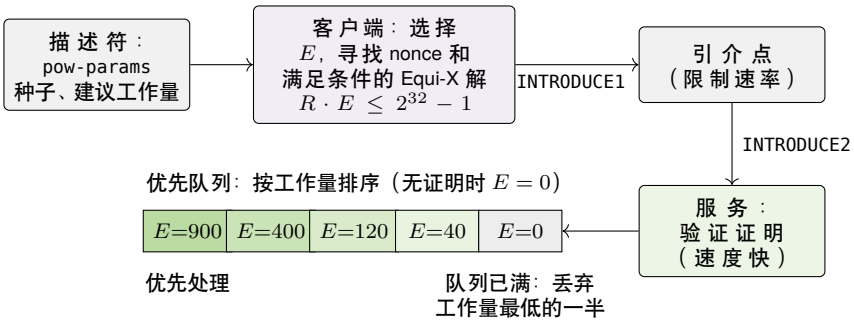


图 15.1: 洋葱服务的工作量证明。客户端从描述符读取种子和建议工作量，寻找满足条件的解，并在 `INTRODUCE1` 的加密部分发送它。服务验证证明并按工作量对请求排序。

深入说明：为什么使用 Equi-X？

Equi-X 由 tevador 专门为此设计，是一种非对称难题：求解需要内存和较多计算，但检验解所需的计算量很小。它基于 HashX——一类实现会随种子变化、能够利用现代 CPU 并行执行能力的哈希函数。这使专用硬件加速更为困难，限制攻击者相对普通电脑用户的优势。现有两种通过自动化测试保持兼容的实现：tor 中采用 C 语言实现，Arti 则使用 Rust 实现。C 代码以 LGPL 许可发布；要将其纳入 tor，编译时必须使用 `--enable-gpl`。如果没有该选项，构建版本就不支持 PoW，`tor --list-modules` 会显示 `pow: no`，而不是我们将在 第 17 章 中见到的 `pow: yes`。不过，这并非常见情况：实验镜像和 Debian 的 `tor` 软件包都启用了该选项，因此都包含这一模块。

15.4.3 线性工作量

为什么还需要 Blake2b 检验？寻找单个 Equi-X 解的成本大致固定。随机解满足 $R \cdot E \leq 2^{32} - 1$ 的概率约为 $1/E$，因此客户端平均需要尝试 E 个解。这样，计算成本就随 E 线性增长，便于细粒度调节。许多其他 PoW 系统要求哈希值开头有一定数量的零比特，每增加一个零比特，成本就会翻倍：这样的刻度对精细反馈控制来说太粗糙。工作量还必须在开始搜索前声明，并成为挑战的一部分。因此，客户端不能先侥幸找到一个结果，再谎称自己进行了远高于实际的工作量。

15.4.4 控制工作量

建议工作量并不是永久不变的。服务最初建议的工作量为零，表示防御尚未主动施加计算成本。此后每五分钟，服务根据队列状态更新建议值：如果队列持续满载，或不得不丢弃具有建议工作量的请求，就提高该值；否则便降低。控制逻辑借鉴了 TCP 拥塞控制（加性增加、乘性减小）。客户端若在尝试连接时遇到超时，也会提高自己的工作量。最终形成一个双反馈环路：愿意投入更多计算资源的用户可以用更高工作量取得队列中的优先位置。

15.5 实验：触发防御机制

实验 9：在实验网络中观察抗 DoS 限制与工作量证明

目标。启用中继的电路创建限制并触发它；启用洋葱服务的工作量证明并读取参数；使用 `labs/zh-Hans/代码/torlab/pow.py` 研究 Blake2b 工作量检验。
前置条件。实验 1–8；实验网络已运行，其镜像编译时启用了 `--enable-gpl`。
预计时间。2 小时。

注意：仅限自己的网络

本实验会有意生成异常流量。请只在本书提供的私有网络中运行。对真实中继或服务执行同样操作属于伤害其他用户的攻击行为，在许多司法管辖区也属于违法行为（第 25 章）。

步骤 **1**：启用中继限制。 我们在 `test003r` 中继的 `torrc` 中加入非常低的阈值，以便快速触发防御，然后通过 `HUP` 信号让它重新加载配置：

```
DoSCircuitCreationEnabled 1
DoSCircuitCreationMinConnections 2
DoSCircuitCreationRate 1
DoSCircuitCreationBurst 10
DoSCircuitCreationDefenseType 2
DoSCircuitCreationDefenseTimePeriod 60 seconds
```

防御类型 2 表示“拒绝创建新电路”。

步骤 **2**：超过阈值。 我们向中继建立一个或两个通道，使用 **`CREATE_FAST`** 连续快速请求 30 条电路。为使每次实验使用独立计数器，连接分别从不同环回地址（127.0.0.2、127.0.0.3）发起，并使用 `open_channel` 的 `source_address` 参数：

```
源地址 127.0.0.2，2 个通道：成功 9，DESTROY(5) 21；首次拒绝：第 10 次
源地址 127.0.0.3，1 个通道：成功 30
源地址 127.0.0.1，2 个通道：成功 9，DESTROY(5) 21；首次拒绝：第 10 次
```

可以得出三个结论：

- 中继通过原因码为 5（`RESOURCELIMIT`）的 **`DESTROY`** 信元拒绝请求。
- 当 *burst* 为 10 时，成功的是九条电路而不是十条。源码 `src/core/or/dos.c` 中的 `cc_has_exhausted_circuits` 会在令牌桶归零时立即标记客户端，因此第十个请求已经被拒绝。
- 只有一条通道时不会触发限制，因为该防御要求至少存在 `MinConnections` 个并发连接。这样就不会惩罚通常只维持一条入口守卫连接的普通客户端。

我们原本预测，从实验中继使用的地址 127.0.0.1 发起的连接应当获得豁免。实验否定了这个预测，而 `dos.c` 也证实电路创建并不存在这种豁免。实验结束后，将 `torrc` 恢复为原始配置。

步骤 **3**：洋葱服务的工作量证明。 在 `test009h` 的 `torrc` 中，将以下配置加在 `HiddenService…` 选项之后：

```
HiddenServicePoWDefensesEnabled 1
```

发送 `HUP` 信号后，服务会发布新的描述符。按照实验 8 下载并解密描述符，就会在第二层看到这一行：

```
pow-params v1 tx2Mtmzw5GHgWjcDoivTvWzwQPNZONwMwSMddOc+ZVQ= 0 2026-09-23T18:20:53
```

它记录方案 v1、以 base64 编码的种子、零建议工作量（服务尚未出现排队）以及种子到期时间：发布后 1 小时 50 分钟，在规定的 105–120 分钟范围内。

步骤 **4**：工作量呈线性增长。 pow.py 模块实现 Blake2b 检验，并用随机字节模拟 Equi-X 解；对于这项检验，它们具有相同的统计分布：

```
def check_value(service_id, seed, nonce, effort, solution):
    challenge = P_STRING + service_id + seed + nonce + struct.pack(">I", effort)
    h = hashlib.blake2b(challenge + solution, digest_size=4).digest()
    return int.from_bytes(h, "big")

def valid_effort(r, effort):
    return r * effort <= UINT32_MAX
```

对于每个工作量，进行 400 次试验并测量求解所需的平均尝试次数，可得到：

```
工作量     1: 1.0 次尝试
工作量    10: 9.7
工作量   100: 102.3
工作量  1000: 1068.8
```

尝试次数如预测所示，随工作量成比例增长。由于每次尝试成功的概率为 $1/effort$，而尝试次数遵循几何分布，因此其期望值等于工作量；标准差也与均值一样大。所以，400 次试验的平均值通常存在 $1/\sqrt{400} = 5\,\%$ 的相对偏差。即使条件完全不变，两次运行的结果也可能相差 10 %；重新测量时，我们确实得到过 969.9，而上面的输出为 1068.8。这提醒我们，解读样本均值时必须同时考虑不确定性。还有一个容易忽略的细节：使用 `digest_size=4` 的 `blake2b` 结果，并不等于 64 字节 Blake2b 结果的前四个字节，因为输出长度会影响函数的初始状态。规范特别强调了这一点。

本章小结

- 在 Tor 中，DoS 也会威胁匿名性：按需使中继失效或发生故障，可以把流量引向攻击者。
- `MaxMemInQueues` 为内存设定内部上限；超过阈值时，tor 会逐出缓存并关闭持续积累数据的电路。
- 入口守卫使用令牌桶限制每个客户端地址的连接数和电路数。这些防御在代码中默认关闭，通过共识启用和调节。
- 洋葱服务面临放大效应：发起请求成本低，响应请求成本高。
- v1 工作量证明将 Equi-X 与 Blake2b 检验结合，使计算成本线性增长。服务优先处理工作量更高的请求，并按队列状态调整建议工作量。

练习

练习 **15.1** ⋆ 按照公开共识，若 `DoSCircuitCreationRate` 为 2、突发容量（`Burst`）为 60，客户端从满令牌桶开始，一分钟内最多能创建多少条电路？一小时内又是多少？假设防御已被触发，即该地址同时建立的连接数达到 `DoSCircuitCreationMinConnections`。低于该阈值时不存在限制。

练习 **15.2** ⋆ 为什么按地址限制只针对客户端，而且只能在第一跳实施？如果中间中继尝试“按地址”限制电路，会发生什么？

练习 **15.3** ⋆⋆ 估计一次引介请求中，客户端工作量与整个网络工作量的比率：服务的响应会产生多少条电路、多少次握手和多少个 cell？

练习 **15.4** ⋆⋆ 将 Tor 的工作量证明与要求哈希值前 k 位均为零的方案比较。两种方案各自能表达哪些工作量取值？为什么线性刻度更适合反馈控制？

练习 **15.5** ⋆⋆ 某客户端声明工作量 $E = 50$，却碰巧很快找到一个 R 极小、甚至满足 $E = 5000$ 条件的解。它能否把这份解作为 $E = 5000$ 的证明提交？为什么？

练习 **15.6** ⋆⋆⋆ 在实验网络上，将一台中间中继的 `MaxMemInQueues` 设置为最小值，并建立经过它的电路，下载一个大文件，但不读取传入的数据（也不发送 SENDME）。积累的信元会发生什么？请在该中继的日志中查找内存管理器的消息。

练习 **15.7** ⋆⋆⋆ 编写一个小程序模拟洋葱服务的优先队列。请求带有随机工作量（多数为零，少数较高）；服务每秒处理一定数量的请求，每当队列超过阈值，就丢弃工作量最低的一半。测量平均等待时间与工作量的关系。

延伸阅读

- *Tor denial-of-service mitigations*（dos-spec）与 *Onion service proof-of-work*（hspow-spec）[Thee]。
- R. Jansen、F. Tschorsch、A. Johnson、B. Scheuermann，*The Sniper Attack: Anonymously Deanonymizing and Disabling the Tor Network* [Jan+14]。
- tevador 的 Equi-X 代码仓库及开发日志，参见规范中的引用。

第 16 章

审查、网桥与可插拔传输

最后核验：*2026 年 9 月 23 日 — tor 0.4.9.13*；*pt-spec v1*；*obfs4proxy 0.0.14*（*Debian*）；*Tor Project* 文档

本章目标

学完本章后，你将能够：

- 描述审查者可以通过哪些方式阻断 Tor；
- 解释网桥是什么，以及为什么网桥分发十分困难；
- 描述 tor 与可插拔传输程序之间的通信协议；
- 区分 obfs4、meek、Snowflake、WebTunnel 和 Conjure 所采用的策略；
- 在实验网络中运行 obfs4 网桥，并观察审查者能够看到什么（实验 10）。

此前介绍的所有机制，都以客户端能够连接到 Tor 网络为前提。但在许多国家，这个前提并不成立：连接 Tor 的流量受到阻断，有时仅在政治紧张时期如此，有时则是长期限制。本章以绕过网络审查为主题，为第二部分画上句号。这也是近年来 Tor Project 最活跃的工作领域之一。

16.1 Tor 如何遭到封锁

审查者通常是能够控制一国互联网接入服务商的政府，其手段包括：

封锁地址 中继列表是公开的，来自网络共识（第 12 章）。审查者可以下载共识并封锁所有中继的 IP 地址，也可以只封锁目录权威节点，以及客户端获取共识所依赖的中继。

封锁网站 即使 Tor 网络本身仍然可以连接，封锁 `torproject.org` 也会妨碍人们下载 Tor Browser。

识别协议 深度包检测（deep packet inspection，DPI）可以根据 TLS 握手、数据包大小或时间特征识别 Tor 连接，即使审查者尚不知道目的地址也一样。

主动探测 当 DPI 怀疑某服务器是未公开的中继时，审查者可以主动连接该服务器并尝试使用 Tor 协议通信。如果服务器响应，就将其封锁。

白名单 在极端情况下，审查者仅允许明确授权的协议或目的地，并阻断其他所有流量。

每种技术都有相应的反制措施，审查者与开发者之间的较量仍在继续。OONI 等项目则测量 Tor 在哪些地方、以何种方式遭到封锁（第 24 章）。

16.2　网桥

对抗地址封锁的方法是使用网桥（bridge）：一种不出现在公开共识中的中继。网桥扮演入口守卫的角色（第 13.3 节）。客户端把它作为第一跳，随后电路正常进入 Tor 网络。由于网桥地址没有公开，审查者不能仅靠读取公开列表就将其封锁。网桥向网桥权威节点发布描述符，由后者检查可达性，并维护一份不公开的列表。在已发布的网桥统计中，系统使用身份摘要的二次摘要，避免泄露身份。

客户端通过一条网桥配置行（bridge line）配置网桥。在实验网络中，客户端 `test011bc` 的 `torrc` 包含：

```
UseBridges 1
Bridge obfs4 127.0.0.1:9909 2557828B596B33AB02075FE7EAD86F2DC3CEAFD5 cert=4sKDQih...
    iat-mode=0
ClientTransportPlugin obfs4 exec /usr/bin/obfs4proxy
```

这一行包含传输类型、地址、网桥身份指纹（用于验证通信对端，参见 第 7 章），以及传输参数。其中 `cert=4sKDQih...` 仅为排版而截断的示例，不能直接复制使用；实际配置必须使用读者自己启动的网桥所生成的完整证书与身份指纹。

16.2.1　分发难题

只要审查者不知道网桥的地址，网桥就有用。但是要让用户使用网桥，就必须把地址交给他们；而审查者也可以假扮普通用户。因此不存在完美方案：分发系统既要让足够多的合法用户得到网桥，又要尽可能减少地址泄露给审查者的机会。Tor Project 通过多种渠道分发网桥，每种渠道都配有防止自动化收集地址的措施 [Theg]：

- 网站 `https://bridges.torproject.org/`；
- 发送邮件到 `bridges@torproject.org`；
- Telegram 机器人 `@GetBridgesBot`；
- 直接使用 Tor Browser：浏览器检测连接受阻后，会为用户所在国家请求合适的网桥（*Connection Assist*，基于名为 *moat* 的分发渠道）；
- 由能够直接识别用户身份的组织运营的私人分发渠道。

多年来，负责把网桥分配给不同渠道的系统是 BridgeDB。随后它被 **rdsys** 取代，迁移于 2024 年 10 月完成 [The24b]。网桥运营者可以通过 `BridgeDistribution` 选项选择分发渠道。

16.2.2　Lox

Lox [TG23] 利用信誉机制解决分发问题。新用户收到一个网桥地址和一个匿名凭证。如果该网桥持续未被封锁，凭证的等级就会逐渐提高，最高达到 4 级；从 2 级开始，用户还可以邀请其他人。如果网桥被封锁，用户可以获取另一个网桥，但会失去信誉。其基本假设是：审查者如果封锁自己拿到的每一个网桥，就无法提升等级；诚实用户则可以。凭证采用密码学技术，使系统本身也无法知道谁邀请了谁，从而保护用户的社交关系图（social graph）。Lox 正在 Tor Browser 的 alpha 版本中测试。

16.3　可插拔传输

只隐藏地址，还不足以抵御协议识别：流量本身也必须表现得不同。Tor Project 没有选择在审查者每次识别出新特征时都修改 tor，而是为转换流量的外部程序定义了一个接口：可插拔传输（pluggable transports，PT）[pt-spec § Architecture Overview]。

16.3.1 系统架构

PT 是由 tor 启动的独立进程，因此被称为托管进程。在客户端一侧，它向 tor 提供本地 SOCKS 代理。tor 把需要发往网桥的连接交给 PT；PT 转换流量后再发送。在网桥一侧，另一个 PT 监听公开端口，逆向转换流量，再把它转发到 tor 的本地 ORPort。PT 还可以通过名为 *Extended ORPort* 的附加协议，将客户端的实际地址告知 tor，用于统计。

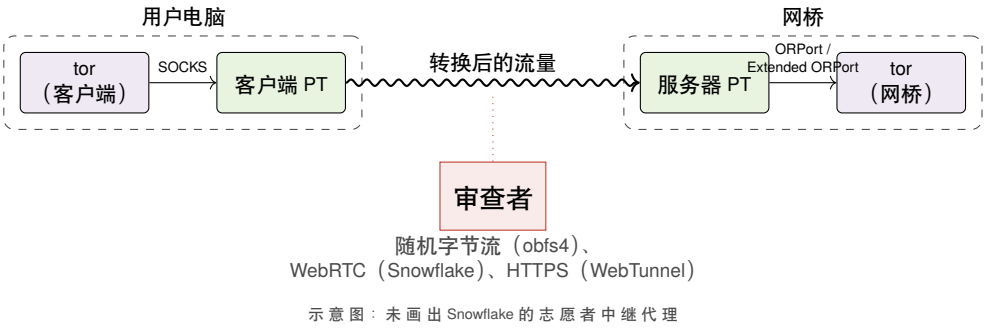


图 16.1: 可插拔传输的简化架构。在客户端一侧，tor 通过本地 SOCKS 代理与 PT 通信；在网桥一侧，PT 通过 ORPort 或配置后的 Extended ORPort 将还原的流量传给 tor。图中省略了 Snowflake 的志愿者中继代理。审查者在所监测的链路上看到转换后的流量，但仍有可能识别该流量。

16.3.2 tor 与 PT 之间的协议

tor 通过环境变量配置 PT，PT 则通过向标准输出写入文本行进行回应 [pt-spec § Specification]。我们可以在实验容器中像 tor 一样手动启动 `obfs4proxy`，验证这一机制。首先以客户端方式启动：

```
$ env TOR_PT_MANAGED_TRANSPORT_VER=1 TOR_PT_STATE_LOCATION=/tmp/pt \
      TOR_PT_EXIT_ON_STDIN_CLOSE=1 TOR_PT_CLIENT_TRANSPORTS=obfs4 obfs4proxy
VERSION 1
CMETHOD obfs4 socks5 127.0.0.1:40057
CMETHODS DONE
```

PT 声明协议版本，并告知 tor：自己提供 `obfs4` 传输，访问入口是一个自行选定本地端口的 SOCKS5 代理。作为服务器运行时，则通过环境变量指定公开监听地址和 ORPort 目的地：

```
$ env ... TOR_PT_SERVER_TRANSPORTS=obfs4 \
      TOR_PT_SERVER_BINDADDR=obfs4-127.0.0.1:19999 \
      TOR_PT_ORPORT=127.0.0.1:5104 obfs4proxy
VERSION 1
SMETHOD obfs4 127.0.0.1:19999 ARGS:cert=5
    W6vcbWbFGGZgabwm4C3WipezwnAfadoBhqfNVpY72xnkjB/Szczbxg8fLWJJ1uRlyjPSg,iat-mode=0
SMETHODS DONE
```

`ARGS:` 后面的参数正是客户端需要知道的内容，它们会出现在网桥配置行中。PT 还会把这些参数写入状态目录内的 `obfs4_bridgeline.txt`。环境变量 `TOR_PT_EXIT_ON_STDIN_CLOSE` 要求 PT 在 tor 关闭输入流时终止，避免子进程比启动它的进程活得更久。（规范中的示例因一处笔误，称 PT 将消息写入 “stdin”；实际上，我们刚刚看到，它写入的是标准输出。）

服务器示例中的 `env ...` 只是示意写法：省略号代表其他必要的环境变量，不能原样输入执行。同样，客户端输出中的 SOCKS5 端口来自一次参考运行，并非固定端口。

16.3.3 Lyrebird

Tor Browser 使用的传输程序是 **lyrebird**，它源自早期的 `obfs4proxy`，并支持 obfs4、meek、Snowflake 和 WebTunnel [Theg]。实验环境使用 Debian 软件包提供的 `obfs4proxy` 0.0.14，它仍与 obfs4 兼容。

16.4 不同的规避策略

当前使用的传输技术大致遵循两种相反的策略：让流量看起来不像任何可识别的协议，或者让流量看起来像一种获准使用的协议。

16.4.1 **obfs4**：看起来不像任何可识别的协议

由 Yawning Angel 设计的 **obfs4** 将流量转换为看似随机的字节序列：没有容易识别的头部、没有 TLS 握手，数据包长度也会变化。它的有效性主要来自三个设计选择：

- 密钥交换采用 ntor 的一种变体（第 9.6 节），其中 Curve25519 公钥通过 Elligator2 编码，使椭圆曲线上的点看起来与随机字符串无异。
- 客户端必须事先知道网桥的 `cert` 参数，其中包含其身份信息和公钥。没有该参数，观察者就无法构造有效握手，而网桥也不会回应。这能够抵御主动探测：好奇的审查者连接网桥时，不会得到可以识别的响应。
- `iat-mode`（*inter-arrival time*，包间到达时间）可以添加随机延迟和分片，以隐藏时间特征，但会牺牲一定性能。

obfs4 的局限也正来自它的优势：如果审查者实施白名单，或者封锁“任何看起来像随机数据的流量”，obfs4 就可能失效。一些国家确实采取了类似做法。

16.4.2 **meek**：隐藏在大型服务之后

meek [Fif+15] 利用域名前置（domain fronting）。客户端向大型内容分发服务建立 HTTPS 连接，在连接的可见部分使用一个无害的域名，而真正的目的地——通往 Tor 的网关服务器——只在加密数据中指定。要封锁 meek，审查者就必须封锁整个服务，并承担大量误伤其他流量的后果。但这一技术依赖服务提供商的许可；近年来，不少提供商已经禁止这种做法。此外，meek 所有流量都经过提供商的服务器，因此速度慢、成本高。

16.4.3 **Snowflake**：大量短暂存在的代理

Snowflake [Boc+24] 依赖成千上万名志愿者运行小型代理，例如浏览器扩展或服务器上的程序。客户端先通过域名前置等难以封锁的渠道联系协调服务器（broker），获得一个代理，然后通过 WebRTC 与代理连接。WebRTC 是浏览器视频通话所使用的协议。代理再把流量转发到 Tor Project 运营的 Snowflake 网桥。由于代理数量很多，地址又不断变化，封锁全部代理地址很不现实；而封锁 WebRTC，则意味着同时阻断视频通话。

16.4.4 **WebTunnel**：看起来像普通网站

Tor Project 于 2024 年 3 月宣布的 **WebTunnel**，把流量封装进行为类似 WebSocket 的 HTTPS 连接 [The24a]。网桥可以隐藏在普通网站后面，与网站共用域名、IP 地址和端口。反向代理只把针对秘密路径的请求转发给网桥，其他请求则由普通网站处理。WebTunnel 主要针对实施白名单的审查者；在这种环境下 obfs4 可能失效，而 WebTunnel 的流量外形却类似被允许的协议。

16.4.5 Conjure：隐藏在地址空间中

Conjure [Fro+19] 是一种折射网络（refraction networking）系统。它不依赖具有固定公开地址的网桥，而是在合作的互联网接入服务商处安装站点。这些站点观察发往未使用地址范围的流量，并将其改道送往 Tor。客户端从数百万个可能的地址中选择一个“幻影”地址建立连接。要封锁 Conjure，审查者就需要阻断互联网地址空间中的大片区域。Tor Project 的文档指出，Conjure 已成功通过测试，但仍需要进行稳定性改进，目前只在 Tor Browser 的 alpha 版本和部分应用中提供 [Theg]。

注意：没有一种传输技术在任何地方都有效

所有传输技术都有弱点，而且效果会因国家和月份而变化。例如，2025 年 4 月有用户在 Tor Project 论坛报告称，obfs4、meek 和 Snowflake 在中国无法使用，而 WebTunnel 虽能连接，却很快遭到封锁 [Tor25]。因此，Tor Browser 提供多种传输技术，并具有根据用户所在国家建议适当方式的功能。

16.5 实验：从审查者的视角观察 obfs4 网桥

实验 10：在实验网络中部署 obfs4 网桥

目标。启动包含 obfs4 网桥和使用该网桥的客户端的网络；捕获并比较普通 Tor 连接与 obfs4 连接的流量。
前置条件。实验网络（第 19 章）。镜像包含 Debian 软件包中的 `obfs4proxy` 和 `snowflake-client`。
预计时间。1–2 小时。

步骤 **1**：启动网桥网络。 文件 `labs/zh-Hans/实验室/chutney/networks/book-bridge` 在本书的测试网络中增加一台网桥权威节点（`test003ba`）、一台 obfs4 网桥（`test009br`），以及一台使用该网桥的客户端（`test011bc`）：

```
$ make -C labs/zh-Hans/实验室 clean
$ make -C labs/zh-Hans/实验室 start NETWORK=book-bridge
```

此前一次参考实验的客户端日志包含以下原始消息：

```
[notice] new bridge descriptor 'test009br' (fresh): $2557828B...~test009br
[warn] Only one bridge (transport: 'obfs4') is configured. You should
       have at least two for conflux, for any transport that is not 'snowflake'.
[notice] Bootstrapped 100% (done): Done
```

这表明客户端通过网桥自身获得了网桥描述符，并完成了引导。警告信息提醒我们，Conflux（第 31 章）使用两条并行电路，只有一台网桥就无法做到这一点。通过此客户端的 SOCKS 端口（9011）发送网页请求，效果与普通客户端相同。

步骤 **2**：审查者能够看到什么？ 在容器内，我们用 `tcpdump` 分别捕获 obfs4 网桥端口（9909）和普通 ORPort（5104）的流量。在客户端访问网桥的同时，使用实验 1 向 `test004r` 建立一个通道。随后用 `tshark` 分析抓包结果。以下是我们一次会话的结果：

```
== ORPort 5104 (TLS), 客户端发送的最初字节 ==
1533  16030105f8010005f403033e1234a0401520d1c52b...
== obfs4 9909, 客户端发送的最初字节 ==
1957  b566475349099523c53005e1004a392160f15d473e...
== 识别为 TLS 的数据包数量 ==
```

```
端口 5104: 83
端口 9909: 0
== obfs4 数据段长度（客户端 -> 网桥）==
1957 1677 1888 1891 1716 1716 3436 1395 1716 1748 1500 1194 ...
```

普通 ORPort 上的首字节十分鲜明：`16 03 01` 是 TLS 握手记录的开头，Wireshark 能把整段通信识别为 TLS。审查者可以检查这条 TLS 连接是否具有 Tor 特征，而中继地址本身也列在公开共识中。相比之下，发往 obfs4 网桥的字节没有可识别结构，没有数据包被识别为 TLS，数据段长度也不再直接对应 514 字节的信元（cell）。审查者只能看到发往未知地址的一条看似随机的数据流。

步骤 **3**：主动探测网桥。 假设我们是怀疑该地址运行着中继的审查者，尝试使用实验 1 连接网桥的 9909 端口：

```
TimeoutError: The handshake operation timed out
```

网桥完全不作响应：不发送错误消息，也不发送任何字节，直到我们的客户端在 15 秒后放弃。对于 obfs4 而言，TLS ClientHello 属于无效握手，而无效握手的响应就是保持沉默。探测该地址的审查者无法获知任何有用信息，甚至无法确认是否存在某个运行其他协议的服务。没有 `cert` 参数，它本来就无法构造有效的 obfs4 握手。

本章小结

- 审查者可以按地址封锁 Tor、识别协议、主动探测可疑服务器，或者只允许明确授权的流量。
- 网桥是不在公开列表中的中继。当前的 rdsys 分发机制和仍在探索中的 Lox 信誉机制，都在用户可访问性与防止审查者发现之间寻找平衡。
- 可插拔传输是转换流量的外部进程；tor 通过环境变量配置它们，并读取其响应（`CMETHOD`、`SMETHOD`）。
- obfs4 使流量看起来像随机数据，并抵御主动探测；meek、Snowflake、WebTunnel 和 Conjure 则把流量隐藏在审查者很难封锁的服务、协议或地址后面。
- 没有一种传输技术能在任何地点永久有效。

练习

练习 **16.1** ⋆ 网桥与普通入口守卫有什么不同？为什么使用网桥的客户端不再另外使用公开入口守卫？

练习 **16.2** ⋆ 根据策略（“看起来不像任何可识别的协议”或“看起来像允许使用的协议”）对 obfs4、meek、Snowflake、WebTunnel 和 Conjure 分类，并说明审查者要阻断每种传输技术需要封锁什么。

练习 **16.3** ⋆⋆ 假设审查者只能观察流量而不能主动探测，如何发现 obfs4 网桥？它可以利用哪些统计特征，`iat-mode` 又如何缓解这些特征？

练习 **16.4** ⋆⋆ 在 Lox 中，为什么审查者无法通过不断封锁网桥来提升信誉等级？它可以尝试如何绕过机制？这样做的成本来自哪里？

练习 **16.5** ⋆⋆ 在实验中分别捕获 obfs4 连接和普通 TLS 连接的前 10 KB，计算其香农熵（每字节的比特数）。你预期得到什么结果？为什么两者的差异可能很小？那么 TLS 究竟凭什么被识别？

练习 **16.6** ⋆⋆⋆ 尝试使用实验 1 访问自己的网桥的 obfs4 端口，会出现什么错误？随后以客户端身份运行 `obfs4proxy`，但提供一个错误的 `cert` 参数：你会在网桥日志中观察到什么？

延伸阅读

- *Pluggable Transport Specification (Version 1)*（pt-spec）[Thee]。
- L. Tulloch 与 I. Goldberg，*Lox: Protecting the Social Graph in Bridge Distribution* [TG23]。
- D. Fifield 等，*Blocking-resistant communication through domain fronting* [Fif+15]。
- C. Bocovich 等，*Snowflake, a censorship circumvention system using temporary WebRTC proxies* [Boc+24]。
- S. Frolov 等，*Conjure: Summoning Proxies from Unused Address Space* [Fro+19]。
- Tor Project 关于 WebTunnel 的公告 [The24a]，以及网桥支持文档 [Theg]。

第三部分

Tor 实践

第 17 章

安装 Tor 与配置 torrc

最后核验：*2026* 年 *9* 月 *23* 日 — *tor 0.4.9.13*；同版本的 *tor(1)* 手册

本章目标

学完本章后，你将能够：

- 根据使用目的选择适当的 Tor 安装方式；
- 阅读和编写 `torrc` 文件，并知道从哪里查找每个选项的文档；
- 把 tor 配置为客户端，设置 SOCKS 端口、流隔离和受保护的控制端口；
- 在投入使用前验证配置，并根据日志诊断问题。

本章开始介绍本书的实践内容。前面的章节解释了协议如何工作；现在我们来学习使用和配置实现这一协议的程序。本章提到的所有选项都收录在 tor 手册（`man tor`）中。手册是权威参考，因为默认值可能随版本改变。

17.1 应该安装哪一种 Tor?

Tor Browser 如果目的是浏览网页，它几乎总是正确选择。它包含预先配置的 tor 和经过修改的浏览器，以避免泄露会削弱网络保护能力的信息（第 23 章）。相反，直接把普通浏览器连接到 tor 仍可能暴露多种可用于识别用户的特征。

`tor` 软件包 适用于需要让其他应用程序通过 Tor 通信，或运营中继、洋葱服务的情况。Linux 发行版通常提供它，但不一定是最新版本。Tor Project 为 Debian 和 Ubuntu 维护自己的软件仓库，并为 Windows 与 macOS 发布包含 tor 和可插拔传输程序的 *Tor Expert Bundle*。

从源代码构建 适合研究 tor 或需要某个精确版本的场景，例如本书的实验网络（第 19.3 节）。

Arti 使用 Rust 编写的客户端（第 30 章），功能正逐渐成熟，但配置方法不同。

无论选择哪一种方法，都只能从官方网站下载软件，并验证其签名。如果 tor 已被攻击者篡改，它所提供的保护将失去意义。

17.2 torrc 配置文件

tor 从文本文件中读取配置；作为系统服务运行时，该文件通常位于 `/etc/tor/torrc`。格式很简单：每行写一个选项，后面跟随参数值。以 `#` 开头的行是注释。有些选项可以出现多次，例如可以指定多个 SOCKS 端口。也可以在命令行中使用双连字符传递选项：

```
$ tor --SocksPort 9150 --DataDirectory /tmp/tor-test
```

手册记录了数百种选项，`tor --list-torrc-options` 可以列出全部选项。发行版还可能通过 `--defaults-torrc` 指定另外一个默认配置文件。使用 `-f` 指定的文件具有更高的优先级。

17.2.1 使用前先检查

下面三个命令可以避免许多意外：

```
$ tor --verify-config -f /etc/tor/torrc     # 检查配置是否有效
Configuration was valid
$ tor --dump-config short -f /etc/tor/torrc # 查看与默认值不同的设置
$ tor --list-modules                        # 查看编译了哪些模块
relay: yes
dirauth: yes
dircache: yes
pow: yes
```

正如 第 15 章 所述，最后一个命令可以显示是否支持洋葱服务的工作量证明等功能。上面保留的是软件的实际英文输出，而非人为翻译后的字符串。

17.3 客户端配置

客户端只需要很少的配置。事实上，最简配置文件可以是空文件：默认情况下，tor 会在本机的 9050 端口启动 SOCKS 代理。下面是一个写得更明确、带注释的系统服务配置示例：运行 tor 的服务账户需要能够访问 `/var/lib/tor` 和 `/var/log/tor`，不能假定普通用户直接运行时也具有这些权限。

```
# tor 存储状态、密钥和缓存的位置
DataDirectory /var/lib/tor

# 供应用程序使用的 SOCKS 端口（只能在本机监听！）
SocksPort 127.0.0.1:9050

# 使用 cookie 保护的控制端口（参见第 18 章）
ControlPort 127.0.0.1:9051
CookieAuthentication 1

# 以 notice 级别记录日志
Log notice file /var/log/tor/notice.log
```

注意：不要暴露 SOCKS 端口

手册明确指出：SOCKS 协议既不加密，也不认证。如果 `SocksPort` 监听网络上其他主机可以访问的地址，这台计算机就会变成开放代理，而且应用程序的请求会以明文形式暴露在本地网络上。

17.3.1 流隔离

多个流可以共用一条电路。这样效率较高，但出口中继也就可能把同一用户进行的不同活动关联起来。如果个人邮箱和使用化名的论坛共用一条电路，出口中继便可以看出两种活动来自同一个人。因此，tor 提供流隔离：用规则决定哪些流可以共用电路。这些规则通过 `SocksPort` 的选项表达：

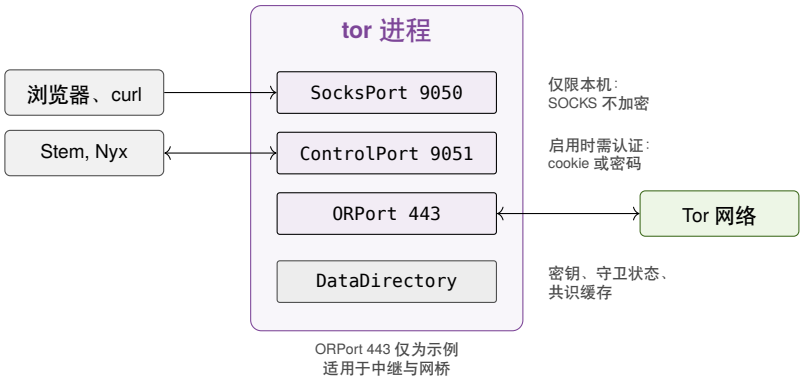


图 17.1: tor 进程的示例端口和目录。9050 上的 SOCKS 服务于本机应用；ControlPort 9051 不是默认开启的，启用后必须进行身份认证。这些本地监听端口不应暴露到网络。DataDirectory 是文件目录而非端口。ORPort 443 只是中继或网桥的一种配置示例，并非固定端口。

`IsolateClientAddr` 来自不同客户端地址的流不能共用电路（默认启用）。

`IsolateSOCKSAuth` 使用不同 SOCKS 凭据的流不能共用电路（默认启用）。

`IsolateDestAddr`、**`IsolateDestPort`** 根据目的地址和目的端口隔离。

`IsolateClientProtocol` 根据客户端协议隔离（SOCKS4、SOCKS5、DNS 等）。

默认情况下，不同 SOCKS 监听端口也互相隔离：每个监听器都有自己的会话组。可以通过 `SessionGroup` 改变这一行为，允许不同端口共用电路。应用最广泛的是 SOCKS 认证隔离：Tor Browser 为不同网站发送不同的 SOCKS 凭据，因此每个网站会使用不同的电路。我们将在 第 18 章 中观察到这一点。

电路也不会被无限期重复使用。经过 `MaxCircuitDirtiness` 规定的时间后——默认从首次使用起计算 10 分钟——该电路就不再接收新流，tor 会建立另一条电路。

17.3.2 谨慎选择中继

`EntryNodes`、`ExitNodes` 和 `ExcludeNodes` 选项可以通过指纹、昵称、地址或国家代码强制选择或排除中继。即使会损害可用性，`StrictNodes` 也可以强制执行 `ExcludeNodes`。这些工具适用于实验和特殊场景。但如果只是为了“感觉更安全”而使用它们，结果通常适得其反：缩小中继选择范围，会降低路径多样性，让用户的行为更加显眼，并破坏加权路径选择和守卫机制原本提供的保护（第 13 章）。

17.3.3 其他端口

`DNSPort` tor 响应 DNS 请求，并通过 Tor 网络解析域名（第 11.1.4 节）。

`TransPort` 接受由系统防火墙重定向的透明代理 TCP 连接。Whonix 等系统使用这一机制，把应用程序流量送入 Tor（第 23 章）。

`AutomapHostsOnResolve` 为经 DNS 查询的 `.onion` 名称分配虚拟地址，使不支持 SOCKS 的应用也可以通过 `TransPort` 访问洋葱服务。

17.4 控制端口

其他程序可以通过 `ControlPort` 在 tor 运行期间控制它：读取状态、请求建立新电路，或者动态创建洋葱服务。该接口功能非常强大，因此必须受到保护。可用的两种认证方式是：

Cookie　当 `CookieAuthentication` 设置为 1 时，tor 在数据目录中写入一个包含 32 个随机字节的文件。任何能读取该文件的进程都能通过认证，因此安全性取决于文件权限。

密码　使用 `HashedControlPassword` 时，tor 存储通过 `tor --hash-password` 生成的密码哈希：

```
$ tor --hash-password test
16:458528FE0C0AF36260B5071446823FB789212185D7EE7F615BC120D55D
```

这里的哈希是使用固定盐值、针对示例密码 `test` 验证过的演示结果。实际执行命令时，tor 会随机生成盐值，因此即使密码不变，输出通常也不同。切勿在真实配置中复用示例密码或哈希。

与 SOCKS 端口一样，控制端口绝不能暴露给网络。有关控制协议的详细内容，参见 第 18 章。

17.5　日志

tor 以不同详细程度记录活动：`err`、`warn`、`notice`、`info` 和 `debug`。日常运行使用 `notice` 即可。研究协议问题时，`info` 和 `debug` 很有价值，正如我们在实验中所做的那样，但它们也会产生大量日志。默认启用的 `SafeLogging` 会把日志中的地址和目的地替换为 `[scrubbed]`。遗留的日志文件不应该成为用户活动记录。为了观察所有细节，实验网络特意关闭这项保护（`SafeLogging 0`）。

启动过程中最有用的是引导（bootstrap）消息。它们显示连接网络的各个阶段，最后到达 `Bootstrapped 100% (done): Done`。如果启动长期停留在某一百分比，相应阶段可以帮助定位故障。例如，连接第一个中继时受阻，通常意味着受到审查或防火墙限制，可以考虑使用网桥（第 16 章）。

本章小结

- 浏览网页应使用 Tor Browser；`tor` 软件包则适合其他应用、中继和洋葱服务。
- `torrc` 每行包含一个选项；手册是权威参考，`--verify-config` 可以验证配置。
- SOCKS 和控制端口只应监听 localhost。控制端口须使用 cookie 或密码保护。
- 流隔离可以避免无关活动共用一条电路。Tor Browser 通过为不同网站使用不同的 SOCKS 凭据来实现这一点。
- 强制选择或排除中继通常会降低安全性；`SafeLogging` 用于保护日志中的敏感信息。

练习

练习 **17.1** ⋆　在实验容器内，创建一个选项名称存在拼写错误的 `torrc`，使用 `--verify-config` 检查。你得到了什么消息？然后在实验网络客户端的 `torrc` 上运行 `--dump-config short`。

练习 **17.2** ⋆⋆　某用户使用同一个 `SocksPort`，既以真实身份登录个人邮箱，也以化名访问论坛。出口中继可能推断出什么？请提出两种能避免此问题的配置。

练习 **17.3** ⋆⋆　为什么 `ExitNodes {it}`（只使用意大利出口中继）可能降低访问意大利网站的意大利用户的匿名性？请结合 第 4 章 中的威胁模型说明。

延伸阅读

- tor 手册 `man tor`，以及源码树中的 `doc/man/tor.1.txt`。
- Tor Project 关于安装和软件仓库的支持文档：`https://support.torproject.org/`。

第 18 章

Control Port 与 Stem

最后核验：*2026 年 9 月 23 日 — tor 0.4.9.13*；*control-spec*；*Stem 1.8.2*

本章目标

学完本章后，你将能够：

- 通过控制端口与 tor 通信，包括手动发送命令；
- 使用主要命令：`GETINFO`、`SETCONF`、`SIGNAL`、`ADD_ONION` 以及事件订阅；
- 使用 Stem 库编写 Python 程序；
- 在运行中的系统上观察流隔离和切换身份的影响（实验 11）。

Tor Browser 可以显示当前网页使用的电路；Nyx 等工具能够显示中继的带宽与连接；网络测量系统则会按需创建电路。这些应用使用的都是同一个接口：通过 `ControlPort`（第 17.4 节）访问 tor 的控制协议。

18.1 控制协议

这是一种基于文本的协议，与电子邮件协议有些相似 [control-spec § Message format]。客户端每行发送一条命令，tor 则用以三位数字代码开头的行回应。代码 `250` 表示成功；代码后的连字符（`250-`）表示回应尚未结束，空格（`250 `）表示这是最后一行；`+` 则引出一个多行文本块，该文本块由只包含一个句点的行结束。代码 `5xx` 表示错误，而 `650` 表示异步事件。

我们可以通过一个简单的 TCP 连接访问实验网络客户端的 8008 端口：

```
> PROTOCOLINFO 1
250-PROTOCOLINFO 1
250-AUTH METHODS=COOKIE,SAFECOOKIE COOKIEFILE="/lab/net/.../008c/control_auth_cookie"
250-VERSION Tor="0.4.9.13"
250 OK
> AUTHENTICATE 52977550adebb73908610d7fa3355aba3fbdbdaef01d5ef86523d3232b87e5fa
250 OK
> GETINFO status/bootstrap-phase
250-status/bootstrap-phase=NOTICE BOOTSTRAP PROGRESS=100 TAG=done SUMMARY="Done"
250 OK
```

这是此前一次示例运行的记录：目录、版本及认证 cookie 均取决于具体实例。切勿公开真正的控制端口 **cookie**，因为持有它便可能通过控制端口认证。`PROTOCOLINFO` 是认证前允许使用的少数命令之一（其他几个是 `AUTHCHALLENGE`、`AUTHENTICATE` 和 `QUIT`）。它会报告支持的认证方法以及 cookie 文件的路径。采用 `COOKIE` 认证时，客户端以十六进制形式发送该文件内容。`SAFECOOKIE` 则避免直接传输 cookie：客户端与 tor 交换随机值，并通过 HMAC 证明自己知道 cookie，从而防止假冒 tor 的程序窃取该凭据。

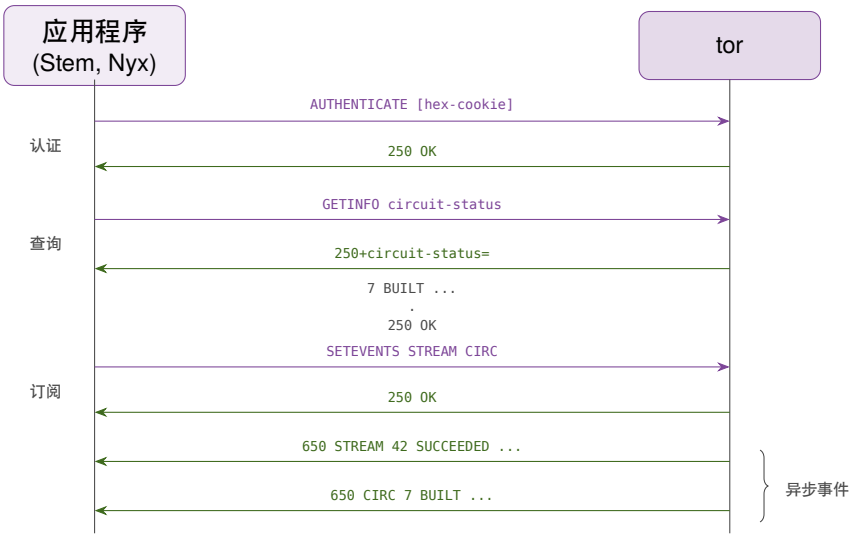


图 18.1: 简化的控制协议会话：COOKIE 认证（图中使用占位符）后，GETINFO circuit-status 可返回多行数据，随后由单独一行的句点和 250 OK 结束。SETEVENTS 以 250 OK 确认订阅；异步的 650 事件可能与其他命令的回应交错。编号和数据仅为示例。

18.1.1 从 tor 的视角观察电路

`GETINFO circuit-status` 列出客户端当前的电路。下面是一条缩写记录：

```
3 BUILT $50160F8F...~test004r,$12415FEB...~test005r,$9449F36E...~test007e
  BUILD_FLAGS=NEED_CAPACITY,NEED_UPTIME PURPOSE=CONFLUX_LINKED
  CONFLUX_ID=DE224696F58F72950D70DDA2D7981726 CONFLUX_RTT=42538
```

每个中继都由 RSA 身份指纹和昵称表示。Conflux 电路成对出现，具有相同的 `CONFLUX_ID`，两个“分支”从不同入口守卫出发。tor 测量它们的往返时延（`CONFLUX_RTT`，单位为微秒），决定由哪个分支传输数据（第 31 章）。tor 的源码 `src/feature/control/control_fmt.c` 中有一个值得注意的细节：`CONFLUX_ID` 只暴露这对电路秘密标识符的一半。知道完整标识符的人，可能从出口端向该电路对添加分支并注入数据。

18.1.2 主要命令

命令	用途
`GETINFO`	读取版本、引导状态、电路、流、共识、描述符等信息
`GETCONF`、`SETCONF`	读取和修改运行中的配置
`SIGNAL`	发送信号：`NEWNYM`（新身份）、`RELOAD`、`SHUTDOWN` 等
`SETEVENTS`	订阅事件：`CIRC`、`STREAM`、`BW`、`HS_DESC` 等
`EXTENDCIRCUIT`	沿控制程序指定的路径建立电路
`ATTACHSTREAM`	将流附加到电路（可配合 `__LeaveStreamsUnattached` 使用）
`ADD_ONION`、`DEL_ONION`	无须编辑 `torrc`，动态创建和移除洋葱服务
`HSFETCH`	下载洋葱服务描述符（用于 第 14.9 节）

注意：掌握控制端口，就能控制 tor

控制端口可以指示 tor 使用哪些中继、建立单跳电路，并暴露所有流的目的地。有权访问控制端口的恶意程序能够破坏用户的匿名性。因此，Tails 和 Whonix 等系统会在应用程序与控制端口之间放置过滤器（onion-grater），仅放行严格必要的命令。

18.2 Stem

Stem 是 Tor Project 为控制协议开发的 Python 库。它封装了网络报文格式的细节，负责解析回应与事件，并提供方便使用的对象。在本书所采用的版本基准中，最新的 Stem 版本为 1.8.2，发布于 2023 年 6 月。该库与当前的 tor 版本兼容，许多工具仍在使用它，但已有一段时间没有发布新版本。有时，较新的功能需要直接发送原始命令（`Controller.msg`）。Arti 使用的是另一种接口：Arti RPC（第 30 章）。

下面是一个最简示例，我们已经在 第 19.7.3 节 中使用过：

```
from stem.control import Controller

with Controller.from_port(port=8008) as c:
    c.authenticate()                    # 自动寻找 cookie 或请求输入密码
    print(c.get_version())
    for circ in c.get_circuits():
        print(circ.id, circ.purpose, [name for _, name in circ.path])
```

注册一个接收事件的函数。下面是可独立运行的示例：在控制连接保持打开时注册监听器，再等待十秒以接收异步事件。等待时长仅用于演示：

```
import time
from stem.control import Controller, EventType

def on_circuit(event):
    print(event.id, event.status, event.purpose)

with Controller.from_port(port=8008) as c:
    c.authenticate()
    c.add_event_listener(on_circuit, EventType.CIRC)
    time.sleep(10)
```

Stem 在自己的事件线程中调用该函数。事件线程只有一个，并且依次调用所有已注册的监听器。因此，执行缓慢的回调会延迟后续事件。如果处理需要进行大量计算，应将计算委托给另一个线程。

18.3 实验：实时观察流隔离

实验 11：流隔离与切换新身份

目标。在实验网络客户端上使用 Stem，根据不同请求的 SOCKS 凭据检查哪些请求共用电路，并观察发送 `SIGNAL NEWNYM` 后发生的变化。
前置条件。实验网络正在运行，并有一台本地 Web 服务器监听 8080 端口。在容器中运行 `labs/zh-Hans/实验室/isolation.py`；容器已包含 Stem 和 `curl`。
预计时间。1 小时。

步骤 **1**：记录流。 我们订阅 `STREAM` 事件，每当一个流成功建立，就记录它被附加到哪条电路。必须按照目的地址过滤事件，因为 tor 也会为自己的任务创建流，例如下载目录文档。

```
assigned = []
def on_stream(ev):
    if ev.status == StreamStatus.SUCCEEDED and ev.target == DESTINATION:
        assigned.append((ev.id, ev.circ_id, ev.target))

c.add_event_listener(on_stream, EventType.STREAM)
```

这段代码摘自 labs/zh-Hans/实验室/isolation.py：完整脚本还会定义 DESTINATION、导入 StreamStatus 并保持控制连接 c 打开。因此应运行完整脚本，而非单独执行此片段。

步骤 **2**：使用不同凭据发起请求。 我们用 curl 发送四个请求：其中两个使用 SOCKS 凭据 site-a.example:x1，一个使用 site-b.example:x2，另一个不带凭据。Tor Browser 以每个页面所属主站的域名作为 SOCKS 用户名：

```
$ curl --proxy-user site-a.example:x1 --socks5-hostname 127.0.0.1:9008 http
    ://127.0.0.1:8080/
```

步骤 **3**：实验结果。 以下数据取自此前的参考实验；Conflux 标识符、电路路径和 NEWNYM 延迟都不是固定值。

```
凭据 site-a.example  -> conflux AADCE71E  ['test003r', 'test004r', 'test007e']
凭据 site-a.example  -> conflux AADCE71E  ['test003r', 'test004r', 'test007e']
凭据 site-b.example  -> conflux 31B1E3E0  ['test003r', 'test004r', 'test007e']
凭据 -               -> conflux 35FD16D0  ['test003r', 'test004r', 'test007e']
 (NEWNYM 延迟 6 秒)
NEWNYM 后, site-a.example  -> conflux F7011ABE
```

上面保留了实验脚本实际使用的英文输出标签，以便读者直接对照执行结果。两个访问“网站 A”的请求虽然发生在不同时间，却使用了同一个隧道。访问“网站 B”的请求和没有认证的请求则分别使用不同隧道。打印出的路径是隧道两个分支中的一个；由于实验网络的中继很少，这些路径恰好相同。下面将解释为什么应该观察隧道，而不是某条单独的电路。

深入说明：单条电路并不够：Conflux 隧道

如果多次重复实验，使用相同凭据的两个请求偶尔会出现在编号不同的电路上。这并非隔离失败。查询 tor 的完整电路列表可以发现，几乎所有客户端电路的用途都是 CONFLUX_LINKED，并且以相同 CONFLUX_ID 成对出现。它们是一条隧道的两个“分支”（第 31.4 节），STREAM 事件可能报告其中任何一个。隔离因此针对的是隧道，而不是单个分支。只要能够获得相关信息，本书脚本就会使用 CONFLUX_ID 对它们进行分组。
还需要注意两点，这两点都能从日志中验证：

- SIGNAL NEWNYM 每十秒最多执行一次（见 src/core/mainloop/mainloop.c 中的 MAX_SIGNEWNYM_RATE）。过早发出的请求会被延迟，tor 在日志中写入“Rate limiting NEWNYM request: delaying by N second(s)”。如果不等待，上一次实验留下的 NEWNYM 可能在新实验进行到一半时才生效。
- STREAM 事件还包括 tor 为自身用途建立的流，例如目录文档下载。因此需要根据目的地址过滤。

即便采取了这些措施，负载较重的小型网络仍可能让 tor 为第二个请求选择刚刚建立的新隧道。这就是本书的自动检查器需要重复进行实验的原因。

注意：实验中犯过的错误

首次运行实验时，我们看到四个请求全部使用相同电路，看起来流隔离似乎没有生效。问题其实出在脚本中：我们把 --proxy-user 紧接在 curl 的 -o 选项之后，后者把它解释成了输出文件名，因此根本没有发送任何凭据。在断言系统行为与规范不一致之前，应先确认实验确实执行了自己以为正在执行的操作。

本章小结

- 控制协议基于文本：单行命令、三位数字回应代码，以及以 650 为代码的异步事件。
- 使用 cookie（优先采用 `SAFECOOKIE`）或密码认证。能控制该端口，就能控制 tor。
- Stem 为命令和事件提供 Python 接口。它运行稳定，但自 2023 年以来没有发布新版本。
- SOCKS 凭据隔离为每个网站建立独立隧道；在 Conflux 下，一条隧道包含两条电路。`NEWNYM` 最多每十秒切换一次新隧道。

练习

练习 **18.1** ⋆ 解释对 `GETINFO pippo` 的回应 `552 Unrecognized key "pippo"`。错误代码 `5xx` 与事件代码 `650` 有什么不同？

练习 **18.2** ⋆⋆ 使用 Stem 的 `create_ephemeral_hidden_service`，在实验网络客户端上创建一个临时洋葱服务，把它的 80 端口转发到本地 8080 端口的服务器。使用 `curl` 访问该服务。服务密钥存放在哪里？

练习 **18.3** ⋆⋆ 编写程序，在通过实验网络下载大文件时，利用 `BW` 事件每秒显示客户端的入站和出站带宽。

练习 **18.4** ⋆⋆⋆ 结合 `__LeaveStreamsUnattached` 1、`EXTENDCIRCUIT` 和 `ATTACHSTREAM`，让一次请求沿你指定的路径传输（例如 `test004r`、`test005r`、`test006e`）。为什么这一能力在真实环境中可能危害匿名性？

延伸阅读

- *Tor's control protocol*（control-spec）[Thee]：所有命令和事件。
- Stem 官方文档：`https://stem.torproject.org/`。

第 19 章

私有 Tor 网络：Chutney 与 Shadow

最后核验：*2026 年 9 月 23 日 — tor 0.4.9.13*；*Chutney* 提交 `6cc1588`；*Shadow 3.3.0*

本章目标

学完本章后，你将能够：

- 解释为什么 Tor 实验应在私有网络中运行，而不是在公共网络上进行；
- 使用两条独立命令构建并启动本书所有实验使用的网络；
- 找出测试网络与真实网络之间不同的配置选项；
- 浏览节点状态文件，包括密钥、共识文档和日志；
- 利用 Stem 检查客户端电路，并通过实验网络发送普通 Web 流量和洋葱服务流量；
- 理解什么时候应选用 Shadow 等模拟器，而不是 Chutney 这样的仿真网络。

本章虽然放在第三部分，但建议提前阅读：本书所有实验——从 第 7 章 的实验 1 开始——都运行在这里搭建的网络中。按照第二部分的顺序学习时，可以先阅读 第 19.1 节和第 19.3 节，以后再回来学习本章的其他内容。

19.1 为什么需要私有网络？

公共 Tor 网络由数千台志愿者运营的中继构成，每天有数百万人使用，其中不少用户依靠它保障自身安全。在该网络上开展实验会产生两个问题：一个涉及伦理，另一个涉及实验方法。

伦理问题。 任何测量、改变，甚至仅仅观察公共网络真实流量的实验，都可能涉及没有同意参与研究的用户，并给他们带来风险。即使看似无害的实验，例如让中继记录超过必要范围的信息，也可能生成一旦泄露就能用于识别某位用户的数据。因此，Tor Project 设立了 *Tor Research Safety Board*（Tor 研究安全委员会），审查研究人员的实验计划，并倡导一项基本原则：如果研究问题可以在不涉及真实用户的情况下得到解答，就应当这样做。我们将在 第 29 章 中再次讨论这些原则。

实践问题。 研究人员几乎无法控制公共网络：无法预测会选择哪些中继，电路不断变化，共识每小时更新，实验也难以在完全相同的条件下重复。私有网络则不同：我们能够选择节点数量、读取详细日志，在几分钟内停止和重新启动整个网络，并反复执行实验，直到理解结果。

注意：实验规则

本书凡涉及自行构造 cell、改变节点行为或捕获流量的实践操作，都必须仅在私有网络中进行。教学用的 `torlab` 代码不能提供实际的匿名保护，绝不可用于公共 Tor 网络。

19.2　仿真还是模拟?

Tor Project 使用两种方法构建实验网络。

仿真（**emulation**）在同一台计算机上运行多个真正的 `tor` 进程，进程之间通过环回接口通信。时间是真实时间，运行的是真实代码，网络通信由操作系统负责。这就是 **Chutney** 的工作方式。Chutney 是 Tor Project 用于 tor 与 Arti 集成测试的工具。

模拟（**simulation**）在离散事件模拟器中运行真实程序，由模拟器拦截系统调用，并提供虚拟网络和虚拟时钟。这是为大规模研究 Tor 而开发的 **Shadow** 所采用的方法 [JH12]。

	Chutney（仿真）	Shadow（模拟）
典型规模	几个到几十个节点	数百到数千个节点
时间	真实时间	虚拟时间、可确定性复现
网络	环回接口，没有真实延迟或丢包	可配置延迟和带宽的拓扑
可重复性	受操作系统调度影响	相同随机种子下可以完整复现
典型用途	开发、测试、教学	性能与攻击研究
本书用途	所有实验	第 19.9 节 与第五部分

表 19.1: Chutney 与 Shadow 的比较。

对于学习协议而言，Chutney 是自然的选择：它轻量、几分钟内便能启动，还允许我们直接与运行中的节点交互。因此本章将重点介绍 Chutney，并在最后简要讨论 Shadow。

19.3　本书的实验环境

为了避免操作系统之间的差异，整个实验环境被封装进一个 Docker 镜像，定义文件位于仓库中的 `labs/zh-Hans/实验室/Dockerfile`。镜像会：

- 从官方源代码编译本书的参考版本 tor 0.4.9.13，并使用 `--enable-gpl` 启用洋葱服务工作量证明模块（第 15 章）；
- 安装固定提交版本的 Chutney、Stem，以及 第 20 章 使用的流量捕获工具（`tcpdump`、`tshark`）。

固定版本非常重要：一本逐字节解释协议消息的书，应尽量确保读者能观察到与书中相同的字节。

深入说明：tor 发布新版本时怎么办？

本章和第二部分展示的输出，来自运行 tor 0.4.9.13（本书参考版本）的实验网络。其中一部分最初在仅早几天发布的上一版本 0.4.9.12 上记录，随后又重新复现：脚本 `scripts/verify_labs.py` 会在实时网络中重新运行全部实验并检查结果，在两个版本上得到相同结论（目前共 53 项检查，网桥实验另有 14 项）。该脚本运行的是意大利语原版的实验，位于 `labs/it`；本版的实验代码译自其中，列表中的名称和消息与本版代码一致。只有与单次运行相关的元素会变化：密钥、指纹、`.onion` 地址、时间戳和证书长度，后者甚至在不同中继之间也不相同。借助同一脚本，你可以在自己使用的版本上验证书中描述的内容是否仍然成立。

19.3.1 运行要求

你需要一台安装了 Docker 的 Linux 计算机，以及大约 1 GB 的可用磁盘空间。macOS 和 Windows 上的 Docker Desktop 也可使用，不过本实验采用的主机网络模式在这些系统上的行为不同；此时请在容器内部执行实验（第 19.7.4 节）。实验网络会占用以 5100、7100、8000 和 9000 开头的主机端口，请先确认相关端口没有被占用。

19.3.2 构建镜像并启动网络

从仓库根目录执行（不要先切换到 `labs/zh-Hans/实验室`）：

```
$ make -C labs/zh-Hans/实验室 image        # 编译 tor 并准备 Chutney（只需首次执行）
$ make -C labs/zh-Hans/实验室 start        # 配置并启动网络，等待引导完成
```

构建镜像需要几分钟。编写本章时使用的计算机，网络启动大约耗时两分半钟，最后得到以下摘要（已简化）：

```
  test006e:
    bootstrap progress: 100 (SUCCESS), done, Done
    desc publishing: 100 (SUCCESS), all nodes, DESC MD MD_CONS NS_CONS
  test008c:
    bootstrap progress: 100 (SUCCESS), done, Done
  test009h:
    bootstrap progress: 100 (SUCCESS), done, Done
    oniondesc publishing: 200 (ONIONDESC_PUBLISHED), hidden service v3
```

所有节点均已完成引导；各中继的描述符已经发布，并被网络共识收录；洋葱服务也发布了自身的描述符。接下来使用 `make -C labs/zh-Hans/实验室 verify` 运行 Chutney 的检查：分别通过出口和洋葱服务传输数据，并验证内容未发生改变：

```
  not-done:0 successes:4 failures:0
Transmission: Success
```

`make -C labs/zh-Hans/实验室 status` 用于查看节点状态；`make -C labs/zh-Hans/实验室 shell` 打开容器中的命令行；`make -C labs/zh-Hans/实验室 stop` 停止网络；`make -C labs/zh-Hans/实验室 clean` 则会一并删除生成的数据。

19.4 网络拓扑

本书实验网络的配置写在一个简短的 Python 文件 `labs/zh-Hans/实验室/chutney/networks/book` 中，使用 Chutney 的配置语言：

```
Authority = Node(tag="a", authority=1, relay=1)
Middle    = Node(tag="r", relay=1)
Exit      = Node(tag="e", relay=1, exit=1)
Client    = Node(tag="c", client=1, launch_phase=2)
Onion     = Node(tag="h", hs=1, launch_phase=2)

NODES = (Authority.getN(3) + Middle.getN(3) + Exit.getN(2)
         + Client.getN(1) + Onion.getN(1))
ConfigureNodes(NODES)
```

每个 `Node` 都是一种模板，`getN` 创建相应数量的实例。客户端和洋葱服务在第二阶段启动（`launch_phase=2`），因为此时目录权威节点已生成共识。Chutney 按顺序为节点编号，组合编号和标记形成名称：`test000a`、…、`test009h`。节点编号还决定端口分配，如 图 19.1 所示。

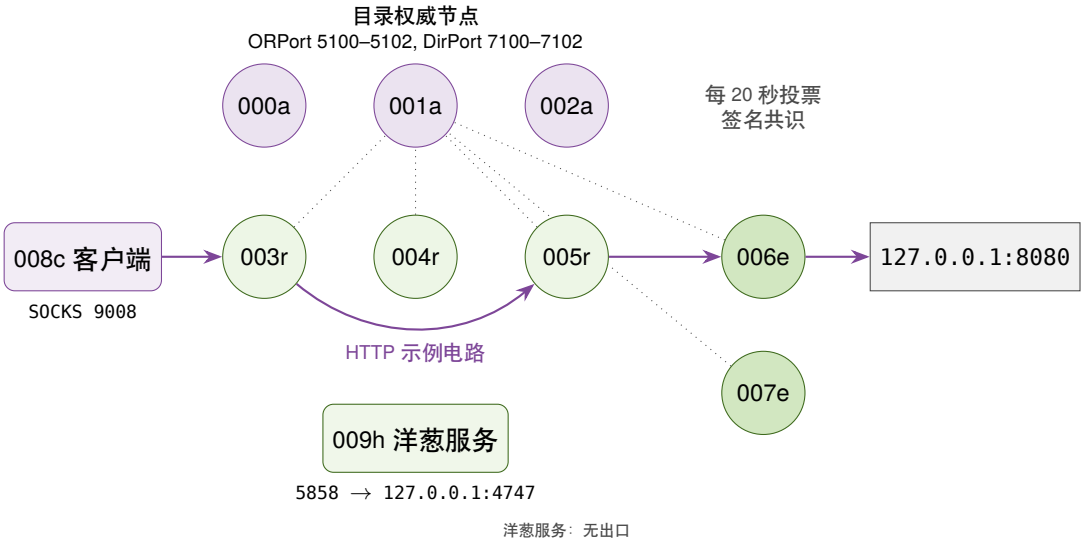


图 19.1: Chutney 实验室在 `127.0.0.1` 上的拓扑：三台兼任中继的目录权威节点、三台中间中继、两台出口中继、客户端和洋葱服务。只有权威节点及其他中继（包括出口）使用 ORPort $5100+n$；所有节点都有 ControlPort $8000+n$。权威节点另有 DirPort $7100+n$，客户端使用 SocksPort 9008，洋葱服务将虚拟端口 5858 映射到 `127.0.0.1:4747`。图中的 HTTP 出口电路与洋葱服务不同；洋葱服务不经过出口中继。绑定环回地址不代表容器网络隔离。

节点	角色	ORPort	ControlPort	其他
`test000a–002a`	目录权威	5100–5102	8000–8002	DirPort 7100–7102
`test003r–005r`	中间中继	5103–5105	8003–8005	
`test006e–007e`	出口	5106–5107	8006–8007	
`test008c`	客户端	—	8008	SocksPort 9008
`test009h`	洋葱服务	—	8009	虚拟端口 5858

深入说明：为什么需要三台目录权威节点？

一份共识必须获得过半数目录权威节点的签名（第 12 章）。如果只有一台权威节点，我们就无法观察投票过程；有了三台，不仅可以观察投票，还能在练习中了解其中一台停止工作后会发生什么。公共网络有九台目录权威节点，此外还有独立的网桥权威节点。它们的列表被编译进 tor 本身（`src/app/config/auth_dirs.inc`）。

19.5　测试网络不等于真实网络

Chutney 为每个节点生成独立的 `torrc`。这些文件清楚地展示了，为使微型 Tor 网络正常运行，必须修改哪些配置，因此也揭示了公共网络中的哪些前提不再适用。下面是一台目录权威节点配置文件 `net/nodes/000a/torrc` 中最重要的选项：

```
TestingTorNetwork 1
AuthoritativeDirectory 1
V3AuthoritativeDirectory 1
AssumeReachable 1
V3AuthVotingInterval 20 seconds
V3AuthVoteDelay 4
V3AuthDistDelay 4
TestingDirAuthVoteGuard *
```

```
TestingDirAuthVoteHSDir *
```

TestingTorNetwork 启用测试模式。该选项只能与显式指定的目录权威列表（DirAuthority）一起使用，防止测试节点意外加入公共网络。它还会修改许多默认值，以加快运行速度。

V3AuthVotingInterval 在真实网络中，目录权威节点每小时为新共识投一次票。而在这里，每 20 秒就投一次；共识本身也反映了这一点：

```
valid-after 2026-09-23 14:37:40
fresh-until 2026-09-23 14:38:00
valid-until 2026-09-23 14:38:20
```

AssumeReachable 中继通常要验证自己能从外部访问，才会公布自身信息。在环回接口上，这项测试没有意义，因此被跳过。

TestingDirAuthVoteGuard * 在真实网络中，中继需要连续数周稳定运行，并具备足够带宽，才会获得 Guard 标志；而这里，权威节点会给所有中继分配该标志。因此，本实验共识中的每台中继都是潜在入口守卫，与公共网络不同。

出口中继的 torrc 还包含：

```
ExitPolicyRejectPrivate 0
ExitPolicy accept private:*
```

出口默认拒绝连接私有地址和环回地址，以免使所在的本地网络暴露于风险之中。但在本实验里，127.0.0.1 就是唯一可用的“互联网”，因此必须反转这条策略。

注意：不要在其他环境中照搬这些选项

Testing… 类选项、AssumeReachable，尤其是 ExitPolicy accept private:*，只能用于测试网络。在公共中继上采用这些设置，不仅配置不当，还可能危及宿主机所在的网络。

19.6 节点目录内部有什么?

每个节点都有自己的数据目录，位于 labs/zh-Hans/实验室/net/nodes/NNNx/。以目录权威节点为例，目录中包括：

文件或目录	内容
torrc	Chutney 生成的配置文件
keys/	节点身份密钥和洋葱密钥（第 5 章和第 6 章）
fingerprint、fingerprint-ed25519	RSA 和 Ed25519 身份指纹
cached-consensus、cached-microdesc-consensus	两种共识格式（第 12 章）
cached-microdescs.new	已下载的微描述符
v3-status-votes	目录权威节点的投票（仅权威节点）
sr-state	共享随机性协议的状态（仅权威节点）
notice.log、info.log	*notice* 和 *info* 级别的日志
control_auth_cookie	控制端口认证 cookie

洋葱服务的数据目录还包含 hidden_service/，其中存储私钥及名为 hostname 的文件。该文件中记录服务的 .onion 地址，例如：

```
rpns2r36zn5u46wctlgnvtwjotxaf7iflsc3k5eas3lqoxeesv4zwyad.onion
```

每次重新生成配置时都会创建新密钥，因此地址也会改变（第 14 章）。

共识本身是人类可阅读的文本文件。提取其中的 `r` 行（中继身份与地址）和 `s` 行（标志），可以看到以下两个中继：

```
r test000a H+9DDHx4rEo6I6Jdn8gIyKyi3jk ... 127.0.0.1 5100 7100
s Authority Fast Guard HSDir Running Stable V2Dir Valid
r test006e dEPHoFFHamQyy6pRzlsLB9/ilzU ... 127.0.0.1 5106 0
s Exit Fast Guard HSDir Running Stable V2Dir Valid
```

可以看到，如预期一样，所有中继都拥有 Guard 标志，而只有两台出口中继拥有 Exit 标志。各字段的含义已在 第 12 章 中说明。

19.7　使用实验网络

19.7.1　通过私有网络浏览网页

以下两个 HTTP 示例都应在实验容器内部的 `/lab` 目录执行。可从另一台宿主终端运行 `make -C labs/zh-Hans/实验室 shell` 进入容器。curl 的 `--noproxy` 使用空字符串，可在设置了 `NO_PROXY` 的情况下，仍强制通过 SOCKS 代理访问 `127.0.0.1`。4747 端口的服务器提供第一个示例所创建的同一个 `index.html`。

客户端 `test008c` 在端口 9008 提供 SOCKS 服务。任何支持 SOCKS5 的程序都能使用它。先启动一个简单的本地 Web 服务器，再通过 Tor 访问：

```
$ echo "来自私有实验网络的问候" > index.html
$ python3 -m http.server 8080 --bind 127.0.0.1 &
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 http://127.0.0.1:8080/index.html
来自私有实验网络的问候
```

这次请求经过三跳中继电路，之后由其中一台出口中继连接到服务器。使用 `--socks5-hostname` 而不是 `--socks5`，可以让出口中继解析目的域名，而不是让本地操作系统执行解析。在真实网络中，这对避免访问目标泄露给本地 DNS 解析器十分重要（第 11 章）。

19.7.2　访问洋葱服务

Chutney 为洋葱服务生成以下配置：

```
HiddenServicePort 5858 127.0.0.1:4747
```

这将 `.onion` 地址的虚拟端口 5858 映射到本机的 4747 端口。我们只需让服务器监听该端口：

```
$ python3 -m http.server 4747 --bind 127.0.0.1 &
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 \
    http://$(cat net/nodes/009h/hidden_service/hostname):5858/index.html
来自私有实验网络的问候
```

这次不再涉及出口中继。客户端与服务在网络内部的汇合点相遇（第 14 章）。

19.7.3　用 Stem 观察电路

控制端口允许我们检查运行中的节点。利用镜像中安装的 Stem（第 18 章），可以列出客户端的电路：

```
from stem.control import Controller

with Controller.from_port(port=8008) as c:
    c.authenticate()                    # 自动使用 cookie 进行认证
    print(c.get_version())
```

```
    for circ in c.get_circuits():
        print(circ.id, circ.status, circ.purpose,
              [name for _, name in circ.path])
```

执行 `make -C labs/zh-Hans/实验室 verify` 后的部分输出如下：

```
0.4.9.13 (git-3c575400909efe65)
10 BUILT GENERAL ['test004r', 'test005r', 'test007e']
124 BUILT CONFLUX_LINKED ['test006e', 'test004r', 'test007e']
114 BUILT CONFLUX_UNLINKED ['test007e', 'test004r', 'test006e']
128 BUILT CIRCUIT_PADDING ['test004r', 'test001a', 'test007e', 'test006e']
163 BUILT HS_CLIENT_REND ['test004r', 'test001a', 'test006e']
166 BUILT HS_CLIENT_HSDIR ['test004r', 'test006e', 'test007e', 'test003r']
181 BUILT HS_VANGUARDS ['test004r', 'test007e', 'test006e']
```

即使在这样小的网络中，这份列表也汇集了本书介绍的多种机制：

- 几乎所有电路都从 `test004r` 开始：它是客户端选择一次、随后重复使用的入口守卫（第 13 章）。例外是某些 Conflux 隧道的分支，它们从第二个守卫开始（第 18 章）。
- 三跳的 `GENERAL` 电路承载普通流量；此处未列出的单跳电路用于获取目录信息。
- `CONFLUX_LINKED` 电路成对出现，共同承载同一份流量：这就是 Conflux 功能（提案 329，第 31 章）。
- `HS_VANGUARDS`、`HS_CLIENT_HSDIR` 和 `HS_CLIENT_REND` 标识访问洋葱服务时使用的电路（第 14 章）。
- 按照控制协议规范，`CIRCUIT_PADDING` 表示为了掩盖电路实际停止使用的时间而保留的电路。因此，它的关闭并不一定能说明用户活动何时结束。

19.7.4 在容器内部操作

容器使用主机网络模式，因此主机上的程序可以直接访问所有节点的端口。如果这种模式不可用（例如某些 Docker Desktop 环境），可以执行 `make -C labs/zh-Hans/实验室 shell`，进入已安装 tor、Chutney、Stem、Python 和抓包工具的容器。实验文件在容器内位于 `/lab`。

19.8 第一次与中继通信

下面的 Python 代码依赖位于 `labs/zh-Hans/代码/` 的教学包 `torlab`。Chutney 容器只将 `labs/zh-Hans/实验室/` 挂载到 `/lab`，因此该包不会自动出现在容器中。在采用宿主网络的 Linux 系统上，可将代码保存为文件，然后从仓库根目录执行 `PYTHONPATH=labs/zh-Hans/代码 python3 script.py`；也可先明确地将代码包提供给容器。该示例仅适用于私有实验网络。

现在回到 第 8 章。使用实验 2 提供的 `labs/zh-Hans/代码/torlab/cells.py`，我们可以向实验中继的 ORPort 建立 TLS 连接、发送一条 **`VERSIONS`** cell，再解码回应。完整的证书验证代码见 第 7 章 的实验 1；这里仅观察初次交换的报文：

```
import socket, ssl
from torlab.cells import *

ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE   # 仅用于私有实验网；不验证 TLS 证书
s = ctx.wrap_socket(socket.create_connection(("127.0.0.1", 5103)))
s.sendall(encode(versions_cell([3, 4, 5]), 0))

```

```
buf, version, received = b"", 0, 0
while received < 4:
    chunk = s.recv(4096)
    if not chunk:
        raise ConnectionError("TLS channel closed early")
    buf += chunk
    while (r := decode(buf, version)) is not None:
        cell, buf = r
        received += 1
        print(describe(cell, version))
        if cell.command == CellCommand.VERSIONS:
            version = negotiate_version([3, 4, 5], cell.body)
s.close()
```

注意：这个简短示例通过 `ssl.CERT_NONE` 有意关闭 TLS 证书验证，并没有真正根据 CERTS 信元验证中继身份。不要用它连接公共中继。完整的身份验证步骤见实验 1。

以下输出来自此前一次参考实验：电路编号、指纹、洋葱地址、时间戳及证书长度都可能因运行而异。

中继 `test003r` 的回应为：

```
CircID=0x0000 VERSIONS 消息体=6 字节
CircID=0x00000000 CERTS 消息体=1451 字节
CircID=0x00000000 AUTH_CHALLENGE 消息体=36 字节
CircID=0x00000000 NETINFO 消息体=509 字节
```

这恰好对应 第 8.8 节 描述的序列：首先，**`VERSIONS`** 使用两字节 CircID；协商到版本 5 后，后续 cell 则使用四字节 CircID。**`CERTS`** 和 **`AUTH_CHALLENGE`** 是变长消息，会保留其消息体的真实长度；**`NETINFO`** 是固定长度消息，其主体包含全部 509 字节。注意，收到 **`VERSIONS`** 后立即更新 `version = ...`：后续 cell 能否被正确解码，取决于刚刚协商出的协议版本。

19.9　Shadow：模拟器中的网络

当我们想研究整个网络的行为时，Chutney 就不够了。例如，不同拥塞控制算法会怎样影响延迟？攻击者需要多少流量才能完成某种攻击？要回答这些问题，需要数千个节点、跨洲的真实延迟特征，最重要的是还需要可复现的实验。

Shadow 最初由 Rob Jansen 和 Nicholas Hopper 开发 [JH12]，能够满足这些需求。当前版本让包括 tor 在内的真实程序作为普通 Linux 进程运行，同时拦截网络与时间相关的系统调用。虚拟网络图提供由研究者定义的延迟和带宽，时间只在模拟器安排时推进。因此，模拟中的一小时可能比现实中的一小时更快，也可能更慢，具体取决于处理能力；用相同随机种子重新执行，便能得到相同的结果。

研究人员通常使用 **tornettools** 在 Shadow 中构建贴近现实的 Tor 网络。该工具根据公开的 Tor Metrics 数据（第 24 章），生成规模较小的网络，同时让其带宽、标志和节点角色分布接近真实网络。

在 第 29 章 中，我们会通过一个引导实验使用 Shadow。现在只需要了解什么时候应选择模拟器即可。

本章小结

- Tor 实验应在私有网络中进行。在公共网络上做实验可能涉及未同意的用户，而且通常难以复现。
- Chutney 使用运行在环回接口上的真实 `tor` 进程仿真 Tor 网络。Shadow 则在离散事件模拟器中运行真实软件，适合大规模且需要重复实验的研究。

- 本书的实验环境（`labs/zh-Hans/实验室/`）从源码构建 tor 0.4.9.13。`make -C labs/zh-Hans/实验室 start` 启动由三台权威节点、三台中间中继、两台出口、一台客户端和一台洋葱服务组成的十节点网络。
- 编号为 n 的节点分别使用 ORPort $5100+n$、ControlPort $8000+n$；客户端另有 SocksPort $9000+n$。
- `TestingTorNetwork`、每 20 秒一次的投票、对所有中继分配 Guard 标志以及允许连接私有地址等设置，让微型网络得以运行，但也使它不同于真实网络。把实验测量结果推广到公共网络时必须谨慎。
- SocksPort 9008 用于通过网络传送应用程序流量；Stem 可以观察电路；`torlab` 则直接与中继通信。

练习

练习 **19.1** ⋆ 不查看 `torrc`，请给出 `test006e` 的 ORPort 和 ControlPort。`test009h` 有 ORPort 吗？为什么？

练习 **19.2** ⋆ 私有网络中的所有中继都有 Guard 标志。哪个选项导致这种现象？为什么真实网络不会这样？它会如何影响实验室中的路径选择测量结果？

练习 **19.3** ⋆ 重复 第 19.7 节 的 `curl` 请求，但用 `--socks5` 代替 `--socks5-hostname`，并用 `localhost` 代替 `127.0.0.1`。出口实际收到的内容有什么不同？为什么这在公共网络中很重要？

练习 **19.4** ⋆⋆ 在几分钟内观察客户端共识的 `valid-after` 字段（`grep valid-after net/nodes/008c/cached-microdesc-consensus`）。它多久变化一次？计算实验网络和真实网络每天分别生成多少份共识。

练习 **19.5** ⋆⋆ 停止三台目录权威节点中的一台（先运行 `make -C labs/zh-Hans/实验室 shell`，再根据该进程的 `pid` 文件使用 `kill` 终止进程）。网络还能继续生成共识吗？如果停止两台又会怎样？请解释，基本原因见 第 12 章。

练习 **19.6** ⋆⋆ 修改 第 19.7.3 节 中的 Stem 脚本，使它能打印每条三跳电路上所有中继的 ORPort，并通过 `get_network_status` 从共识中获取这些端口。

练习 **19.7** ⋆⋆ 创建文件 `labs/zh-Hans/实验室/chutney/networks/book-large`，配置五台目录权威节点、十台中间中继和四台出口，然后在 `make` 命令中设置 `NETWORK=book-large` 启动它。与本书原来的实验网络相比，引导时间有什么变化？

练习 **19.8** ⋆⋆⋆ 在客户端的 `info.log` 中找到与电路建立有关的日志，从选择节点直到构建完成。把每条日志与 图 8.5 中的一种 cell 对应起来。

练习 **19.9** ⋆⋆⋆ 在容器中运行 第 19.8 节 的脚本，同时用 `tcpdump -i lo` 捕获发往 5103 端口的流量。能直接看到 Tor cell 吗？为什么不能？哪些信息仍然可以观察到——数据大小、时间特征？第 20 章 将介绍进一步分析的方法。

延伸阅读

- Chutney：`https://gitlab.torproject.org/tpo/core/chutney`。项目的 README 和 `networks/` 目录提供大量网络配置示例，包括网桥和 Arti。
- R. Jansen、N. Hopper，*Shadow: Running Tor in a Box for Accurate and Efficient Experimentation* [JH12]。
- Shadow 与 tornettools 文档：`https://shadow.github.io/`。

- Tor Research Safety Board：`https://research.torproject.org/safetyboard/`，介绍 Tor 研究的伦理与安全指南。

第 20 章

观察流量：Wireshark 与日志

最后核验：*2026 年 9 月 23 日 — tor 0.4.9.13*；*tshark（Debian trixie）*；实验网络

本章目标

学完本章后，你将能够：

- 使用 `tcpdump` 捕获 Tor 流量，并借助 `tshark` 或 Wireshark 分析；
- 区分外部观察者能够看到的信息与仍受保护的信息；
- 利用会话密钥解密自己建立的通道上的 TLS，并检查其中的信元（实验 12）；
- 通过 tor 的日志，逐步跟踪协议的运行过程。

前面几章，我们通过编写直接使用协议的代码，从“内部”研究协议。本章则从“外部”观察它，如同一个监听网络流量的人一样。这项练习有两个方向相反的目的：了解攻击者能从流量中推断出什么，以及获得一种独立验证自己实现的方法。

注意：只观察自己的流量，而且只在实验环境中

在意大利以及大多数其他国家，未经授权捕获他人的流量都可能违法（第 25 章），同时也违背匿名研究的伦理原则（第 19.1 节）。本章所有抓包操作，都只针对本书的私有网络或我们自己建立的连接。

20.1 观察者能够看到什么？

观察客户端与入口守卫之间连接的人，例如互联网接入服务商，能够看到：

- IP 地址和端口：客户端连接到某个地址；如果它不是网桥，该地址通常会出现在公开共识中，因此可以被识别为 Tor 中继；
- TLS 握手：在 TLS 1.3 下，证书经过加密传输，但客户端最初发送的消息仍有部分特征可见；
- TLS 记录的大小和时间特征。

观察者无法看到 cell 的内容、电路信息或目的地。然而，网站指纹识别和流量关联攻击正是依靠数据大小与时间特征（第 26 章）。内容被加密，并不意味着流量完全不泄露信息。

在 第 16.5 节 中，我们已经比较了能够被识别为 TLS 的普通 Tor 流量，以及不具有明显结构的 obfs4 流量。现在再进一步，尝试查看 TLS 内部。

20.2 使用的工具

`tcpdump` 从网络接口捕获数据包，并保存到 `pcap` 文件。例如，在实验容器内，捕获发往 `test004r` ORPort 的流量：

```
$ docker exec -u 0 torbook-network tcpdump -i lo \
    -w /lab/net/tls.pcap "tcp port 5104"
```

Wireshark 和 **`tshark`** 分析抓包文件：前者提供图形界面，后者通过命令行使用。它们能够识别 TLS，但没有官方的 Tor 协议解析器。即使解密 TLS，Tor 信元仍然显示为原始字节。我们将使用 `labs/zh-Hans/`代码`/torlab/cells.py` 模块来解析这些字节。

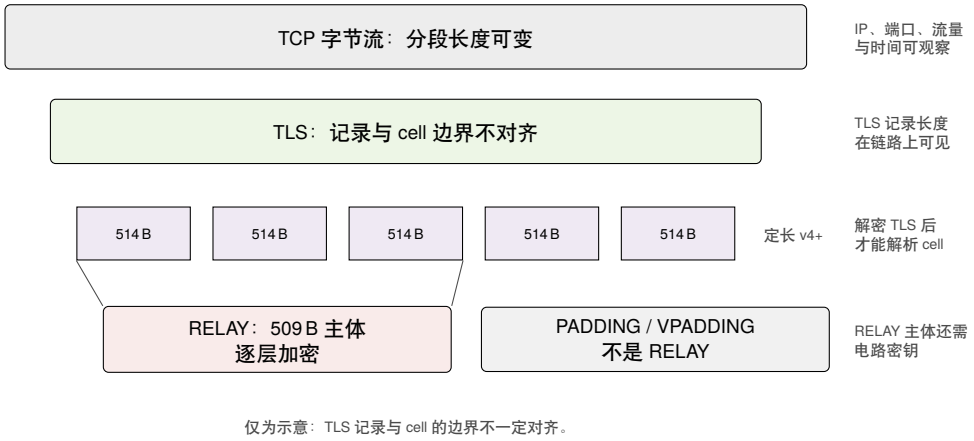


图 20.1: Tor 通道示意：TCP 分段承载 TLS 记录，记录边界不一定与 cell 边界对齐。IP 地址、端口、长度和时间特征均可观察；解密 TLS 后才能读取 cell。链路协议 v4+ 的固定长度 cell 为 514 字节（首部 5 字节、主体 509 字节），但也存在可变长度 cell。只有 `RELAY` cell 主体由电路密钥逐层加密；填充 cell 并不是 RELAY 消息。

20.2.1 解密自己建立的 TLS 连接

TLS 使用外部观察者不具备的会话密钥。但是，直接参与连接的一方可以导出这些密钥。浏览器和启用密钥日志功能的 Python 应用可以将会话密钥写入文件。本书的 `channel.py` 会在设置 `SSLKEYLOGFILE` 时显式配置 `SSLContext.keylog_filename`，从而导出密钥。Wireshark 和 `tshark` 可以使用该文件解密抓包数据。实验 1 的 `channel.py` 遵循这一约定：运行前设置好环境变量即可。文件中可能包含以下内容：

```
# TLS secrets log file, generated by OpenSSL / Python
SERVER_HANDSHAKE_TRAFFIC_SECRET 4878224d3f9fb9aedb96d... 0da89dff6feb...
```

这些记录包含客户端随机数，以及对应 TLS 1.3 会话中各个阶段的秘密值。

深入说明：为什么这不构成对 Tor 的攻击？

解密自己通道上的 TLS，不会揭示客户端原本不知道的信息。即使获得 TLS 会话密钥，**`RELAY`** 信元仍然使用每一跳的独立密钥进行多层加密（第 10 章）。要读取它们，还需要只有客户端和对应中继才持有的电路密钥。TLS 只是洋葱的最外层，而不是唯一一层。

20.3 实验：TLS 内部的信元

实验 12：捕获、解密并解析一条通道

目标。捕获我们自己的客户端与中继建立的通道，使用会话密钥解密，并通过 `labs/zh-Hans/代码/torlab/traffic.py` 还原信元序列。
前置条件。实验 1–3；实验网络已经运行；容器中已安装 `tcpdump` 和 `tshark`。
预计时间。1–2 小时。

执行实验 12 时，请先启动私有实验网络，并在仓库根目录打开两个终端。下面的抓包命令应在宿主机上运行：它会在实验容器内部以 root 身份执行 `tcpdump`，因为 Makefile 启动的普通容器 shell 使用非特权用户。第二个终端运行客户端时，保持抓包运行，最后使用 Ctrl+C 结束。捕获文件写入共享卷 `labs/zh-Hans/实验室/net/`。

步骤 **1**：抓包。 先在容器中启动抓包，然后从主机向 `test004r` 打开一条通道，在导出 TLS 会话密钥的同时建立一跳电路：

```
$ PYTHONPATH=labs/zh-Hans/代码 SSLKEYLOGFILE=labs/zh-Hans/实验室
    /net/tls-keys.log python3 -c "
from torlab.laboratory import relay
from torlab.circuit import build
build([relay('004r')], cc=False)"
```

前述命令已在同一行指定 `PYTHONPATH`，因此无需另外设置导入路径。

请在采用主机网络的 Linux 系统上，从仓库根目录执行 Python 命令。`PYTHONPATH` 使 `labs/zh-Hans/代码/` 下的 `torlab` 包可以导入；`labs/zh-Hans/代码/torlab/channel.py` 会显式读取 `SSLKEYLOGFILE` 并开启 TLS 会话密钥导出。密钥日志包含敏感会话秘密，不得公开，并应在实验后删除。

步骤 **2**：识别自己的连接。 实验网络中的其他中继也会连接同一个端口，但它们的会话密钥不同。通过 `tshark` 尝试解密并逐条跟踪 TCP 流，找出能够成功解密的连接：

```
$ docker exec torbook-network tshark -r /lab/net/tls.pcap \
    -o tls.keylog_file:/lab/net/tls-keys.log \
    -q -z follow,tls,raw,2
```

TCP 流编号 2 仅来自以前的一份示例抓包，并非固定编号。可利用 `tshark` 的 `-T fields -e tcp.stream` 查看候选流，再选择能够成功解密的流。下面的输出是历史示例，证书长度和电路 ID 会随着运行而改变。输出会以十六进制给出两个方向上的解密数据，其中一个方向通过制表符缩进。

步骤 **3**：检查 **cell**。 `cells_from_follow` 函数会重建两个方向上的字节流，并把它们交给实验 2 的 `decode`。每个方向都有自己的协议版本协商：第一个信元是带有两字节 CircID 的 **`VERSIONS`**。只有在版本协商完成后，CircID 才使用四字节格式。结果如下：

```
client -> relay  CircID=0x0000 VERSIONS 消息体=4 字节
relay  -> client CircID=0x0000 VERSIONS 消息体=6 字节
relay  -> client CircID=0x00000000 CERTS 消息体=1443 字节
relay  -> client CircID=0x00000000 AUTH_CHALLENGE 消息体=36 字节
relay  -> client CircID=0x00000000 NETINFO 消息体=509 字节
client -> relay  CircID=0x00000000 NETINFO 消息体=509 字节
client -> relay  CircID=0xc244f1f6 CREATE2 消息体=509 字节
relay  -> client CircID=0xc244f1f6 CREATED2 消息体=509 字节
```

从外部观察到的序列，恰好就是 图 8.5 和 第 7 章 所描述的过程。客户端的 **VERSIONS** 信元消息体长四字节，因为 `channel.py` 只提供版本 4 和 5。电路 ID `0xc244f1f6` 的最高有效位为 1，因为通道由客户端发起（第 8.4 节）。

注意：初版实验中的两个错误

最初的分析犯过两个错误，直到把输出与预期序列比较后才发现。首先，我们错误地把同一个协商版本用于两个方向，因而误解了中继的 **VERSIONS** 信元。其次，我们靠猜测判断 `tshark` 对哪个方向做了缩进，结果将数据块归入了错误方向。正确的判断规则很简单：第一个数据块始终是客户端发送的 **VERSIONS** 信元。

20.4　tor 日志

抓包能展示网络中传输了什么，而 tor 的日志则帮助解释为什么。在 `info` 级别，tor 会记录电路建立阶段、守卫选择、描述符下载和发布等活动；在 `debug` 级别，记录还可以细化到单个 cell。我们已经利用日志确认客户端查询了哪个 HSDir（第 14.9 节），以及洋葱服务何时更新描述符。

下面是几点实用建议：

- `debug` 会产生大量文本。可只为特定模块启用，例如使用 `Log [circ]debug file circ.log`，把详细记录限制在电路事件。
- 在真实网络中，`SafeLogging` 会掩盖地址信息。客户端日志不应记录用户访问的目的地。
- 公开中继的日志可能包含其他人的个人信息，并产生 第 25 章 讨论的法律后果；只应保留严格必要的数据。

本章小结

- 连接观察者可以看到 IP 地址、TLS 握手、数据包大小和时间特征，但无法直接看到 Tor 信元或目的地。
- `SSLKEYLOGFILE` 可以让我们解密自己建立的通道上的 TLS，从而查看 cell；不过 **RELAY** 信元仍受多层加密保护。
- 分析抓包数据为验证协议实现提供了独立方法。
- tor 的 `info` 和 `debug` 日志可解释程序所做的选择；真实使用时应尽量减少日志，并妥善保护。

练习

练习 **20.1** ⋆　在一次 Tor 下载流量抓包中，TLS 记录的长度并不全是 514 字节的整数倍。既然 cell 长度固定，为什么会出现这种情况？

练习 **20.2** ⋆⋆　扩展实验 12，使用 `circuit.py` 提供的密钥解密自己电路上的 **RELAY** 信元。构建三跳电路的过程中，能够观察到哪些 relay 消息？

练习 **20.3** ⋆⋆　在不产生应用程序流量的情况下，捕获实验客户端通道上持续一分钟的通信。统计两个方向的 TLS 记录数量，并将观察结果与 第 7 章 中的通道填充机制联系起来。

练习 **20.4** ⋆⋆⋆　使用 Lua 编写一个简单的 Wireshark 协议解析器，识别解密后数据中的 Tor 信元，并显示 CircID 和命令类型。

延伸阅读

- Wireshark 文档中有关利用会话密钥日志文件解密 TLS 的说明。
- tor 手册的 `Log` 条目，介绍日志模块和详细程度。

第 21 章

发布洋葱服务

最后核验：*2026 年 9 月 23 日 — tor 0.4.9.13*；*Onionbalance* 文档

本章目标

学完本章后，你将能够：

- 配置洋葱服务并保护它的密钥；
- 避免泄露服务器真实位置的配置错误；
- 通过 Onion-Location 向普通网站的访问者提供 `.onion` 版本；
- 通过受限发现机制将访问权限限制给获得授权的客户端（实验 13）；
- 评估个性化地址、单跳洋葱服务及 Onionbalance 负载均衡机制。

第 14 章 已经解释了洋葱服务的工作原理。本章将介绍对应的实践操作：发布一个服务，避免可能破坏匿名性的常见错误，并根据实际运维需求选择适当工具。

21.1 第一个洋葱服务

只需在 `torrc` 中添加两行：

```
HiddenServiceDir /var/lib/tor/my_service/
HiddenServicePort 80 127.0.0.1:8080
```

`HiddenServiceDir` 路径必须允许 tor 进程读写；在本书实验中，它是 Chutney 容器内部的路径。这个入门示例配置公开的洋葱服务，不需要 `alice.auth` 文件。客户端授权及其 `authorized_clients/*.auth` 文件将在本章后文单独介绍。第一行指定 tor 存储服务密钥的目录。第二行将其 `.onion` 地址的虚拟端口 80 映射到本地服务器的端口 8080。当 tor 重启时，会在该目录内创建以下文件：

`hostname` `.onion` 地址（第 14.2 节）。

`hs_ed25519_public_key`、**`hs_ed25519_secret_key`** 服务的身份密钥对。谁拥有私钥，谁就控制这个地址：可以发布描述符，并冒充服务。因此应像保护证书私钥一样保护它，并妥善备份。如果丢失私钥，原地址就无法恢复。

只有运行 tor 的用户才能访问该目录，否则 tor 会拒绝使用它。一个 tor 进程可以托管多个服务，每个服务使用自己的目录。在实验中，我们通过再添加两行配置，为 `test009h` 创建了第二个服务。

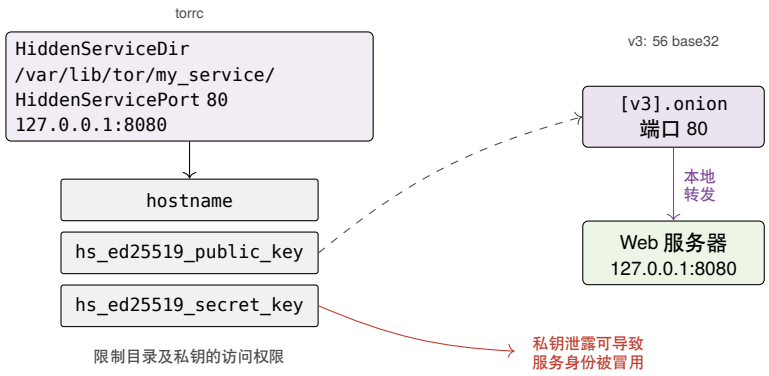


图 21.1: 配置公开的 v3 洋葱服务只需 `HiddenServiceDir` 与 `HiddenServicePort` 两条指令。tor 在服务目录生成 `hostname` 和 Ed25519 身份密钥；地址由公钥、校验和及版本字节编码成 56 个 base32 字符（图中使用占位符）。虚拟端口 80 转发到 `127.0.0.1:8080`。私钥泄露可导致服务被冒用；本例无需 `alice.auth`。

21.1.1 避免泄露服务器位置

协议虽然会隐藏服务器地址，但配置不当仍然可能泄露它。常见错误包括：

- 同时允许从外网直接访问服务。如果 Web 服务器监听所有网络接口，而不只是 `127.0.0.1`，扫描互联网的人就可能发现它，并把页面内容与洋葱服务进行比较。应让服务器仅绑定到 localhost；更理想的做法是使用 Unix 套接字（`HiddenServicePort 80 unix:/path/socket`）。

- 在响应中泄露细节。错误页面可能包含服务器名称，HTTP 头可能列出内部地址，TLS 证书可能对应某个公开域名，或者 Web 服务器的状态页面可能未经保护。

- 使用外部资源。如果页面从普通网站加载图片或脚本，这些请求就会离开洋葱服务。在 Tor Browser 中，访问仍经过 Tor，因此不会直接泄露访客 IP，但这些外部请求不再享有洋葱地址提供的端到端身份认证与保护；出口中继可以观察连接元数据以及未加密的 HTTP 内容，但 HTTPS 仍能保护应用数据。服务器端应用若主动联系外部服务以获取更新或发送分析数据，则可能暴露服务器自身的地址。

- 时钟与时间特征。如果服务器显示与某个时区对应的本地时间，或者总在某台已知公开服务器维护时不可用，就可能留下可供关联的线索。

在 第 28 章 中，我们将研究这些错误曾如何帮助调查人员定位真实服务。

21.2 Onion-Location

许多公开网站，包括大型新闻机构、社交平台和 Tor Project，都提供 `.onion` 版本。普通网站可以通过以下 HTTP 响应头向使用 Tor Browser 的访客介绍这一版本：

```
Onion-Location: http://<address>.onion/path
```

也可以在网页中使用等效的 `meta` 元素。Tor Browser 随后会在地址栏显示按钮，供用户切换到洋葱版本。这样，访客不再经过出口中继，并且可以通过地址本身验证服务的身份；网站收到的 Tor 流量也无须经过有时会被封锁或受到额外审查的出口中继。

21.3 受限发现

某些个人服务，例如连接家用服务器的 SSH 或管理面板，不一定需要向所有知道地址的人开放。受限发现（第 14.7 节）会加密描述符的第二层，使只有获得授权的客户端才能解密。其配置采用 X25519 密钥对 [Thee]：

服务端 在服务的 `authorized_clients/` 目录中为每位客户端放置一个 `.auth` 文件。文件内只有一行：`descriptor:x25519:`，后接客户端经过 base32 编码的公钥。

客户端 `ClientOnionAuthDir` 指定的目录中，为每个服务保存一个 `.auth_private` 文件。内容由不带 `.onion` 后缀的地址、`:descriptor:x25519:` 和客户端经过 base32 编码的私钥组成。

Tor Browser 允许用户在访问需要授权的服务时输入私钥。

实验安全提示：自动脚本 `labs/zh-Hans/实验室/restricted_discovery.py` 在开始清理时会删除已有的 `restricted_service/` 与 `onion_auth/` 目录。只能在专用、可丢弃的 Chutney 私有网络中运行，并事先确认目录中没有需要保留的数据。不要对真实洋葱服务执行此脚本。手动操作时应先写入授权文件，再重新加载相关节点。

实验 13：具有受限发现功能的洋葱服务

目标。在 `test009h` 上配置第二个洋葱服务，使其只允许持有授权密钥的客户端访问，并分别测试有密钥和无密钥的连接。
前置条件。实验网络已经运行；本地有一台 Web 服务器监听 8080 端口。
预计时间。1 小时。

步骤 **1**：生成客户端密钥对。 使用 `cryptography` 库：以下代码是能够独立运行的 Python 示例，会输出用于服务端的公钥授权行。私钥 `priv` 必须单独安全保存，以便随后创建 `.auth_private`；不要将其泄露给他人。

```
import base64
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from cryptography.hazmat.primitives.serialization import (
    Encoding, PrivateFormat, PublicFormat, NoEncryption
)

sk = X25519PrivateKey.generate()
priv = sk.private_bytes(Encoding.Raw, PrivateFormat.Raw, NoEncryption())
pub = sk.public_key().public_bytes(Encoding.Raw, PublicFormat.Raw)
b32 = lambda b: base64.b32encode(b).decode().rstrip("=")
print("descriptor:x25519:" + b32(pub))
```

base32 字符串必须去掉末尾用作填充的 `=` 字符。此代码只打印公钥授权行，不会保存私钥。如果作为独立脚本运行，进程结束后就会丢失 `priv`。手动操作时必须在进程退出前以严格权限安全保存私钥，或者使用随附的自动化实验脚本，由它生成客户端授权文件并在实验结束后清除临时密钥。不要把手动和自动化步骤混合运行。

步骤 **2**：配置服务。 创建文件 `restricted_service/authorized_clients/alice.auth`，写入客户端公钥。在 `test009h` 的 `torrc` 中添加：

```
HiddenServiceDir /lab/net/nodes/009h/restricted_service
HiddenServicePort 80 127.0.0.1:8080
```

然后发送 HUP 信号。tor 将生成服务地址并发布描述符。这里的 /lab/ 是容器内挂载的实验目录。

下面的命令从 tor 创建服务时一并生成的 hostname 文件中读取真实洋葱地址：每个网络都有自己的地址。下面的 SOCKS 错误和客户端日志来自此前的一次参考会话，可能有所不同。

步骤 **3**：不提供密钥。 在网络客户端上执行：

```
$ ONION_ADDR=$(cat labs/zh-Hans/实验室/net/nodes/009h/restricted_service/hostname)
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 \
    "http://${ONION_ADDR}/index.html"
(curl: SOCKS proxy error 97)
```

客户端日志报告：

```
[notice] Fail to decrypt descriptor for requested onion address.
         It is likely requiring client authorization.
```

客户端已找到并下载描述符，却无法解密第二层。它甚至不知道引介点在哪里，因此无法尝试建立连接。

注意：空目录可能使服务无法访问

如果 authorized_clients/ 存在但其中没有任何文件，服务仍会继续发布描述符，但 HSDir 会拒绝它：

```
[warn] Uploading hidden service descriptor: http status 400 ("Invalid HS
       descriptor. Rejected.") response from dirserver 127.0.0.1:5106.
```

相关规范要求，描述符经过二次加密的层中至少要有一条 auth-client 记录 [rend-spec §Hidden service descriptors: encryption format]。实验中启用了受限发现，却没有授权客户端，因此没有生成这一记录，导致描述符无效。这是在撤销最后一个客户端的访问权限，或者先建目录、后放入密钥时可能遇到的运维陷阱。实验脚本若意外中断，也可能留下不完整的测试配置。

步骤 **4**：提供密钥。 在 ClientOnionAuthDir 指定的目录中写入 .auth_private 文件，重新加载客户端配置，然后再次请求网页。这次页面成功送达。labs/zh-Hans/实验室/restricted_discovery.py 会自动执行实验、清理临时内容并恢复 torrc。

21.4 其他运维需求

21.4.1 个性化地址

以可识别单词开头的地址，可以帮助用户记忆。使用 *mkp224o* 等专用程序不断生成随机密钥，直到地址具有所需前缀。base32 前缀每增加一个字符，期望尝试次数就乘以 32。普通计算机可能在合理时间内找到六到七个字符的前缀，但再增加一个字符，计算成本就会增加数十倍。如 第 14.2 节 所述，攻击者也可以生成具有相同前缀的地址。因此，前缀有助于识别，却不能用来认证服务。第三方程序生成的密钥，也必须像任何其他秘密密钥一样谨慎保护。

21.4.2 单跳服务

某些不需要隐藏服务器位置的公开网站，例如知名报纸的洋葱版本，可以牺牲服务端匿名性来换取速度。启用 HiddenServiceSingleHopMode 和 HiddenServiceNonAnonymousMode 后，服务会通过单跳电路连接引介点和汇合点。客户端仍然像平常一样保持匿名。发布描述符仍使用三跳电路：正如我们在 第 14.9 节 中发现的，HSDir 会拒绝直接请求，tor 手册也明确提到

了这一点。选项名称中的 “NonAnonymous” 并非偶然：以此模式运行的 tor 实例不能同时用作匿名客户端。而且，这一选择很难逆转。一旦服务以非匿名模式运行过，它的 IP 地址就可能与 `.onion` 地址建立关联。简单地关闭选项并不会恢复匿名性。要重新获得匿名性，必须创建使用新密钥、新 `HiddenServiceDir` 和新地址的服务，并使用干净的新实例。

21.4.3 Onionbalance

单个 tor 进程既可能成为性能瓶颈，也可能形成单点故障。Onionbalance [Theb] 允许多个服务器共用一个公开地址。每台服务器运行自己的洋葱服务，并拥有自己的引介点。Onionbalance 持有主地址的密钥，发布汇集所有服务器引介点的描述符。客户端在无感知的情况下被分配到不同服务器。Tor Project 自己的网站也使用 Onionbalance。

本章小结

- 洋葱服务只需要两行配置。其私钥控制服务地址，必须得到保护并安全备份。
- 服务器应只监听 localhost 或 Unix 套接字，而且不应通过响应或外部连接泄露自身信息。
- Onion-Location 向 Tor Browser 访问者告知网站的洋葱版本。
- 受限发现使用 X25519 密钥；没有授权的客户端甚至无法解密描述符。
- 个性化地址、单跳服务和 Onionbalance 分别针对易识别性、速度和可扩展性。

练习

练习 **21.1** ⋆ 平均需要尝试多少次，才能生成具有六字符 base32 前缀的地址？八字符前缀呢？如果一台计算机每秒尝试一千万组密钥，分别需要多长时间？

练习 **21.2** ⋆⋆ 某洋葱服务在错误页面显示 `Apache/2.4.62 (Debian) Server at 192.0.2.10 Port 80`。这会泄露哪些信息？应如何避免？

练习 **21.3** ⋆⋆ 在实验 13 中，你会如何撤销 Alice 的访问权限？已经下载描述符的客户端会受到什么影响？

练习 **21.4** ⋆⋆⋆ 修改实验服务，让连接被转发到 Unix 套接字，而不是本地 TCP 端口。除了避免暴露端口之外，这还有哪些好处？

延伸阅读

- tor 手册中的 `HiddenServiceDir`、`HiddenServicePort` 和客户端授权相关条目。
- Tor Project 关于洋葱服务和 Onionbalance 的文档 [Theb]。

第 22 章

运行中继或网桥

最后核验：*2026 年 9 月 23 日 — tor 0.4.9.13*（手册）；实验网络

本章目标

学完本章后，你将能够：

- 根据自己的资源和能够承担的风险，选择运行中间中继、出口中继或网桥；
- 编写各类节点所需的基本配置，包括带宽限制与离线密钥存储；
- 阅读、编写出口策略，并在实验网络中验证其作用（实验 14）；
- 正确声明中继家族。

Tor 网络之所以存在，是因为数千名个人和组织贡献了带宽与计算机。前面的章节从客户端视角观察中继；现在我们换到运营者的一侧。本章并不是系统管理手册——Tor Project 已维护一份持续更新的运营指南——而是解释主要的运维选择在协议层面意味着什么。

22.1 应该运行哪种节点？

中间中继 （之后也可能成为入口守卫）在其他中继之间转发 cell，但不向互联网外部目的地建立连接。这是运营要求最低的一种角色。其地址会出现在共识中，但网站不会看到来自它的出口流量。

出口中继 代表客户端向目标网站或服务建立连接。由于出口相对于需求仍然稀缺（第 13.2 节），这一角色尤其有价值，但也带来最高的风险暴露：如果出现滥用流量，接收方看到的表面来源就是出口的 IP 地址。

网桥 不列入公开列表的中继，供无法直接访问 Tor 公共网络的用户作为第一跳（第 16.2 节）。即使只是普通住宅宽带，只要配合 obfs4 或 WebTunnel 等可插拔传输，也能有所帮助。

注意：开始之前

运行出口中继会产生法律和运营后果，包括滥用投诉以及有关部门的查询，参见 第 25 章。可以考虑先从中间中继或网桥开始，事先告知网络接入服务商，并避免在同一条网络连接上既运行中继，又以 Tor 客户端身份开展敏感活动。

22.2 基本配置

中间中继只需要在 `torrc` 中设置几个选项：

```
Nickname        MyRelay
ContactInfo     operator@example.org
ORPort          443
ExitRelay       0
SocksPort       0
RelayBandwidthRate  4 MBytes
RelayBandwidthBurst 8 MBytes
```

`ContactInfo` 用于在中继出现问题时让其他人联系运营者。手册提醒，描述符会被归档和公开，因此联系地址可能无限期地保留在公共记录中。`ExitRelay 0` 明确禁止该节点作为出口；对于近年的版本，这本已是默认行为：设置为 `auto` 时，只有运营者编写出口策略，tor 才会成为出口。`RelayBandwidthRate` 和 `RelayBandwidthBurst` 分别限制平均与峰值带宽。相比之下，`AccountingMax` 限制一个统计周期（例如一个月）内的总流量。一旦达到上限，中继便停止接受新流量，直到下一个周期开始。

运维注意：当 `ExitRelay` 设为 `auto` 时，不仅显式的 `ExitPolicy`，`ReducedExitPolicy` 或 `IPv6Exit` 配置也可能使节点成为出口。示例中的 `ExitRelay 0` 则会禁止全部出口流量。部署公开中继之前应检查最终生效的配置。

22.2.1 新中继的成长过程

刚启动的中继会向目录权威节点发布描述符，随后由权威节点将其纳入共识并开始测量（第 12.4 节）。起初，它获得的流量很少：客户端根据带宽权重选择中继，测量结果稳定前，新节点的权重仍然较低。各种标志需要随着运行时间逐渐获得（第 12.4.3 节）。`Stable` 需要长期持续、不中断地运行，而 `Guard` 还要求节点达到一定年龄并保持高可用性。当一台中继刚成为入口守卫时，它的流量甚至可能暂时下降：已经选好守卫的客户端，不会像过去那样频繁把它用作中间中继。随着新客户端逐渐选择它，流量才会缓慢增长（第 13.3 节）。

两种故障的后果并不相同。频繁重启但不改变身份的中继，仍保留带宽测量数据，却会降低获得 `Stable` 和 `Guard` 所需的持续可用性，因此可能需要重新取得这些标志。相反，改变身份密钥会让目录权威节点将其视为一个全新的中继，就连带宽测量也必须从零开始。

22.2.2 密钥

中继的身份由其主 Ed25519 密钥决定（第 7.2 节）。默认情况下，tor 将它存放在中继磁盘上。如果设置 `OfflineMasterKey 1`，就可以把主密钥保留在另一台计算机上，只把使用 `tor --keygen` 签发证书的临时签名密钥传输给服务器。即使服务器遭到攻破，攻击者获得的也是会过期的密钥，而不是中继的长期身份。不过这种方法需要严格管理：临时密钥必须在到期前更新，否则中继将停止正常运行。

22.3 出口策略

出口策略是一份按顺序排列的 `accept` 与 `reject` 规则列表。每条规则指定一个地址或网络，以及一个端口或端口范围。Tor 按顺序检查规则，并采用第一条匹配规则。运营者自行定义的规则优先于手册列出的默认规则。默认情况下，Tor 会拒绝用于电子邮件的端口（25）、NNTP（119）、NetBIOS 与 SMB（135–139、445），以及部分文件共享应用使用的端口，其余流量则允许通过。另外，tor 默认拒绝私有目的地址，例如 `127.0.0.0/8`、`10.0.0.0/8`、`192.168.0.0/16`，以及中继自身的公开地址。这可以保护运营者的内部网络（`ExitPolicyRejectPrivate`）。我们的实验网络关闭了这一保护，以允许流量连接到 `127.0.0.1`。

保护规则的先后顺序：默认开启的 `ExitPolicyRejectPrivate` 会把拒绝私有网络和本地地址的规则放在运营者自定义规则之前。Chutney 私有实验网络故意关闭了这项保护；不要将这种设置照搬到公开出口中继。

只允许 Web 流量通过的出口可以配置为：

```
ExitRelay  1
ExitPolicy accept *:80
ExitPolicy accept *:443
ExitPolicy reject *:*
```

Tor Project 还建议将 `ReducedExitPolicy` 作为一种常见折中方案：它允许若干广泛使用的服务端口，但排除更容易产生滥用投诉的端口。

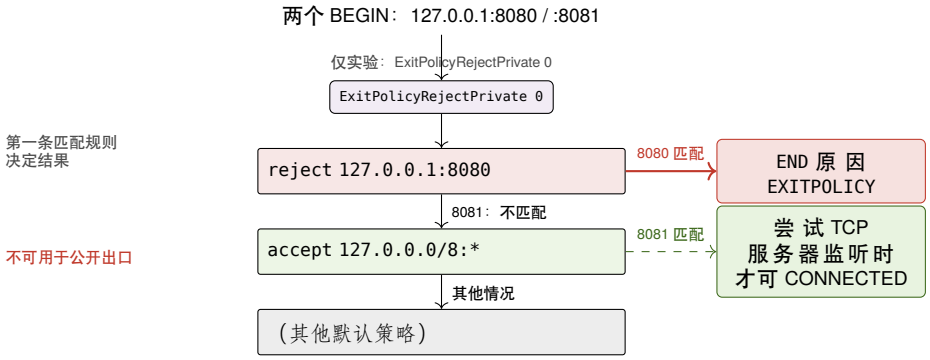


图 22.1: Chutney 实验室仅为连接私有地址而设置 `ExitPolicyRejectPrivate 0`；正常情况下 Tor 会在运营者自定义规则之前拒绝这些地址。到 `127.0.0.1:8080` 的 `BEGIN` 命中 `reject` 并产生 `END EXITPOLICY`；端口 8081 跳过该规则并命中 `accept`。允许仅代表可以尝试 TCP 连接，服务器实际监听时才可能收到 `CONNECTED`。颠倒规则顺序会使拒绝无效。

客户端得到什么信息？ 中继描述符中包含完整的出口策略，而共识和微描述符中只提供按端口汇总的摘要，例如 `p accept 80,443`。客户端借此判断某个出口能否处理请求（第 13.1 节）。因为该摘要省略了目的地址，客户端可能只有在尝试建立连接后，才能发现某个地址受到特定规则的拒绝，正如下面的实验所示。

22.4 实验：测试出口策略

实验 14：出口规则及其后果

目标。给实验出口增加一条规则，再用 `labs/zh-Hans/代码/torlab` 观察允许和拒绝不同目的地时的回应。
前置条件。实验 3 和 4；实验网络已经运行。
预计时间。1 小时。

步骤 **1**：添加规则。 在 `test006e` 的 `torrc` 中，在原先允许 `127.0.0.0/8` 的规则前面加入一条拒绝某个端口的规则：

```
ExitPolicy reject 127.0.0.1:8080
ExitPolicy accept 127.0.0.0/8:*
```

这里的两行配置是相对顺序示意：修改已有的 `torrc` 时，只需把新的 `reject` 插入第一条 `ExitPolicy` 之前，切勿重复添加原有的 `accept` 规则。顺序非常重要。如果把拒绝规则放到 `accept` 后面，它就不会产生作用，因为第一条匹配规则已经决定了结果。在之前的参考实验中，我们曾犯过这个错误。重新启动出口中继后，在容器的 8080 和 8081 端口各启动一个简单的 HTTP 服务器。

步骤 **2**：发送三个请求。 使用 `build` 和 `stream.py` 建立一条以 `006e` 为出口的电路，分别向端口 8080、8081，以及没有服务监听的 9999 发送 `BEGIN` 消息。在我们的一次会话中，响应如下（原因码、地址和 TTL 可能随版本与环境变化）：

```
127.0.0.1:8080 -> END  reason=EXITPOLICY address=127.0.0.1 TTL=1800
127.0.0.1:8081 -> CONNECTED
127.0.0.1:9999 -> END  reason=MISC
```

步骤 **3**：解释回应。 策略拒绝会产生 `EXITPOLICY` 原因码。`END` cell 还会包含解析后的地址和 TTL。客户端因此可以记住该出口不能访问这个目的地，转而选择另一个出口，而无须等待新描述符。相比之下，没有服务器监听的端口返回通用的 `MISC`，而不是规范针对连接被拒绝所列出的 `CONNECTREFUSED` [tor-spec § Closing streams]。

深入说明：为什么得到 `MISC` 而不是 `CONNECTREFUSED`？

答案出现在出口中继的 `info` 日志中。追踪这一问题，可以看到研究规范与实际实现存在差异时应如何调查：

```
[info] connection_handle_write_impl(): in-progress connect failed.
       Removing. (Connection refused)
[info] errno_to_stream_end_reason(): Didn't recognize errno 115 (Operation
       now in progress); telling the client that we are ending a stream
       for 'misc' reason.
```

出口中继实际上知道连接被拒绝，却依据另一个数值确定原因码。到目的地的连接采用非阻塞方式：`connect()` 最初返回 `EINPROGRESS`（errno 115），实际结果要等后续写事件发生时才会得知。此时，`connection.c` 通过 `getsockopt(SO_ERROR)` 获取真正的错误，记录 “Connection refused”，随后调用 `connection_edge_end_errno()`。然而，该函数又使用 `errno_to_stream_end_reason(tor_socket_errno(…))` 重新计算原因码。在 Unix 上，`tor_socket_errno` 是展开为 `(errno)` 的宏（`src/lib/net/socket.h`），读取的仍是保留了 `EINPROGRESS` 的全局变量。数值 115 不在 `src/core/or/reasons.c` 的已识别列表中，因此默认分支返回 `MISC`。

正确信息本已存在，却没有被使用。相关代码中的两个分支也有所区别：中继到中继的连接分支会传递刚获取的错误，而流处理分支则忽略它。根据已有的 `docs/it/segnalazioni/` 分析与补丁记录，此前在私有网络中对 tor 0.4.9.13 进行的测试表明，正确传递错误值后第三个请求返回 `CONNECTREFUSED`。这是一个不影响匿名性的轻微缺陷：客户端仍能知道流建立失败，但失去了更有用的信息。`CONNECTREFUSED` 会提示目的地明确拒绝了连接，因此换一个出口重试也可能没有意义。分析报告与补丁存放在 `docs/it/segnalazioni/`，可供 Tor Project 审阅。这也说明为什么必须用真实行为检验协议规范。

实际脚本为 `labs/zh-Hans/实验室/exit_policy.py`。在 Linux 宿主机的专用 Chutney 网络启动后，可从仓库根目录执行 `PYTHONPATH=labs/zh-Hans/代码 python3 labs/zh-Hans/实验室/exit_policy.py`。该脚本运行三项测试，并在 `finally` 中尝试恢复 `006e` 的原始 `torrc`。它会暂时修改出口配置并开启本地 HTTP 服务器，只能用于可丢弃的私有实验网络。常规的 `make -C labs/zh-Hans/实验室 verify` 只执行 Chutney 检查，并不会调用此脚本。本轮审核未运行脚本，也未动态验证恢复过程。有一个重要细节：电路必须包含三跳。出口会拒绝转发来自单跳电路的流量，并返回原因码 `TORPROTOCOL` 的 `END`（第 15.3 节）。

22.5　中继家族

运营多个中继的人必须声明一个家族（family）。客户端不会将同一家族中的两个中继放入同一条电路（第 13.1 节），因为同一运营者可能关联所观察到的流量。传统机制是 `MyFamily`：每

台中继都列出其他所有中继的指纹，只有双方声明互相认可时，家族关系才成立。如果维护十台中继，就需要管理十份一致的列表，每次加入新中继时都要修改它们。

tor 0.4.9 引入的新机制使用家族密钥。运营者通过 `tor --keygen-family MyFamilyKey` 生成密钥，将生成的 `MyFamilyKey.secret_family_key` 文件复制到每台中继的密钥目录（`KeyDirectory`，文件名须保留 `.secret_family_key` 后缀），并在每台中继的 `torrc` 中写入生成的 `FamilyId` 配置行。必须保护并备份这份共享的私钥；若密钥泄露，应为整个家族轮换密钥。每台中继都用该密钥签署把自身身份绑定到家族的证书。验证方因而无须逐一比较列表。在过渡阶段，建议同时配置传统机制和新机制。

深入说明：为什么声明家族有帮助？

没有什么强制运营者把自己的中继声明为同一家族；攻击者甚至可能故意不这样做。不过，监测网络的目录权威节点和志愿者会留意共同特征，例如相同的网络、软件版本、同时启动的时间和联系信息。未声明的家族可能引起额外审查，相关中继甚至可能被排除在共识之外。主动声明家族，是整个网络信任安排的一部分。

不要将网桥列入公开的 **`MyFamily`** 列表：这可能暴露网桥与公开中继的关联。Tor 官方运维指南明确建议不要在网桥上设置该选项。此外，`tor --keygen-family MyFamilyKey` 可能覆盖同名密钥文件；请使用未被占用的名称，并严格保护生成的秘密文件。

22.6 运行网桥

可以按照实验 10（第 16.5 节）配置一台 obfs4 网桥：

```
BridgeRelay 1
ORPort 9001
ServerTransportPlugin obfs4 exec /usr/bin/obfs4proxy
ServerTransportListenAddr obfs4 0.0.0.0:443
ExtORPort auto
BridgeDistribution any
```

`BridgeDistribution` 告诉当前的分发系统 rdsys，运营者希望如何分发网桥。`any` 将选择交给系统，`none` 则完全排除公开分发，适合仅向可信的人私下分享网桥。网桥信息页面列出了可用渠道，包括网站、电子邮件和 Telegram。网桥虽然不出现在公开共识中，仍会把描述符发送给网桥权威节点，由后者检查可达性。

传输程序应监听不同于 ORPort 的端口；条件允许时，最好使用 443 等常见端口。这才是客户端使用、也是审查者观察到的端口（第 16.4 节）。

本章小结

- 运行中间中继是门槛最低的贡献方式；出口中继特别有价值，但风险暴露更高；网桥可以帮助面临审查的用户。
- 基本配置只需要身份、联系信息、端口和带宽限制等少数选项。主身份密钥可以离线保存。
- 出口策略是按顺序排列的规则列表，第一条匹配规则决定结果；默认拒绝私有目的地址。
- 策略拒绝以 `END` 消息传给客户端，包含 `EXITPOLICY` 原因码、地址和 TTL。
- 多中继运营者应使用 `MyFamily` 或较新的家族密钥机制声明家族关系。

练习

练习 **22.1** ⋆　给定策略 `accept *:80`、`reject 192.0.2.0/24:*`、`reject *:*`，客户端能否通过该出口连接 `192.0.2.7:80`？请修改策略，使这条连接被拒绝。

练习 **22.2** ⋆　为什么共识中仅包含端口信息的摘要，不足以判断出口是否允许某次连接？客户端收到原因码为 `EXITPOLICY` 的 `END` 后会怎么做？

练习 **22.3** ⋆⋆　某运营者使用 `MyFamily` 管理五台中继，又加入第六台，却只更新了新中继的配置。客户端会看到什么？改用家族密钥后有什么变化？

练习 **22.4** ⋆⋆　验证技术分析中的解释：在出口中继启用 `info` 日志，重新执行三个请求。如果 `connection_edge_end_errno()` 收到的是 `getsockopt` 返回的错误值，会发生什么变化？哪些其他流关闭原因也可能因此被正确报告？

练习 **22.5** ⋆⋆⋆　一台中继的月流量上限为 `AccountingMax 1 TBytes`，带宽为 10 MB/s。全速运行多少天后会耗尽配额？一台在半个月内处于离线状态的中继，与另一台整月在线但带宽减半的中继相比，会对网络产生什么不同影响？

延伸阅读

- Tor Project 的中继运营指南：`https://community.torproject.org/relay/`。
- tor 手册中的 `ExitPolicy`、`ReducedExitPolicy`、`FamilyId`、`OfflineMasterKey` 与 `BridgeDistribution` 条目。
- 流关闭原因规范：[tor-spec §Closing streams]。

第 23 章

Tor Browser、Tails 与 Whonix

最后核验：*2026* 年 *9* 月 *23* 日 — *Tor Browser* 设计文档（*2018*）；*Tor Browser* 手册；*Tails* 与 *Whonix* 文档

本章目标

学完本章后，你将能够：

- 解释为什么仅把普通浏览器配置为使用 Tor 仍然不够；
- 列出 Tor Browser 设计所依据的安全与隐私要求；
- 将 Tor Browser 的按站点隔离机制与 tor 的流隔离关联起来；
- 比较 Tor Browser、Tails 和 Whonix 分别提供哪些保护，以及各自的代价。

2004 年的威胁模型就明确指出，Tor 不会统一规范应用层协议（第 4.2 节）。如果浏览器自己泄露了用户身份，洋葱路由也无能为力。本章介绍专为保护这一层而设计的工具：让浏览器行为与网络保护目标保持一致的 Tor Browser，以及把保护范围扩展到整个计算机的两个操作系统——Tails 和 Whonix。

23.1 为什么只有代理还不够？

让普通浏览器通过 tor 的 SocksPort 通信，虽然通常可以隐藏 IP 地址，但并非在所有情况下都有效，而且除此之外几乎不能隐藏其他信息：

- 某些组件可能绕过代理直接联网，例如本地 DNS 解析、WebRTC、浏览器扩展或浏览器启动的外部程序。
- Cookie 和其他浏览器状态可以将今天的访问与昨天的访问关联起来，即使昨天的访问没有通过 Tor。
- 网站能够收集数十种浏览器特征，包括窗口尺寸、安装的字体、语言、时区和渲染细节，并将其组合为可以从数百万用户中辨认出某人的指纹。
- 如果所有标签页共用一条电路，嵌入多个网站的跟踪器便可能同时观察到来自同一个出口的请求。

第一个问题威胁的是地址隐私；其他问题则威胁不同活动之间的不可关联性，而这正是 Tor 的核心目标之一。

23.2 Tor Browser

Tor Browser 是由 Tor Project 连同 tor 一起发布、基于 Mozilla Firefox 长期支持分支修改的浏览器。其设计原则记录在一份文档中。尽管文档的部分实现细节已经过时，它仍然是理解项目原则的重要资料 [Per+18]。文档把要求分为两组。

安全要求。 这些要求保障 Tor 的安全使用，违反其中任何一项，都可能立即暴露用户：

遵守代理设置 任何内容都不能绕过 tor 发送。

状态分离 不得与 Tor 之外的其他浏览器或会话共享信息。

不留下持久痕迹 除非用户明确选择保留，否则会话结束后不应在磁盘或内存中遗留能暴露浏览活动的信息。

应用隔离 浏览器在系统中留下的所有数据，都应能够通过删除被清除。

隐私要求。 这些要求旨在防止不同活动被关联起来，其核心是地址栏所显示的站点——即用户正在访问的网站：

标识符 当用户浏览某个网站时，嵌入其中的第三方（例如跟踪器）保存的数据，不应在同一第三方出现在另一个网站时继续供其访问。

浏览器指纹 浏览器的可观察特征不应允许不同网站关联用户。

长期不可关联性 用户应能通过一个简单操作清除全部状态，并取得新身份。

文档还提出一条有用的设计原则：保护机制不应破坏用户对浏览器正常行为的预期。Tor Browser 的前身 Torbutton 扩展允许用户在同一浏览器内切换 Tor 的启用状态，用户很容易混淆在不同状态下打开的标签页。容易诱使用户犯错的界面，本身就是安全弱点。

23.2.1 每个网站使用独立电路

来源隔离也延伸到网络层。Tor Browser 在发送给 tor 的 SOCKS 凭据中标识地址栏中的网站。默认情况下，tor 利用 `IsolateSOCKSAuth` 把携带不同凭据的流分配到不同电路。我们在实验 11（第 18.3 节）中验证了这一机制。按照用户手册的解释 [Thed]，即使两个网站嵌入相同的跟踪器，它们也会使用不同电路，而同一网站的不同标签页则会共用一条电路。

两个菜单命令可以改变这种安排：

为此网站使用新电路（**New Circuit for This Site**）通过新电路重新加载当前网站，例如出口中继无法连接该网站时。此操作不会清除浏览器状态，也不会改变其他网站的电路。

新身份（**New Identity**）关闭全部标签页，清除 Cookie、历史记录和其他状态，并为后续活动使用新电路，从而实现长期不可关联性的要求。

这两个操作都不会改变入口守卫；出于 第 13.3 节 所解释的原因，守卫通常会稳定使用数月。

按站点隔离的主要目标是隔离不同站点，并不保证一个站点在整个会话中始终只有一条电路；电路会老化和更换。“为此网站使用新电路”会重新加载当前标签页；同一网站的其他标签页要等重新加载后才使用新电路，已有连接未必立即中断。“新身份”还会中止当前活动和下载。两种操作都不用于强制轮换入口守卫，守卫由 tor 另行管理。

23.2.2 抵御浏览器指纹识别

Tor Browser 的反指纹策略，并不是让每个用户显得不同，而是让用户尽可能相似。如果所有 Tor Browser 用户都暴露相同的值，指纹最多只能说明 “这是一名 Tor Browser 用户”。这种思路体现在多个可见的设计选择中：窗口采用标准化尺寸，调整窗口时会在页面内容周围增加留白，使网页报告的尺寸保持在标准间隔上（*letterboxing*）；报告的时区为 UTC；许多暴露操作系统细节的接口被关闭或标准化。这也解释了为什么不鼓励安装扩展或随意更改设置：每个不同点都会缩小用户能够藏身的人群，即 第 1 章 中介绍的匿名集原则。

防指纹保护的目标是尽量减少可观察差异，并非保证所有用户的指纹完全相同。平台、语言、安全级别等属性仍可能把用户分成可区分的群组；因此应把这种保护理解为降低独特性，而不是完全一致的保证。

23.2.3 安全级别

许多浏览器漏洞会利用 JavaScript 和复杂的网页功能。Tor Browser 提供三个安全级别 [Thed]：

Standard（标准）启用所有功能，为默认级别。

Safer（更安全）在非 HTTPS 网站禁用 JavaScript；限制部分字体和数学符号；音频、视频需要点击后才能播放。

Safest（最安全）在所有网站禁用 JavaScript，部分图片及其他内容也可能受到限制。静态网站通常仍可使用。

选择安全级别，就是在安全性与易用性之间取舍。所选级别本身也可能影响浏览器指纹，因为网站有时能够推断用户使用了哪个级别。

深入说明：可复现构建

下载 Tor Browser 的用户需要确信，获得的二进制程序确实对应已公开的源代码。Tor Project 因此采用可复现构建（reproducible builds）：在相同源码和构建环境下，应当得到逐比特一致的文件。设计文档介绍了为了消除不确定性而需要采取的措施，包括固定日期、文件路径和归档成员顺序 [Per+18]。独立重新构建并比较输出，可以发现有人在软件分发的构建环境中恶意插入的改动。

23.3 浏览器之外的保护

Tor Browser 只保护它自己的流量，不能保护同一台计算机上其他应用程序的流量。那些应用仍可直接联网，用户失误或浏览器漏洞也可能泄露真实地址。两个操作系统采用不同的架构，把保护扩展到整台机器（图 23.1）。

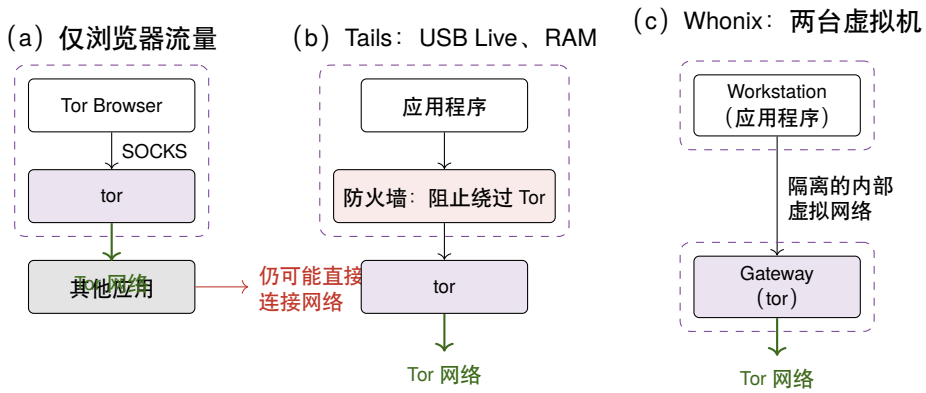


图 23.1: 三种路由和隔离方式，并非绝对匿名保证。(a) Tor Browser 只让自身流量经过 tor，宿主机的其他程序仍可能直接联网。(b) Tails 通常通过防火墙阻止应用程序绕过 Tor；用于 captive portal 的 *Unsafe Browser* 是明确的不匿名例外。Tails 从 USB 启动，主要运行在 RAM 中，可选择持久存储。(c) Whonix 使用彼此通过隔离虚拟网络连接的两台虚拟机：运行应用的 Workstation 和运行 tor 的 Gateway；宿主机或虚拟机管理程序遭到攻破仍可能破坏隔离。

23.3.1 Tails

Tails 是一个从 USB 设备启动、基于 Debian 的操作系统 [Taia]。它有两个关键特征：

一切流量经过 **Tor** 任何试图绕过 tor 连接互联网的应用程序都会被阻断。

遗忘性（**amnesia**）系统在内存中运行，不向计算机硬盘写入数据，并在关机后忘记此前的状态。如果用户选择保存数据，可以把它们放在 USB 设备上可选的加密持久存储区。

Tails 内置 Tor Browser 和其他经过谨慎配置的应用程序。该项目于 2024 年 9 月加入 Tor Project [The24c]。它的文档明确说明了局限：Tails 无法抵御遭到篡改的硬件，例如实体键盘记录器，也无法解决在已感染恶意软件的计算机上制作 USB 启动盘所带来的风险。

这是 Tails 在通常运行条件下的性质，并不绝对保证所有存储设备都不会留下痕迹。用户仍可能主动挂载磁盘或建立持久文件；恶意软件、受损硬件以及 Tails 之外的操作也不受这种保护保证。

23.3.2 Whonix

Whonix 把功能分离到两台虚拟机中。**Gateway** 运行 tor，并且是唯一连接外部网络的虚拟机；**Workstation** 运行应用程序，只能通过内部虚拟网络连接 Gateway。因此，Workstation 的所有流量都必须通过 tor：这种保证来自网络架构，而不只是各应用的代理配置。与 Tails 相比，Whonix 提供了更强的隔离：即使攻击者获得 Workstation 的管理员权限，也不能直接从该虚拟机获知真实地址，因为这台机器根本不知道它。代价是需要依赖虚拟化软件的安全性；此外，Whonix 不具备 Tails 式的遗忘性，除非在另一个提供该特性的系统内运行。

Gateway 与 Workstation 的分离是重要的安全边界，但即使工作站被入侵，也不意味着绝对安全。攻击者仍可能通过应用数据识别用户；若宿主机、虚拟化管理程序或网关失守，网络隔离也可能被突破。防火墙的完整性和环境的正确配置同样关键。

经过过滤的控制端口。 Workstation 上的某些应用，包括 Tor Browser 的新身份功能，需要与 Gateway 上 tor 的 ControlPort 通信。但是，控制协议（第 18.1 节）还允许使用 `GETINFO address` 查询 tor 的公开地址，而这恰恰是 Workstation 不应该知道的信息。因此，Whonix 在两者之间放置了名为 *onion-grater* 的代理，只允许明确批准的命令通过 [Who]。某项功能在一种环境中无害，在另一种环境中却可能成为信息泄露渠道。

注意：工具不能取代良好的使用习惯

Tor Browser、Tails 和 Whonix 能减少技术性失误，却无法消除行为失误：登录个人账户、在受保护环境之外打开下载的文件，或者主动透露可识别身份的细节。第 28 章 将专门研究这些问题。

本章小结

- 即使普通浏览器通过 Tor 代理联网，仍可能有流量绕过代理，更重要的是，不同活动可能通过共享状态、浏览器指纹或电路被关联起来。
- Tor Browser 遵循明确的设计要求：强制遵守代理配置、分离状态、限制持久存储、隔离不同网站，并提供易于使用的新身份功能。
- 按网站隔离依靠 SOCKS 凭据和 tor 的流隔离机制。
- Tor Browser 通过标准化用户可观察特征来抵御指纹识别；安全级别则用于减小攻击面。
- Tails 强制所有流量通过 Tor，并避免保留持久痕迹；Whonix 将应用限制在不知道真实网络地址的虚拟机中。

练习

练习 **23.1** ⋆ 从电路和浏览器状态两个方面解释，“为此网站使用新电路”与“新身份”有什么不同？为什么这两种操作都不会更换入口守卫？

练习 **23.2** ⋆ 某用户认为广告拦截扩展能够提高隐私，于是在 Tor Browser 中安装了一个扩展。这可能给他带来哪些好处，又可能失去哪些保护？

实验方法说明：实验 11 只模拟不同的 SOCKS 凭据，并不会真正打开三个 Tor Browser 标签页。相同凭据可能重用电路，但并非任何时刻都保证如此：电路会老化，其他隔离规则也可能起作用。

练习 **23.3** ⋆⋆ 使用实验网络与实验 11 的脚本模拟三个标签页：其中两个访问同一网站，第三个访问另一个网站，并把网站名称作为 SOCKS 凭据。通过 ControlPort 检查哪些流共用电路。

练习 **23.4** ⋆⋆ 对于以下情况，从 Tor Browser、Tails 和 Whonix 中选择最适合所述需求的工具，并说明理由：（a）在家用计算机上浏览遭到审查的网站的人；（b）在他人计算机上处理机密文件的记者；（c）运行多个长期在线程序，且这些程序必须全部通过 Tor 的研究人员。

练习 **23.5** ⋆⋆⋆ 选出五种网站能够通过 JavaScript 读取的浏览器特征，估计普通浏览器中每种特征可能暴露多少比特信息。Tor Browser 如何将这些特征标准化？用户之间还有哪些差异？

延伸阅读

- M. Perry、E. Clark、S. Murdoch、G. Koppen，*The Design and Implementation of the Tor Browser* [Per+18]。
- Tor Browser 使用手册 [Thed]。
- Tails 文档 [Taia] 和 Whonix 文档，特别是 onion-grater 页面 [Who]。

第 24 章

测量网络：Tor Metrics 与 OONI

最后核验：*2026* 年 *9* 月 *23* 日 — 截至 *2026* 年 *9* 月 *20* 日的 *Tor Metrics* 数据；*2026* 年 *9* 月 *23* 日 *15:00* 的共识；*OONI* 规范 *ts-023* 与 *ts-030*

本章目标

学完本章后，你将能够：

- 描述 Tor Project 发布的数据及其来源；
- 解释一个刻意不识别用户身份的网络如何估算用户数量，以及这些估计存在多大的不确定性；
- 从真实共识中提取网络统计信息（实验 15）；
- 解释 OONI 如何从用户视角测量 Tor 的封锁情况。

匿名网络给运营者提出了一个特别的问题：改进网络需要测量，但测量意味着收集的恰恰是网络本应保护的信息。Tor Project 的做法是尽量少收集数据，将结果汇总后公开，并记录计算过程的每一步，使任何人都能复现结果 [Thec]。本章从数据来源出发，一直追踪到统计图表，最后介绍 OONI 如何从网络外部进行测量。

24.1 数据来源

公众所了解的 Tor 网络情况，几乎都来自我们已经见过的文档：

共识与描述符 （第 12 章）：中继列表、标志、带宽权重、版本和出口策略。

额外信息描述符（**extra-info**）中继在发布普通描述符时同时发布这些文档，其中包含历史字节传输计数，以及按国家分组的目录请求数量等汇总统计。

主动测量 OnionPerf 定期从世界各地选定的测量点通过 Tor 下载文件，记录传输时间和失败情况。

CollecTor 从网络早期起便持续归档这些文档，并提供下载。**Tor Metrics** 基于它们生成图表和数据表，附有可下载的 CSV 数据及方法说明。**Relay Search** 则支持查询单个中继。本书实验使用的共识文件（`fonti/collector`）即来自 CollecTor。

24.2 统计匿名用户

Tor 不要求用户注册，任何中继也无法知道两条连接是否属于同一个人。那么，网络是如何估计约三百万名用户的呢？方法是间接的（图 24.1）。每个客户端都会定期下载共识，而无需识别请求者身份就可以统计目录请求数量。提供目录文档的中继每天统计成功的请求，并根据来

源 IP 地址推断国家，对各国请求分别汇总。这些总数被发布在额外信息描述符中；为了避免小而精确的计数泄露更多信息，各国计数会向上取整为 8 的倍数 [dir-spec § Extra-info document format]。

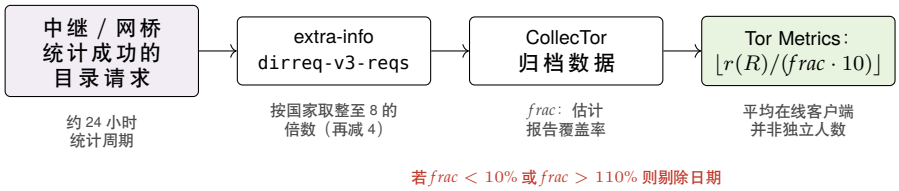


图 24.1: 从成功的目录请求间接估算平均在线客户端数，而不是独立人数。目录中继和网桥分别汇总 extra-info 描述符中的请求数量；`dirreq-v3-reqs` 按国家计数并向上取整到 8 的倍数。CollecTor 归档这些数据；Tor Metrics 对有效计数减去 4 来修正舍入，再估计报告覆盖率 $frac$，并计算 $\lfloor r(R)/(frac \cdot 10) \rfloor$。其中 10 是单个客户端每天平均目录请求次数的假设；若 $frac < 10\%$ 或 $frac > 110\%$，则排除对应日期。

Tor Metrics 收集这些数字，并按三个步骤处理 [Thec]：

1. 从各项计数中减去舍入区间的一半，以补偿向上取整造成的误差。
2. 估算提供报告的中继覆盖全网目录请求的比例 $frac$，同时考虑每台中继实际运行的时长及其处理的目录流量。
3. 将报告中的请求数量除以该比例，估计全网总请求数，再除以 10：

$$r(N) = \left\lfloor \frac{r(R)}{frac \cdot 10} \right\rfloor .$$

除数 10 是一种假设。始终在线的客户端每天大约请求 15 次共识，但许多客户端一天只在线一段时间，所以 Tor Project 选择 10 作为平均值。报告覆盖比例低于 10 % 或高于 110 % 的估计结果会被丢弃。

注意：“用户数”究竟代表什么？

计算结果估计的是平均在线客户端数量，并不是独立的自然人数量。每天只运行两小时的客户端通常比全天在线的客户端产生更少的目录请求，但二者并不存在固定的比例关系；同一台机器上运行多个 Tor 程序，也会被计为多个客户端。软件行为变化，例如降低下载共识的频率，即使实际使用人数不变，也会改变估计结果。因此，这些图表更适合用来解读变化趋势，而不是绝对人数。

本书基准日期附近的数据。 2026 年 8 月 1 日至 9 月 20 日，直接连接 Tor 的估计日用户数在 280 万到 350 万之间波动，报告覆盖比例为 57 %–73 %。意大利的日均估计值约为 60 000，但每日结果的预期区间很宽。例如 9 月 20 日，区间约为 34 000–111 000。在这样的范围内，数字变化本身不足以证明发生了特定事件。同一天，按传输技术估计的网桥用户数分别约为：obfs4 98 000、Snowflake 60 000、WebTunnel 35 000。

24.2.1 发现网络审查

Metrics 使用相同的时间序列标记可能的审查事件。对于用户数量足够多的国家，它将该国的周环比变化与其他国家的典型变化比较，并突出显示超出预期区间的日期 [Thec]。直接连接的用户数骤降，同时网桥用户数上升，是遭到封锁的典型信号。但这只是一条线索，不能视为证据。确认情况需要来自该国境内的测量结果，而这正是 OONI 的工作。

24.3 性能测量

OnionPerf 从固定位置测量通过 Tor 下载不同大小文件所需的时间、失败情况，以及电路延迟：从发送 HTTP 请求到收到响应头之间的时间 [Thec]。2026 年 9 月 20 日，从德国测量点访问公共服务器的延迟中位数约为 110–140 ms，美国测量点约为 280 ms，香港测量点则超过 500 ms。测量点与欧洲、北美主要 Tor 网络资源之间的地理距离具有重要影响。部分测量点使用 第 31 章 介绍的多路径机制 Conflux，从而可以在相近条件下进行比较。

24.4 实验：读取真实共识

实验 15：从共识中推导网络统计信息

目标。使用新模块 `labs/zh-Hans/代码/torlab/metrics.py`，从 CollecTor 的真实共识中提取关键网络统计指标。
前置条件。实验 6（`directory.py`）和实验 7（`path_selection.py`）；无需启动私有网络。
预计时间。1 小时。

该模块新增 `summary` 函数，复用实验 6 的共识解析器，同时提供两个用户估计函数。当数值相同时，统计结果也会按名称排序。否则，Python 集合的迭代顺序可能导致每次执行的结果顺序不同。要复现 2026 年 9 月 23 日 15:00 的历史共识结果，请先在仓库根目录运行 `make -C labs/zh-Hans data` 准备数据（下载时需要联网）。应用于该共识（输出有节选）：

```
$ PYTHONPATH=labs/zh-Hans/代码 python3 -m torlab.metrics labs/zh-Hans/数据
    /collector/consensus-microdesc-2026-09-23-15
relay: 9358
flag: {'Running': 9358, 'Valid': 9358, 'Fast': 8961, 'Stable': 8199,
       'V2Dir': 7674, 'Guard': 5306, 'HSDir': 4050, 'Exit': 3177,
       'BadExit': 32, 'MiddleOnly': 28, 'StaleDesc': 21, 'Authority': 10}
total_bandwidth: 198927900
exit_bandwidth_share: 0.3308947161257923
guard_bandwidth_share: 0.7010271560701139
versions: [('Tor 0.4.9.12', 6027), ('Tor 0.4.9.11', 2520), ...]
networks16: [('64.65', 771), ('192.42', 636), ('204.8', 598), ...]
```

可以观察到以下几点：

- 共识列出的 9 358 台中继，与 Tor Metrics 图表相符：9 月 20 日，图表显示 9 416 台中继和 2 333 台网桥。
- 大约三分之一的中继拥有 `Exit` 标志，约占所测量带宽的三分之一；拥有 Guard 标志的中继约占 70 % 的带宽。这两个比例可以重叠，因为一台中继可能同时拥有两个标志。第 13.2 节 中介绍的带宽权重用于处理这些不平衡。
- `BadExit`（32 台中继）和 `MiddleOnly`（28 台）反映目录权威节点对可疑或配置不当中继的判断。
- 有十台中继带有 `Authority` 标志，但只有九台权威节点参与共识投票（第 6.1 节）。第十台是网桥权威节点 `Serge`，虽带有该标志，却不参与投票。
- 多数中继运行较新的 0.4.9 系列版本。
- 部分 /16 网络包含数百台中继，通常是大型出口运营者。第 13.1 节 中的每个 /16 网络最多选择一台中继的规则，可以减少电路对单一网络的依赖。

共识带宽权重的总和代表目录权威节点测量的带宽单位，而不是实际传输的流量。要估算真实流量，Metrics 使用额外信息描述符中的历史字节计数。

24.5 OONI

Open Observatory of Network Interference（OONI，开放网络干扰观测站）从用户视角测量互联网审查。世界各地的志愿者安装 OONI Probe，定期执行测试并公开结果。其中有些测试专门针对 Tor [OON]：

`tor` 检查选定目录权威节点和网桥的可达性。测试不仅仅尝试建立 TCP 连接，还执行协议特有的操作，例如 obfs4 握手。

`torsf` 尝试通过 Snowflake 启动 tor，记录引导是否成功及耗时。

这些测试规范包含防止泄露私人网桥地址的保护措施：如果将这些地址写入公开结果，就等于直接交给审查者。Metrics 和 OONI 互为补充。Metrics 可能显示某个国家的用户数量下降；OONI 则能调查该国境内是否真的无法连接中继或网桥，并帮助识别封锁方式。

深入说明：测量不能造成伤害

中继发布的每项统计数据，都是在实用性与隐私之间进行权衡。细节越多，数据越有用，泄露信息的风险也越大。因此，请求计数会进行舍入，并按日期和国家汇总。网络研究也应遵循同样原则（第 19.1 节）：只收集必要的最少数据，在发布前进行汇总，不保留不必要的资料。

本章小结

- 网络数据来自 CollecTor 归档的共识、描述符与额外信息文档，以及 OnionPerf 主动测量。Tor Metrics 使用公开、可复现的方法分析它们。
- 用户数量依据目录请求估算：先用报告数量除以覆盖比例，再除以 10。结果表示平均在线客户端数，尤其适合研究变化趋势。
- 同样的数据还可能提示审查事件，但需要当地测量来确认。
- 真实共识可以提供中继数量、标志、带宽占比、版本和网络集中程度。
- OONI 从不同国家境内测量权威节点与网桥的可达性，以及 Snowflake 的运行情况。

练习

练习 **24.1** ⋆　如果所有客户端的共识下载频率都减少一半，而实际使用人数保持不变，用户数量估计会如何变化？解读图表的人可能怎样察觉这一变化？

练习 **24.2** ⋆　某一天，提供报告的中继记录了两千万次请求，估计报告覆盖比例为 60 %。Metrics 会估算出多少用户？如果覆盖比例是 8 %，又会怎样处理？

练习 **24.3** ⋆⋆　扩展 `summary` 函数，计算同时带有 `Guard` 和 `Exit` 标志的中继所占的带宽比例，并估算电路出口属于中继数量最多的五个 /16 网络之一的加权概率。

练习 **24.4** ⋆⋆　选择一个发生过明确记录的 Tor 封锁事件的国家，从 Metrics 下载直接连接用户和网桥用户的时间序列。描述封锁发生时两个序列如何变化。

练习 **24.5** ⋆⋆⋆　各国请求计数会向上舍入为 8 的倍数。描述一种因此更难实施的攻击，并解释为什么这一措施无法抵御控制大量中继的攻击者。

延伸阅读

- Tor Project 的 *Reproducible Metrics* 页面 [Thec]，介绍各类图表背后的方法。
- OONI 测试规范 [OON]，以及 OONI Explorer：`https://explorer.ooni.org/`。
- 额外信息描述符格式：[dir-spec § Extra-info document format]。

第 25 章

意大利和欧盟的法律与伦理问题

最后核验：*2026* 年 *9* 月 *23* 日 — 欧盟条例（*EU*）*2022/2065*；意大利第 *196/2003* 号立法法令（意大利个人数据保护机构公布的合并文本）；*Tor Research Safety Board* 指南

本章目标

学完本章后，你将能够：

- 区分工具本身的合法性和使用工具的行为是否合法；
- 根据 Tor 出口中继的运行方式，理解欧盟的“单纯传输通道”（mere conduit）规则；
- 识别意大利中继运营者面临的尚未解决的问题，包括责任、数据保留和设备扣押；
- 将 Tor Research Safety Board 的安全原则应用于 Tor 研究。

注意：本章不构成法律意见

法律会发生变化，适用结果取决于每个案件的具体事实，而专门涉及 Tor 的司法裁判仍相对有限。本章旨在梳理相关问题，指出已经对照官方文本研究的法律规定，并明确说明不确定性。任何准备作出实际运营决策的人，例如准备运行出口中继的运营者，都应咨询熟悉技术法律的专业律师。

25.1 工具及其用途

在意大利以及欧盟其他地区，法律通常并不禁止使用 Tor 或其他保护通信机密性的工具。相反，通信与信函的保密（意大利宪法第 15 条）、表达自由（第 21 条）均受到基本权利保护；《欧盟基本权利宪章》第 7 条和第 8 条同样保护私人生活与个人数据。

人们通过 Tor 进行的活动，仍然受不使用 Tor 时同样适用的法律约束。未经授权访问计算机系统（意大利刑法典第 615-*ter* 条），不会因为流量经过三台中继就变成合法行为。本章的核心是区分工具与其用途；对基础设施提供者也是如此：中继运营者与网络接入服务商类似，提供的是传输能力，而不是选择被传输的内容。

25.2 中继运营者

25.2.1 单纯传输通道（Mere Conduit）

欧盟《数字服务法》（Digital Services Act，DSA）自 2024 年 2 月 17 日起适用，取代了原先由 2000 年《电子商务指令》建立的中介责任规则。意大利曾通过第 70/2003 号立法法令实施该指令 [22]。DSA 第 4 条规定了单纯传输通道的责任豁免：对于传输他人提供的信息的服务，如果运营者没有发起传输、没有选择传输对象，也没有选择或修改传输的信息，则在满足适

用条件时，无须为该信息承担责任。图 25.1 从 cell 协议层面，将这些条件与 Tor 出口中继的行为进行比较。

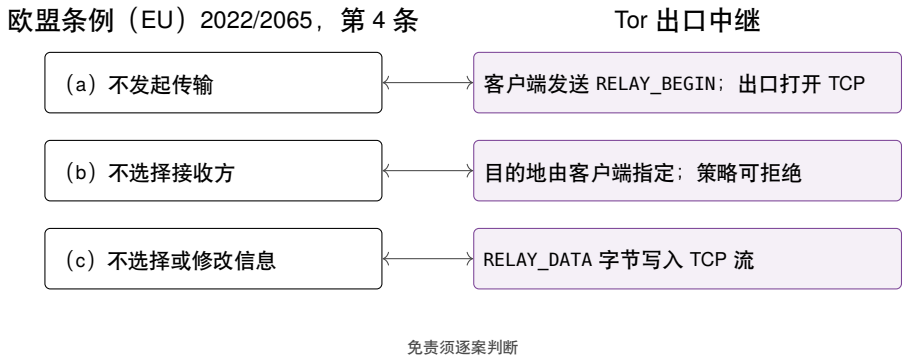


图 25.1: 示意比较欧盟条例（EU）2022/2065 第 4 条的单纯传输条件与 Tor 出口的技术行为，不构成自动免责结论。客户端通过 `RELAY_BEGIN` 指定目的地；出口建立 TCP 连接，并可按照出口策略拒绝请求。`RELAY_DATA` 中的应用数据字节写入 TCP 流，而不是将 Tor cell 原样转发。是否免责还取决于全部法律条件与具体情况。

还有两项规定值得关注。第 8 条排除了对所传输信息进行一般性监控，或主动寻找违法行为证据的义务：运营者通常没有无差别检查流量的法律义务；中间中继也无法读取被加密的应用层明文内容。另一方面，第 4 条第 3 款保留了司法或行政主管机关命令服务提供者预防或终止侵权行为的权力。

尚未解决的问题。 该条例适用于“信息社会服务”。按照欧盟法律定义，此类服务通常是有偿提供的。完全免费、由志愿者运营的中继是否属于这一范畴？一种观点主张应作较宽泛的解释，而且责任豁免反映了法院可能考虑的原则：仅负责传输信息的人，不应为信息内容承担责任。但是，本书并未找到能够直接确立意大利志愿者 Tor 中继法律地位的适用判决。这种不确定性也有助于理解，为什么有些出口中继由本来就提供网络服务的协会或组织运营。

25.2.2 民事责任与刑事调查

豁免规则涉及对所传输信息承担的责任，却不能阻止刑事调查从出口 IP 地址开始。受害者看到的通过 Tor 实施的违法行为，表面上来自出口中继。调查人员在初期可能搜查或扣押运营者的计算机。Electronic Frontier Foundation 为中继运营者编写的法律常见问题解答虽以美国法为背景，但也总结了实务经验：该组织表示不知道有人仅因运行中继而被判有罪，却记载了由于误解 Tor 而开展的调查与扣押 [Ele]。以下措施可以降低实际风险：

- 在数据中心的专用服务器上运行出口，不要使用家用计算机或个人工作电脑。
- 清楚表明该 IP 地址属于出口中继，例如设置说明性的反向 DNS，或使用 `DirPortFrontPage` 在 tor 的 DirPort 上提供介绍页面。
- 事先通知网络接入服务商，并约定如何处理滥用投诉。
- 回应投诉时解释 Tor 的工作方式，并保存相关通信记录。

25.2.3 数据保留

意大利《个人数据保护法典》第 132 条要求受其约束的服务提供者为刑事调查和起诉保留电子通信流量数据十二个月，而电话流量数据的保留期为二十四个月 [03]。第 5-*bis* 款保留了特殊规定：2017 年 11 月 20 日第 167 号法律第 24 条，对于《刑事诉讼法典》第 51 条第 3-*quater*

款和第 407 条第 2 款（a）项列明的罪行——包括恐怖主义及其他严重犯罪——将调查所需的保留期限延长为七十二个月 [17]。《个人数据保护法典》的相关部分适用于通过公共通信网络向公众提供的电子通信服务（第 121 条）。

志愿者中继运营者是否属于此类服务提供者，仍然是一个未解决的问题，并且可能取决于服务的具体提供方式。运营者，特别是原本就提供通信服务的组织，在运行出口前应请律师评估适用情况。从技术上说，在推荐的日志设置下，tor 通常不会保存用户目的地的历史记录。出口中继能看到它所转发连接的目的地，却不知道原始客户端的 IP 地址。实际能够获得的数据取决于中继的角色和配置，因此不能假定出口中继可以识别 Tor 的具体用户。

25.2.4 日志属于个人数据

IP 地址属于个人数据。《通用数据保护条例》（GDPR）要求仅处理必要的数据，并且仅在必要的期限内保留（第 5 条，数据最小化原则）。对于中继而言，符合安全目标的选择是仅以 `notice` 级别记录日志，保持 `SafeLogging` 启用，并避免抓包（第 20.4 节）。

25.3 研究人员

Tor 研究历史上曾出现一些存在伦理争议的实验，部分案例将在 第 27 章 中讨论。有些研究人员运行中继，监控或操纵真实用户的流量，以展示某种攻击。这些实验除了产生伦理问题，还可能触及意大利刑法，包括关于非法截获计算机通信的第 617-*quater* 条，以及个人数据保护方面的规定。

Tor Research Safety Board 是一个为实验安全提供建议的研究人员组织，并制定了相关指南 [Tor]：

- 尽可能使用测试网络，只攻击自己产生的流量。
- 仅收集真正需要的数据，并且只收集适合安全公开的数据。
- 保护收集到的数据以及实验工具。
- 通过舍入或加入噪声降低数据精度。
- 考虑第三方数据集与自己收集的数据组合后可能带来的信息泄露。
- 评估用户是否有理由认为相关数据应保持私密。
- 确保预期收益足以证明风险是合理的。

安全委员会要求研究人员说明希望了解什么、准备如何实施、实验会引入哪些风险，以及为什么预期收益能够抵消风险。本书遵循同样的原则：需要产生实际 Tor 流量的实验使用私有测试网络（第 19.1 节）；而 第 24 章 中的公共文档分析则离线进行，使用公开的汇总数据。

本章小结

- 使用 Tor 本身是合法的；通过 Tor 开展的活动仍受一般法律约束。
- DSA 第 4 条在满足规定条件时，为单纯传输他人信息的服务提供责任豁免。出口中继的运行方式与这些条件有关，但志愿者中继能否适用该条例，仍存在法律不确定性。
- 责任豁免不能阻止调查或扣押。使用专用服务器、明确标识出口身份，以及与网络接入服务商协调，可以减少实际问题。
- 数据保留要求适用于法律涵盖的、向公众提供的通信服务提供者；这些要求是否以及如何适用于志愿者中继，需要逐案进行法律分析。

- Tor 研究应尽可能避免涉及真实用户的实验：使用测试网络、最小化数据、只发布安全的汇总数据，并权衡收益与风险。

练习

练习 **25.1** ⋆　针对 DSA 第 4 条的三项条件，分别描述一种出口中继可能采取的、会违反该条件的行为变化。

练习 **25.2** ⋆　通过一个实例解释工具合法性与使用行为合法性的区别，并分别将其应用于 Tor 用户和中继运营者。

练习 **25.3** ⋆⋆　假设某网站投诉你的出口 IP 地址发起了多次访问尝试，网络接入服务商因此向你发送滥用报告。请草拟一份回应，说明你能够解释哪些情况，以及可以采取哪些实际措施。

练习 **25.4** ⋆⋆　某研究人员提议记录出口中继观察到的域名，以估计有多少 Tor 用户访问意大利新闻网站。请根据 Tor Research Safety Board 的指南评估这一计划，并提出更安全的替代方案。

练习 **25.5** ⋆⋆⋆　查找意大利及其他欧洲国家法院涉及 Tor 中继运营者的司法裁判。分别总结争议的法律问题与结果，并引用一手来源。

延伸阅读

- 欧盟条例（EU）2022/2065，尤其是第 4、8 和 9 条 [22]。
- 意大利第 196/2003 号立法法令第 121 条及第 132 条，以意大利个人数据保护机构发布的合并文本为准 [03]。
- Electronic Frontier Foundation，*Legal FAQ for Tor Relay Operators* [Ele]。
- Tor Research Safety Board 网站 [Tor]。

第四部分

攻击与防御

第 26 章

流量关联、指纹识别与 Sybil 攻击

最后核验：*2026* 年 *9* 月 *23* 日 — 本章引用的研究论文；*tor 0.4.9.13*；实验网络

本章目标

学完本章后，你将能够：

- 根据攻击者的观察位置对针对 Tor 匿名性的攻击分类；
- 解释端到端流量关联，并在实验网络中测量自己产生的流量（实验 16）；
- 描述网站指纹识别以及相关评估的局限性；
- 识别 Sybil 攻击及 Tor 的防御机制，从入口守卫到带宽测量。

第 4 章 的威胁模型已经表明：Tor 无法抵御同时观察通信两端的攻击者。二十多年的研究量化了这一警告，并发现实际可用的观察点比最初想象的更多，也更容易接近。本章从防御者的视角分析这些攻击：它们需要什么条件、能够取得什么效果，以及我们已经研究过的哪些协议机制可以提高攻击成本。

26.1 攻击者的位置

按照攻击者能够观察到的内容，可以将攻击分为：

一个端点 互联网接入服务商、本地网络或审查者可以观察客户端到入口守卫的连接；攻击者也可能观察出口到目的地的连接。在客户端与入口守卫之间，它能观察加密流量的时间、数据量和方向；出口与目的地之间是否有应用层加密则取决于协议。网站指纹识别可以利用客户端侧的元数据，而无需读取内容。

两个端点 攻击者通过在入口和出口同时运营中继，或者控制二者路径上的网络，观察通信两端，从而进行流量关联。

网络内部 恶意中继，包括中间中继，可以测量、延迟、标记或丢弃 cell。

底层互联网 中继连接经过的自治系统（AS）和互联网交换节点（IXP）等底层网络基础设施。

26.2 端到端流量关联

为了保持可用性，Tor 有意避免对 cell 施加混合网络式的长时间延迟或重新排序（第 2.2 节）。因此，流量的形状——每个时间区间内通过多少字节——通常仍可在电路两端被识别，但会受到延迟、缓冲、多路复用和其他流量的影响（图 26.1）。同时观察两端的人，不需要破解任何加密，就可以比较这些形状并判断两条流是否对应。

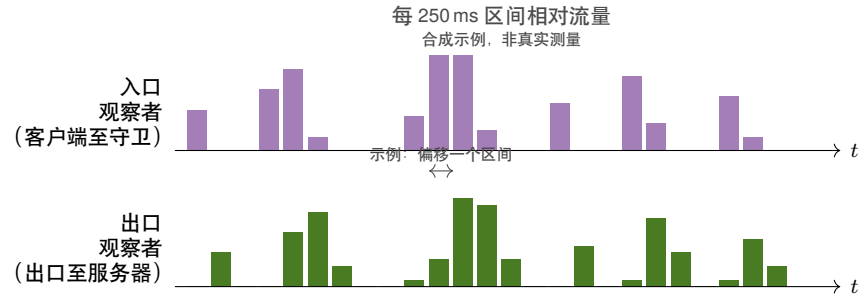


图 26.1: 单向流量关联的两条合成示例曲线，并非真实测量：分别表示客户端到入口守卫、出口到服务器链路的相对流量。突发模式相似但并不完全相同，图中的一个 250 ms 区间偏移仅为示例，而非 Tor 的固定延迟。加密可以保护内容，但未必隐藏流量时间和规模特征。缓冲、开销、多路复用和并发流量可能改变两端模式；统计相似不能单独证明流量关联成立。

匹配有多容易？ 2018 年，DeepCorr 训练神经网络来识别 Tor 流量对。在每条流约有 900 个数据包的条件下，它达到了 96 % 的准确率；相同条件下，之前的统计方法只有 4 % [NBH18]。对攻击者而言，最困难的部分通常不是比较流量，而是占据适当的观察位置。

攻击者同时观察到两端的概率有多高？ Johnson 等人使用 Tor 实际的路径选择算法，对典型用户进行了持续数月的模拟 [Joh+13]。假设攻击者运行具有中等带宽的恶意中继，在研究所建模的不同用户类型中，有 80 % 的用户在六个月内至少发生过一次去匿名化。面对单个自治系统，部分常见位置的用户在三个月内暴露概率接近 100 %。这些历史模拟结果并不意味着每条连接都遭到监视；真正的含义是攻击机会随时间积累。正是这一推理推动了更长寿命的入口守卫和每位客户端使用更少守卫的设计（第 13.3 节）。

以互联网基础设施为攻击者。 RAPTOR 识别出自治系统增加观察机会的三种途径 [Sun+15]：利用互联网路径的不对称性，因为在通信两端各观察一个方向的流量（包括 TCP 确认报文）就可能足够；利用随时间自然发生的路由变化；以及通过 BGP 劫持或拦截地址前缀。这些技术都无须修改 Tor。防御方向包括监控互联网路由，以及可能采用考虑网络拓扑的路径选择。

主动攻击。 控制通信一端的攻击者，可以故意通过延迟或集中发送数据包，为流量刻入可识别的节奏，并在另一端寻找这一模式。第 10.3 节 中讨论的加密层标记攻击，是这种思想的极端例子。早在 2005 年，Murdoch 和 Danezis 就表明，攻击者并不总需要直接观察真实链路。恶意服务器可以为发往受害者的流量施加特定节奏；攻击者再测量自己通过候选中继所建立电路的延迟，如果某些中继的延迟也随该节奏变化，就说明它们可能位于受害者电路上 [MD05]。

26.3 实验：关联自己产生的流量

实验 16：端到端流量关联

目标。在实验网络中证明，流量模式能够穿过电路而保留下来，并用于识别某条出口侧流量属于哪个客户端。
前置条件。实验 11；实验网络正在运行。所有流量都由实验脚本自行生成。
预计时间。1 小时。

`labs/zh-Hans/实验室/correlation.py` 脚本在容器内的端口 8095 启动服务器，并启动六个客户端。各客户端使用不同的 SOCKS 凭据，在 `IsolateSOCKSAuth` 启用时，不同客户端的

流不能共用同一电路（第 18.3 节）。在 20 秒内，客户端随机交替发送 2 KB 数据块和保持静默。两个“观察者”分别记录数据：

- 入口观察者记录每个客户端何时写入数据。这里简化了实际入口守卫链路观察者所能看到的内容：真实数据还会被封装进 cell 和 TLS，但关联所需的时间模式会穿过这两层。
- 出口观察者就是我们运行的服务器。它记录各条连接上的字节到达时间，同时收到一个用于核对结果的四字节标识符，但匹配算法不使用该标识符。

两个观察者都按 250 ms 的时间区间汇总字节。对每个客户端，我们计算其时间序列与全部六条出口侧流量之间的 Pearson 相关系数。相关性最高的流被选为可能的对应项。脚本实际上知道正确答案，因为客户端在最初四个字节中提供了身份信息，但这一信息只用于评估匹配结果，不用于选择最相关的流。

```
$ docker exec -w /tmp torbook-network python3 /lab/correlation.py 1
客户端 0:  +0.79 -0.11 +0.12 +0.17 +0.11 -0.01  -> 出口流 0 匹配
客户端 1:  -0.11 +0.80 -0.02 +0.07 +0.18 +0.08  -> 出口流 1 匹配
客户端 4:  +0.13 +0.02 +1.00 +0.21 +0.10 +0.21  -> 出口流 2 匹配
客户端 3:  +0.21 +0.09 +0.33 +0.81 -0.07 +0.03  -> 出口流 3 匹配
客户端 2:  +0.17 +0.05 +0.10 -0.09 +0.85 -0.03  -> 出口流 4 匹配
客户端 5:  -0.00 +0.01 +0.23 +0.06 -0.13 +0.92  -> 出口流 5 匹配
正确匹配数: 6/6
```

列对应按建立连接的到达顺序排列的出口侧流量，因此其顺序与客户端编号并不一致。每行的正确匹配都很明显，相关系数在 0.79 到 1.00 之间，其余通常接近零。这是一次参考运行的节选，而不是精确预测：实际运行时流的顺序与系数可能不同，也可能只匹配成功 5/6。这里既不需要解密密钥，也不需要查看解密后的 cell，只需要数据量和时间信息。实验网络规模很小，负载也较轻，因此延迟极小。DeepCorr 的历史研究表明，在所研究的条件下，更复杂的分类器甚至可以利用真实网络中几百个数据包进行匹配。此实验包含在本书的实验验证中。

26.4 网站指纹识别

如果攻击者只能观察客户端到入口守卫的链路，就不能执行端到端关联，但仍可能识别正在访问的网站。每个网页都会在双向连接上产生具有一定特征的 cell 序列，包括数量、顺序和集中发送的模式。攻击者先通过 Tor 访问目标网站，记录这些序列并训练分类器，再将受害者的流量与已学习的模式比较。

相关研究一直是分类器与防御机制之间的持续较量：

- 2011 年，Panchenko 等人利用流量大小、时间和方向特征，在包含 775 个网站的封闭集合中，将识别率从 3 % 提升到 55 %。他们还展示了有效的开放世界攻击，即受害者可能访问训练集之外的网站 [Pan+11]。
- 2018 年，Deep Fingerprinting 使用卷积神经网络，在 95 个未受保护的网站上取得超过 98 % 的准确率；面对曾被考虑用于 Tor 的自适应填充防御 WTF-PAD，准确率仍达到 90 % [Sir+18]。

深入说明：忽视基础发生率的谬误

极高的准确率数字需要谨慎解释。Juarez 等人指出，许多研究采用了有利于攻击者的假设：用户一次只访问一个网页、受害者与攻击者使用相同版本的 Tor Browser，而且训练数据在攻击发生前不久收集 [Jua+14]。更根本的问题是，在开放世界中，受监视的网站只占所有访问的一小部分。即使分类器的误报率很低，在数百万次无关访问中产生的误报也可能多于正确识别。这与针对罕见疾病的筛查测试面临的统计陷阱相同；真实攻击者同样无法回避。

防御机制。 通道填充（第 7 章）使空闲时段更难识别，却并非专门为抵御网页指纹攻击而设计。tor 还实现了更一般的电路填充框架：状态机可以按计划发送伪装用 cell。在本书参考日期，已经部署的状态机可以帮助洋葱服务引介与汇合电路看起来更像普通电路（第 14 章，提案 302）。能在可接受带宽开销下广泛有效防御网站指纹识别的方法，仍是尚待解决的研究问题。

26.5 Sybil：大量伪造中继身份

Sybil 攻击是指向网络引入大量身份。在 Tor 中，攻击者添加大量中继，希望它们频繁被选为入口守卫和出口中继。三类防御会提高攻击成本：

测量带宽 中继如果虚报可用带宽，就可能吸引过多客户端。Bauer 等人在 2007 年展示了这种基于描述符的欺骗手段 [Bau+07]。如今，共识带宽权重以目录权威节点的测量为基础，而不是仅信任中继的声明（第 12.4 节）；因此，吸引大量流量通常需要真正提供带宽。

路径约束 属于同一家族或同一个 IPv4 /16 网络的两台中继，不能同时出现在一条电路上（第 13.1 节）。诚实运营者会声明家族；不诚实的运营者至少仍受到 IP 地址分布的约束。

网络监控 权威节点可以排除中继或为其设置 `BadExit` 标志。志愿者与研究人员调查具有共同特征的中继群组，并监控出口行为。例如，2014 年一项出口扫描发现了 25 台操纵流量或配置不当的中继，其中包括 HTTPS 拦截、DNS 篡改与 *sslstrip* [WL14]。

第 27 章 将研究此后发现的恶意中继群组，并说明为什么监控必须持续进行。

针对洋葱服务的 **Sybil** 攻击。 在 v2 中，负责某个服务的 HSDir 位置可以预测。攻击者因此能够生成特定中继身份，让它们占据这些位置。Biryukov 等人在 2013 年利用这种手法，测量服务热度、使服务不可访问，或协助定位攻击 [BPW13]。在 v3 中，位置取决于盲化密钥和每天更新的共享随机值（第 14 章），因此显著增加了提前预计算和定向布置恶意 HSDir 的难度；但当期随机值公布后仍可算出负责的 HSDir，并不能完全消除恶意目录节点。

26.6 发现入口守卫

入口守卫保护客户端，却也成为一个有吸引力的攻击目标：一旦知道它是谁，攻击者就知道应该在哪里观察。面对洋葱服务，攻击者可以通过不断请求连接等方式触发大量电路建立，直到某个恶意中继被选为紧邻守卫的中间中继。2006 年发表的早期同类攻击，也推动了入口守卫机制的发展 [ØS06]。Vanguards（第 13.4 节）则额外引入一层在一段时间内保持固定的中间中继。这会拖慢攻击者逐层追踪至真正入口守卫的过程，但并不保证守卫无法被发现。

本章小结

- 攻击的区别在于观察位置：一个端点、两个端点、网络内部，或底层互联网路由。
- 流量关联无需破解加密，只需比较两端的流量形状。攻击难点在于占据适当位置，而观察机会会随时间积累。
- 网站指纹识别仅根据一端的加密流量识别网页。评估时必须考虑基础发生率和现实条件。
- Sybil 防御包括带宽测量、家族声明、/16 约束，以及对出口和中继群组的监控。在 v3 中，提前操纵 HSDir 位置更难，但当期负责的 HSDir 仍可计算。
- 入口守卫和 Vanguards 使发现第一跳的攻击更加缓慢且昂贵。

练习

练习 **26.1** ⋆ 针对本章每种攻击，确定攻击者至少需要哪种观察位置，并找出能够提高该攻击成本的 Tor 防御机制。

练习 **26.2** ⋆ 某分类器以 0.95 的概率正确识别受监视的网站，并以 0.001 的概率把无关访问误报为该网站。如果每 100 000 次访问中只有一次真正访问了被监视的网站，那么所有警报中有多少比例是正确的？

练习 **26.3** ⋆⋆ 重复实验 16，先把时间区间改为两秒，再把实验总时长缩短为五秒，最后让所有客户端采用相同的流量节奏。哪种修改对匹配结果的破坏最大？为什么？

练习 **26.4** ⋆⋆ 修改实验 16 的客户端，让它们以固定速率发送数据块，即使没有实际数据也发送填充数据。这会消耗多少额外带宽？流量关联结果会怎样变化？

练习 **26.5** ⋆⋆⋆ 使用 `path_selection.py` 和实验 15 的真实共识，计算如下概率：攻击者控制 5 % 的守卫带宽和 5 % 的出口带宽，在六个月内至少一次同时占据某客户端电路的两端。比较固定入口守卫与每条电路都重新选择守卫的情形。请明确假设六个月内的电路数量，并仅在教学模型中按照带宽权重进行独立抽样；解释该模型为何不能预测真实网络。

延伸阅读

- A. Johnson 等，*Users Get Routed* [Joh+13]。
- Y. Sun 等，*RAPTOR: Routing Attacks on Privacy in Tor* [Sun+15]。
- M. Juarez 等，*A Critical Evaluation of Website Fingerprinting Attacks* [Jua+14]。
- *Anonymity Bibliography*：`https://www.freehaven.net/anonbib/`，可用于检索后续研究。

第 27 章

针对真实 Tor 网络的攻击案例

最后核验：*2026* 年 *9* 月 *23* 日 — *Tor Project* 公告（*2014*、*2022*、*2023* 年）；*tor 0.4.9.13*；实验网络

本章目标

学完本章后，你将能够：

- 重建 2014 年涉及 **RELAY_EARLY** cell 的攻击与随后加入 tor 的防御，并在运行中的实验网络验证防御机制（实验 17）；
- 描述 KAX17 事件以及后来加强的中继监控工作；
- 将 2022–2023 年拒绝服务攻击与 第 15 章 介绍的防御联系起来；
- 总结保护开放网络安全的一般经验。

上一章从研究角度讨论攻击：模型、实验和统计比例。本章根据 Tor Project 公开发表的声明，重建真实网络上发生的三起安全事件。对于 KAX17，运营者的意图及是否开展了具体恶意活动并未得到证实。我们的重点不是操作性的攻击细节，而是这些事件能帮助我们理解哪些协议问题，以及如何管理一个开放网络。

27.1 2014 年：利用 Cell 类型传递信号

发生了什么？ 2014 年 7 月 4 日，Tor Project 发现并移除了一个中继群组，认为它们试图使发布洋葱服务描述符或请求其信息（当时称为隐藏服务）的用户失去匿名性。这些中继从 1 月 30 日起加入网络：大约 115 台高速、非出口中继，分布在两个 /16 网络中，总共占入口守卫容量的约 6.4 %。7 月 30 日，Tor Project 发布安全公告，介绍攻击及应对措施 [Din14]。

攻击如何运作？ 攻击将 Sybil 身份部署（第 26.5 节）与主动流量确认（第 26.2 节）结合起来。这些中继保持足够长时间的稳定运行，从而同时获得 `HSDir` 和 `Guard` 标志。当恶意 HSDir 收到描述符请求时，会使用交替出现的 **RELAY** 和 **RELAY_EARLY** 类型构成一串 cell，以此编码被请求的服务名称（图 27.1）。如果同一群组中的另一台中继恰好是该客户端的入口守卫，它就可以读取这串信号，将客户端与对该服务描述符的请求关联起来，但不能据此证明客户端实际访问了该服务。

为什么偏偏使用这个字段？ **RELAY_EARLY** cell 原本用于限制电路的长度：客户端必须使用它们发送扩展请求，每条电路至多允许八个（[tor-spec § Handling RELAY_EARLY cells]）。然而，旧版检查只适用于客户端发往网络的方向。正常情况下没有人反向发送这种 cell，也没有人检查它们。与分层加密的载荷不同（第 10 章），外层 cell 命令对每台中继都可见。因此，写入该字段的信号能够在传输中保持不变。公告还指出，攻击者以可直接读懂的形式编码了服务名称，因此收到此类 cell 的中继可以理解该信号；仅持有网络抓包的外部观察者，则必须先解密中继之间的 TLS 链路，才能事后恢复该信号。

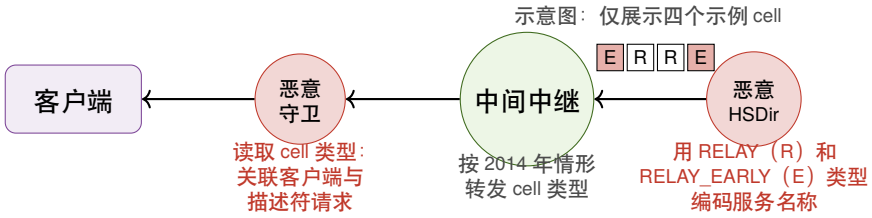


图 27.1: 2014 年的历史攻击：恶意 HSDir 在外层 RELAY 和 RELAY_EARLY cell 命令中编码所请求的服务。该命令对路径上的中继可见，但在链路上受 TLS 保护。同伙控制的恶意入口守卫能够把客户端与描述符请求关联，不能据此证明客户端实际访问了服务。图中四个 cell 仅为示意，并非完整 onion 地址编码。

应对措施。 随后采取了三个步骤：排除恶意中继；从 tor 0.2.4.23 和 0.2.5.6-alpha 开始，一旦收到发往客户端的 **RELAY_EARLY**，就关闭电路；并逐步推动将每位客户端的守卫数量从三台降为一台，以减少攻击者观察流量的机会。公告强调，被动流量确认依然是尚未解决的问题。利用 cell 类型的信道使确认变得更容易，而不是第一次让流量确认成为可能。

27.2 实验：验证 RELAY_EARLY 的数量限制

实验 17：允许八个，第九个不允许

目标。使用 `labs/zh-Hans/代码/torlab` 验证当前中继如何执行 **RELAY_EARLY** cell 数量限制。
前置条件。实验 4 和实验 5；实验网络正在运行。
预计时间。30 分钟。

我们的 `circuit.py` 已经阻止发送超过八个 **RELAY_EARLY** cell。`labs/zh-Hans/实验室/relay_early.py` 脚本则有意绕过这一保护。它先与 `test004r` 建立一跳电路，发送 n 个包含 `DROP` 消息的 **RELAY_EARLY** cell（中继会直接丢弃这些消息，不作回应），随后使用 `BEGIN_DIR` 请求目录连接，以检查电路是否仍然有效：

```
$ PYTHONPATH=labs/zh-Hans/代码 python3 labs/zh-Hans/实验室/relay_early.py 0 8 9
0 RELAY_EARLY cells -> CONNECTED
8 RELAY_EARLY cells -> CONNECTED
9 RELAY_EARLY cells -> circuit destroyed, reason 1
```

第九个 cell 会导致电路关闭，中继返回原因码为 1、即 `PROTOCOL` 的 **DESTROY** cell。该列表来自一次参考运行：地址、电路编号和日志内容会因运行而异。一份参考中继日志显示：

```
[warn] Received too many RELAY_EARLY cells on circ 4045911407 from
       127.0.0.1:58048.  Closing circuit.
```

该检查位于 `src/core/or/command.c`，发生在 cell 解密之前。再往上几行就是 2014 年引入的检查：如果收到发往客户端的 **RELAY_EARLY**，就记录警告并关闭电路。源码注释指出，旧版 tor 曾因缺陷产生这种 cell，但受影响版本早已不再使用。如今它被视为“异常”输入，并按攻击处理。此实验也包含在 `make verifica-lab` 中。

27.3 2020–2021 年：KAX17 与网络健康

第二起事件展示了防御中不太显眼的一面：首先得发现攻击者。Tor Project 于 2022 年 4 月在介绍其网络健康团队的文章中讨论了该事件 [The22]。

2020 年 12 月，中继统计图显示异常模式：每月第一天的午夜，数十台中继总会在同一时刻加入网络，此后逐渐消失。这看上去像休眠机制（`AccountingMax`，第 22.2 节）存在缺陷。到 2021 年 9 月，另一批似乎由同一运营者控制的中继开始出现，却没有家族声明和联系信息。目录权威节点排除了它们，但替代中继不断加入。10 月下旬，一份匿名报告帮助识别了后来称为 KAX17 的中继群组，该群组随即被移除。此后，每月的异常高峰和中继加入浪潮都停止了，Tor Project 因而认为这些现象可能来自同一运营者。运营者的身份和目标至今不明。

文章坦率地反思了响应延迟的原因：多年来监控工具缺乏维护，志愿者、员工和目录权威节点之间的信任不足，组织结构也不适合开展这种工作。它还解释了治理程序：网络监测人员可以建议排除中继，但最终由目录权威节点依照多数决定，而不是由 Tor Project 员工直接决定。这正是 第 12 章 中介绍的分布式信任在网络运营中的体现。

深入说明：没有声明家族的中继

KAX17 事件的最初警讯不是观察到某种漏洞利用，而是违反了一条社会性规则：大量显然由同一运营者控制的中继没有声明家族（第 22.5 节）。更深层的经验在于，如果攻击者能够绕过路径选择所依赖的运营惯例，有时根本不需要攻破协议。

27.4 2022–2023 年：拒绝服务攻击

从 2022 年夏季起，Tor 网络连续数月遭遇多种拒绝服务攻击。2023 年 2 月，Tor Project 报告称，用户有时无法加载网页或访问洋葱服务；攻击方法和目标不断变化，也无法可靠确认攻击者身份 [The23b]。洋葱服务尤其容易受到攻击：请求通过匿名电路到达，因此它们无法直接封锁攻击者的 IP 地址。

更长期的应对措施，是从 2023 年 8 月的 tor 0.4.8 起引入的工作量证明系统（第 15.4 节）[The23a]。该防御通常处于休眠状态；在服务遭到攻击时，它会要求客户端解答计算难度不断增加的问题，并优先处理投入更多工作量的请求。Tor Project 将正常用户所需的成本描述为几毫秒，而严重攻击期间可能达到约一分钟。这与实验 9 的原则相同：让攻击者原本很廉价的行为变得昂贵，同时不要求用户提供身份信息。

27.5 经验总结

- 每个可见字段都可能成为通信信道。2014 年的攻击没有破解密码学，而是利用中继能够读取并转发的字段。设计匿名协议时，必须针对每个未加密的比特追问：谁能写入它，谁能读取它？这也是新一代中继加密设计（第 10.4 节）的动机之一。
- **Sybil** 攻击可能很廉价，监控却不廉价。添加中继的成本，相比维护检测恶意协同行为所需的工具、专业知识和信任关系，可能很低。
- 防御必须考虑时间维度。使用单个守卫、延长守卫寿命以及引入 Vanguards，都能减少攻击者随时间获得的机会（第 26.2 节）。
- 透明性也是一种防御。在这三起事件中，Tor Project 都同时公布了已知与未知的信息，让用户、运营者和研究人员能够评估风险。

本章小结

- 2014 年，一组同时担任入口守卫和 HSDir 的恶意中继利用 cell 类型编码客户端所请求描述符对应的洋葱服务名称。现在，tor 收到发往客户端的 **`RELAY_EARLY`** 时会关闭电路；守卫数量从三台减为一台，随后在 2018 年调整为两台，也减少了选中恶意守卫的机会（第 13.3 节）。

- 实验网络可以验证最多八个 **RELAY_EARLY** cell 的限制：第九个会导致电路销毁。
- KAX17 在数月的间接线索（包括每月异常高峰与未声明家族的中继）以及一份匿名报告后被识别；排除中继的决定由目录权威节点负责。
- 2022–2023 年的拒绝服务攻击推动了洋葱服务工作量证明的引入。

练习

练习 **27.1** ⋆　恶意 HSDir 如果把信号写在 cell 的加密载荷中，为什么入口守卫无法读取，而写在外层 cell 类型中却可以被读取？

练习 **27.2** ⋆　解释将守卫数量从三台减少到一台，如何缓解类似 2014 年的攻击，以及这种改变带来什么缺点。

练习 **27.3** ⋆⋆　修改实验 17，证明即使一个 **RELAY_EARLY** cell 没有包含电路扩展请求，它仍会计入数量限制。然后检查 tor 源码，判断一条 **RELAY_EARLY** 在发往客户端的方向到达时会发生什么。

练习 **27.4** ⋆⋆　利用 CollecTor 数据，哪些迹象可能暴露同一运营者控制、却未声明家族的中继？提出三个检测指标，并讨论误报。

练习 **27.5** ⋆⋆⋆　列出中间中继能以明文看到的 cell 和 relay 消息字段（第 8 章）。逐项讨论恶意中继能否利用这些字段向电路上的另一台中继发送信号，并识别已有或正在提议的防御。

延伸阅读

- Tor Project，*Tor security advisory: "relay early" traffic confirmation attack* [Din14]。
- Tor Project，*Malicious relays and the health of the Tor network* [The22]。
- Tor Project，*Introducing Proof-of-Work Defense for Onion Services* [The23a]。

第 28 章

操作安全失误与去匿名化：经验教训

最后核验：*2026* 年 *9* 月 *23* 日 — *Tor Project* 支持文档；*Tails* 安全警告；*tor 0.4.9.13*；实验网络

本章目标

学完本章后，你将能够：

- 区分身份信息可能泄露的不同层次，并判断哪些工具能够保护每一层；
- 识别常见的操作安全失误，包括 DNS 泄露以及匿名集过小；
- 使用 tor 本身检查应用程序是否把域名解析交给 tor（实验 18）；
- 根据威胁模型分析真实事件。

第 26 章 末尾指出了一个令人不安的事实：攻击 Tor 协议有时成本高昂，而且要求攻击者占据特殊观察位置。但在许多有记录的事件中，匿名性也因工具使用失误而受损，仅靠网络协议无法避免这类问题。本章从试图保护自己的人所处的视角研究这些错误，以明确 Tor 的保护边界，以及用户本人需要承担的责任，而不是介绍如何识别他人身份。

注意：本章的讨论范围

我们只从自我保护的角度讨论失误与防御习惯，不提供识别个人身份的操作流程。提到的事件都已公开，并依据引用来源重建。

28.1 四个层次

身份可能在四个层次泄露（图 28.1）。第一层是网络，也是 tor 直接保护的层次：在常规 Tor 连接中，目的地看不到客户端的原始 IP 地址，单个电路中继通常也不掌握通信两端的完整信息；不过这不能消除流量关联等风险。第二层是应用程序，需要 Tor Browser、Tails 和 Whonix（第 23 章）提供保护。最后两层是内容和行为，tor 并不会自动保护它们。专门工具可以减轻部分风险，但用户所写、所上传和所做的事情也至关重要。

28.2 应用程序层面的失误

流量绕过 **Tor**。 tor 只保护已经配置为使用它的应用程序 [Thef]。直接联网的软件，即使只是获取更新或生成预览，也可能暴露真实 IP 地址。Tor Project 的支持文档特别提到两种常见情况。BitTorrent 程序可能忽略代理设置，或者仍向跟踪器透露自己的地址。下载后在联网状态下打开的文档也是一种风险：DOC 或 PDF 文件可能包含远程资源，被不使用 Tor 的程序自动获取。

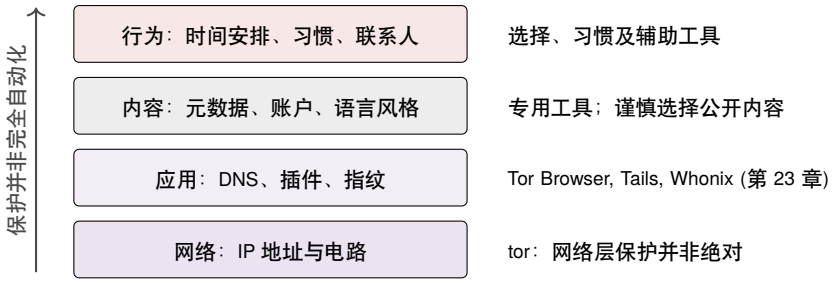


图 28.1: 身份线索可能暴露的四个层次（教学示意图）。Tor 能减少部分网络风险，应用层防护与内容处理工具有所帮助，但用户披露的信息及行为在各层次都很重要。任何一层都不能单独保证匿名。

DNS 泄露。 更隐蔽的问题与 SOCKS 有关。应用程序可以将目的地的域名交给 tor，让出口进行解析（第 11.1.4 节）；也可以先调用系统的 DNS 服务自行解析，然后只向 tor 提供 IP 地址。后一种情况下，后续连接仍经过 Tor，但“这个网站在哪里?”的解析可能已在 Tor 电路之外向解析器暴露域名，并与客户端的网络连接关联。但并非必然发生明文 DNS 查询：答案也可能来自缓存、本地文件或加密 DNS。tor 无法证明此前已经发生泄露，只能提示 SOCKS 请求仅携带 IP 地址这一风险信号，下面的实验将展示这一点。

28.3 实验：tor 识别 DNS 泄露的警讯

实验 18：携带域名与不携带域名的 SOCKS 请求

目标。观察 tor 如何区分让自己解析域名的应用程序与自行解析的应用程序，并了解 tor 如何拒绝后一类请求。
前置条件。实验 11；实验网络正在运行。
预计时间。30 分钟。

`curl` 支持两种行为。`--socks5` 在本地解析域名，只向 tor 提供 IP 地址；`--socks5-hostname` 则直接传递域名。`labs/zh-Hans/实验室/dns_leak.py` 脚本连接客户端 `008c` 的 ControlPort，启用 `TestSocks`，让 tor 记录每个 SOCKS 请求的类型。随后脚本监听日志消息，使用两种方式访问 `http://localhost:8080/`。

本实验展示的是机制，而不是真正从网络中观察到了 DNS 泄露。`localhost` 在本机解析，不会联系外部解析器，因此没有 DNS 查询离开计算机。tor 发现收到的是 IP 地址而非域名，这是此前本地解析的可能迹象，却不能证明发生了 DNS 泄露（程序也可能直接使用 IP 地址）。若换成普通域名，类似的本地解析可能产生外部查询，其是否为明文取决于 DNS 配置。为了避免此类流量，我们有意只使用私有网络。脚本还会覆盖可能让 `localhost` 绕过代理的 `NO_PROXY` 设置，强制使用 SOCKS。最后，它启用 `SafeSocks`，即使没有 DNS 泄露也拒绝仅携带 IP 地址的请求：

```
--socks5           HTTP 200  tor: Your application (using socks5 to port
                   8080) is giving Tor only an IP address. Applications
                   that do DNS resolves themselves may leak information...
--socks5-hostname  HTTP 200  tor: Your application (using socks5 to port
                   8080) instructed Tor to take care of the DNS resolution
                   itself if necessary. This is good.
--socks5 SafeSocks HTTP 000
```

在示例输出中，前两种请求都能成功，但只有第二种将域名解析交给 tor；第一种使用 `localhost`，不足以证明 DNS 泄露。在 tor 源码（`src/core/proto/proto_socks.c`）中，第一条警告以 `warn` 级别记录，并进行速率限制以免日志被淹没；同时 ControlPort 会产生动作

类型为 `DANGEROUS_SOCKS` 的 `STATUS_CLIENT` 事件。因此，控制程序能够向用户发出警告。当 `SafeSocks` 启用后，相同请求会遭到拒绝，`curl` 也得不到响应。此实验包含在 `make verificalab` 中。

28.4 内容层面的失误

元数据。 许多文件包含作者甚至没有注意到的信息：照片可能泄露位置或相机型号，文档可能透露作者姓名和创建时间。Tails 内置元数据清除工具，其安全警告也提到过照片元数据泄露拍摄者所在位置的情况 [Taib]。

账户与身份。 登录个人账户就等于直接告诉网站自己是谁，不论连接使用什么 IP 地址 [Thef]。在同一网站、同一会话中使用两个身份，也可能因 Cookie、会话标识符、应用层特征等被关联；但网站无法直接读取 Tor 电路标识符。Tails 建议在需要彼此隔离的活动之间重启系统 [Taib]。Tor Browser 的不可关联性设计（第 23.2 节）保护不同网站之间的活动，并不自动分离同一网站上的多个身份。

写作风格。 即使没有元数据，语言习惯、话题、反复出现的错误以及发布时间，也可能将匿名文本与署名作品联系起来。没有任何自动化工具能够消除所有此类线索。

28.5 行为失误：匿名集过小

第 1 章 以在某个集合内无法彼此区分来定义匿名性。Tor 能减少某些使网络活动容易区分的特征，但无法保证在特定时间、特定地点有足够多人同时使用 Tor。如果使用者寥寥无几，仅仅知道谁使用了 *Tor*，就可能大幅缩小候选范围。

一起事件。 2013 年 12 月，美国一所大学在考试前夕收到虚假的炸弹威胁电子邮件。发送者使用临时电子邮件服务和 Tor。根据多家媒体对随刑事诉状附上的 FBI 宣誓书的报道，校方调查了相关邮件发出前几小时内，哪些人在校园无线网络上使用过 Tor [WBU13; San13]。随后，这名学生在接受询问时承认发送了邮件。调查并不需要破解 Tor 的加密：本地网络观察者可以看到 Tor 的使用情况，但这并不表示他们能看到连接内容或目的地；他们可以知道谁在那段时间使用 Tor，于是将调查范围缩小到少数人。

这起事件说明威胁模型中的三个原则（第 4 章）：

- 不使用网桥时，Tor 不会隐藏正在使用 Tor 这一事实：本地网络观察者可能识别其连接 [Taib]。带可插拔传输的网桥（第 16 章）有助于减少这种暴露。
- 如果候选集合很小，且事件发生在狭窄时间窗口内，即使没有全球性观察能力，攻击者也可能通过集合交集缩小候选范围。
- 匿名需要同伴 [DM06]。同一环境中使用 Tor 的人越多、使用原因越多样，任何一个人使用 Tor 这一事实就越不具有识别力。Tor Project 也明确鼓励更广泛地使用 Tor [Thef]。

28.6 自我检查清单

这份清单并不穷尽所有问题，但按照不同层次提供了主要建议：

1. 使用 Tor Browser，或者 Tails、Whonix 等系统，而不是手动设置代理的普通浏览器。
2. 避免安装扩展或改变会让浏览器显得独特的设置。
3. 对其他应用程序，验证它们是否把域名交给 tor（`SafeSocks`、`TestSocks`）。

4. 联网期间不要直接打开下载的文档，或者在隔离环境中处理。
5. 优先使用 HTTPS 和洋葱服务，因为出口能够看到未加密的内容。
6. 分享文件前清除元数据。
7. 不要在同一会话中混合必须分离的身份。
8. 如果仅仅被发现使用 Tor 就有风险，应考虑适当的混淆网桥，但不能假定它能使活动完全不可见。
9. 记住时间安排、习惯和写作风格也会传递信息。

本章小结

- 身份可能从网络、应用、内容和行为四个层次泄露；tor 直接保护的只有第一层。
- 绕过 Tor 的应用程序、在联网时打开下载文档，以及 DNS 泄露，都属于应用层失误。tor 可以针对只提供 IP 地址的 SOCKS 请求发出警告并拒绝它们。
- 元数据、账户和写作风格，可能把内容与特定个人联系起来。
- 如果 Tor 用户集合很小，知道谁在何时使用了 Tor，就可能足以缩小识别范围。网桥与广泛的用户群体有助于降低这一风险。

练习

练习 **28.1** ⋆ 针对本章讨论的每一种失误，在 图 28.1 中找出对应层次，并指出有助于预防该失误的工具或习惯。

练习 **28.2** ⋆ 解释为什么即使后续连接通过 Tor，DNS 泄露仍然会暴露信息。谁能够看到这些信息？

练习 **28.3** ⋆⋆ 在实验 18 中，通过 Stem 订阅 `STATUS_CLIENT` 事件，显示仅提供 IP 地址的请求所触发的 `DANGEROUS_SOCKS` 动作。图形界面可以如何利用它？

练习 **28.4** ⋆⋆ 重新分析 2013 年事件，假设当事人从同一网络使用 obfs4 网桥。本地网络观察者可能看到什么？还有哪些信息可能缩小候选集合？

练习 **28.5** ⋆⋆⋆ 选择一名因正当理由需要匿名的真实或虚构人物，例如新闻消息来源、活动人士或研究人员，编写一份操作安全计划，涵盖威胁、相关层次、工具与习惯，以及残余风险。

延伸阅读

- Tor Project 关于匿名性的支持指南 [Thef]。
- Tails 警告，*Tails is safe but not magic* [Taib]。
- R. Dingledine 和 N. Mathewson，*Anonymity Loves Company* [DM06]。

第五部分

研究与未来

第 29 章

Tor 学术研究

最后核验：*2026* 年 *9* 月 *23* 日 — 本章引用的研究论文；*Tor Project* 研究门户；*2026* 年 *9* 月 *23* 日的共识

本章目标

学完本章后，你将能够：

- 找到 Tor 研究的重要发表渠道，以及 Tor Project 提供的资源；
- 从熵到首次失守时间，定量衡量匿名性；
- 选择合适的实验方法：分析建模、模拟、仿真或实际测量；
- 使用真实共识模拟数千个客户端在数月内的活动（实验 19）。

从一开始，Tor 的发展就与学术研究紧密相连。洋葱路由诞生于研究实验室（第 3 章）；Tor 的设计曾在学术会议发表 [DMS04]；本书介绍的众多机制，从入口守卫到拥塞控制，都是在回应已发表的研究成果。本章面向希望自己开展研究的读者：到哪里发表成果、如何衡量匿名性、应该选择哪些实验工具，以及必须遵守哪些安全原则。

29.1 Tor 研究发表在哪里？

匿名网络研究主要发表在两类渠道：

Privacy Enhancing Technologies Symposium（**PETS**）以及对应的期刊 *Proceedings on Privacy Enhancing Technologies*（PoPETs）。这是隐私技术领域的专门发表平台；下文讨论的匿名性测量奠基研究发表于 2002 年的 PET 研讨会，早于 PoPETs 期刊创办 [Día+02]。

重要安全会议 包括 USENIX Security、ACM CCS、IEEE Symposium on Security and Privacy，以及 NDSS。第 26 章 引用的多篇论文发表于这些会议。

Free Haven 的 *Anonymity Bibliography* 已经收录这一领域二十多年的研究成果，是文献综述的自然起点。

Tor Project 还维护研究门户，提供数据、工具、技术报告、研究选题以及 Research Safety Board 的相关信息。其建议强调，协议和网络一直在变化：如果不了解内部工作方式就进行测量，很容易得出错误结论。因此，研究人员应尽早与项目联系，了解同时进行的实验和计划中的修改。

29.2 如何衡量匿名性？

在 第 1 章 中，我们用某个集合内的不可区分性来定义匿名性。然而，从攻击者的视角看，集合中每个成员成为目标的可能性并不相同。观察流量后，攻击者可能认为某些用户的概率远高于其他人。仅仅知道集合中有多少人并不够。

匿名程度。 2002 年，Díaz 等人提出通过攻击者赋予潜在发送者的概率分布的熵来衡量匿名性 [Día+02]。如果 p_i 是攻击者认为用户 i 为发送者的概率，熵为

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i,$$

当所有 N 个用户的概率相同时，熵达到最大值 $H_M = \log_2 N$。匿名程度定义为

$$d = \frac{H(X)}{H_M}, \qquad 0 \le d \le 1:$$

当 $N \ge 2$ 且后验分布均匀时，$d = 1$（这不一定表示攻击者没有获得新信息）；只有一名发送者的概率为 1 时，$d = 0$。若 $N = 1$，由于 $H_M = 0$，此比值无定义。

理解熵的一种直观方式是计算 $2^{H(X)}$：它表示在相同不确定性下，概率相等的用户数量。假设匿名集有 1 000 人，但攻击者已将嫌疑范围缩小到两人，各有 0.5 的概率，那么不确定性就与只有两人的集合相同：$H = 1$ 比特。

单次测量的局限。 熵描述单次观察，但正如 第 26.2 节 所示，Tor 的暴露风险会随时间累积：每条新电路都为攻击者带来一次新机会。因此 Johnson 等人提出纵向指标，包括在特定时间内至少一次失去匿名性的概率，以及首次失守时间的中位数。这些指标可通过结合真实网络数据模拟用户行为计算 [Joh+13]。下面的实验采用类似方法。

29.3 四种研究方法

分析建模 建立数学模型，例如匿名程度模型，或 第 26.5 节 中的 Sybil 攻击模型。它速度快、适用范围广，但依赖各种假设，而实际系统总会在一定程度上偏离假设。

行为模拟 使用真实数据模拟系统某个简化的部分，例如路径选择。Johnson 等人开发的路径模拟器 TorPS [Joh+13] 和本书实验 19 都采用这种方式。其运行速度快，却无法涵盖模型遗漏的因素。

网络仿真与模拟 在本书贯穿始终使用的 Chutney 私有网络中运行真实软件，或者在 Shadow 等离散事件模拟器内运行它 [JH12]。Shadow 可以通过虚拟时间，在一台机器上执行数千台中继和客户端（第 19.9 节）。这种方法尤其适合性能研究，因为新的协议设计可以先在逼真的网络中测试，再部署到公共网络。

公共网络测量 只有直接测量才能反映真实网络正在发生的情况，但它对真实用户也有最高风险，因此需要严格的实验设计（第 29.5 节）。

优秀研究通常结合多种方法：数学分析产生假设，大规模模拟检验假设，精心设计的测量再考察模型是否符合现实。

深入说明：在不观察个体的情况下测量

Jansen 和 Johnson 开发的 PrivCount 展示了测量与隐私保护可以共存 [JJ16]。参与测量的中继虽然会统计相关事件，但计数器使用与其他参与者共享的随机值进行掩蔽，并依据差分隐私原则加入噪声。最终可见的只是经过聚合且刻意带有噪声的结果，从而限制人们推断单个用户信息的能力。2016 年，研究人员利用 PrivCount 估计约有 710 000 名同时在线客户端，其中约 550 000 名在某一时刻处于活跃状态；流向 Web 端口的流量占参与测量的出口所记录数据字节的 91%。这些是 2016 年研究的历史数据，并非当前网络统计。这比 第 24.2 节 中通过舍入中继统计保护隐私的做法更进一步。

29.4 实验：首次失守需要多长时间？

实验 19：模拟数月的电路

目标。使用真实共识估算：攻击者控制一定比例带宽时，分别在使用与不使用持久守卫的情况下，随时间能使多少客户端失去匿名性。
前置条件。实验 7 和实验 15；无须运行私有网络。
预计时间。1–2 小时。

`labs/zh-Hans/代码/torlab/simulation.py` 模块复用实验 7 的带宽权重。攻击者增加一台守卫和一台出口，两者各占对应位置加权带宽的比例 q。我们模拟 2 000 名客户端运行 180 天，每个客户端每天建立 20 条电路。

本模型按照已确认（confirmed）守卫的近似寿命安排持久守卫的轮换，而不是完整复现实际客户端行为。根据 guard-spec 和 `entrynodes.c` 中的代码，已确认守卫在确认后达到 `GUARD_CONFIRMED_MIN_LIFETIME`（60 天）时会从样本中移除，而 `CONFIRMED_ON_DATE` 会随机向前回溯最长 `GUARD_LIFETIME`/10（12 天）。因此，模拟中的轮换间隔为 48–60 天。120 天的 `GUARD_LIFETIME` 用于管理未确认守卫在候选样本中的到期时间。真实客户端还会考虑守卫样本和确认列表、可达性及替换策略，无法直接用这一间隔预测。实验的早期版本误将守卫轮换间隔设为 120 天，从而在这个简化模型中低估了结果。作为对照，我们还模拟每条电路都重新选择守卫的客户端。当某个客户端第一次建立同时包含恶意守卫与恶意出口的电路时，就将其判定为失守。

这个模型有意保持简单：忽略中间中继、家族、/16 网络、备用守卫和节点故障。为了保证速度，它没有使用每次抽样都重新计算数千个权重的 `random.choices`，而是在累积权重数组上进行二分搜索，从而无需每次都重新计算全部权重；具体运行时间取决于机器。

要复现这些数字，请先获取公开的历史共识（如执行 `make -C labs/zh-Hans data`）；该文件未直接存入 Git。模拟是确定性的：在模型、共识、比例和随机种子都相同的情况下，结果总是如下：

```
$ PYTHONPATH=labs/zh-Hans/代码 python3 -m torlab.simulation labs/zh-Hans/数据
    /collector/consensus-microdesc-2026-09-23-15 0.05
攻击者：守卫带宽 5%、出口带宽 5%；2000 名客户端、180 天、每天 20 条电路
持久守卫              1d:  2.6%   7d:  4.4%   30d:  4.4%   90d:  9.4%  180d: 18.4%  中
    位数：None
每条电路重新选守卫    1d:  4.3%   7d: 27.9%   30d: 76.5%   90d: 98.9%  180d:100.0%  中
    位数：15
```

图 29.1 显示了两条曲线。如果不使用持久守卫，十五天内就有一半客户端失守，三个月内几乎全部失守，因为每条新电路都提供一次新的独立机会。使用持久守卫时，风险主要在选择守卫的时刻确定。约 4% 的客户端早早选择了恶意守卫并失守；其余客户端在轮换守卫前受到保护。在该模型中，各客户端约每 48–60 天分别轮换守卫、并非同步发生，新的抽样机会使更多客户端暴露，六个月后总失守比例达到约 18%。如果攻击者控制的带宽比例降至 1%，持久守卫情况下的六个月失守比例约为 4%，而不使用持久守卫时约为 31%。

这些基于 2026 年 9 月参考快照的结果，说明了守卫设计的理由。在这一简化模型中，守卫的主要作用是改变不同客户端之间的风险分布，而不是保证每条电路的风险都会下降。模型中，大多数用户一开始的风险接近零，但少数用户的风险很高。因此，守卫寿命是一种权衡：轮换越慢，使用诚实守卫的用户就能受到更长时间的保护，但选中恶意守卫的用户也会更长时间处于暴露状态。能够在网络中潜伏数月的攻击者，每次轮换都会获得新机会。

29.5 如何安全地开展研究

第 25 章 介绍了 Tor Research Safety Board 的指南 [Tor]。设计实验时，首先应当问：我的研究问题真的需要公共网络吗？许多性能和攻击问题，都可以用 Shadow 或私有网络研究，并

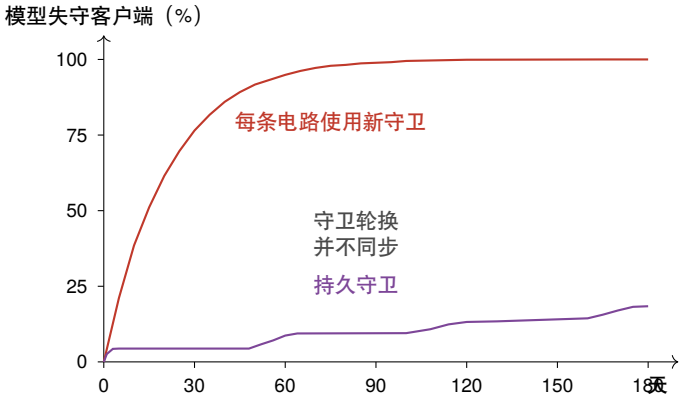


图 29.1: 实验 19 的简化模拟：使用 2026 年 9 月 23 日的共识快照，模拟 2000 名客户端在 180 天内每天建立 20 条电路；攻击者分别控制 5% 的守卫和出口加权带宽。累计比例表示在模型中至少有一条电路同时使用恶意守卫与恶意出口的客户端。持久守卫轮换根据客户端独立发生（模拟间隔 48–60 天），不存在全体同步的三个轮换日期。这不是公共网络真实去匿名化事件的观测值。

得到可重复的实验和可控的变量。当公共网络测量确实不可替代时，应遵循前面介绍的原则：只针对自己产生的流量操作，只收集最少的数据，在发布前聚合结果，并在开始前咨询安全委员会和 Tor Project。

近期历史提供了形成鲜明对比的案例。第 27 章 描述了一种通过接收中继可见、但外部观察者无法直接从加密 TLS 流量中读取的信号影响真实用户的实验或攻击。PrivCount 则表明，可以在不暴露个体用户的前提下细致测量网络。两者的区别不仅在于工具，更在于实验设计。

本章小结

- Tor 研究常发表于 PETS 及重要安全会议。Tor Project 提供数据、工具和 Research Safety Board。
- 熵衡量单次观察下的匿名性，匿名程度则把结果标准化到零与一之间。对长时间风险，还需要特定时间段内失守概率等指标。
- 分析模型、行为模拟、网络仿真或模拟，以及公共网络测量，分别用于回答不同问题，也具有不同风险。
- 基于参考共识的模拟显示，持久守卫通过将暴露风险集中到较少客户端身上，显著降低了随时间失守的客户端比例。

练习

练习 **29.1** ⋆ 某攻击者观察一个包含四名用户的系统，赋予各用户的发送概率依次为 1/2、1/4、1/8 和 1/8。计算熵、匿名程度，以及具有相同不确定性的等概率用户数量。

练习 **29.2** ⋆ 为每个问题选择最合适的研究方法并说明理由：(a) 如果电路使用四跳，延迟会如何变化？(b) 某个国家有多少用户访问 Tor？(c) 一种新的守卫选择算法能否延长首次失守时间？

练习 **29.3** ⋆⋆ 运行实验 19 的 `simulate`，分别将 `guard_lifetime` 从默认 60 天改为 30 天和 180 天，比较 180 天后客户端失守比例，并解释结果。

练习 **29.4** ⋆⋆　重复实验 19：先假设攻击者控制 10% 的出口带宽，却不控制任何守卫带宽；然后假设控制 10% 的守卫带宽，却不控制出口带宽。各自能实现什么效果？为什么？

练习 **29.5** ⋆⋆⋆　为模拟器增加目前忽略的三项功能：客户端对电路复用十分钟而不是不断新建电路；守卫与出口之间的 /16 约束；攻击者控制某大型网络所属的整个自治系统。哪些修改对结果影响最大？

延伸阅读

- C. Díaz、S. Seys、J. Claessens、B. Preneel，*Towards Measuring Anonymity* [Día+02]。
- R. Jansen、A. Johnson，*Safely Measuring Tor* [JJ16]。
- Tor Project 研究门户：`https://research.torproject.org/`。
- *Anonymity Bibliography*：`https://www.freehaven.net/anonbib/`。

第 30 章

Arti：用 Rust 实现 Tor

最后核验：*2026* 年 *9* 月 *23* 日 — *Arti 2.6.0*（*2026* 年 *9* 月 *1* 日）；*tor 0.4.9.13*；实验网络

本章目标

学完本章后，你将能够：

- 解释为什么 Tor Project 要使用 Rust 重写 tor，并评估项目进展；
- 理解 Arti 的架构，识别本书第二部分介绍过的概念；
- 阅读其支持的子协议表格，并了解目前仍缺少哪些功能；
- 把 Arti 配置为本书私有网络的客户端（实验 20）。

本书一直以 *tor* 为基础——这是从 2002 年开始开发的 C 语言实现。然而，Tor 客户端的未来是 **Arti**，一个使用 Rust 完全重写的实现。Tor Project 已将 Arti 指定为开发新功能时优先使用的客户端，并宣布今后所有新的客户端功能都将提供 Arti 实现。本章面向已经在实验中用 Python 编写过简易 Tor 客户端的读者：Arti 的许多设计决策与我们曾面临的问题相同，只是将它们提升到了生产级规模。

30.1 为什么重写 tor？

项目介绍文档列出了四个动机：

安全性 安全 Rust 能避免许多内存安全错误，但 unsafe 代码、依赖库与逻辑缺陷仍需审查。Tor Project 估计，tor 历史上记录的至少一半安全漏洞如果使用 Rust 就不可能出现，另外许多漏洞发生的概率也会大幅降低。

开发速度 语言层面的保证有助于更有信心地编写新代码，减少开发所需时间。

灵活性 tor 最初只是一个 SOCKS 代理，后来才加入其他集成功能。Arti 则从一开始就设计为可由其他应用嵌入的模块化库。

可维护性 二十年的开发使 tor 代码库的组件之间形成复杂联系。重写提供了更清楚地分离职责的机会。

重要里程碑。 Arti 第一个公开版本 0.0.1 发布于 2021 年 10 月。2022 年 9 月 1 日发布的 1.0.0 宣布 Arti 已成为稳定的客户端。2023 年 8 月的 1.1.7 默认启用了编译客户端洋葱服务支持的选项，但使用者仍须主动开启相应功能。2026 年 2 月，2.0.0 发布。本章采用的参考版本为 2026 年 9 月 1 日发布的 2.6.0，它永久启用了拥塞控制（第 11.3 节）和 CGO 加密（第 10.4 节）。

目前的功能范围。 根据参考日期时的项目文档，Arti 已是完整的通用客户端，支持网桥和可插拔传输，也可以访问及发布洋葱服务。它尚不能作为中继、网桥或目录权威节点运行。这些功能仍在开发中，近期版本的许多工作都集中在中继与权威节点方面。

30.2 架构

Arti 由数十个 Rust *crate*（Rust 的编译单元）组成，2.6.0 版本有七十多个。名称约定区分其角色：直接面向用户的 crate 以 `arti` 开头；Tor 协议专用 crate 以 `tor-` 开头；可在 Tor 之外复用的 crate 则使用更通用的名称。设计中的一个指导原则是：尽量减少直接读写网络数据的代码；多数组件应能表现得“仿佛网络不存在”，以便更容易测试。

图 30.1 将主要 crate 与实现类似功能的 `torlab` 模块放在一起。它们有意形成对应：两者都遵循协议的分层，从底层原语一直延伸到高层客户端。

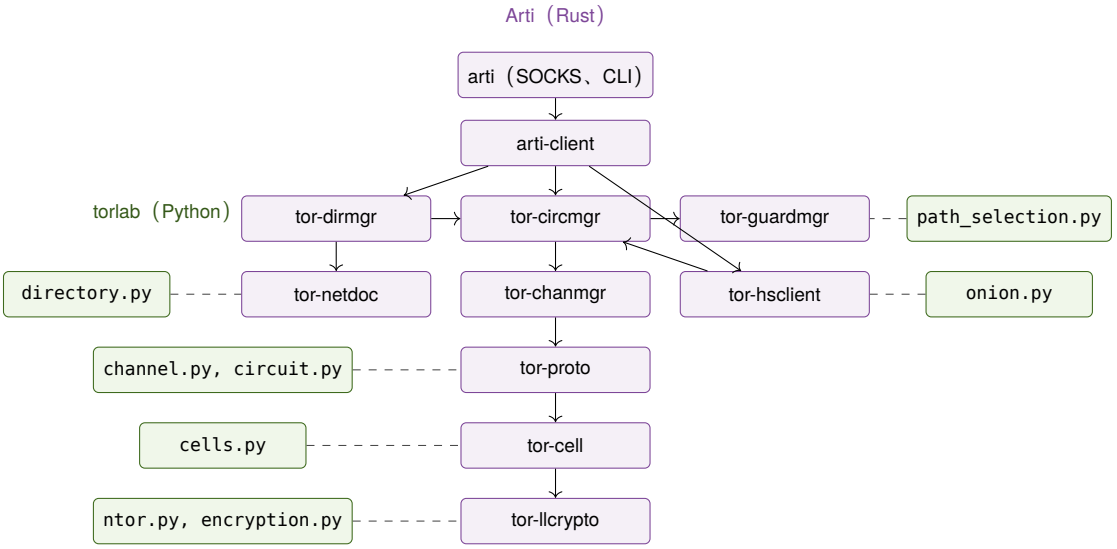


图 30.1: Arti 的部分 crate 与教学用 `torlab` 模块的简化功能示意图。实线表示概括的架构关系，而非精确的 Cargo 依赖图；虚线表示功能类比，不是跨项目依赖或完整等价。例如，`onion.py` 只解密描述符，不能替代完整的洋葱服务客户端；图中未包含 `tor-bytes`、`tor-hsservice` 等 crate。

`tor-llcrypto`、`tor-bytes` 密码学原语和二进制数据的解析与序列化。它们提供了 `ntor.py` 和 `encryption.py` 借助 Python 的 `cryptography` 与 `struct` 模块实现的基础功能。

`tor-cell` Cell 和 relay 消息，相当于 Arti 中的 `cells.py`（第 8 章）。

`tor-proto` 通道、电路、分层加密与流，也就是我们使用 `channel.py`、`circuit.py` 和 `stream.py` 实现的全部内容。

`tor-chanmgr`、`tor-circmgr`、`tor-guardmgr` 管理多条通道和电路、选择路径并维护守卫。教学客户端几乎没有实现这些功能，但它们是真实客户端中最复杂的部分之一。

`tor-netdoc`、`tor-dirmgr` 读取和更新目录文档，对应 `directory.py`。

`tor-hsclient`、`tor-hsservice` 洋葱服务的客户端和服务端功能；我们的 `onion.py` 只实现了描述符解密。

`arti-client`、`arti` 分别是供其他应用嵌入的高层库，以及提供 SOCKS 代理的命令行程序。

30.2.1 对比：编写一个 Cell

实验 2 中，我们曾编写 cell 编码函数。Arti 在 `crates/tor-cell/src/chancell/codec.rs` 中的对应函数 `write_cell` 遵循相同步骤：先写 CircID，再写命令。对于变长 cell，预留两字节长度字段，编码消息体，然后返回填写实际长度；对于定长 cell，则先编码消息体，再确认它没有超过 509 字节（`CELL_DATA_LEN`）。其中有两点值得注意：

- CircID 始终写作四个字节。Arti 不实现版本 4 之前的链路协议，因此不需要处理两字节 CircID 的特殊情况（第 8 章）。只有协商前发送的 **`VERSIONS`** cell 会单独处理。

- 超出长度限制不会作为调用者错误返回，而会被视为内部错误：如果生成了过长的 cell，就意味着 Arti 自身代码存在缺陷，而不是收到恶意对端的数据。错误类型本身可以区分实现缺陷与恶意输入。

Arti 明确不打算支持过时协议。它的目标是兼容当前推荐的版本，而不是永久保留所有历史格式。

30.3 支持的功能

描述 Tor 实现能力最准确的方式，是查看它支持哪些子协议（[tor-spec §Subprotocol-based versioning]）。每台中继和每个客户端都要声明自己支持各个功能族的版本：`Link`、`Relay`、`FlowCtrl`、`HSIntro` 等。Arti 的兼容性文档列出了以下客户端能力：

- `Link=4` 和 `Link=5`：四字节 CircID 与现代通道填充（第 7 章）；

- `Relay=4`（ntor-v3，第 9 章）、`Relay=5`（子协议协商）和 `Relay=6`（CGO 加密）；

- `FlowCtrl=1`（带认证的 SENDME）和 `FlowCtrl=2`（拥塞控制）；

- `Desc=4`：第 22.5 节 所述的基于密钥的家族机制。

注意：Arti 表格中的差一错误

Arti 的兼容性文档将上述两种流量控制子协议编号为 `FlowCtrl=0` 和 `FlowCtrl=1`。然而规范将带认证的 SENDME 定义为 1，将拥塞控制定义为 2 [tor-spec §Subprotocol-based versioning]；公共网络共识中的中继也声明 `FlowCtrl=1-2`。这是文档编号偏差，并非代码实现错误。它还说明了一项实用原则：当资料来源互相矛盾时，应核对规范，并寻找共识等独立证据。

在本章的参考日期，C 客户端的两项功能仍未在 Arti 客户端中实现。`Conflux=1` 提供多路径功能：`tor-proto` crate 已支持它，但 Arti 尚不会构建 Conflux 电路。`Padding=2` 对应旨在掩盖洋葱服务电路构建过程的填充状态机（第 26.4 节）。虽然 `tor-proto` 包含一个基于 *maybenot* 库的填充框架，但尚未实现这些状态机。中继侧的子协议还没有被标记为已支持：相当一部分中继代码已经存在，但只有当 Arti 能够提供实际可运行的中继时，相关功能才会被正式声明为支持。

30.4 实验：让 Arti 接入私有网络

实验 20：第二个客户端：Rust 实现

目标。编译 Arti，将它配置为本书私有网络的客户端，并使用其 SOCKS 代理代替 `test008c`。
前置条件。实验网络正在运行，并已安装 Rust 工具链（`cargo`）。
预计时间。1–2 小时，其中大部分用于编译。

步骤 **1**：编译。 从本书仓库根目录获取参考版本 Arti 2.6.0，放在 `labs/zh-Hans/vendor/arti`，并构建 `arti` 程序：

```
$ mkdir -p labs/zh-Hans/vendor
$ git clone https://gitlab.torproject.org/tpo/core/arti.git labs/zh-Hans/vendor/arti
$ git -C labs/zh-Hans/vendor/arti checkout arti-v2.6.0
$ (cd labs/zh-Hans/vendor/arti && cargo build --release -p arti --locked)
$ labs/zh-Hans/vendor/arti/target/release/arti --version
Arti 2.6.0
```

第一次构建需要几分钟，因为它必须编译整套 crate 及其依赖。

步骤 **2**：配置。 Arti 不读取 `torrc`，而是使用 TOML。为了连接非公开 Tor 网络，Arti 必须知道目录权威节点的身份，以及用来获取最初目录文档的后备缓存；这与 Chutney 生成的 `torrc` 中 `DirAuthority` 配置的作用相同（第 19.6 节）。`labs/zh-Hans/实验室/arti_config.py` 脚本读取每台权威节点本地文件中的 RSA 指纹、Ed25519 密钥、v3 证书身份和监听端口，并将这些信息写入 `labs/zh-Hans/实验室/net/arti/arti.toml`。下面只是省略内容的示例，不是可直接使用的 TOML：权威身份由本地网络生成，省略号仅作占位：

```
[tor_network]
fallback_caches = [
  { rsa_identity = "7B465200C885ED566C6CD4F46DBB0492076D203A",
    ed_identity = "JwhsA9VS01mRVnQseL9xsKZfstoBae7OcgL3jBvDzis",
    orports = ["127.0.0.1:5100"] },
  ...
]
[tor_network.authorities]
v3idents = ["390998046DEFF1E68AC346D7577319791B32C91A", ...]
```

还有三项配置是为所有节点均位于 `127.0.0.1` 的网络准备的。`allow_local_addrs` 允许连接本地地址，而公共网络客户端通常禁止这样做。两个不可能的网络前缀长度（IPv4 为 33 位，IPv6 为 129 位）用于关闭同网段约束；否则实验网络里的所有电路都会遭到阻止。最后，`hsdir_interval` 缩短洋葱服务时间周期，与 tor 的 `TestingTorNetwork` 效果类似。这些设置参照了 Chutney 自身的 Arti 配置。

步骤 **3**：启动。 先从本书仓库根目录运行 `python3 labs/zh-Hans/实验室/arti_config.py` 生成配置，再执行 `labs/zh-Hans/vendor/arti/target/release/arti proxy -p 9150 -c labs/zh-Hans/实验室/net/arti/arti.toml`。引导耗时取决于机器；在我们的一次运行中：

```
INFO tor_dirmgr::bootstrap: 1: Looking for a consensus.
INFO tor_dirmgr::bootstrap: 1: Downloading certificates for consensus
     (we are missing 3/3).
INFO tor_dirmgr::bootstrap: 1: Downloading microdescriptors (we are missing 8).
INFO tor_dirmgr: Marked consensus usable.
WARN tor_netdir: Requested 60 relays, but only 8 were usable, ...  role=Guard
```

```
INFO arti::subcommands::proxy: Sufficiently bootstrapped; proxy now functional.
INFO tor_guardmgr::guard: We have found that guard [scrubbed] is usable.
```

从中可以看出三个重要细节。引导流程逐步对应 第 12.7 节：先获取共识，再获取三台权威节点的证书，最后下载八个微描述符。`tor_netdir` 的警告反映了实验网络太小：守卫管理器希望得到 60 个候选节点，但可用的只有八个。日志中的守卫身份被显示为 `[scrubbed]`：Arti 利用 `safelog` crate 标记敏感值，避免将其写入日志，效果类似于 tor 的 `SafeLogging`。在后续目录更新中，其他日志可能出现 `Applying a consensus diff`（这一行未出现在上方摘录中）。如果差异文件可用且通过 `DirCache=2` 协商，Arti 可以只下载变更部分；并非每次更新都必然如此。

步骤 **4**：使用。 Arti 在端口 9150 提供 SOCKS 代理。`curl` 可以像使用 `test008c` 一样使用它：

```
$ curl -s -o /dev/null -w '%{http_code} in %{time_total}s\n' \
       --noproxy "" --socks5-hostname 127.0.0.1:9150 http://127.0.0.1:8080/
200 in 0.047224s
```

空的 `--noproxy` 参数可防止 `NO_PROXY` 使本地目标绕过 SOCKS。电路经过私有网络中的 C 语言中继：Rust 客户端与 C 中继讲的是完全相同的字节级协议，也就是第二部分研究的协议。最后还有一个与实验 18 有关的发现：仅包含 IP 地址的 SOCKS 请求（`--socks5`）同样会成功，而 Arti 默认日志没有显示与 tor 类似的警告。不过，`arti-client` crate 的文档提醒 API 使用者，本地域名解析可能向 DNS 解析器暴露目的地，具体取决于配置；但 SOCKS 请求只携带 IP 地址本身并不能证明 DNS 已经泄露（第 28.3 节）。

Arti 编译完成后，本实验包含在 `make verifica-lab` 中；否则该测试会被跳过并给出提示。

本章小结

- Arti 用具有内存安全保证的 Rust 重写 tor。它采用模块化库设计，自 2022 年起已是稳定的客户端，也是开发新客户端功能的优先平台。
- Arti 的架构沿协议层次组织：密码学原语和 cell、通道与电路，以及目录、路径和守卫管理。
- 子协议版本精确描述 Arti 的能力。客户端实现了 ntor-v3、CGO、拥塞控制和基于密钥的家族机制；在本章参考日期，Conflux 和填充状态机仍不完整，中继功能仍在开发。
- 与 Chutney 节点类似，Arti 通过配置目录权威节点和缓存接入私有网络。

练习

练习 **30.1** ⋆ 针对第二部分的每一章，找出实现其主要概念的 Arti crate。

练习 **30.2** ⋆ Arti 决定不实现过时协议版本，这有什么优点和缺点？哪些用户或节点可能因此无法使用？

练习 **30.3** ⋆⋆ 阅读 `crates/tor-cell/src/chancell/codec.rs` 中的 `decode_cell`，并与实验 2 的 `decode` 比较。它如何识别变长 cell？如果输入尚未包含完整 cell，会发生什么？

练习 **30.4** ⋆⋆ 使用 Arti 作为 SOCKS 代理，重复实验 11 的流隔离测试。不同的 SOCKS 凭据能否使流避免共用电路？请区分流隔离与“每组凭据永久对应唯一电路”的错误假设，并查找 Arti 的相关配置方法。

练习 **30.5** ⋆⋆⋆ 使用 `arti-client` 编写一个简单的 Rust 程序，从文件读取实验 20 的配置，并通过私有网络访问实验服务器上的网页。

延伸阅读

- 项目网站 `https://arti.torproject.org/` 与源码仓库，特别是 README、*Architectural overview*（`doc/dev/Architecture.md`）和兼容性指南（`doc/Compatibility.md`）。
- Arti 的更新日志，记录每个月发布版本中的变化。
- Chutney 自动生成 Arti 配置的源码（`lib/chutney/arti/config.py`），也是实验 20 的设计来源。

第 31 章

开放问题与正在推进的项目

最后核验：*2026* 年 *9* 月 *23* 日 — 截至第 *369* 号的 *torspec* 提案；*tor 0.4.9.13*；*Arti 2.6.0*

本章目标

学完本章后，你将能够：

- 跟踪 Tor 协议的修改流程，并正确解读提案状态；
- 解释为什么后量子密码学要求改变消息格式，而不只是替换算法；
- 描述 Conflux、Walking Onions 等旨在改善性能和可扩展性的项目；
- 找到参与 Tor Project 的具体途径。

本书描述的内容没有一成不变的部分。2026 年研究的协议，已不同于缺少 ntor-v3、拥塞控制和 CGO 的 2016 年版本；到 2036 年，它必然又会有所变化。最后一章展望未来。对于每个开放问题，我们都要问：现在哪些方面还不够好、已经提出了什么方案，以及相关工作进展到哪里。以下内容都不可避免地具有暂时性：提案状态以本章参考日期时规范仓库中的记录为准。

31.1 协议如何发生变化？

自 2007 年以来，Tor 协议的修改通过编号提案（proposal）来发展，有些类似于 IETF 的 RFC。贡献者先在开发者邮件列表讨论设想，再按规定格式编写文档；当大家广泛同意这个想法值得进一步研究时，编辑者就将它以特定编号加入仓库。随后不断讨论和完善，直到足够详细，可以实现。功能完成后，其具有规范约束力的内容会被纳入正式协议规范。与 RFC 不同，Tor 提案可能在保持编号不变的情况下继续修改；而且，对已经实现的功能来说，提案并不是最终权威文档，该角色属于协议规范（提案 001）。

阅读提案时，首先应该检查它的状态（图 31.1）。本书介绍过不同阶段的提案。ntor-v3（提案 332）处于 *Closed*：已实现并纳入规范。拥塞控制（提案 324）和 Conflux（提案 329）处于 *Finished*：已经实现，但尚未完全纳入规范。CGO（提案 359）虽然已出现在 tor 和 Arti 中，状态仍为 *Open*。移除 TAP 的计划（提案 350）为 *Accepted*，而 Walking Onions（提案 300）属于 *Informational*。

31.2 量子计算机

问题。 Tor 的全部公钥机制，包括用于握手的 X25519、用于签名的 Ed25519，以及仍在使用的 RSA，如果出现足够强大的量子计算机，都将面临被攻破的风险（第 5 章）。目前没有人知道这样的计算机是否、或者何时会出现。然而对匿名系统来说，威胁可能更早开始：攻击者可以今天保存加密流量，未来再解密。



图 31.1: 依据 Proposal 001 绘制的 Tor 提案状态示意图（简化且非穷尽）。*Draft–Open–Accepted–Finished–Closed* 只是可能的路径，不是必经顺序；*Finished* 表示已实现，*Closed* 还表示内容已纳入规范。*Needs-Revision* 与 *Needs-Research* 是不同障碍，*Dead* 可以重新开启，*Rejected* 是不采纳所述方案；*Informational* 是最终的信息性文档，本身不属于规范。图中省略 *Meta*、*Reserve*、*Obsolete* 等其他状态。

两种情景。 2025 年的 提案 355 区分两种环境。在过渡环境中，只需防护未来的量子计算机：今天捕获的流量必须能抵御未来解密，但传统签名仍可接受，因为今天要主动伪造签名，就需要目前已经存在强大量子计算机。在下一代环境中，防护还必须涵盖量子计算机已经存在，能够伪造签名或主动攻击握手的情况。

代价。 2024 年，NIST 标准化了后量子密钥建立机制 ML-KEM，以及签名算法 ML-DSA 和 SLH-DSA [NIS24]。相关提案将其数据大小与现有算法比较：

算法	公钥	密文或签名
X25519	32 字节	32 字节
ML-KEM-768	1 184 字节	1 088 字节
Ed25519	32 字节	64 字节
ML-DSA-65	1 952 字节	3 309 字节

第 8 章 已说明，一条 relay 消息最多承载 498 字节。因此，即使不考虑握手的其他字段，单个 ML-KEM-768 公钥就需要三个 cell。后量子密码学并非简单地替换算法：它会改变消息在 cell 中的装载方式。现有 relay 消息格式无法直接携带如此大的消息：三条 cell 仅代表假设新格式能够分片时的有效载荷容量下界。

当前方向。 提案的结论相当谨慎。过渡阶段可能只需按照先前提案，把后量子机制加入 ntor。对于下一代系统，优先方案是在成本可接受的前提下，使用两种后量子密钥建立机制，同时提供前向保密。如果把后量子公钥加入微描述符，还会抵消通过移除 TAP 密钥（提案 350）和引入基于密钥的家族机制（第 22.5 节）所节省的部分带宽。最终选择还依赖尚待完成的测量：有多少电路需要从零建立？握手消耗多少中继带宽和 CPU 时间？这仍是开放的研究问题，也是贡献测量结果的机会（第 29 章）。

31.3 更大的消息，以及更小的消息

在参考日期处于 *Open* 状态的 提案 340，直接针对一条消息只能装入一个 cell 的限制。它建议将 relay 消息与 cell 分离：较长的消息可以跨越多个 cell，而几个较短的消息可以共享一个 cell。其动机既包括后量子握手、UDP 传输和 Walking Onions 所需的超过 498 字节的消息，也包括 `SENDME`、`XON` 和 `XOFF` 等很短的消息。这些短消息目前也要占据整整一个 cell；将来则

可以藏在其他消息的填充空间里，使观察者无法直接辨认。新格式旨在与新版 relay 加密配合，而其 CGO 设计已在 第 10.4 节 中讨论。协调这些修改，有助于避免产生越来越多互不兼容的格式。

31.4 性能：多路径传输

拥塞控制（第 11.3 节）提高了单条电路的效率。下一步是 **Conflux**（提案 329，*Finished*）。客户端建立两条通向同一个出口的电路，使用不同的守卫与中间中继；然后把它们关联起来，并根据每条电路测量得到的往返时延分配流量。这一思想源于 AlSabah、Bauer、Elahi 和 Goldberg 的研究；Tor 的实现利用拥塞控制信息，并借鉴多路径 TCP 的经验。我们已在实验网络（第 19.7.3 节）和通过 Stem（第 18 章）观察过 Conflux 电路。提案建议，除出口外，两条分支不要共用其他中继，以避免共享瓶颈。这是建议而非严格规则，在规模很小的实验网络中经常无法满足。tor 从 0.4.8 版本开始使用 Conflux；如 第 30.3 节 所述，本章参考日期时 Arti 尚未实现客户端 Conflux。使用或不使用 Conflux 的 OnionPerf 测量点可以用于比较其对真实网络的影响（第 24.3 节）。

31.5 可扩展性：**Walking Onions**

目前，每个 Tor 客户端都知道全部中继：它下载共识和微描述符，目录流量会随中继数量增长。微描述符与共识差分能降低成本，却没有改变根本结构。启动时的开销还高度集中：新客户端必须先下载完整目录，才能建立第一条电路（提案 300）。

直观的解决办法都有已知缺点。只向每位客户端展示部分网络，会产生认知差异（epistemic）攻击：客户端视图之间的区别本身就可能识别用户。让每台中继选择下一跳，又会像某些去中心化系统一样，使恶意中继容易控制路径。Walking Onions 提出另一种方式：目录权威节点发布经过签名的结构，把每台中继与一个数值区间关联起来。客户端随机选择一个数值，电路中的上一跳中继在扩展时提供一份签名证明，说明哪个中继占据对应区间。客户端能够验证签名，而不必持有完整中继列表。原始提案的状态为 *Informational*；更详细的后续提案（提案 323）试图弥补其剩余缺陷。包括后量子密码学在内的近期提案也会考虑这一设计。

31.6 其他开放问题

UDP Tor 只传输 TCP。视频通话、游戏以及 HTTP/3 所使用的 QUIC 因此无法直接支持，或者需要使用可能不安全的变通方式。提案 348（*Open*）探索了从经出口建立隧道到原生数据报传输的多种方案，同时也讨论新风险：应用程序指纹、对等通信元数据收集和端口耗尽。

指纹识别 第 26.4 节 以一个开放问题收尾：目前还没有一种普遍有效、开销又可接受的网站指纹防御。tor 已部署用于洋葱服务电路的填充状态机；Arti 则在基于 *maybenot* 库的框架中继续开发，这些都为未来防御提供基础。

流量关联 第 26 章 的核心问题仍未解决：低延迟网络可能保留可关联的时间与流量规模特征，但缓冲及并发流量也会改变这些模式。恒定速率填充与增加延迟等已知方法成本较高，研究人员仍在寻找更高效的折中方案，以及考虑互联网拓扑的路径选择。

拒绝服务 工作量证明保护洋葱服务（第 15 章）；如何自动调节防御机制，是 提案 362 的研究主题。

Arti 作为中继 当 Arti 能够运行中继和目录权威节点之后，C 语言代码库最终可能退役。这是 Tor Project 的长期目标之一（第 30 章）。

31.7 如何参与贡献?

读到这里的读者有多种途径可以贡献力量，并不是每一种都需要编写代码：

- 贡献带宽：运行中间中继或网桥；在采取 第 25 章 所述防护措施的前提下运行出口（第 22 章）；也可以运行 Snowflake 代理，甚至仅通过浏览器参与。
- 阅读和讨论提案：许多提案处于 *Open*，因为还需要反馈、分析和测量。开发者讨论列表是公开的。
- 贡献代码：tor 与 Arti 在 Tor Project 的 GitLab 上开发，有专门适合新贡献者的 issue 标签。Arti 对新增功能尤其重要。
- 开展研究：使用 Shadow、Chutney 和 Metrics 数据，并遵守 Research Safety Board 的指南（第 29 章）。
- 翻译与编写文档：志愿者帮助翻译 Tor Browser、手册和网站。包括意大利语在内的各种语言都需要更多文档。
- 使用 **Tor**：更多用户可能扩大匿名集合，但若时间特征、行为或其他元数据可区分，用户数量本身并不能保证不可区分性。

本章小结

- Tor 协议通过编号提案演进；随着修改被实现并纳入正式规范，提案状态可从 *Draft* 发展到 *Closed*。
- 即使强大量子计算机尚未存在，后量子密码学也很重要。它需要比单个 cell 更大的消息，因而推动了关于消息分片与打包的第 340 号提案。
- Conflux 将流量分配到两条电路；Walking Onions 则可能免去每个客户端下载完整网络目录的需要。
- UDP、指纹识别和流量关联防御、DoS 自动调节，以及 Arti 发展为中继实现，都仍是开放挑战。

练习

练习 **31.1** ⋆ 选择本书提到的三个提案，分别确认它们当前的状态以及状态的含义：功能是否已部署？是否已纳入正式规范？

练习 **31.2** ⋆ 假设每条 relay 消息最多承载 498 字节，并且不同消息不能共用一个 cell，那么传输一个 ML-KEM-768 公钥与一个 ML-KEM-768 密文，各需要多少个 cell？采用 提案 340 后会有什么变化？请区分不计分片封装开销的理论最少 cell 数与实际需要的数量；后者取决于提案定义的封装方式。

练习 **31.3** ⋆⋆ 举例说明认知差异攻击：如果不同客户端只知道网络中不同的中继子集，攻击者可以如何利用这些差异？

练习 **31.4** ⋆⋆ 利用 Stem 找出实验网络中具有相同 `CONFLUX_ID` 的 Conflux 电路对。确认两条分支是否以同一个出口结束，以及是否像 提案 329 建议的那样避免共用其他中继。你观察到什么？如何解释？

练习 **31.5** ⋆⋆⋆ 从本章选择一个开放问题，按照 Tor Project 的格式编写简短提案：动机、设计、安全性分析、兼容性与实施计划。

延伸阅读

- `torspec` 仓库中按状态分类的提案索引，以及提案流程文档（提案 001）。
- N. Mathewson，*Options for postquantum circuit extension handshakes*（提案 355）。
- *Walking Onions: Scaling and Saving Bandwidth*（提案 300）及后续提案（提案 323）。
- Tor Project 博客，用于了解协议变更公告及解释。

附录 A

Tor 术语表

本术语表解释简体中文版中使用的技术术语，并为每个条目列出首次系统介绍它的章节。

术语	含义	章
流量分析	不读取通信内容，仅根据时间、数据量和地址信息推断相关事实。	1
匿名性	在称为匿名集的主体集合内，无法识别某个特定主体。	1
Sybil 攻击	向网络展示大量身份（例如多个中继），以提高被选中的概率。	26
目录权威节点	参与投票并签署网络共识的少数服务器之一。	12
网桥（bridge）	不出现在公开共识中的中继，为面临审查的用户提供网络入口。	16
通道（channel）	相邻两台 Tor 节点之间的 TLS 连接，承载来自多条电路的 cell。	7
cell	Tor 通道上的传输单位：通常为 514 字节，部分命令采用变长格式。	8
盲化密钥	根据洋葱服务身份密钥和特定周期派生的密钥，用于向 HSDir 隐藏服务身份。	14
家族密钥	由同一运营者管理的中继共享、用于证明其家族归属的密钥。	22
电路 ID（CircID）	仅在单条通道内有效的电路标识符；每一跳的值都不同。	8
电路（circuit）	逐跳建立、经过多个中继的多层加密路径。	9
共识（consensus）	由目录权威节点签署、列出中继及其标志和权重的文档。	12
拥塞控制	根据测得的往返时延调整电路上尚未确认的数据量。	11
流量控制	利用窗口和 SENDME 消息防止发送方使接收方负载过高。	11
流量关联	判断网络上不同位置观察到的两条流是否属于同一次通信。	26
描述符（descriptor）	中继或洋葱服务发布密钥、地址和属性的文档。	12
出口中继	电路的最后一跳，代表客户端连接最终目的地。	6
中继家族	由同一运营者管理、不能同时出现在同一电路中的中继集合。	13
标志（flag）	由目录权威节点赋予中继的属性，例如 `Guard`、`Exit` 和 `Stable`。	12
前向保密	即使长期密钥以后遭到泄露，也不能据此解密过去流量的安全属性。	5
入口守卫（guard）	客户端电路的第一跳中继，被持久选中并在较长时间内重复使用。	13

术语	含义	章
握手（handshake）	双方交换消息并协商密钥的过程，例如 ntor 或 ntor-v3。	9
隐藏服务目录（HSDir）	存储并提供洋葱服务描述符的中继。	14
中间中继	位于电路中间、不知道最初客户端和最终目的地的中继。	6
流隔离	根据 SOCKS 凭据等条件，将不同应用流分配到不同电路。	18
链路协议	通道上使用的带版本协议，通过 **`VERSIONS`** cell 协商。	7
relay 消息	发给特定跳点的 **`RELAY`** cell 内部消息，例如 `BEGIN` 或 `DATA`。	8
微描述符	只包含客户端所需中继信息的精简描述符。	12
威胁模型	明确描述系统旨在抵御哪些攻击者和威胁的文档。	4
不可关联性	观察者无法判断两项活动是否来自同一个主体的性质。	1
洋葱服务（旧称隐藏服务）	可通过 Tor 访问、其地址编码了公钥的服务，过去称为隐藏服务。	14
填充（padding）	在 cell 或通道中加入额外数据，用于掩盖长度和时间特征。	7
带宽权重	共识中用于平衡不同电路位置的中继选择概率的系数。	13
可插拔传输（PT）	转换 Tor 流量以抵御识别或封锁的独立程序，例如 obfs4、Snowflake 和 WebTunnel。	16
出口策略	按顺序指定出口中继允许访问哪些目的地的规则集合。	22
工作量证明（PoW）	客户端为获得遭受攻击的洋葱服务优先处理权而求解的计算难题。	15
提案（proposal）	用于讨论和设计 Tor 协议变更的编号文档。	31
假名	不同于真实姓名的标识符，能够关联活动但不必披露真实身份。	1
引介点	洋葱服务指定的、用于接收首次联系请求的中继。	14
汇合点	客户端选择的中继，客户端与洋葱服务的电路在此连接。	14
中继（洋葱路由器）	Tor 网络中负责转发 cell 的节点。	6
SENDME	确认已经收到电路数据、并重新打开流量控制窗口的消息。	11
子协议	中继和客户端声明支持的带版本功能族，例如 `Link` 或 `Relay`。	30
流（stream）	通过电路承载的应用程序数据流，例如一条 TCP 连接。	11
Vanguard	在一段时间内保持不变、用于保护洋葱服务入口守卫的中间层中继。	13
网站指纹识别	仅根据加密流量的大小和时间特征识别访问的网页。	26

附录 B

密码学进阶

最后核验：*2026* 年 *9* 月 *23* 日 — *dir-spec* 与 *tor-spec*（*torspec* 提交 `8e1f245`）；`cryptography` 库；实验网络

第 5 章 以最少的数学预备知识介绍了 Tor 的密码学基础。本附录进一步探讨此前只是简要提及的三个主题，并提供可执行的示例：X25519 密钥与经过钳制处理的比特；同一椭圆曲线的 Montgomery 与 Edwards 两种表示之间的转换，并通过真实证书验证；以及流密码的可篡改性，它使 第 10.3 节 中的标记攻击成为可能。

B.1 同一条曲线的两种表示

X25519 与 Ed25519 使用的是有限域 $p = 2^{255} - 19$ 上的同一条椭圆曲线，只是采用两种方程形式：

Montgomery 形式 $v^2 = u^3 + 486662\,u^2 + u$。X25519 使用这种形式：公钥只包含曲线上点的 u 坐标，以 32 个小端序字节编码。这种表示允许使用简单、高效、执行时间不依赖秘密密钥的标量乘法算法——“Montgomery 梯形算法”。

扭曲 **Edwards** 形式 $-x^2 + y^2 = 1 + d\,x^2 y^2$，其中 $d = -121665/121666$。Ed25519 用它实现签名：公钥存储 y 坐标，并将 x 坐标的符号写入最后一个字节的最高位。

两种形式之间存在显式转换，我们将在 B.2 中使用它。

B.1.1 X25519 密钥的比特钳制

X25519 私钥最初是 32 个字节，但实际使用前，实现会强制修改其中某些比特：将最低三位清零，将最高位清零，再把次高位置为 1。清除最低三位，使标量成为 8 的倍数，8 正是这条曲线的余因子，因此收到的点的小阶分量不会泄露标量信息。固定高位还确定了 Montgomery 梯形算法执行的步数。可以使用与实验相同的库来验证：

```
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from cryptography.hazmat.primitives import serialization as s
import os
def public_key(k):
    return X25519PrivateKey.from_private_bytes(k).public_key().public_bytes(
        s.Encoding.Raw, s.PublicFormat.Raw)
a = bytearray(os.urandom(32)); b = bytearray(a)
b[0] ^= 0x07                    # 仅改变最低三位
print(public_key(bytes(a)) == public_key(bytes(b)))      # True
```

如果两份私钥字节串仅在钳制时会被覆盖的比特上不同，它们会产生相同公钥。

B.1.2 无穷远点

在 ntor 握手中，客户端和中继都会检查各自的 EXP() 运算是否产生无穷远点。规范指出，对于 Curve25519，这种检查足以代替验证 Y 属于指定群的条件 [tor-spec § The "ntor" handshake]。否则，攻击者提供的小阶公钥点可能生成可预测的共享秘密，例如全零字节串。`cryptography` 库会自行执行该检查：如果使用全零的 32 字节公钥调用 `exchange`，它会抛出异常（`Error computing shared key`），而不是返回全零秘密。

B.2 从 Montgomery 到 Edwards：ntor 交叉证书

每台中继在描述符中公布自己的 ntor 公钥，即 X25519 密钥，并通过 `ntor-onion-key-crosscert` 证明拥有对应私钥：这是一个类型为 `0a` 的 Ed25519 证书，用于认证中继的主 Ed25519 身份，却由 *ntor* 密钥签名 [dir-spec § ntor-onion-key-crosscert]。这带来一个问题：ntor 密钥使用 Montgomery 形式，而 Ed25519 签名需要 Edwards 形式。规范定义了如下转换 [dir-spec § Converting a curve25519 public key to an ed25519 public key]：

$$y = \frac{u - 1}{u + 1} \bmod p.$$

u 坐标能够决定 y，却不能决定 x 的符号：每个 Montgomery 坐标对应两个对称的 Edwards 公钥。这个符号无法从公开的 Montgomery 密钥恢复，因此描述符会在交叉证书字段的第一个参数中明确提供：0 表示正，1 表示负。中继根据自己的私钥计算该符号。

`labs/zh-Hans/代码/torlab/conversion.py` 模块只用几行代码就实现了这一转换：

```
P = 2**255 - 19
def montgomery_to_edwards(u_bytes, sign):
    u = int.from_bytes(u_bytes, "little") & ((1 << 255) - 1)
    y = (u - 1) * pow(u + 1, P - 2, P) % P      # 用模 p 的逆元进行除法
    compressed = bytearray(y.to_bytes(32, "little"))
    compressed[31] |= (sign & 1) << 7          # 最高位存储 x 坐标的符号
    return bytes(compressed)
```

随后，模块使用实验 1 的证书解码器验证交叉证书。应用于实验网络的描述符（为便于阅读，公钥作了截断；每建立一个新网络，密钥都会改变）：

```
$ PYTHONPATH=labs/zh-Hans/代码 python3 -m torlab.conversion labs/zh-Hans/实验室
    /net/nodes/000a/cached-descriptors test004r
test004r: ntor-onion-key DaxPE+Grnjex2Hmv+5qutl5dIX4La2jF6MIL7IK6+3E
  Edwards 公钥（符号位 0）：  c369d08f8083002a15fe22f65dca14bf...
  交叉证书验证成功；已认证的身份：  kjPIgDB2OFsGJzuwsT+KMS/El9qrlE2/4P5WvIvmzC8
```

在参考示例中，签名有效，所认证的身份与中继 `fingerprint-ed25519` 文件中的值一致；实验环境的其他中继也是如此。如果提供错误的符号，验证会失败，这也是该模块自动化负面测试中的一种情况。模 p 除法通过费马小定理实现，即乘以 $(u+1)^{p-2}$。

B.3 流密码的可篡改性

计数器模式的 AES（第 5.2 节）产生一条字节流，并通过 XOR 与消息组合。任何人修改密文中的一个比特，都能在不知道密钥的情况下修改明文中的对应比特；攻击者只需要知道或猜出原始内容。下面是 Python 示例：

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
k, n = os.urandom(16), bytes(16)
cipher = lambda d: Cipher(algorithms.AES(k), modes.CTR(n)).encryptor().update(d)
```

```
c = bytearray(cipher(b"BEGIN www.example.org:443"))
delta = bytes(x ^ y for x, y in zip(b"www.example.org", b"www.example.net"))
for i, d in enumerate(delta):
    c[6 + i] ^= d                 # 不知道密钥也可以实施定向修改
print(cipher(bytes(c)))            # b'BEGIN www.example.net:443'
```

Tor 原来的分层 relay 加密 tor1 使用 AES-CTR，只在电路终点通过四字节摘要检查完整性。因此，恶意中继可能以可预测方式修改信元，让路径下游的同伙识别这种修改。这就是 第 10.3 节中的标记攻击，也是开发 CGO（第 10.4 节）的原因之一；CGO 的设计使被篡改的信元及后续信元都无法正常恢复。

练习

练习 **B.1** ⋆　比特经过钳制后，一个 X25519 私有标量可以取多少种不同值？将结果与 2^{256} 比较。

练习 **B.2** ⋆⋆　推导并实现从 Ed25519 公钥的 y 坐标到 Montgomery u 坐标的逆转换。保留原始公钥的 x 符号位，再使用 `conversion.py` 中的 `montgomery_to_edwards` 检查两次转换是否还原原来的 Ed25519 公钥（要求 $y \neq 1$）。

练习 **B.3** ⋆⋆⋆　在 tor1 中，一条 cell 包含四字节摘要，以及两字节全零的 `recognized` 字段。如果随机修改某个信元，最终跳点将其接受为有效信元的概率是多少？为什么标记攻击仍然可能成功？

附录 C

torspec 协议规范阅读指南

最后核验：*2026* 年 *9* 月 *23* 日 — *torspec* 提交 `8e1f245`

Tor Project 的规范是本书几乎每一章的主要资料来源。它们内容全面，却不是按照教科书形式编写的。二十多年来，规范最初以互相独立的文本文件维护，直到较近时期才汇总到同一个网站。规范自己的约定文档也承认，其一致性仍在持续完善中。本附录为希望读完本书后继续研究规范的读者提供一份导航图。

C.1 从哪里找到规范？

规范发布在 `https://spec.torproject.org/`，并与提案一起在 Tor Project 的 `torspec` GitLab 仓库维护。规范是 `spec/` 目录中的 Markdown 文件：较小的主题由单独文件描述，内容较多的文档则使用目录。文件 `permalinks.md` 列出了文档的稳定地址，例如 `https://spec.torproject.org/tor-spec`；引用规范时应优先使用这些链接。本书一直采用 [document §section] 形式引用规范文档及章节，保留网站上使用的原始章节标题；提案则写作“提案 NNN”。

C.2 约定

规范的引言约定文档规定了一些通用规则，除非具体章节另有说明：

- `MUST`、`SHOULD`、`MAY` 及其否定形式，采用 RFC 2119 中的含义，分别表示强制要求、强烈建议和允许的选项。
- 长度以八比特字节为单位，“kibibyte” 等二进制前缀代表 1024 的倍数。
- 多字节整数采用大端序，即 “网络字节序”：十进制整数 4660 以两个字节表示为 `12 34`。

本书前面遇到过两个重要例外：Curve25519 与 Ed25519 的密钥及点采用小端序（附录 B）；部分目录格式则是包含以 base64 编码的二进制数据的可读文本。

C.3　规范文档地图

文档	内容	相关章节
`tor-spec`	核心协议：通道、信元、电路、握手、中继加密、流、流量控制和拥塞控制	7–11
`cert-spec`	Tor 的 Ed25519 证书	7
`dir-spec`	目录权威、投票、共识、描述符、微描述符与 `extra-info` 文档	12, 24
`path-spec`	路径选择及其约束	13
`guard-spec`	守卫选择算法	13
`vanguards-spec`	完整与轻量级 Vanguard 保护机制	13
`rend-spec`	v3 洋葱服务：密钥、描述符、引介及汇合	14, 21
`hspow-spec`	洋葱服务工作量证明	15
`dos-spec`	拒绝服务攻击防御	15
`pt-spec`	tor 与可插拔传输程序之间的接口	16
`padding-spec`	通道与电路填充	7, 26
`control-spec`	ControlPort 控制协议	18
`socks-extensions`	Tor 专用的 SOCKS 扩展	11, 28
`param-spec`	共识网络参数及其默认值	11, 15
`address-spec`	特殊地址名称，例如 `.onion` 地址	14
`srv-spec`	目录权威节点的共享随机值	14

C.4　如何阅读规范章节?

以下建议来自本书的编写过程：

1. 先了解格式。描述二进制消息的章节会以表格列出字段名称及长度。阅读规则之前，应像本书的格式示意图那样，先在纸上重建消息格式。

2. 区分当前功能与过时内容。许多文档仍保留 TAP、链路协议版本 1–3 和 v2 洋葱服务等历史功能。过时资料通常集中在 `obsolete-channels.md`、`historical/` 等章节或文件，但并非总是如此。存在疑问时，应检查源码对应分支是否仍然存在。

3. 与实现相比较。规范表达设计意图，tor 和 Arti 的源码揭示实际行为。本书研究期间，一些差异很有启发：单跳电路中未出现的通道填充（第 7 章），以及出口中继返回的流关闭原因（第 22.4 节）。

4. 检查提案状态。有些提案在纳入正式规范前就已经实现，另一些仍在讨论中，尚无实现（第 31.1 节）。

5. 进行实验验证。本书的私有实验网络可用于检查某个字段是否按文档描述运行。这也是全部实验共同采用的方法。

C.5　Arti 与规范

Arti 的源码经常在注释和文档中引用规范，并指出其中模糊的要求或与 tor 的差异。它支持的子协议表格（第 30.3 节）说明两种实现共同支持哪些功能，因此也显示了哪些规范内容描述的是仍在实际运行的“活协议”。

附录 D

习题解答

第 1 章：匿名、监控与元数据

解答 **1.1**。 使用 HTTPS 时，互联网接入服务商能够看到网站的 IP 地址，有时还可以通过 DNS 解析以及 TLS 的 SNI 字段获得域名。它还可以观察时间特征、连接持续时间和流量大小。使用 VPN 时，接入服务商只能看到通向 VPN 服务器的连接，但 VPN 提供者能够看到此前接入服务商可能看到的全部信息。使用 Tor 时，接入服务商只能看到连接到了 Tor 中继或网桥，以及相应的时间特征和数据量。Tor 网络内部没有任何单个节点同时了解用户与目的地。

解答 **1.2**。 使用虚构姓名的账户属于假名性。公告栏上一条没有署名的消息，在没有其他信息能识别作者的前提下属于匿名性。只写姓名首字母的信件属于较弱的假名性，往往仍可能识别作者。匿名现金捐款是匿名性的例子。

解答 **1.3**。 匿名集从五十名 Tor 用户缩小到其中会说意大利语、并且在凌晨三点仍然清醒的人，可能只剩寥寥数人。Tor 隐藏的是 IP 地址，而不是行为。语言、时间、写作风格和个人信息都可能在网络流量未泄露身份时，依然使用户被识别。

解答 **1.4**。 VPN 提供者同时知道用户身份和访问目的地。所有信任都集中在一个运营者身上；它可能记录活动、依法被要求交出数据，或者遭到入侵。Tor 把信任分散到三个独立运营的中继，任何一个都看不到通信两端。要关联用户与目的地，必须攻破或同时观察第一跳与最后一跳。

第 2 章：起源

解答 **2.1**。 随机猜测时，正确概率为百分之一。如果观察者提供某批次一百条消息中的九十九条，就能识别自己送出的消息，并将唯一剩余的消息判定为受害者的消息。匿名性随之消失。这就是 $n-1$ 攻击。真实的混合网络通过更复杂的批处理、伪装流量以及对进入消息的控制来应对这种攻击。

解答 **2.2**。 使用 Chaum 意义上的匿名回复地址。用户为一串混合节点准备经过多层加密的“返回洋葱”，最内层包含用户地址及加密回复所需的密钥。回复者把消息附在洋葱上；每个混合节点剥去一层，但没有任何单个节点知道最终目的地。无须集中式路由表。

解答 **2.3**。 每一步以 $3/4$ 的概率把请求转发给另一位群体成员，以 $1/4$ 的概率发送到网站。经过的成员数量服从几何分布，平均为 $1/(1/4) = 4$，并计入第一次转发。接收请求的成员无法知道上一位转发者是否也是原始发起者，因为中间转发也遵循同样规则。

第 3 章：从 NRL 洋葱路由到 Tor Project

解答 **3.1**。 在三跳结构中，恶意中间节点立刻就能知道通信两端所属的两个组织网络，因为入口和出口节点位于这些组织的网络内部。使用四跳时，攻破一端附近节点的人可以知道需要继续攻击哪一台节点才能重建路径；使用五跳则还需额外攻破一个节点。Tor 使用三跳，是因为中继不位于用户自己的网络，而且增加跳数会提高延迟，却无法大幅缓解入口与出口流量关联的主要威胁。守卫机制更适合解决这一问题。

解答 **3.2**。 如果只使用一份通过节点长期公钥加密的洋葱，就没有前向保密。之后获得这些密钥的人可以解密以前保存的洋葱。逐跳扩展式构建则让每一跳执行临时密钥交换，使用后销毁临时秘密。客户端还可以一次扩展一跳，立即识别没有回应的中继并更换它。使用单个预先构造的洋葱，只有尝试完整路径后才会发现失败。

解答 **3.3**。 这是开放式论证题。可以独立验证的因素包括公开的代码与规范、可复现构建、科学文献中的独立分析，以及由不同运营者运行的网络。仍然需要信任的方面包括：发布的二进制程序是否与公开源码一致（虽然可以部分验证）；目录权威节点是否串通；是否存在未知漏洞。应根据可验证性和分散信任评估安全性，而不是仅凭项目资金来源作出假设。

第 4 章：威胁模型

解答 **4.1**。 （a）可以：网络运营者只能看到连接到了 Tor 中继。（b）可以：网站看到的是出口中继的地址。（c）就身份匿名而言不可以：用户登录账户时主动标识自己。Tor 只能隐藏用户从哪里连接。（d）很可能不可以：同时观察用户端与报社端的攻击者，能够尝试进行时间关联，而这正是 Tor 威胁模型明确不保证抵御的情形。

解答 **4.2**。 难用的系统只能吸引少数动机强烈的人，而这些人可能恰恰最需要保护。仅仅使用该系统，就足以使他们显得独特，留下一个很小的匿名集。易用的系统则可以吸引数百万普通用户，让有敏感需求的人更容易融入其中。

解答 **4.3**。 这是开放式练习。潜在攻击者包括能够观察消息来源所在地网络的政府或雇主、电子邮件或消息服务提供商，以及控制通信两国之间网络的实体。Tor 可以阻止本地网络得知记者与谁通信，也防止服务端直接看到记者的 IP 地址。但如果没有端到端加密，Tor 无法保护消息内容；它也无法隐藏服务可见的账户和时间戳等元数据，不能抵御设备遭到攻破，亦不能防止同时观察两端的攻击者进行流量关联。使用 SecureDrop 等洋葱服务可以减轻其中若干风险。

第 5 章：密码学基础

解答 **5.1**。 $C_1 \oplus C_2 = P_1 \oplus P_2$，因为重复使用的密钥流会相互抵消。若已知 P_1，就能恢复 $P_2 = C_1 \oplus C_2 \oplus P_1$。

解答 **5.2**。 16 字节输出正好等于 32 字节输出的前 16 字节：SHAKE 生成一条可以读取任意所需长度的输出流。因此，密钥派生函数可以申请较长的流，并将其切分为多个部分。然而，为不同用途请求不同长度并不能实现域分离；仍然需要使用不同的域标签。

解答 **5.3**。 两次计算出的共享秘密相同。全零公钥对应小阶点，不论私钥是什么，都可能产生全零的共享秘密，也就是所有人都知道的秘密。`cryptography` 库拒绝返回这样的结果。ntor 明确要求这种检查（第 9.6 节）；否则攻击者能够强制生成可预测的共享秘密。

解答 **5.4**。 没有域分离时，一种上下文中生成的值可能被另一个上下文错误接受。例如，用来认证握手消息的 MAC，可能被当作另一阶段的密钥或证明，也可能被使用相同函数与密钥的其他协议接受。域标签使不同用途的输入在设计上彼此区分。

第 6 章：网络架构

解答 **6.1**。 入口守卫：能看到用户 IP 地址、流量时间与数据量，但不知道目的地。中间中继：既不知道用户，也不知道目的地。出口中继：知道目的地，并可看到没有在应用层加密的内容，却不知道用户身份。HSDir：不会获得用户访问普通网站的信息。目录权威节点：通常不知道某人访问了哪个网站，但如果客户端直接向权威节点获取目录文档，尤其在引导时，权威节点可能看到客户端 IP。如果客户端从后备目录或缓存获取文档，则看到请求的是这些服务器，而非权威节点。

解答 **6.2**。 `Link=3-5` 表示支持最高到版本 5 的链路协议（第 7 章）。`Relay=2-6` 包括 ntor、ntor-v3（`Relay=4`）和 CGO（`Relay=6`）。`FlowCtrl=1-2` 表示支持经过认证的 SENDME 与拥塞控制。如果中继同时具有 `Exit` 标志，并协商使用拥塞控制与 CGO，就可以充当 CGO 出口（第 9.7 节）。

解答 **6.3**。 100 KB 应用数据约需 $102400/498 \approx 206$ 个 DATA cell，链路上相应的数据量约为 $206 \times 514 \approx 106$ KB。还要加上少量请求、流建立和 SENDME cell（大约七个，即每 31 个数据 cell 发送一个），以及 TLS 记录开销。不计初始握手，总体开销约比有效载荷高 5–7%。

第 7 章：通道与链路协议

解答 **7.1**。 客户端总是主动发起连接，包括连接自己的入口守卫和提供目录信息的中继。没有人需要主动联系客户端，监听端口反而会让它更容易被识别。中继主要在收到 `EXTEND2` 请求、需要连接电路下一跳时主动建立通道；此外，也会像其他节点一样，为下载目录文档或检查自身可达性建立电路。

解答 **7.2**。 `0x000772F7` $= 488183$ 小时，对应 UTC 时间 2025 年 9 月 9 日 23:00。以 32 位无符号整数计数秒，会在 2106 年溢出；改为计数小时则可覆盖数十万年，而精确到小时已经足以表示有效期为数天或数周的证书。规范中有一处需要注意：`cert-spec` 声称四字节可以使用到 10136 年。这是 提案 338 明确指出的错误，原因是把计数器误解成按分钟计数，因为 2^{32} 分钟大约是 8 166 年。

解答 **7.3**。 $38 = 32 + 2 + 2 \times 2$ 对应一种包含两种方法的挑战。如果 N_Methods 为 2，消息体就必须包含 $32 + 2 + 4 = 38$ 字节。长度为 36 字节的消息体声明的方法数量多于实际包含的数量，因此格式不正确，必须拒绝。规范允许忽略多余字节，却不允许缺少必要字节。

解答 **7.4**。 类型 5 证书认证的公钥，与攻击者 TLS 证书的 SHA-256 指纹之间的比较会失败：攻击者的证书不同于真实中继的证书。如果攻击者提供完全相同的真实 TLS 证书，没有对应私钥就无法完成 TLS 握手。

解答 **7.5**。 `695661c0`（UTC 时间 2026 年 1 月 1 日 12:00:00）；`04 04 c6336407`（客户端地址 198.51.100.7）；`01`；以及 `04 04 cb007105`（中继地址 203.0.113.5），之后填充零字节，直到消息体达到 509 字节。

解答 **7.6**。 首先，当用户在家庭、工作单位和移动网络之间切换时，入口守卫始终看到同一个身份，因此能够关联这些环境中的会话与 IP 地址。其次，客户端向某些中继获取目录信息时，也会提供这个身份。串通的中继，或者同时观察它们的攻击者，便能将用户在不同中继上的活动关联起来。两种情况都会让持久标识符破坏 Tor 所追求的不可关联性。

解答 **7.7**。 思路：使用 `cryptography.x509` 等 X.509 库解析类型 2 证书，提取 RSA 公钥，验证证书自签名，并检查公钥指数为 65537。然后按照格式解析类型 7 交叉证书，确认被认证的 Ed25519 公钥与已验证的身份一致，再使用 PKCS#1 v1.5 验证 RSA 签名；签名针对规定前缀与字段的 SHA-256 哈希，不携带哈希算法 OID。中继指纹是 RSA 公钥按 PKCS#1 格式 DER 编码后的 SHA-1 摘要。

解答 **7.8**。 不能。客户端 `test008c` 的流量经过的是它自己与守卫之间的通道，而不是我们的通道。我们的通道还没有承载过电路，因此 `channelpadding_decide_to_pad_channel` 决定不添加填充。要观察通道填充，必须让多跳电路经过我们自己的通道，如 第 10 章 所示。

解答 **7.9**。 TLSSECRETS 根据 A 与中间人的 TLS 会话秘密派生，A 的签名也涵盖它。B 则根据自己与中间人的 TLS 会话秘密重新计算 TLSSECRETS。两组秘密不同，因此即使中间人忠实转发每条信元，使 SLOG 与 CLOG 相同，验证也会失败。如果没有 TLSSECRETS 和 SCERT，中间人就能在 B 面前冒充 A 并转发 A 的签名，误导 B 相信自己已经与 A 建立了经过认证的通道。

第 8 章：信元

解答 **8.1**。 **`CREATE2`** 是定长信元：在使用两字节 CircID 的链路协议版本 3 中总长 512 字节；在使用四字节 CircID 的版本 5 中总长 514 字节。**`VERSIONS`** 始终采用两字节 CircID，因此总长为 $2+1+2+4=9$ 字节。

解答 **8.2**。 命令 0（**`PADDING`**）是定长；7（**`VERSIONS`**）是变长；9（**`RELAY_EARLY`**）是定长；12（**`PADDING_NEGOTIATE`**）是定长；129（**`CERTS`**）是变长。按照命令编号 ≥ 128 的规则，200 应当采用变长格式，但它未定义，因此不允许使用。接收方可以丢弃它并关闭通道。

解答 **8.3**。 只有 `0x92345678` 合法。CircID 零值保留给不属于任何电路的信元。在链路协议版本 ≥ 4 中，发起通道的一方（此处为客户端）必须选择最高有效位为 1 的 CircID；而 `0x12345678` 的最高有效位为 0。

解答 **8.4**。 第一个信元：CircID 为 `00000000`，命令 `0x81`=129（**`CERTS`**），因此采用变长格式；长度为 `0003`，消息体为 `aa bb cc`。第二个信元：CircID 为 `0x80000007`，命令 `0x04`（**`DESTROY`**），属于定长格式，消息体包含 509 字节。消息体的第一个字节 `00` 对应 NONE 原因码。`aabbcc` 本身不是有效的 CERTS 对象：本题只讨论信元的外层封装格式。

解答 **8.5**。 `02 0000 0005 00000000 012c`：DATA 命令（2）、Recognized 为 0、StreamID 为 5、Digest 暂时为零，数据长度 $300 = $ `0x012c`。剩余 $509-11-300=198$ 字节为填充：前四字节为零，其余 194 字节为随机数据。

解答 **8.6**。 对于仍受到另一层加密的信元，Recognized 恰好为零的概率是 2^{-16}，因此平均每一百万个这样的信元中约有 $10^6/65536 \approx 15$ 个。只检查 Recognized 不够，因为存在误判。只有进一步比较运行中的摘要才能确认中继是该信元的目标；非目标中继想碰巧匹配摘要，还需要额外的 2^{-32} 概率。

解答 **8.7**。 假设攻击者控制入口守卫与出口。当出口希望标记一条电路时，可以用 `NOSUCHSERVICE` 等罕见原因码关闭它。如果中间中继和守卫把该原因值原样转发给客户端，守卫就能在同一时刻于自己的某条电路上看到这一不寻常的值，将守卫已知的客户端与出口已知的目的地关联起来。如果所有中继只传输和传播 NONE，原因字节就无法承载这样的信号。通过电路关闭时间进行关联仍有可能，但这种信道的噪声大得多。

解答 **8.8**。 传统格式：$\lceil 1048576/498 \rceil = 2106$ 个信元，在通道上占 $2106 \times 514 = 1\,082\,484$ 字节，开销约为 3.2%。CGO：$\lceil 1048576/488 \rceil = 2149$ 个信元，占 $1\,104\,586$ 字节，开销约为 5.3%。这里没有计入 SENDME 和流建立消息。

解答 **8.9**。 思路：使用 Hypothesis 生成由 CircID、命令和消息体组成的元组列表。从 `CellCommand` 中选择命令，将定长信元的消息体限制在 509 字节以内。将 `encode` 的输出拼接起来，再切分为随机长度的碎片，逐个送入缓冲区。反复调用 `decode` 提取信元，直到缓冲区不再包含完整信元。检查定长信元的消息体是否正确补零至 509 字节。

解答 **8.10**。 在 Arti 2.6.0 中，通道编解码器为 `crates/tor-cell/src/chancell/codec.rs` 中的 `ChannelCodec`，通过 `ChannelCodec::new` 以链路协议版本初始化。它的 `write_cell` 和

`decode_cell` 方法分别序列化和提取信元。与我们的 `decode` 一样，`decode_cell` 在收到不完整信元时不会报错，而是返回 `Ok(None)`，直到收到足够字节，并且只在获得完整信元时消耗缓冲区数据。不同之处在于，Arti 使用可变的 `BytesMut` 缓冲区，并以通道消息类型 `M: ChanMsg` 为泛型，因此可以限制允许的消息集合。另一个显著区别是，`decode_cell` 从偏移 4 处读取命令，并假设定长信元为 514 字节，也就是使用四字节 CircID。Arti 只支持链路协议版本 4 和 5，见 `crates/tor-proto/src/channel/handshake.rs` 中的 `LINK_PROTOCOLS`；首次握手阶段的两字节 CircID **`VERSIONS`** 信元会单独处理。

第 9 章：电路建立

解答 **9.1**。 ntor 需要 $20{+}32{+}32 = 84$ 字节。带三字节消息的 ntor-v3 需要 $32{+}32{+}32{+}3{+}32 = 131$ 字节。HDATA 最多包含 $509 - 4 = 505$ 字节，因此 CM 最多可占 $505 - 128 = 377$ 字节。

解答 **9.2**。 **`CREATE_FAST`** 的秘密是 $X \| Y$，在通道内部以明文形式发送。如果用于第二跳，X 和 Y 就必须穿过第一台中继。该中继知道自己那一层的密钥，因此能够读取这些值、派生第二跳的密钥，并解密客户端与第二跳交换的一切数据，破坏不同跳点之间的隔离。

解答 **9.3**。 AES-CTR 将明文与只由密钥和计数器决定的密钥流进行 XOR。如果两个方向使用相同密钥，客户端发送的第 n 个信元和中继发送的第 n 个信元就会使用相同密钥流。观察两份密文的中间中继，可以将它们异或，得到两份明文的异或值，往往足以恢复两条消息。

解答 **9.4**。 中继 `test004r` 的 ORPort 为 5104，即 `0x13f0`。`03` 表示有三个链路说明符；IPv4 地址与端口说明符为 `00 06 7f000001 13f0`。全部消息包含 $1 + (2+6) + (2+20) + (2+32) = 65$ 字节说明符、$2 + 2 = 4$ 字节 HTYPE 与 HLEN，以及 $32 \cdot 3 + 1 + 32 = 129$ 字节 HDATA。总共 198 字节，远低于中继消息 498 字节的容量上限。

解答 **9.5**。 缺少 $\mathrm{EXP}(B, x)$ 时，中继身份就得不到认证。转发握手的中间中继可以使用自己的 y 生成 Y 和 $AUTH$，冒充下一跳，读取后续发往该跳的流量，即实施中间人攻击。缺少 $\mathrm{EXP}(Y, x)$ 时，前向保密就会丧失。以后获得长期洋葱密钥 b 的任何人，都能根据保存的握手重新计算 $\mathrm{EXP}(X, b)$，并派生全部会话密钥。

解答 **9.6**。 `VER` 把握手与使用场景绑定；Tor 的值是 `"circuit extend"`。由于所有派生都加入了这一值，一个上下文中生成的握手，不会在另一个上下文中产生有效密钥或 $AUTH$。如果缺少 `VER`，不同协议使用相同计算时，攻击者可能把其他协议的握手消息重放给 Tor 中继，或者反过来。这属于跨协议攻击。

解答 **9.7**。 在实验网络中，支持 `Relay=2-6` 的 `test003r` 会拒绝仅包含 `[02 06]` 的请求，并发送原因码为 `PROTOCOL` 的 **`DESTROY`**。包含 `CC_FIELD_REQUEST` 的组合请求则会被接受，中继返回 `CC_FIELD_RESPONSE`。tor 源码只会在同时启用拥塞控制的跳点启用 CGO：`src/core/or/extendinfo.c` 中写着 `enable_cgo = supports_cgo && use_congestion_control`。提案 359 解释了原因：新中继消息格式只有部分命令带 StreamID，因此与旧的流级 SENDME 流量控制不兼容。它要求使用 提案 324 的流量控制，即 `FlowCtrl=2`。

解答 **9.8**。 这是动手练习。`ntor_ref.py` 包含客户端与服务器端函数。使用固定密钥而不是每次随机生成新密钥，可以得到可复现的结果，并与 `ntor.py` 比较。

解答 **9.9**。 思路：`server_part1` 接收 HDATA，验证 `ID` 与 B，计算 $X_b = \mathrm{EXP}(X, b)$，派生第一阶段密钥，验证 MAC 并解密 CM。随后 `server_part2` 生成 (y, Y)，计算 *key_seed* 和 $verify$，派生 SM 使用的密钥并计算 $AUTH$。如果把 y 固定为提案测试向量中的值，那么回应必须与 `server_handshake` 一致。

第 10 章：洋葱式加密

解答 **10.1**。 客户端更新第二跳的正向运行摘要，使用 Kf_2 加密，再使用 Kf_1 加密。第三跳不会看到该信元：第二跳识别后就不会继续转发它。

解答 **10.2**。 每条信元消耗 509 字节密钥流。发送 1 000 个信元后，密钥流位置为 $509 \times 1000 = 509\,000$ 字节，已超过 AES 的第 31 812 个 16 字节数据块。如果为新电路重复使用该密钥，却将计数器重置为零，就会重复使用密钥流；异或两份用相同密钥流加密的密文，会暴露明文的异或值。因此，每条电路都必须通过包含临时随机值的握手派生新密钥。

解答 **10.3**。 中继的运行摘要将加入一条客户端并未计入该跳的信元。此后双方的摘要状态会分离，后续信元都无法被识别，电路最终失效。对于每个经过该中继、却不是发给它的信元，发生这种情况的概率为 2^{-16}；平均约每 65 536 个信元出现一次，相当于约 33 MB 的流量。

解答 **10.4**。 攻击者应修改明文预期内容已知或可猜测的字节，以便出口处的同伙识别特定变化，例如中继消息命令（`DATA` 为 `02`）、StreamID，或填充开头的四个零字节。在检查摘要前，串通的出口会先判断这些字节是否符合预期的修改；诚实的出口则只会发现摘要无效并销毁电路。

解答 **10.5**。 在 CGO 中，出口去掉自己的加密层后，被篡改的信元会产生伪随机的标签与内容，不存在可识别的标记：修改会以不可预测的方式扩散至整个信元。由于标签还会影响下一条信元的处理，电路后续的信元也无法解密，电路只能被丢弃。因此，攻击者至多能破坏电路，而不需要这种机制也能做到。

解答 **10.6**。 实际观测间隔为 4.1、3.0、5.7、8.7 和 6.8 秒，平均为 5.7 秒。如果取区间 $[a, b]$ 上两个独立均匀随机变量的最大值，其期望为 $a + \frac{2}{3}(b-a) = 1.5 + \frac{2}{3} \cdot 8 \approx 6.8$ 秒。仅有五个样本，观测均值与期望存在这种差异并不意外。还要注意，每当任何信元经过通道时，计时器都会重新开始，而不只是填充信元。

解答 **10.7**。 在实验网络中，中间中继不会回应 `DROP`：消息被识别后直接丢弃。电路仍可正常使用是间接证据：发送两条 `DROP` 后，客户端依然可以把电路扩展到出口。如果中间中继没有识别这些信元，那么它当时作为电路的终点，会销毁电路。如果客户端与中间中继对信元的计数不同，双方运行摘要会失去同步，后续扩展也会失败。

解答 **10.8**。 在私有网络测试中，客户端修改一条发给守卫的 `EXTEND2` 信元中的一个比特，而守卫当时是电路的唯一一跳，摘要检查因此失败。作为电路终点，守卫返回原因码为 1（`PROTOCOL`）的 **`DESTROY`**。在较长电路上，客户端若修改发往第三跳的信元，修改后的信元会先经过前两跳，直到第三跳才被检测到。

第 11 章：流、SENDME 与拥塞控制

解答 **11.1**。 主机名必须使用小写：`example.com:443` 后接一个零字节，即 `6578616d706c652e636f6d3a343433 00`，然后是 FLAGS，设置第 1 位表示接受 IPv6：`00000001`。

解答 **11.2**。 IPv4 地址 `c0000201` 表示 192.0.2.1；TTL `0000012c` 表示 300 秒。客户端可以忽略该地址。如果选择保存，就不能跨电路重复使用：出口可能撒谎，或者返回专门生成的唯一地址来识别客户端。

解答 **11.3**。 如果不使用乐观发送，客户端必须等到收到 `CONNECTED` 才能发送请求，因此在至少需要两个 RTT 的操作中额外增加一次往返时间，约 600 ms。连接失败时，排队的数据必须丢弃，因为没有目的地可以接收它们；不能把这些数据通过另一条连接发送。

解答 **11.4**。 一条流最多允许 500 个尚未确认的信元，相当于每个 RTT 约 $500 \times 498 \approx 249$ KB。如果 RTT 为 400 ms，速率约为 620 KB/s。即使存在多条流，单条电路也只能在一个 RTT 内发送至多 1 000 个信元，约 1.2 MB/s。使用拥塞控制时，窗口开始只有 124 个信元（约 62 KB），随后依据 RTT 测量增长，直到发现拥塞；不存在固定的窗口上限。

解答 **11.5**。 恶意客户端可以向发送大量数据的服务器（也许是它自己运营的服务器）打开许多流，只读取少量数据，却不断发送伪造的 `SENDME`。出口因此继续传输，而信元会在守卫、中间中继和客户端的队列中积累，消耗中继内存。经过认证的 `SENDME` 要求客户端证明自己收

到了触发该消息的信元。如果信元内容可预测，例如是熟悉的下载文件，客户端就可以在未收到信元的情况下计算摘要；随机填充字节能够阻止这一捷径。

解答 **11.6**。 思路：在新的 StreamID 上发送 `RESOLVE`，消息体是以零字节结尾的主机名，然后将 `RESOLVED` 解释为一组包含类型、长度、值和 TTL 的元组。对于不存在的域名，出口返回错误类型条目（`0xF0` 或 `0xF1`），应当忽略其载荷。

解答 **11.7**。 出口发送 `END`，原因码为 4（`EXITPOLICY`），后接目的地址（IPv4 地址占四字节）以及四字节 TTL。客户端可以根据这些信息避免再次通过同一个出口访问该目的地。

解答 **11.8**。 我们测量 `SENDME` 到达出口所需的时间，加上出口解除阻塞后发送额外信元、这些信元到达客户端所需的时间。当出口正在等待 `SENDME` 时，这可以近似反映客户端观察到的电路 RTT。第二个并行下载会增加排队，抬高测量值：这恰好是拥塞控制用于缩小窗口的信号。

第 12 章：目录系统

解答 **12.1**。 投票周期为一小时，即从 `valid-after` 到 `fresh-until`。客户端在 $11{:}00 + \frac{3}{4}\cdot 60' = 11{:}45$ 与 $11{:}45 + \frac{7}{8}\cdot 75' \approx 12{:}50$ 之间随机选择一次更新时间。如果到 13:00 仍未获得新共识，旧共识在形式上已经过期。客户端仍可依据 `networkstatus.c` 中的 `REASONABLY_LIVE_TIME` 把它视为合理地仍可使用——过期不足 24 小时。此时客户端会不断尝试获取新共识，同时继续建立电路；直到旧共识过期约 24 小时后，才会停止依赖它。

解答 **12.2**。 很可能有 `Running` 和 `Valid`，同时因为 500 KB/s 高于 100 KB/s，可能获得 `Fast`。如果该中继也提供目录缓存，还可能有 `V2Dir`。它没有 `Stable`、`Guard` 或 `HSDir`，因为这些标志需要连续运行数天至数周。它也没有 `Exit`，因为出口策略只允许端口 443，而该标志要求同时允许端口 80 和 443 的出口流量。

解答 **12.3**。 缓存向某个特定客户端发送两小时前、但仍有效的旧共识，其中不包括最近加入的中继。该客户端因而在与其他人稍有不同的节点集合中选择路径，可能让自己的流量变得容易识别。攻击受到三项限制：共识只有三小时有效期；必须有真正的权威签名（缓存不能伪造共识）；客户端随机选择缓存。客户端按摘要而不是中继名称请求微描述符，因此缓存不能为这名客户端单独提供另一版本的中继密钥，因为摘要已经由签名的共识固定。

解答 **12.4**。 三次测量的中位数为 5200。单个被人为抬高的测量值不会改变中位数。如果第三台权威节点是唯一测量此中继的节点，系统就得不到取中位数所需的三次测量。此时共识会使用声明或投票带宽，设置 `Unmeasured=1` 并施加上限，因此被抬高的测量值不会影响结果。

解答 **12.5**。 投票文档包含共识中不存在或与共识不同的行。例如，带宽权威节点提供的 `Measured` 评估、标识所支持共识方法的 `m` 行、该权威节点自行分配的标志、`flag-thresholds` 以及共享随机性承诺。共识只保留综合后的结果。

解答 **12.6**。 对从微描述符开头的 `onion-key` 行开始，到下一微描述符前一个换行符或文档结尾为止的文本计算哈希，包含 PEM 编码的 RSA 密钥行。SHA-256 摘要按 base64 编码并移除末尾 `=` 后，必须与共识中的 `m` 行匹配。

解答 **12.7**。 一台权威节点停止后，剩余两台仍可投票，生成带两个签名的共识，满足所需多数 $\lfloor 3/2 \rfloor + 1 = 2$。如果两台停止，仅存的一台无法达到多数，因此不会生成新共识。客户端仍可在旧共识过期后约 24 小时内，把它视为合理地仍可使用而继续工作。实验网络中一份共识只持续 40 秒，但上述宽限期仍为 24 小时，因此故障不会立即显现。

解答 **12.8**。 `ns` 格式的共识通常包含 SHA-1 签名，规范也只允许该格式使用这类签名。它的 `r` 行还包含完整的中继描述符摘要，每个中继条目中另有汇总出口策略的 `p` 行。

第 13 章：路径选择

解答 **13.1**。 出口具有最严格的条件：必须允许连接所请求的目的地。首先选择出口可以确保存在合适的候选。如果先选择守卫，家族或网络限制可能排除唯一合适的出口，迫使算法重新开始。统计学上也有理由：排除限制作用于后续位置，所以应该尽量让它们影响候选集合更大的位置。

解答 **13.2**。 作为守卫：$20000 \times 0/10000 = 0$（`Wgd=0`）。作为中间中继：$20000 \times 0 = 0$（`Wmd=0`）。作为出口：$20000 \times 10000/10000 = 20000$（`Wed=10000`）。根据这份共识的权重，该中继只会用作出口。

解答 **13.3**。 没有持久守卫时，当 $1-(1-c^2)^C > 0.5$ 时，失守概率超过 50%，也就是 $C > \ln 0.5 / \ln(1-c^2)$：当 $c = 1\%$ 时大约需要 6932 条电路，$c = 20\%$ 时大约需要 17 条电路。固定使用一个守卫时，概率仍接近 c，分别为 1% 和 20%。面对控制能力极强的攻击者，守卫机制也不够，但两种策略的差异仍很显著。

解答 **13.4**。 攻击者必须真正提供带宽，因为选中概率依赖权威节点测量的带宽，而不是自行声明的数值。出口还受到严格监控，包括 `BadExit` 标志。每个 IP 地址最多两台中继的限制，迫使攻击者获取大量地址。声明家族会阻止其多台中继同时出现在同一条电路；不声明则有被目录权威节点发现并排除的风险。最后，带宽权重会把稀缺的出口容量保留给实际承担出口角色的中继，而攻击者提供的带宽也只在适当位置发挥作用。

解答 **13.5**。 如果没有样本大小限制，控制客户端本地网络的攻击者可以阻断所有诚实守卫的连接。客户端便会不断抽取新的守卫，直到找到一台可连接的守卫，最终可能选中攻击者控制的中继。限制样本后，客户端会耗尽尝试次数并停止添加守卫，即使因此失去连接，也不会自动选择攻击者的守卫。

解答 **13.6**。 这是实践练习。应从 CollecTor 下载与共识对应的微描述符。针对每台中继，使用 `family-ids` 或相互声明的 `family` 列表构建家族标识符集合；如果一个候选中继与已选中任何中继属于同一家族，就拒绝它。

解答 **13.7**。 这是研究练习。大型家族总体上通常占有比任何单台大型中继都高得多的出口选择概率。因此，实际集中风险取决于运营者，而不只是中继数量。这也是 Tor 社区关注运营者多样性和未声明中继家族的原因（第 27 章）。

第 14 章：v3 洋葱服务

解答 **14.1**。 $32 + 2 + 1 = 35$ 字节，即 280 比特。Base32 每个字符编码五比特，因此不带填充时需要 $280/5 = 56$ 个字符。随机输入错误恰好匹配校验和的概率约为 2^{-16}。版本字节还必须等于 `0x03`，使随机错误未被发现的概率进一步降低。

解答 **14.2**。 2026 年 1 月 1 日 UTC 09:00，距离 Unix 纪元已经过 29 454 300 分钟。减去 720 分钟的偏移得到 29 453 580，因此 $\lfloor 29453580/1440 \rfloor = 20453$。第 20 453 个周期从 2025 年 12 月 31 日 UTC 12:00 开始，因为 09:00 仍早于中午。

解答 **14.3**。 攻击者需要该周期的盲化密钥，因此首先需要知道服务地址；还要知道该周期的 SRV，以及自己控制的中继在环上的位置。之后可利用这些数值生成中继身份，使其恰好位于服务索引之后。SRV 只有在权威节点公布时才会得知，通常距离相关周期开始还有大约半天。攻击者不能提前几天精确部署中继；新中继还必须获得 `HSDir` 标志，而这至少需要运行 96 小时。

解答 **14.4**。 HSDir 看到的是盲化密钥，而不是服务地址，请求通过匿名电路到达。引介点看到认证密钥以及服务发起的电路，并能统计引介请求。汇合点看到 cookie、两条电路和在它们之间传输的加密流量。引介点特别适合估算热度，因为它能看到每次连接尝试，虽然不知道对应服务地址。不过，已经知道地址的人可以下载描述符并建立对应关系。

解答 **14.5**。 单跳电路中的 HSDir 就是第一跳，因此能够看到客户端 IP 地址。它至少能通过盲化密钥，或者在已知地址时直接通过地址，将谁正在搜索哪个服务联系起来。拒绝非匿名请求会迫使客户端通过其他中继获取描述符。

解答 **14.6**。 这是实践练习。当 HSDir 数量大于 2×4 时，服务不再把描述符上传到所有位置。通过三跳电路向每台 HSDir 请求描述符，应当只在为当前或下一周期计算出的指定 HSDir 上找到它，因为服务会为两个周期发布描述符。

解答 **14.7**。 base64 值中首先是链路说明符数量，随后是符合 `EXTEND2` 格式的说明符（第 9.2.2 节）。例如，类型 `00` 包含地址与 ORPort，类型 `02` 包含 RSA 指纹，类型 `03` 包含 Ed25519 身份。可以通过 ORPort 找到中继，它等于 5100 加节点编号。`auth-key` 行包含类型 9 证书，由描述符签名密钥签署，用于认证服务在该引介点的认证公钥。

第 15 章：拒绝服务防御

解答 **15.1**。 在第一分钟内，初始满令牌桶提供 60 个令牌，补充机制增加 $2 \times 60 = 120$ 个，因此理论上约可建立 180 条电路；实际可能稍少，因为令牌桶一旦降到零，客户端便被标记。一小时内按持续速率计算，合计约 $60 + 2 \times 3600 = 7260$ 条。正常客户端通常只建立几十条电路。

解答 **15.2**。 只有第一跳知道客户端真实的 IP 地址。对于后续跳点，通道上可见的“地址”其实属于上一台中继。如果中间中继按地址限制电路，它实际上会限制来自某个入口守卫的全部电路，而该守卫可能服务数千名用户。这样会伤及所有人，而不是针对攻击者实施限制。因此，按地址限制只用于未经认证的客户端连接。

解答 **15.3**。 客户端在可重复使用的电路上发送一条 `INTRODUCE1`。服务必须解密请求、执行 hs-ntor 握手，并向汇合点新建至少三跳的电路。这需要与中继进行多次 ntor 握手，还要发送创建和扩展各跳所需的信元。因此，一条客户端信元可能触发数次公钥运算和数十条网络信元，形成十倍到百倍量级的放大效应。

解答 **15.4**。 要求前 k 位为零，只能表达 2 的幂次工作量等级：1、2、4 等。每多要求一个零位，预期计算成本就翻倍。线性检验则允许任意整数工作量 E。以较小步长调整工作量的反馈控制器需要这种更细致的刻度；倍增式刻度容易使控制器在工作量过低与过高之间来回振荡。

解答 **15.5**。 不能。工作量 E 是计算 Equi-X 解和哈希 R 所使用挑战的一部分。把声明值改为 $E = 5000$，挑战随之改变，因此最初针对 $E = 50$ 找到的解会失效。客户端必须在开始搜索前就选择 $E = 5000$。

解答 **15.6**。 这是实验练习。信元会在中间中继的电路队列中积累，直到内存占用超过 `MaxMemInQueues`。内存管理器随后关闭排队数据最旧的电路，包括我们的电路，并在中继日志中记录释放了多少内存。

解答 **15.7**。 这是编程练习。提供较高工作量的请求，预期具有短而稳定的等待时间。工作量为零的请求，主要在队列几乎为空时才会得到处理；一旦队列需要缩减，就可能被丢弃。这正是机制想实现的效果：保护愿意支付适度计算成本的用户。

第 16 章：审查、网桥与可插拔传输

解答 **16.1**。 网桥与入口守卫承担相同角色，都是电路的第一跳，但网桥不出现在公开共识中。客户端通过私下获得的网桥配置行知道它的地址。启用 `UseBridges` 后，客户端使用网桥代替公开守卫。再增加一台公开守卫就需要直接连接已公布的地址，而审查者恰恰会封锁或监视这类连接。

解答 **16.2**。 “看起来不像任何可识别的协议”：obfs4。要封锁它，审查者需要拒绝所有无法识别为获准协议的流量。“看起来像允许的协议”：meek，需要封锁大型内容分发服务商；Snowflake，需要干扰 WebRTC、协调服务器和大量志愿者代理；WebTunnel，需要封锁发往陌生网站或托管网桥网站的 HTTPS；Conjure，需要封锁合作网络接入服务商的大片地址空间。

解答 **16.3**。 审查者可能检查：发往没有对应主机名的地址的长时间连接、从首个数据包开始就具有很高熵的字节、类似 Tor 信元集中发送的大小与时间分布，以及类似网页浏览的上下行流量比例。`iat-mode` 增加随机延迟和分片，使时间与长度特征更难识别，代价是性能下降。

解答 **16.4**。 每获得一台网桥就立即封锁它的审查者，无法积累信誉：网桥被封锁后，其凭据就无法升到更高级别，也不能获得邀请权限。攻击者可以改为暂时不封锁网桥，先累积足够多的高级凭据，再同时发动封锁。然而，这需要维护大量耐心等待数月的身份；在这段时间里，被发现的网桥仍可以帮助合法用户。每份高级凭据能够访问的网桥数量也是有限的。

解答 **16.5**。 两份抓包的每字节熵都应接近八比特：TLS 完成握手后也传输加密数据。差别主要在少数可识别字节，例如 TLS 记录头部（`16 03 01`、`17 03 03`，后面是长度字段）和最初未加密的握手。深度包检测不需要计算统计熵，也可以识别这些特征。

解答 **16.6**。 实验 1 会超时（`TimeoutError`），因为网桥不作回应。错误的 `cert` 参数会使客户端 `obfs4proxy` 握手失败。网桥无法识别客户端的认证标记，因此不会回应，延迟一段时间后关闭连接。根据日志详细程度，网桥日志可能只有握手失败消息，也可能没有相关记录。

第 17 章：安装与 **torrc** 配置

解答 **17.1**。 tor 会报告无法识别的选项，例如显示 `Unknown option '...'. Failing.`，然后不启动就退出。对客户端的 `torrc` 运行 `--dump-config short`，会显示所有不同于默认值的设置，包括实验网络特有的 `TestingTorNetwork`、`DirAuthority` 条目以及监听端口。

解答 **17.2**。 如果邮箱与论坛流量共用电路，出口就能看到同一电路在接近的时间连接了用户的邮箱服务商（可能暴露身份）与使用假名的论坛，从而关联假名和真实用户。解决方法包括：给两个应用分别设置 SOCKS 端口，默认情况下不同端口彼此隔离，除非设为同一个 `SessionGroup`；使用不同 SOCKS 凭据并启用 `IsolateSOCKSAuth`；或者在共用端口上使用 `IsolateDestAddr`。

解答 **17.3**。 监控意大利网络的攻击者，可能同时看到意大利用户进入 Tor，以及意大利出口访问意大利目的地。通信两端于是位于同一个司法辖区或可观察的网络区域，使 Tor 威胁模型所排除的流量关联成为可能。意大利出口数量也相对有限，强行限制出口选择会让用户更显眼，也更容易成为恶意中继的目标。

第 18 章：**Control Port** 与 **Stem**

解答 **18.1**。 `552` 是对命令的永久性错误回应：请求的键 `pippo` 不存在。与之不同，`650` 不是命令回应，而是客户端通过 `SETEVENTS` 订阅后收到的异步事件。它可以在任何时刻到达，包括另一条命令的回应尚未完成时。

解答 **18.2**。 思路：调用控制器的 `create_ephemeral_hidden_service`，将虚拟端口 80 映射到本机端口 8080，并设置 `await_publication`。方法返回洋葱地址。通过 `curl --socks5-hostname 127.0.0.1:9008 http://<address>.onion/` 访问。tor 将临时服务密钥保存在内存中，并把它们返回给控制器；控制器可以保存密钥，以便将来重建相同地址。默认情况下，除非设置为脱离控制连接运行，否则控制连接关闭时临时服务也会消失。

解答 **18.3**。 思路：注册处理 `EventType.BW` 的回调函数。它每秒收到的已读和已写字节数分别位于 `event.read` 与 `event.written`。下载大文件期间显示这些值。入站速率会逐渐上升，直到接近拥塞控制与可用系统资源所允许的上限。

解答 **18.4**。 启用 `__LeaveStreamsUnattached` 后，tor 不再自动把流附加到电路。`EXTENDCIRCUIT 0 test004r,test005r,test006e` 会建立电路并返回其标识符。当状态为 `NEW` 的 `STREAM` 事件到达时，调用 `ATTACHSTREAM`。手工选择路径很危险，因为它绕过加权选择与守卫策略：不寻常的路径可能让用户更容易被识别，恶意软件也可能故意通过攻击者控制的中继传输流量。

第 19 章：私有 Tor 网络

解答 **19.1**。 `test006e` 的节点编号为 6，因此 ORPort 是 5106，ControlPort 是 8006。`test009h` 没有 ORPort：洋葱服务不是中继，不接受其他网络节点主动发起的连接，而是自己向引介点和汇合点建立出站电路。

解答 **19.2**。 相关选项是目录权威节点 `torrc` 中的 `TestingDirAuthVoteGuard *`。在公共网络中，获得 Guard 标志需要经过一定时间的稳定性与带宽验证（第 12 章）。在实验网络中，任何中继都可以充当守卫，因此在其中测得的第一跳选择统计，不能代表公共网络的情况。

解答 **19.3**。 使用 `--socks5` 时，`curl` 会自己解析 `localhost`，再向 Tor 客户端提供 IP 地址。使用 `--socks5-hostname` 时，它会把域名传给出口解析。在公共网络上访问普通互联网域名时，本地解析会把访问的网站泄露给客户端的 DNS 解析器，以及任何观察该解析器的人，即使后续连接仍通过 Tor。这是典型的 DNS 泄露。

解答 **19.4**。 按照 `V3AuthVotingInterval` 的配置，每 20 秒更新一次。因此实验网络每天生成 $86400/20 = 4320$ 份共识；公共网络中的权威节点每小时投票一次，每天只产生 24 份。

解答 **19.5**。 停止一台权威节点后，网络仍能产生共识，因为三台中的多数，即两个签名，已经足够。停止两台后，最后一台无法独自生成具有足够签名的新共识。不过，客户端仍可在旧共识过期后约 24 小时内，将其视为合理地仍可使用（第 12 章），因此一段时间内仍可建立电路。实验网络的共识有效期只有 40 秒，但这一宽限时间仍然是 24 小时。

解答 **19.6**。 思路：`c.get_network_status(fingerprint)` 返回对应的共识条目，其中含有 `address` 和 `or_port` 属性。遍历 `circ.path` 中的各个元组，取其第一项作为指纹，然后逐个查询。

解答 **19.7**。 采用与本书网络定义相同的结构，将节点数量改成 `getN(5)`、`getN(10)` 和 `getN(4)`。引导时间取决于硬件。随着网络增长，新增时间主要用于等待中继出现在共识中，以及客户端下载微描述符。

解答 **19.8**。 这是探索性练习。查找涵盖路径选择、发送 **`CREATE2`** 和 `EXTEND2`、接收回应以及电路完成的日志。*info* 级别不会记录每一条信元。要获得更详细的信息，可以在客户端 `torrc` 中按模块启用 *debug* 日志。

解答 **19.9**。 不能。TLS 协商完成后，Tor 信元在加密后传输，`tcpdump` 只能看到 TLS 记录。地址、端口、记录长度和时间特征仍然可见。这些恰好是流量分析攻击（第 26 章）能够利用的信息。

第 21 章：发布洋葱服务

解答 **21.1**。 六字符前缀平均需要约 $32^6 \approx 1.07 \times 10^9$ 次尝试。每秒尝试一千万次时，耗时约 107 秒。八字符前缀需要约 $32^8 \approx 1.1 \times 10^{12}$ 次尝试，约 1.1×10^5 秒，略超过 30 小时。每多一个字符，预期工作量就乘以 32。

解答 **21.2**。 错误页面暴露了 Web 服务器的软件名称和版本，这可能有助于识别漏洞；更重要的是，页面显示了 IP 地址 192.0.2.10，可能是服务器真实的公开地址，使服务位置暴露。应关闭 HTTP 头部和错误页面中的服务器签名，让服务器仅监听 localhost 或 Unix 套接字，并通过实验客户端检查响应。

解答 **21.3**。 删除 `alice.auth` 并重新加载配置。服务发布新描述符，排除 Alice 的记录，并使用她无法解密的新描述符 *cookie* 进行加密。仍持有旧描述符的客户端，只要此前获知的引介点仍然有效，就可能继续使用它们。若要更彻底地撤销权限，可以通过重启服务等方式轮换引介点。但要注意边界情况：如果 Alice 是唯一获得授权的客户端，删除后目录将变为空目录，原实验中 HSDir 会拒绝这样的描述符（第 21.3 节）。若要彻底关闭受限发现，应删除 `authorized_clients/` 目录，再重新加载配置。

解答 **21.4**。 设置 `HiddenServicePort 80 unix:/path/socket`，同时让 Web 服务器监听对应 Unix 套接字。除了避免开放 TCP 端口之外，文件系统套接字还可以使用文件权限保护，不能直接通过网络接口访问，也能防止误将服务暴露在公开接口上。

第 20 章：观察流量

解答 **20.1**。 TLS 记录不会与 Tor 信元的边界对齐。tor 将信元写入缓冲区，TLS 库再将它们组成最大约 16 KB 的记录，这与每 31 个信元的 `sendme_inc` 值有关（第 11.3 节）。TLS 还会添加自身的头部和认证标签。随后 TCP 又可能把 TLS 记录拆成不同的数据段，或把多个记录合并到同一段。固定长度的约束适用于 Tor 信元，而不是网络数据包。

解答 **20.2**。 思路：收到 **`CREATED2`** 后，使用 `circuit.py` 中保存的各跳密钥，按正确顺序逐层解密电路上的 **`RELAY_EARLY`** 和 **`RELAY`** 信元。AES-CTR 密钥流状态必须严格按照真实使用顺序推进。建立电路时可以看到 `EXTEND2` 和 `EXTENDED2`，之后还可能看到 `BEGIN`、`CONNECTED`、`DATA` 和 `SENDME`。

解答 **20.3**。 如果客户端通道曾承载多跳电路，即使没有应用流量，也可能每隔几秒出现 TLS 记录。这些记录包含通道级 **`PADDING`** 信元，其发送间隔在 1.5 到 9.5 秒之间随机变化（第 10.5 节）。相比之下，从未承载过多跳电路的通道不会出现这种填充流量。

解答 **20.4**。 这是编程练习。注册为解密后 TLS 数据的启发式解析器的 Lua 程序，可以在最初的 **`VERSIONS`** 信元中读取前两个字节作为 CircID，之后则读取四个字节，再读取命令及变长信元的长度字段。应像 `traffic.py` 那样，为每条连接、每个方向分别保存状态。

第 22 章：运行中继或网桥

解答 **22.1**。 可以。第一条匹配规则 `accept *:80` 会允许连接，后续规则不会再被检查。只需对调前两条规则：`reject 192.0.2.0/24:*`、`accept *:80`、`reject *:*`。

解答 **22.2**。 摘要只列出端口，不包含地址，因此必然是一种近似。只有出口允许该端口连接到“大多数”地址时，摘要才会将其列入。dir-spec 对此有精确定义，并非简单的超过 50%：排除非公开地址后，出口不能阻断超过 2^{25} 个 IPv4 地址（一个 /7）或 2^{112} 个 IPv6 地址（一个 /16）。客户端收到原因码为 `EXITPOLICY` 的 `END` 后，会在指定 TTL 内记住该出口拒绝访问对应地址与端口，并通过包含另一出口的新电路重试流。

解答 **22.3**。 对于 `MyFamily`，只有相互声明才有效。原来的五台中继仍然互为家族成员，但第六台不属于该家族，因为旧中继没有任何一台把它列入。客户端因此可能同时选中第六台与旧家族成员。如果使用家族密钥，只需把相同家族密钥安装到新中继：其签名证书会将它绑定到家族，无须修改其他中继。

解答 **22.4**。 日志依次报告“in-progress connect failed ...(Connection refused)”和“Didn't recognize errno 115”。如果流关闭原因直接使用 `getsockopt(SO_ERROR)` 返回的错误，客户

端就会收到 `CONNECTREFUSED`。同理，在连接最初返回 `EINPROGRESS` 后失败时，`ETIMEDOUT` 可以正确映射为 `TIMEOUT`，`ENETUNREACH` 或 `EHOSTUNREACH` 映射为 `NOROUTE`，另外还可能正确报告 `CONNRESET`，而不是全部变为 `MISC`。对客户端的实际影响很小：错误消息变化，但流失败的处理过程不变。

解答 **22.5**。 以 10 MB/s 的带宽运行，中继每天约传输 864 GB，以发送和接收流量中较大的一项计入配额。因此，1 TB 的月配额在略超过一天后就会耗尽。如果一台中继整个月的大部分时间处于离线状态，它会失去 `Stable` 和 `Guard` 标志，并在关闭时破坏现有电路。相比之下，整月在线但速度较慢的中继对网络更有价值，因此应该通过 `RelayBandwidthRate` 把流量配额分配到整个统计周期。

第 23 章：Tor Browser、Tails 与 Whonix

解答 **23.1**。 “为此网站使用新电路”只更换当前标签页所属网站的电路，不会清除 Cookie、浏览历史或其他网站的状态；网站仍可能凭借已存储的状态识别用户。“新身份”则关闭所有标签页、清除浏览器状态，并为后续活动重新建立电路。二者都不更换守卫，因为频繁轮换会提高最终选中恶意守卫的概率（第 13.3 节）。无论如何，入口守卫都看不到客户端访问了哪些网站。

解答 **23.2**。 用户可能阻止更多跟踪器加载，但 Tor Browser 已按网站隔离跟踪器，并限制持久标识符。安装扩展会牺牲用户之间的统一性：网站可能检测扩展是否存在或采用何种过滤列表，使用户与 Tor Browser 的其他大量用户区别开来。扩展还会增加一批能够访问网页内容的代码。

解答 **23.3**。 如果访问同一网站的两个标签页使用相同 SOCKS 凭据，tor 会把它们的流附加到同一电路。使用不同凭据的第三个标签页会获得另一条电路。`GETINFO stream-status` 可以显示每条流对应的电路标识符。

解答 **23.4**。 （a）Tor Browser 已经足够，可能还需要使用网桥。（b）Tails：从 USB 设备启动，通常不会在他人计算机上留下痕迹，只保留用户主动放入加密持久存储的数据，但仍需考虑遭到攻破的硬件。（c）Whonix：Workstation 上的应用在架构上只能通过 tor 联网，而且系统支持长期运行的工作，不要求像 Tails 一样具有遗忘性。

解答 **23.5**。 这是开放式练习。可选择的特征包括屏幕与窗口尺寸、时区、语言、可用字体和 Canvas 渲染结果。在普通浏览器中，这些特征可能泄露从几个比特（例如时区）到大量比特（例如字体和 Canvas 特征）的信息。Tor Browser 利用 letterboxing 对尺寸取整、报告 UTC 与统一语言、限制字体，并标准化或禁用易泄露信息的接口。仍然存在的差异包括操作系统、选择的安全级别以及取整后的显示尺寸。

第 24 章：网络测量

解答 **24.1**。 目录请求数量会减少一半，而用户估计仍使用固定除数 10，因此估计用户数也会减少一半。随着新版 tor 普及，图表会显示多个国家几乎同时发生、幅度很大的下降。如果这种全球性变化与软件采用率相关，就应怀疑客户端行为发生了变化，而不是用户大量流失。估算方法需要采用修正后的除数。

解答 **24.2**。 估计用户数为 $20\,000\,000/(0.6 \cdot 10) \approx 3\,333\,333$。如果报告覆盖比例只有 8%，由于低于 10% 的最低要求，估计结果会被丢弃。

解答 **24.3**。 思路：筛选同时具有两个标志的中继，将其带宽之和除以全网带宽。计算出口选择概率时，使用实验 7 的 `probabilities(consensus, "exit")`，找出出现次数最多的五个 /16 网络，再对所属中继的选择概率求和。

解答 **24.4**。 这是开放式练习。有明确记录的封锁事件通常会在开始封锁当天，使直接连接用户估计数明显下降。接下来几天，随着受影响用户改用其他传输技术，网桥用户估计数，尤其是 Snowflake 或 WebTunnel 的用户数，可能上升。Metrics 可能把最初下降的日期标为异常。

解答 **24.5**。 如果没有舍入，攻击者知道自己是某个国家唯一访问某中继的用户，或精确知道其他用户数量时，就可以检查额外信息统计，以计数是否增加 1 判断这个国家的另一个人是否连接。舍入可以隐藏这种小幅变化。但控制大量中继的攻击者并不需要公开计数，因为他可以直接观察进入自己中继的连接。

第 25 章：法律与伦理

解答 **25.1**。 （a）出口如果主动向网站发起请求，例如为了“混合”流量，就构成主动发起传输。（b）出口如果把连接重定向到不同于 `BEGIN` 指定的目的地，就参与选择接收方。（c）出口如果向 HTTP 页面插入内容，或者按语义过滤信息，就参与选择或修改信息。这些行为也可能导致中继被标记为 `BadExit`。

解答 **25.2**。 厨房刀具本身合法，但用它实施伤害行为违法。通过 Tor 阅读遭到审查的报纸，可能属于合法使用合法工具；但未经授权访问计算机，不论是否使用 Tor，都可能构成犯罪。中继运营者只提供传输能力，不选择内容，因此更接近承运者而非作者，前提是运营者没有参与违法活动。具体法律结论始终取决于司法辖区和案件事实。

解答 **25.3**。 说明该地址属于 Tor 出口，可通过 Tor Project 的公开工具核实；运营者不知道、也不记录客户端身份，因此无法识别用户；投诉网站如果愿意，可以对公开出口地址的连接采取区别处理措施。在实践中，如果运营收益超过因此带来的限制，运营者也可以增加一条拒绝该网站地址的出口策略规则。

解答 **25.4**。 该计划在未获得同意的情况下观察真实用户的流量，收集访问域名等可能泄露个人信息、不适合公开的数据，而预期收益有限。因此，它违反多项研究安全原则，在意大利还可能涉及刑法与数据保护问题。更安全的替代方案包括使用已公开的 Metrics 汇总数据，只用研究人员自己的 Tor 客户端测量新闻网站可达性，或者请 Research Safety Board 审查一种采用聚合与噪声、类似中继统计的隐私保护计数方案。

解答 **25.5**。 这是研究练习。应检索官方司法数据库，例如意大利法院公开的裁判或欧盟法院文书，而不只是依赖第三方摘要。对于每份裁判，记录法院、日期、适用法律和结果，并区分涉及中继运营者的案件与涉及 Tor 用户的案件。

第 26 章：关联、指纹识别与 Sybil 攻击

解答 **26.1**。 流量关联需要通过中继或底层网络同时观察通信两端；守卫、路径约束和测量带宽可以提高攻击成本。网站指纹识别只需要观察客户端到守卫的链路；目前只有效果有限的填充防御，相关研究仍在进行。发现守卫需要占据中间中继位置并诱发电路建立；守卫与 Vanguards 有助于防御。Sybil 攻击者运营大量中继；防御包括带宽测量、家族、/16 约束和网络监控。路由攻击需要影响自治系统或互联网交换点；应对措施包括 Tor 协议之外的 BGP 监控。

解答 **26.2**。 在 100 000 次访问中，预期正确警报约为 0.95 次，而误报约为 $0.001 \cdot 99\,999 \approx 100$ 次。因此正确警报占比为 $0.95/(0.95 + 100) \approx 0.9\%$。几乎所有警报都是误报。

解答 **26.3**。 如果以两秒为一个区间，却只观测五秒，每条序列只有两三个样本，相关性会变得很不稳定。如果让所有客户端采用相同的流量节奏，例如使用相同随机种子，时间序列会几乎完全相同，匹配就无法区分。统一流量节奏尤其会破坏关联，这也是 PipeNet 恒定速率通信的基本思想（第 2.4 节）。

解答 **26.4**。 恒定传输速率使入口流量时间序列不再变化。此时方差为零，Pearson 相关系数在数学上没有定义，或者主要由网络噪声决定，因此这种攻击失效。代价是即使没有有效数据，也必须按选定的最高速率持续发送。实验中，这可能意味着每名客户端每秒发送数十或数百 KB，比真正的应用数据多出数倍。

解答 **26.5**。 固定守卫时，只有当选中恶意守卫，攻击者才可能成功，其概率约为 5%。发生这种情况后，约 5% 的电路还会经过恶意出口。在六个月的高强度使用中，至少一次失守的概率因此接近 5%；在这个简化模型中，选择诚实固定守卫的用户始终不会失守。每条电路都重新随机选择守卫时，单条电路失守概率约为 $0.05 \cdot 0.05 = 0.0025$；经过数千条电路后，至少一次失守几乎必然发生。这说明持久守卫的意义，实验 7 的 `Adversary` 类也可通过模拟验证。

第 27 章：真实网络攻击案例

解答 **27.1**。 在发往客户端的路径上，每台中继会给信元载荷增加自己的加密层（第 10 章）。守卫收到的载荷受到中间跳点的密钥保护，因此无法读取 HSDir 写入其中的内容。外层信元命令则属于每台中继都会读取、并原样转发的头部。

解答 **27.2**。 使用三台守卫时，至少一台为恶意中继的概率大约是只用一台时的三倍，每台恶意守卫还会观察客户端的部分流量。只用一台守卫可以降低选择风险；如果选中诚实守卫，客户端会长期受到保护。缺点是，如果唯一守卫恶意，它就能观察该客户端的全部入口流量；而守卫过慢或不可访问，也会影响连接质量。

解答 **27.3**。 实验本来就使用 `DROP` 信元，而不是扩展请求。`command.c` 中的计数器在解密前检查外层信元命令，因此与内部中继消息无关。在相反方向，源码会记录警告并使用 `PROTOCOL` 原因码关闭电路，无论电路属于客户端还是中继。

解答 **27.4**。 可能的迹象包括：中继在完全相同的分钟加入共识、描述符中罕见但相同的 tor 版本与操作系统、同一网络或托管商的地址、类似的端口和昵称，以及缺少联系信息。误报情况包括合法大型运营者同时更新全部中继、广泛使用的托管服务商，以及使用相同配置的自动部署。没有单一信号能够证明所有权相同；应组合证据，并尽可能联系运营者。

解答 **27.5**。 中间中继能看到明文外层元数据：CircID（每跳都会变化，因此不会原样穿过电路）；信元命令，包括 **`RELAY`** 和 **`RELAY_EARLY`**，这是 2014 年攻击利用的信道：如今朝向客户端的方向已被封闭，反方向则被限制为八个信元；**`DESTROY`** 及其原因码；还有信元的时间和数量。最后两项仍能用于被动确认，目前没有完全有效的防御。载荷采用分层加密，但在 tor1 中，恶意中继可以修改它并让另一台恶意中继识别（第 10.3 节）。CGO（第 10.4 节）使此类修改导致后续跳点得到无法恢复的明文，从而阻止这种标记。

第 28 章：操作安全失误

解答 **28.1**。 流量绕过 Tor、联网期间打开文档和 DNS 泄露都属于应用层问题；防御措施包括使用 Tor Browser、Tails 或 Whonix、启用 `SafeSocks`，以及断网后再打开文件。元数据、登录账户和混用身份属于内容层问题；应使用元数据清理工具并分离会话。写作风格、发布时间以及在匿名集很小时使用 Tor 属于行为层问题；需要调整使用习惯，考虑网桥并审慎安排发布时间。

解答 **28.2**。 本地 DNS 查询从用户计算机发往系统配置的解析器，通常由网络接入服务商运营，并包含目标网站的域名。接入服务商以及能够观察到解析器通信路径的任何人，都可能获知用户准备访问哪个网站，即使后续连接通过 Tor。

解答 **28.3**。 通过 Stem 订阅 `EventType.STATUS_CLIENT`。只包含 IP 地址的请求会产生严重程度为 `WARN`、动作为 `DANGEROUS_SOCKS` 的事件，参数包含 `PROTOCOL` 与 `ADDRESS`。图形界面可以在警告中标明应用程序与目的地，并说明应该修正该应用的 SOCKS 配置。

解答 **28.4**。 使用 obfs4 时，本地观察者看到的会是一条通向普通外观地址的加密连接，数据没有明显结构，因此无法确定这一定是 Tor 流量。其他线索仍可能存在：流量大小、时间、异常持续时长、相似连接，以及消息内容或可能动机。这些信息依然可能将候选人缩小到预计参加相关考试的人。

解答 **28.5**。 这是开放式练习。好的答案应遵循 第 4 章 的结构：列明攻击者及其能力、他们在 图 28.1 各层能观察到什么、所选择的防御及理由，以及残余风险清单，例如同时观察通信两端的攻击者进行关联，或者用户设备遭到扣押。

第 29 章：Tor 学术研究

解答 **29.1**。 $H = \frac{1}{2}\cdot 1 + \frac{1}{4}\cdot 2 + 2\cdot\frac{1}{8}\cdot 3 = 1.75$ 比特。$H_M = \log_2 4 = 2$ 比特，因此 $d = 0.875$。具有相同不确定性的等概率用户数量为 $2^{1.75} \approx 3.4$。

解答 **29.2**。 （a）使用 Shadow 模拟网络，或在较小规模下通过 Chutney 仿真：需要真实 tor 的行为，但无须涉及真实用户。（b）像 Tor Metrics 一样利用公开的汇总数据测量真实网络（第 24.2 节）。（c）使用跨较长时间的真实共识开展 TorPS 式行为模拟，并可辅以理论分析。

解答 **29.3**。 将 `guard_lifetime` 设为 30 天后，时间戳回溯使实际寿命为 18–30 天，客户端在 180 天内约选择七到八次守卫，模拟得到 32.1% 的失守比例。默认的 60 天对应 48–60 天实际寿命，大约选择三次守卫，失守比例为 18.4%。寿命为 180 天时，只选择一到两次守卫，失守比例为 8.6%。近似地，经过 k 次独立守卫选择，失守比例趋近于 $1-(1-q)^k$，其中 $q = 5\%$；$k \approx 7.5$ 时约为 31%，与 30 天实验结果接近。守卫寿命越长，新选择越少，新增暴露用户越少，但已经选中恶意守卫的人会更长时间处于风险之中。

解答 **29.4**。 根据本实验需要同时观察两端的失守判定标准，两种攻击者都无法使客户端失守。不过，它们观察到的信息仍不同：恶意出口知道访问目的地和未受应用层加密保护的内容，却不知道客户端身份；恶意守卫知道选择它的客户端的 IP 地址，却不知道目的地，还可能尝试网站指纹识别（第 26.4 节）。

解答 **29.5**。 这是研究练习。复用电路会减少每天需要进行的新选择，在没有持久守卫时减缓失守曲线上升。除非恶意中继集中在同一网络中，/16 约束的影响通常较小。自治系统级攻击者会从根本上改变模型，因为它能够在不运营中继的情况下观察流量，因此需要 [Joh+13] 等研究使用的互联网拓扑数据。

第 30 章：Arti

解答 **30.1**。 第 5 章：`tor-llcrypto`。第 7 章：处理通道的 `tor-proto` 和 `tor-chanmgr`。第 8 章：`tor-cell`。第 9–11 章：处理电路、加密、流与流量控制的 `tor-proto`。第 12 章：`tor-netdoc`、`tor-dirmgr` 和 `tor-dirclient`。第 13 章：`tor-circmgr`、`tor-guardmgr` 和 `tor-relay-selection`。第 14 章：`tor-hsclient`、`tor-hsservice` 和 `tor-hscrypto`。第 15 章：工作量证明使用的 `equix` 与 `hashx`。第 16 章：`tor-ptmgr`。

解答 **30.2**。 优点：代码更少、攻击面更小、不必处理两字节 CircID 等特殊情况，也无须维护过时且薄弱的密码学。缺点：Arti 无法与非常旧的中继通信。不过，公共 Tor 网络已经排除了不再被推荐的版本，因此实际影响主要出现在私有网络和长期不更新的安装环境中。

解答 **30.3**。 `decode_cell` 从第五个字节读取命令，据此判断信元是否为变长格式。如果是变长，后两个字节表示消息体长度，总长度为 7+ 声明的数值；否则总长度为 514 字节。收到的输入不足时，函数返回 `Ok(None)`，而不把不完整的信元当成错误；调用方在更多字节到达后重试。这与实验 2 所写的流式解码器要求相同。

解答 **30.4**。 是的。Arti 的 SOCKS 代理默认将使用不同 SOCKS 凭据的流隔离到不同电路，并且也隔离不同的监听器。可以从中继一侧观察电路，例如通过出口的 ControlPort，或者在

Arti 的调试日志（debug）中查看。作为库使用时，可以通过 `TorClient::isolated_client` 或 `StreamPrefs` 隔离令牌明确指定隔离。截至 Arti 2.6.0，其源码 `cfg.rs` 仍把等效的逐端口配置记录为未来工作。

解答 **30.5**。 这是编程练习。把实验 TOML 文件读取为 `TorClientConfig`，通过 `TorClient::create_bootstrapped` 启动客户端，再调用 `connect(("127.0.0.1", 8080))` 建立流，并通过它发送 HTTP 请求。配置必须允许访问本地地址，就像实验 20 生成的文件一样。

第 **31** 章：开放问题

解答 **31.1**。 以本章参考日期为例：ntor-v3 的 提案 332 处于 *Closed*，表示已经实现且内容已纳入正式规范；拥塞控制的 提案 324 为 *Finished*，已实现，但其文本尚未完全纳入规范；CGO 的 提案 359 为 *Open*，虽然功能已存在于代码中，但提案仍在讨论。应检查每份提案头部及仓库状态索引，以确认是否有更新。

解答 **31.2**。 对于传统格式，简单计算需要 $\lceil 1184/498 \rceil = 3$ 个信元来承载 1 184 字节 ML-KEM-768 公钥，以及 $\lceil 1088/498 \rceil = 3$ 个信元承载 1 088 字节密文，总共六个。但是，现有协议甚至不允许这样分片单条中继消息。提案 340 引入了所需的封装方式，允许消息跨越多个信元，也允许多个消息共用一个信元。其建议格式的中继信元消息体为 493 字节，而非 498 字节，因为新格式消耗 16 字节用于认证，即 $509 - 16 = 493$，参见 第 8 章。一条消息的第一个分片还包含封装头部，因此最多只能承载 490 字节有效数据。提案目前仍为普通消息体保留 493 字节上限，因此若要承载更大的后量子 `EXTEND2`，还需要新版本的 `Relay` 子协议解除该限制。即使扩展完成，公钥仍占三个信元，密文另占三个。而且两者传输方向相反——分别从客户端发往中继，再由中继返回客户端——因此不能打包在一起。总数仍是六个信元。

解答 **31.3**。 如果每个客户端都下载不同的中继子集，已经知道或能够影响某个客户端子集的攻击者，就可以通过电路识别它。使用只对该客户端可见的中继构建电路，会泄露电路由谁建立。即使事先不知道子集，长期观察客户端使用哪些中继，也能逐步缩小可能的用户集合。

解答 **31.4**。 筛选用途为 `CONFLUX_LINKED` 的电路，并按 `CONFLUX_ID` 分组（第 18 章），随后比较每对电路的路径。两条分支都会以相同出口结束，符合预期。但在实验网络中，它们经常还共用其他中继。例如 `[test007e, test003r, test006e]` 和 `[test003r, test007e, test006e]` 只是在不同顺序下使用完全相同的节点。这不是 tor 的缺陷。私有网络中合格中继很少，而提案 329 使用的是 `SHOULD NOT` 建议，不是绝对禁止。在拥有数千台中继的公共网络中，这种共享情况罕见得多。

解答 **31.5**。 这是开放式练习。参考格式见 提案 001：包含编号、标题、作者、日期和状态的头部，以及动机、设计、安全性与兼容性分析、实施计划和部署计划。

附录 **B**：密码学进阶

解答 **B.1**。 256 个比特中有五位被固定：最低三位、最高位和次高位。因此，钳制后有 2^{251} 种不同取值，而钳制前有 2^{256} 种。这种减少不会削弱方案：无论如何，该椭圆曲线的安全级别约为 128 比特。

解答 **B.2**。 根据 $y = (u-1)/(u+1)$，可解得 $u = (1+y)/(1-y) \bmod p$。在 Python 中，先从 32 个小端序字节读取 y，并清除符号位；然后计算 `(1 + y) * pow(1 - y, P - 2, P) % P`，再将结果编码为 32 个小端序字节。保留原始 x 符号位，使用 `montgomery_to_edwards` 即可恢复最初的 Edwards 公钥（要求 $y \neq 1$）。

解答 **B.3**。 随机修改后的信元同时通过两项检查的概率约为 $2^{-16} \cdot 2^{-32} = 2^{-48}$：两字节全零 `recognized` 字段，以及四字节正确摘要。但标记攻击并不要求修改后的信元被接受。能够观

察它的恶意同伙，或者拒绝它的终点，可以通过电路关闭、可预测的内容变化等结果识别修改。这种观察可能足以关联电路上的两个位置。

参考文献

[03] *Decreto legislativo 30 giugno 2003, n. 196, Codice in materia di protezione dei dati personali*. 意大利个人数据保护机构（Garante per la protezione dei dati personali）发布的合并文本. 2003-06-30. 网址：`https://www.garanteprivacy.it/`（访问日期 2026-09-23）.

[17] *Legge 20 novembre 2017, n. 167, art. 24 (termine di conservazione dei dati di traffico per finalità di accertamento e repressione dei reati)*.《意大利共和国官方公报》普通系列第 277 号，2017 年 11 月 27 日. 2017-11-20. 网址：`https://www.normattiva.it/uri-res/N2Ls?urn:nir:stato:legge:2017-11-20;167`.

[22] *Regolamento (UE) 2022/2065 del Parlamento europeo e del Consiglio, del 19 ottobre 2022, relativo a un mercato unico dei servizi digitali (regolamento sui servizi digitali)*. 欧盟官方公报 L 277，2022 年 10 月 27 日. 2022-10-19. 网址：`https://eur-lex.europa.eu/eli/reg/2022/2065/oj`.

[Bau+07] Kevin Bauer, Damon McCoy, Dirk Grunwald, Tadayoshi Kohno 和 Douglas Sicker. “Low-Resource Routing Attacks Against Tor”. 载于: *ACM Workshop on Privacy in the Electronic Society (WPES)*. 2007.

[Boc+24] Cecylia Bocovich, Arlo Breault, David Fifield, Serene 和 Xiaokang Wang. “Snowflake, a censorship circumvention system using temporary WebRTC proxies”. 载于: *Proceedings of the 33rd USENIX Security Symposium*. 2024.

[BPW13] Alex Biryukov, Ivan Pustogarov 和 Ralf-Philipp Weinmann. “Trawling for Tor Hidden Services: Detection, Measurement, Deanonymization”. 载于: *IEEE Symposium on Security and Privacy*. 2013.

[Cha81] David L. Chaum. “Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms”. 载于: *Communications of the ACM* 24.2 (1981), 第 84–88 页.

[Cha88] David Chaum. “The Dining Cryptographers Problem: Unconditional Sender and Recipient Untraceability”. 载于: *Journal of Cryptology* 1.1 (1988), 第 65–75 页.

[DDM03] George Danezis, Roger Dingledine 和 Nick Mathewson. “Mixminion: Design of a Type III Anonymous Remailer Protocol”. 载于: *Proceedings of the 2003 IEEE Symposium on Security and Privacy*. 2003, 第 2–15 页.

[Día+02] Claudia Díaz, Stefaan Seys, Joris Claessens 和 Bart Preneel. “Towards Measuring Anonymity”. 载于: *Privacy Enhancing Technologies (PET 2002)*. LNCS. 2002.

[Din14] Roger Dingledine. *Tor security advisory: “relay early” traffic confirmation attack*. 2014-07-30. 网址：`https://blog.torproject.org/tor-security-advisory-relay-early-traffic-confirmation-attack/`（访问日期 2026-09-23）.

[DM06] Roger Dingledine 和 Nick Mathewson. “Anonymity Loves Company: Usability and the Network Effect”. 载于: *Proceedings of the Fifth Workshop on the Economics of Information Security (WEIS 2006)*. 2006.

[DMS04] Roger Dingledine, Nick Mathewson 和 Paul Syverson. “Tor: The Second-Generation Onion Router”. 载于: *Proceedings of the 13th USENIX Security Symposium*. 2004, 第 303–320 页.

[EDG09] Nathan S. Evans, Roger Dingledine 和 Christian Grothoff. “A Practical Congestion Attack on Tor Using Long Paths”. 载于: *Proceedings of the 18th USENIX Security Symposium*. 2009.

[Ele] Electronic Frontier Foundation. *Legal FAQ for Tor Relay Operators*. 网址: `https://community.torproject.org/relay/community-resources/eff-tor-legal-faq/` (访问日期 2026-09-23).

[Fif+15] David Fifield, Chang Lan, Rod Hynes, Percy Wegmann 和 Vern Paxson. “Blocking-resistant communication through domain fronting”. 载于: *Proceedings on Privacy Enhancing Technologies* 2015.2 (2015), 第 46–64 页.

[Fro+19] Sergey Frolov, Jack Wampler, Sze Chuen Tan, J. Alex Halderman, Nikita Borisov 和 Eric Wustrow. “Conjure: Summoning Proxies from Unused Address Space”. 载于: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2019.

[GRS96] David M. Goldschlag, Michael G. Reed 和 Paul F. Syverson. “Hiding Routing Information”. 载于: *Information Hiding, First International Workshop*. 卷 1174. LNCS. Springer, 1996, 第 137–150 页.

[GSU13] Ian Goldberg, Douglas Stebila 和 Berkant Ustaoglu. “Anonymity and One-Way Authentication in Key Exchange Protocols”. 载于: *Designs, Codes and Cryptography* 67.2 (2013), 第 245–269 页.

[Jan+14] Rob Jansen, Florian Tschorsch, Aaron Johnson 和 Björn Scheuermann. “The Sniper Attack: Anonymously Deanonymizing and Disabling the Tor Network”. 载于: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2014.

[JH12] Rob Jansen 和 Nicholas Hopper. “Shadow: Running Tor in a Box for Accurate and Efficient Experimentation”. 载于: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2012.

[JJ16] Rob Jansen 和 Aaron Johnson. “Safely Measuring Tor”. 载于: *ACM Conference on Computer and Communications Security (CCS)*. 2016.

[Joh+13] Aaron Johnson, Chris Wacek, Rob Jansen, Micah Sherr 和 Paul Syverson. “Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries”. 载于: *ACM Conference on Computer and Communications Security (CCS)*. 2013.

[Jua+14] Marc Juarez, Sadia Afroz, Gunes Acar, Claudia Diaz 和 Rachel Greenstadt. “A Critical Evaluation of Website Fingerprinting Attacks”. 载于: *ACM Conference on Computer and Communications Security (CCS)*. 2014.

[Jus14] Just Security. *Video clip of former director of NSA and CIA: “We kill people based on metadata”*. Johns Hopkins Foreign Affairs Symposium，2014 年 4 月 1 日. 2014. 网址: `https://www.justsecurity.org/10318/video-clip-director-nsa-cia-we-kill-people-based-metadata/` (访问日期 2026-09-24).

[MD05] Steven J. Murdoch 和 George Danezis. “Low-Cost Traffic Analysis of Tor”. 载于: *IEEE Symposium on Security and Privacy*. 2005.

[NBH18] Milad Nasr, Alireza Bahramali 和 Amir Houmansadr. “DeepCorr: Strong Flow Correlation Attacks on Tor Using Deep Learning”. 载于: *ACM Conference on Computer and Communications Security (CCS)*. 2018.

[NIS24] NIST. *FIPS 203* (*ML-KEM*), *FIPS 204* (*ML-DSA*), *FIPS 205* (*SLH-DSA*). 三项后量子标准，同时发布. 2024-08-13. 网址：`https://csrc.nist.gov/pubs/fips/203/final` (访问日期 2026-09-24).

[OON] OONI. *Specification: Tor* (*ts-023*) *and Tor using Snowflake* (*ts-030*). 网址：`https://github.com/ooni/spec/tree/master/nettests` (访问日期 2026-09-23).

[ØS06] Lasse Øverlier 和 Paul Syverson. "Locating Hidden Servers". 载于: *Proceedings of the 2006 IEEE Symposium on Security and Privacy*. 2006, 第 100–114 页.

[Pan+11] Andriy Panchenko, Lukas Niessen, Andreas Zinnen 和 Thomas Engel. "Website Fingerprinting in Onion Routing Based Anonymization Networks". 载于: *ACM Workshop on Privacy in the Electronic Society* (*WPES*). 2011.

[Per+18] Mike Perry, Erinn Clark, Steven Murdoch 和 Georg Koppen. *The Design and Implementation of the Tor Browser*. 草案. 2018-06-15. 网址：`https://2019.www.torproject.org/projects/torbrowser/design/` (访问日期 2026-09-23).

[PH10] Andreas Pfitzmann 和 Marit Hansen. *A terminology for talking about privacy by data minimization: Anonymity, Unlinkability, Undetectability, Unobservability, Pseudonymity, and Identity Management*. 版本 0.34. 2010.

[RR98] Michael K. Reiter 和 Aviel D. Rubin. "Crowds: Anonymity for Web Transactions". 载于: *ACM Transactions on Information and System Security* 1.1 (1998), 第 66–92 页.

[San13] Runa A. Sandvik. *Harvard Student Receives F For Tor Failure While Sending "Anonymous" Bomb Threat*. 2013-12-18. 网址：`https://www.forbes.com/sites/runasandvik/2013/12/18/harvard-student-receives-f-for-tor-failure-while-sending-anonymous-bomb-threat/` (访问日期 2026-09-24).

[Sir+18] Payap Sirinam, Mohsen Imani, Marc Juarez 和 Matthew Wright. "Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning". 载于: *ACM Conference on Computer and Communications Security* (*CCS*). 2018.

[Sun+15] Yixin Sun, Anne Edmundson, Laurent Vanbever, Oscar Li, Jennifer Rexford, Mung Chiang 和 Prateek Mittal. "RAPTOR: Routing Attacks on Privacy in Tor". 载于: *USENIX Security Symposium*. 2015.

[Syv11] Paul Syverson. "A Peel of Onion". 载于: *Proceedings of the 27th Annual Computer Security Applications Conference* (*ACSAC*). 2011, 第 123–137 页.

[Taia] Tails. *How Tails works*. 网址：`https://tails.net/about/` (访问日期 2026-09-23).

[Taib] Tails. *Warnings: Tails is safe but not magic!* 网址：`https://tails.net/doc/about/warnings/` (访问日期 2026-09-23).

[TG23] Lindsey Tulloch 和 Ian Goldberg. "Lox: Protecting the Social Graph in Bridge Distribution". 载于: *Proceedings on Privacy Enhancing Technologies* 2023.1 (2023), 第 494–509 页.

[Thea] The Tor Project. *History*. 网址：`https://www.torproject.org/about/history/` (访问日期 2026-09-23).

[Theb] The Tor Project. *Onionbalance: documentazione e guida per gli onion service v3*. 网址：`https://onionservices.torproject.org/apps/base/onionbalance/` (访问日期 2026-09-23).

[Thec] The Tor Project. *Reproducible Metrics*. 网址：`https://metrics.torproject.org/reproducible-metrics.html` (访问日期 2026-09-23).

[Thed] The Tor Project. *Tor Browser User Manual*. 网址：`https://tb-manual.torproject.org/` (访问日期 2026-09-23).

[Thee] The Tor Project. *Tor Specifications*. 网址：https://spec.torproject.org/（访问日期 2026-09-23）.

[Thef] The Tor Project. *Tor Support: Staying anonymous*. 网址：https://support.torproject.org/faq/staying-anonymous/（访问日期 2026-09-23）.

[Theg] The Tor Project. *Tor Support: Using Bridges; Unblocking Tor*. 网址：https://support.torproject.org/little-t-tor/circumvention/using-bridges/（访问日期 2026-09-23）.

[The22] The Tor Project. *Malicious relays and the health of the Tor network*. 2022-04-25. 网址：https://blog.torproject.org/malicious-relays-health-tor-network/（访问日期 2026-09-23）.

[The23a] The Tor Project. *Introducing Proof-of-Work Defense for Onion Services*. 2023-08-23. 网址：https://blog.torproject.org/introducing-proof-of-work-defense-for-onion-services/（访问日期 2026-09-23）.

[The23b] The Tor Project. *Tor is slow right now. Here is what is happening*. 2023-02-07. 网址：https://blog.torproject.org/tor-network-ddos-attack/（访问日期 2026-09-23）.

[The24a] The Tor Project. *Hiding in plain sight: Introducing WebTunnel*. https://blog.torproject.org/introducing-webtunnel-evading-censorship-by-hiding-in-plain-sight/. 2024 年 3 月 12 日. 2024.

[The24b] The Tor Project. *Making new connections: from BridgeDB to Rdsys*. 2024. 网址：https://blog.torproject.org/making-connections-from-bridgedb-to-rdsys/（访问日期 2026-09-24）.

[The24c] The Tor Project. *Uniting for Internet Freedom: Tor Project & Tails Join Forces*. 2024-09-26. 网址：https://blog.torproject.org/tor-tails-join-forces/（访问日期 2026-09-24）.

[Tor] Tor Research Safety Board. *Tor Research Safety Board*. 网址：https://research.torproject.org/safetyboard/（访问日期 2026-09-23）.

[Tor25] Tor Project Forum. *Feedback from China [April 2025]: increased GFW censorship; obfs4/meek/snowflake unusable, webtunnel connects but is quickly blocked*. 2025. 网址：https://forum.torproject.org/t/feedback-from-china-april-2025-increased-gfw-censorship-obfs4-meek-snowflake-unusable-webtunnel-connects-but-is-quickly-blocked/18233（访问日期 2026-09-24）.

[WBU13] WBUR News. *Student Charged For Bomb Threats At Harvard*. 2013-12-17. 网址：https://www.wbur.org/news/2013/12/17/harvard-bomb-hoax-student-charges（访问日期 2026-09-23）.

[Who] Whonix. *onion-grater*. 网址：https://www.whonix.org/wiki/Onion-grater（访问日期 2026-09-23）.

[WL14] Philipp Winter 和 Stefan Lindskog. “Spoiled Onions: Exposing Malicious Tor Exit Relays”. 载于: *arXiv:1401.4917*. 后经其他作者参与，收录于 *Privacy Enhancing Technologies Symposium*（PETS 2014）. 2014.

索引

AUTH_CHALLENGE, 33, 37, 39, 41, 47, 48, 51, 146
AUTHENTICATE, 33, 37, 41, 47, 48
AUTHORIZE, 47

CERTS, 29, 33, 34, 36, 37, 39, 41, 47, 48, 51, 146, 230
CREATE, 47
CREATE2, 29, 46–48, 51, 55, 57, 58, 60, 64, 65, 230, 237
CREATE_FAST, 23, 47, 59, 60, 65, 113, 231
CREATED, 47
CREATED2, 47, 48, 51, 57–59, 65, 238
CREATED_FAST, 47

DESTROY, 47, 48, 51, 54, 55, 58, 64, 68, 76, 79, 80, 113, 192, 230–232, 241

NETINFO, 33, 38–41, 46–48, 51, 146

PADDING, 39, 41, 46–48, 64, 72, 73, 230, 238
PADDING_NEGOTIATE, 47, 48, 230

RELAY, 39, 47–49, 51, 70, 150, 152, 191, 220, 238, 241
RELAY_EARLY, 39, 47–49, 51, 54, 55, 59, 70, 72, 73, 191–194, 230, 238, 241

torrc
__LeaveStreamsUnattached, 134, 137
AccountingMax, 162, 193
AssumeReachable, 143
AutomapHostsOnResolve, 129
BridgeDistribution, 118, 165, 166
ClientOnionAuthDir, 157, 158
ContactInfo, 162
ControlPort, 129, 133
CookieAuthentication, 130
DirAuthority, 143, 236
DirPortFrontPage, 180
DNSPort, 77, 129
EntryNodes, 129
ExcludeNodes, 129
ExitNodes, 129
ExitPolicy, 166
ExitPolicyRejectPrivate, 162
ExitPolicyRejectPrivate 0, 163
ExitRelay 0, 162
FamilyId, 166
HashedControlPassword, 130
HiddenServiceDir, 155, 156, 159
HiddenServiceNonAnonymousMode, 158
HiddenServicePort, 104, 156, 159
HiddenServiceSingleHopMode, 158
HSLayer2Nodes, 96
HSLayer3Nodes, 96
IsolateSOCKSAuth, 168, 186
KeepalivePeriod, 31
Log, 153
MaxCircuitDirtiness, 129
MaxMemInQueues, 110, 114, 115, 235
MyFamily, 164–166, 238
OfflineMasterKey, 166
OfflineMasterKey 1, 162
ReducedExitPolicy, 163, 166
RelayBandwidthBurst, 162
RelayBandwidthRate, 110, 162, 239
SafeLogging, 130, 152, 181, 211
SafeSocks, 196, 197, 241
SessionGroup, 129, 236
SigningKeyLifetime, 36
SocksPort, 128, 130
StrictNodes, 129
TestingDirAuthVoteGuard, 143, 237
TestingLinkCertLifetime, 36
TestingTorNetwork, 143, 147, 210, 236
TestSocks, 196, 197
TransPort, 129
UseBridges, 235
V3AuthDistDelay, 85
V3AuthNIntervalsValid, 85

V3AuthVoteDelay, 85
V3AuthVotingInterval, 85, 143, 237

VERSIONS, 33, 39, 40, 45–48, 51, 53–55, 145, 146, 151, 152, 209, 220, 230, 231, 238
VPADDING, 33, 40, 47, 48

发起方, 31
响应方, 31

提案
001, 213, 217, 243
214, 47
216, 60, 63
220, 33
224, 100
236, 95
271, 95
289, 49, 70, 77, 80, 81
291, 95
300, 213, 215, 217
302, 188
323, 215, 217
324, 50, 62, 78, 79, 81, 213, 231, 243
327, 111
329, 50, 145, 213, 215, 216, 243
332, 61, 63, 65, 213, 243
333, 96
338, 38, 229
339, 50
340, 50, 214, 216, 243
346, 30
348, 215
350, 59, 213, 214
355, 214, 217
359, 50, 65, 69, 213, 231, 243
362, 215